

SPCA 113N

**MASTER OF
COMPUTER APPLICATIONS**

FIRST YEAR

SECOND SEMESTER

INTER DISCIPLINARY - II

**WEB BASED APPLICATION
DEVELOPMENT**



**INSTITUTE OF DISTANCE EDUCATION
UNIVERSITY OF MADRAS**

**MASTER OF COMPUTER APPLICATIONS
FIRST YEAR - SECOND SEMESTER**

**INTER DISCIPLINARY - II
WEB BASED APPLICATION
DEVELOPMENT**

WELCOME

Warm Greetings.

It is with a great pleasure to welcome you as a student of Institute of Distance Education, University of Madras. It is a proud moment for the Institute of Distance education as you are entering into a cafeteria system of learning process as envisaged by the University Grants Commission. Yes, we have framed and introduced as per AICTE Choice Based Credit System(CBCS) in Semester pattern from the academic year 2020-21. You are free to choose courses, as per the Regulations, to attain the target of total number of credits set for each course and also each degree programme. What is a credit? To earn one credit in a semester you have to spend 30 hours of learning process. Each course has a weightage in terms of credits. Credits are assigned by taking into account of its level of subject content. For instance, if one particular course or paper has 4 credits then you have to spend 120 hours of self-learning in a semester. You are advised to plan the strategy to devote hours of self-study in the learning process. You will be assessed periodically by means of tests, assignments and quizzes either in class room or laboratory or field work. In the case of PG (UG), Continuous Internal Assessment for 20(25) percentage and End Semester University Examination for 80 (75) percentage of the maximum score for a course / paper. The theory paper in the end semester examination will bring out your various skills: namely basic knowledge about subject, memory recall, application, analysis, comprehension and descriptive writing. We will always have in mind while training you in conducting experiments, analyzing the performance during laboratory work, and observing the outcomes to bring out the truth from the experiment, and we measure these skills in the end semester examination. You will be guided by well experienced faculty.

I invite you to join the CBCS in Semester System to gain rich knowledge leisurely at your will and wish. Choose the right courses at right times so as to erect your flag of success. We always encourage and enlighten to excel and empower. We are the cross bearers to make you a torch bearer to have a bright future.

With best wishes from mind and heart,

DIRECTOR

**MASTER OF COMPUTER APPLICATIONS
FIRST YEAR - SECOND SEMESTER**

**INTER DISCIPLINARY - II
WEB BASED APPLICATION
DEVELOPMENT**

COURSE WRITER

Dr. B. LAVANYA

Associate Professor in Computer Science
University of Madras
Chepauk, Chennai - 600 005.

CO-ORDINATION & EDITING

Dr. S. SASIKALA

Associate Professor in Computer Science
Institute of Distance Education
University of Madras
Chepauk, Chennai - 600 005.

MASTER COMPUTER APPLICATIONS
FIRST YEAR
SECOND SEMESTER
CORE PAPER - X
INTER DISCIPLINARY - II
WEB BASED APPLICATION DEVELOPMENT
SYLLABUS

Objective of the course: Student will be familiar with client server architecture and able to develop a web application using java technologies. They will gain the skills and project-based experience needed for entry into web application and development careers.

Course Outcomes: After successful completion of this course, Students are able to develop a dynamic webpage by the use of java script and DHTML and write a well formed / valid XML document. They are able to connect a java program to a DBMS and perform insert, update and delete operations on DBMS table. Students will be able to write a server-side java application called Servlet to catch form data sent from client, process it and store it on database.

Unit – I: OVERVIEW OF ASP.NET - The .NET framework – The C# Language: Data types – Declaring variables- Scope and Accessibility- Variable operations- Object Based manipulation- Conditional Structures- Loop Structures- Methods. Types, Objects and Namespaces: The Basics about Classes- Value types and Reference types- Understanding name spaces and assemblies - Advanced class programming.

Unit – II: Developing ASP.NET Applications - The Anatomy of a Web Form – Writing Code - Visual Studio Debugging. Web Form Fundamentals: The Anatomy of an ASP.NET Application - Introducing Server Controls - HTML Control Classes - The Page Class - Application Events - ASP.NET Configuration. Web Controls: Web Control Classes - List Controls - Web Control Events and AutoPostBack - A Simple Web Page.

Unit – III: Error Handling, Logging, and Tracing: Common Errors - Exception Handling - Handling Exceptions - Throwing Your Own Exceptions - Logging Exceptions - Page Tracing. State Management: View State - Transferring Information Between Pages – Cookies - Session State - Session State Configuration - Application State. Validation: Understanding Validation - The Validation Controls.

Unit – IV: Rich Controls: The Calendar - The AdRotator - Pages with Multiple Views - User Controls and Graphics - User Controls - Dynamic Graphics. Website Navigation: Site Maps - URL Mapping and Routing - The SiteMapPath Control - The TreeView Control - The Menu Control. ADO.NET Fundamentals: The Data Provider Model - Direct Data Access - Disconnected Data Access.

Unit – V: Data Binding: Single-Value Data Binding - Repeated-Value Data Binding - Data Source Controls - The Data Controls: The GridView - Formatting the GridView - Selecting a GridView Row - Editing with the GridView - Sorting and Paging the GridView - Using GridView Templates - The DetailsView and FormView – XML: The XML Classes - XML Validation - XML Display and Transforms. Website Security: Security Fundamentals - Understanding Security - Authentication and Authorization - FormsAuthentication - Windows Authentication.

Recommended Texts

- 1) Matthew MacDonald, “Beginning ASP.NET 4 in C# 2010”, Apress 2010.

Reference Books

- 1) Crouch Matt J, “ASP.NET and VB.NET Web Programming”, Addison Wesley 2002.
- 2) Mathew Mac Donald, “ASP.NET Complete Reference”, TMH 2005
- 3) J.Liberty, D.Hurwitz, “Programming ASP.NET”, Third Edition, O’REILLY, 2006.

E-learning resources

- 1) [https://msdn.microsoft.com/en-in/library/aa288436\(v=vs.71\).aspx](https://msdn.microsoft.com/en-in/library/aa288436(v=vs.71).aspx)
- 2) <http://www.asp.net/>

MASTER COMPUTER APPLICATIONS

FIRST YEAR

SECOND SEMESTER

CORE PAPER - X

INTER DISCIPLINARY - II

WEB BASED APPLICATION DEVELOPMENT

SCHEME OF LESSONS

Sl.No.	Title	Page
1..	net and Asp.net Framework	001
2.	C#	036
3.	Asp.Net Application Essentials	094
4.	Web page development	140
5.	Error handling	180
6.	State Management	227
7.	Rich Controls, User Control and Graphics	274
8.	Sitemaps and ado.Net Fundamentals	319
9.	Data Binding	385
10.	The data Control	433
11.	XML	472

LESSON - 1

.NET AND ASP.NET FRAMEWORK

Structure

- 1.1 Introduction**
- 1.2 Objectives**
- 1.3 Evolution of .Net framework**
- 1.4 Benefits of .Net framework**
- 1.5 Overview of .Net framework**
- 1.6 Features of .Net framework**
- 1.7 Asp .net Framework**
- 1.8 Understanding Service Lifetime**
- 1.9 ASP.Net Web Forms**
- 1.10 DataModel**
- 1.11 ASP.Net Life Cycle**
- 1.12 ASP.Net WebServices**
- 1.13 Summary**
- 1.14 Keywords**
- 1.15 Check your progress**
- 1.16 Model Questions**

1.1 Introduction

.NET is a developer platform made up of tools, programming languages, and libraries for building many different types of applications. It is platform independent, Cross platform and

also language insensitive. There are various implementations of .NET. Each implementation allows .NET code to execute in different places—Linux, macOS, Windows, iOS, Android. It is a virtual machine for compiling and executing programs written in different languages like C#, VB.Net etc.

1.2 Learning objectives

- ✓ Learn the evolution and benefits of .net framework
- ✓ Understand the overview of .net framework
- ✓ Learn the features of .net framework 4.0
- ✓ Understand the ASP .Net core services
- ✓ Understand the ASP.Net web forms
- ✓ Learn the dynamic data framework
- ✓ Understand the ASP.net lifecycle
- ✓ Learn the Asp.net web services

1.3 Evolution of .Net framework

Until the arrival of .NET, the Microsoft programming ecosystem had been ruled by a few classic languages, Visual Basic and C++. However, the big changes proposed by .NET were started using a totally different component model approach. Up until 2002, when .NET officially appeared, such a component model was COM (Component Object Model), introduced by the company in 1993. COM is the basis for several other Microsoft technologies and frameworks, including OLE, OLE automation, ActiveX, COM+, DCOM, the Windows shell, DirectX, UMDF (User-Mode Driver Framework), and Windows runtime. At the time of writing this, COM is a competitor with another specification named CORBA (Common Object Request Broker Architecture), a standard defined by the Object Management Group (OMG), designed to facilitate the communication of systems that are deployed on diverse platforms. CORBA enables collaboration between systems on different operating systems, programming languages, and computing hardware. In its life cycle, it has received a lot of criticism, mainly because of poor implementations of the standard.

HTML and HTML Forms

HTML and HTML Forms, it would be difficult to describe early websites as web applications. Instead, the first generation of websites often looked more like brochures, consisting mostly of fixed HTML pages that needed to be updated by hand. A basic HTML page is a little like a word-processing document—it contains formatted content that can be displayed on your computer, but it doesn't actually do anything. The following example shows HTML at its simplest, with a document that contains a heading and a single line of text:

```
<html>

<head>

<title>Sample Web Page </title>

</head>

<body>

<h1>Sample Web Page Heading </h1>

<p> This is a sample web page </p>

</body>

</html>
```

An HTML document has two types of content: the text and the elements (or tags) that tell the browser how to format it. The elements are easily recognizable, because they are designated with angled brackets (<>). HTML defines elements for different levels of headings, paragraphs, hyperlinks, italic and bold formatting, horizontal lines, and so on. For example, <h1> Some Text </h1> uses the <h1> element. This element tells the browser to display Some Text in the Heading 1 style, which uses a large, bold font. Similarly, <p> This is a sample web page, </p> creates a paragraph with one line of text. The <head> element groups the header information together and includes the <title> element with the text that appears in the browser window, while the <body> element groups together the actual document content that's displayed in the browser window.

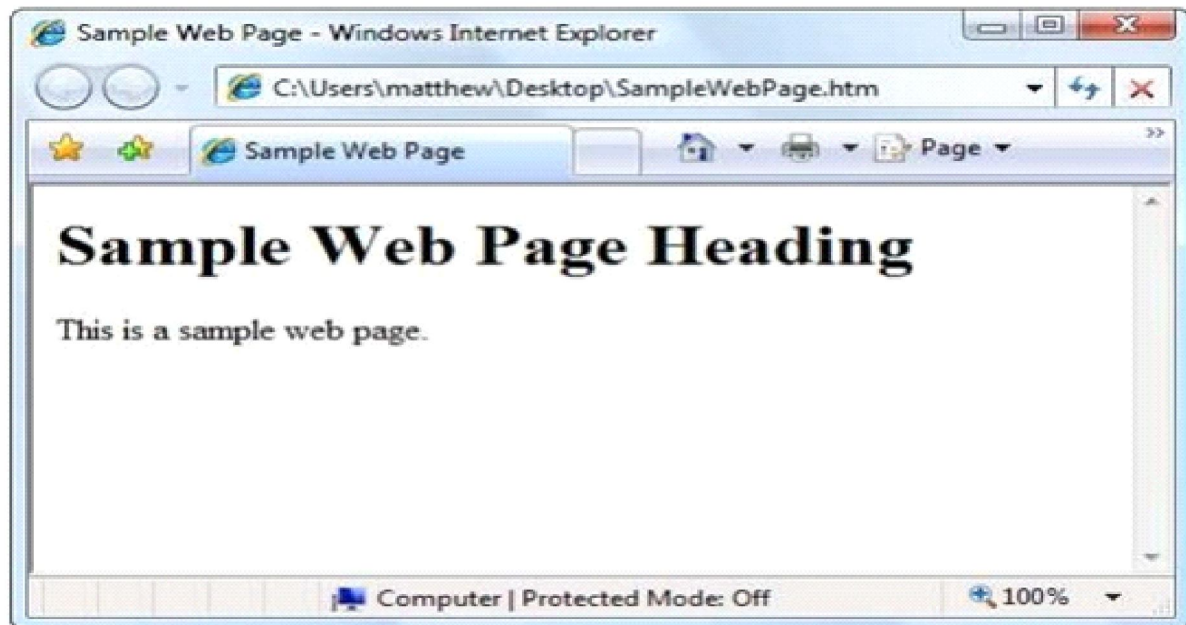


Figure1: Ordinary HTML

HTML 2.0 introduced the first seed of web programming with a technology called HTML forms. HTML forms expand HTML so that it includes not only formatting tags but also tags for graphical widgets, or controls. These controls include common ingredients such as drop-down lists, text boxes, and buttons. Here's a sample web page created with HTML form controls:

```
<html>
```

```
<head>
```

```
<title> Sample Web Page </title>
```

```
</head>
```

```
<body>
```

```
<form>
```

```
<input type =" checkbox"/> This is choice #1<br/>
```

```
<input type =" checkbox"/> This is choice #2<br/>
```

```
Input type = "submit" value = "submit"/>
```

```
</form>
```

```
</body>
```

```
</html>
```

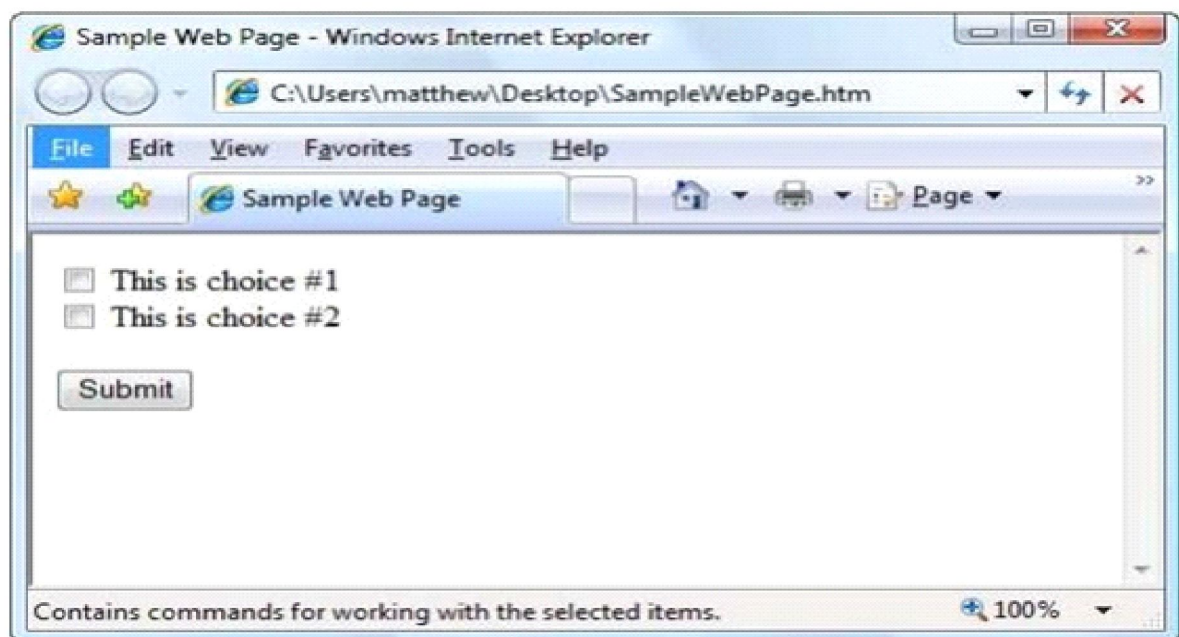


Figure 2: A HTML form

HTML forms allow web developers to design standard input pages. When the user clicks the Submit button on the page shown in Figure 1-2, all the data in the input controls (in this case, the two check boxes) is patched together in to one long string of text and sent to the web server. On the server side, a custom application receives and processes the data. Amazingly enough, the controls that were created for HTML forms more than ten years ago are still the basic foundation that you'll use to build dynamic ASP.NET pages! The difference is the type of application that runs on the server side. In the past, when the user clicked a button on a form page, the information might have been e-mailed to a set account or sent to an application on the server that used the challenging Common Gateway Interface (CGI) standard.

Server-Side Programming

If you wanted higher-level features, such as the ability to authenticate users, store personalized information, or display records you've retrieved from a database, you needed to write pages of code from scratch. Building a web application this way is tedious and error-prone. To counter these problems, Microsoft created higher-level development platforms first ASP and then ASP.NET. Both of these technologies allow developer stop program dynamic web pages without worrying about the low-Level implementation details. For that reason, both platforms have been incredibly successful.

The original ASP platform garnered a huge audience of nearly 1 million developers, becoming far more popular than even Microsoft anticipated. It wasn't long before it was being wedged into all sorts of unusual places, including mission-critical business applications and highly trafficked e-commerce sites. Because ASP wasn't designed with these uses in mind, performance, security, and configuration problems soon appeared. That's where ASP.NET comes into the picture. ASP.NET was developed as an industrial-strength web application framework that could address the limitations of ASP.

Client-Side Programming

At the same time that server-side web development was moving through an alphabetsoup of technologies, a new type of programming was gaining popularity. Developers began to experiment with the different ways they could enhance web pages by embedding miniature applets built with JavaScript, ActiveX, Java, and Flash into web pages. These client-side technologies don't involve any server processing. Instead, the complete application is downloaded to the client browser, which executes it locally. The greatest problem with client-side technologies is that they aren't supported equally by all browsers and operating systems. One of the reasons that web development is so popular in the first place is because web applications don't require setup CDs, downloads, client-side configuration, and other tedious (and error-prone) deployment steps. Instead, a web application can be used on any computer that has Internet access. But when developers use client-side technologies, they encounter a few familiar headaches. Suddenly, cross-browser compatibility becomes a problem. Developers are forced to test their websites with different operating systems and browsers and to deal with a wide range of browser quirks, bugs, and legacy behaviours. In other words, the client-side model sacrifices some of the most important benefits of web development. For that reason, ASP.NET is designed first and foremost as a server-side technology. All ASP.NET code executes

on the server. When the code is finished executing, the user receives an ordinary HTML page, which can be viewed in any browser. Figure-3 shows the difference between the server-side and client-side models.

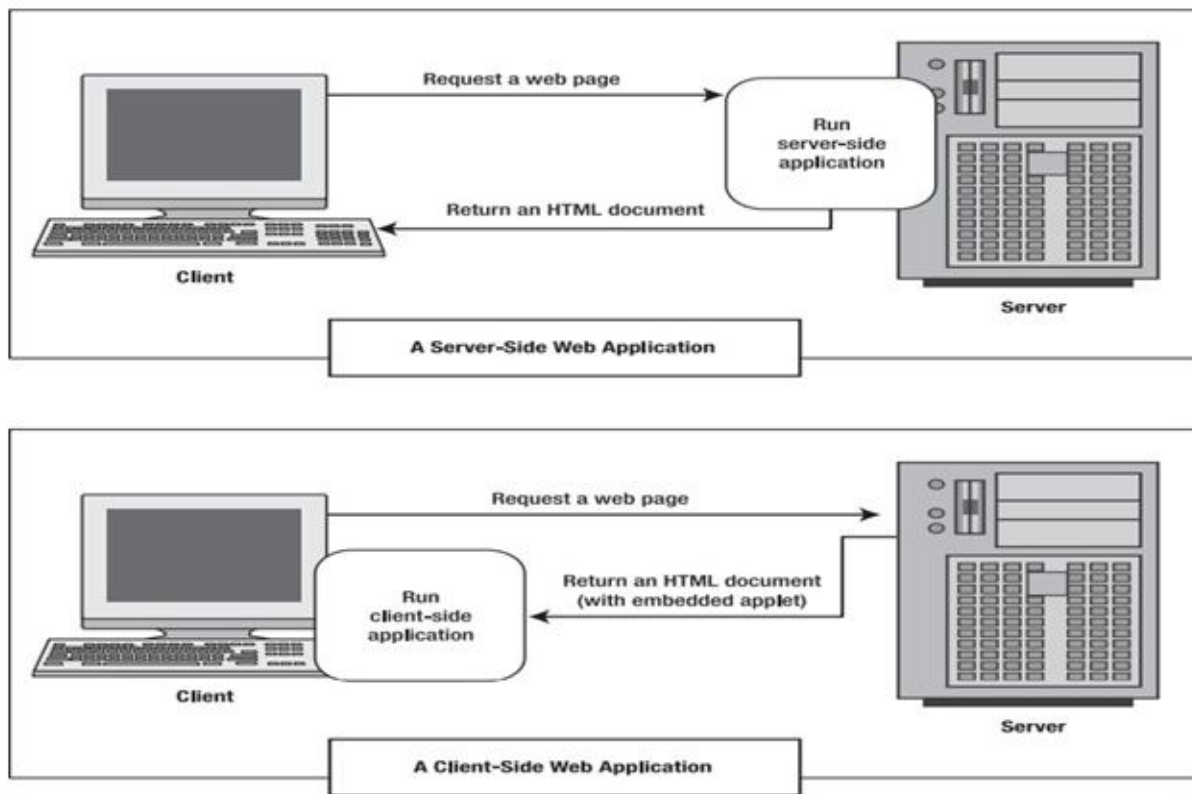


Figure 3: Server-side and client-side web application

These are some other reasons for avoiding client-side programming:

Isolation: Client-side code can't access server-side resources. For example, a client-side application has no easy way to read a file or interact with a database on the server (at least not without running into problems with security and browser compatibility).

Security: End users can view client-side code. And once malicious users understand how an application works, they can often tamper with it.

Thin clients: In today's world, web-enabled devices such as mobile phones, handheld computers, and personal digital assistants (PDAs) are pervasive. These devices usually have some sort of

built-in web browsing ability, but they don't support all the features of traditional desktop-based browsers. For example, thin clients might not support client-side features such as JavaScript and Flash.

1.4 Benefits of .Net framework

a) Less Coding and Increased Reuse of Code: This framework works on object-oriented programming which eliminates unnecessary codes and involves less coding for the developers. .NET consists of re-useable code and many re-useable components. This translates into less time and consequently less cost to develop applications.

b) Deployment: With features such as no-impact applications, private components, controlled code sharing, side-by-side versioning and partially trusted code, the .NET framework makes deployment easier post development. The code execution environment supports safe code execution for reduced conflicts in software deployment and versioning, and minimized performance problems of scripted or interpreted environments.

c) Reliability: Since its release in 2002, .NET has been used to develop thousands of applications. Its performance on Microsoft® Windows Server™ 2003 and Windows 2000 Server is also very stable and reliable.

d) Security: .NET offers enhanced application security as web applications developed using ASP.NET have Windows confirmation and configuration. Managed code and CLR offer safeguard features such as role-based security and code access security.

e) Use Across Platforms and Languages: .NET allows developers to develop applications for a desktop, a browser, a mobile browser (like on your cell phone), or an application running on PDA. .NET is promoted as a language-independent framework, which implies that development can take place in different compliant languages that include C#, managed C++, VB.NET, Visual COBOL, Iron Python, Iron Ruby and more.

f) Use for Service-Oriented Architecture: .NET is often used for Web Services, which are a solution for executing an SOA strategy. Through Web Services, applications which are designed in different programming languages or platforms are able to communicate and transmit data utilizing standard Internet protocols.

g) Integration with Legacy Systems: The capability of .NET to process all types of XML documents and write any format of file with swiftness and ease, provides multiple routes for integration.

1.5 Overview of .Net framework

Version number	CLR version	Release date	Development tool	Included in		Replaces
				Windows	Windows Server	
1.0	1.0	2002-02-13	Visual Studio .NET ^[22]	XP ^[x]	N/A	N/A
1.1	1.1	2003-04-24	Visual Studio .NET 2003 ^[22]	N/A	2003	1.0 ^[23]
2.0	2.0	2005-11-07	Visual Studio 2005 ^[24]	N/A	2003, 2003 R2, ^[25] 2008 SP2, 2008 R2 SP1	N/A
3.0	2.0	2006-11-06	Expression Blend ^{[26][x]}	Vista	2008 SP2, 2008 R2 SP1	2.0 ^[26]
3.5	2.0	2007-11-19	Visual Studio 2008 ^[27]	7, 8 ^[x] , 8.1 ^[x] , 10 ^[x]	2008 R2 SP1	2.0, 3.0 ^[26]
4.0	4	2010-04-12	Visual Studio 2010 ^[28]	N/A	N/A	N/A
4.5	4	2012-08-15	Visual Studio 2012 ^[29]	8	2012	4.0 ^[29]
4.5.1	4	2013-10-17	Visual Studio 2013 ^[30]	8.1	2012 R2	4.0, 4.5 ^[29]
4.5.2	4	2014-05-05	N/A	N/A	N/A	4.0–4.5.1 ^[29]
4.6	4	2015-07-20	Visual Studio 2015 ^[31]	10	N/A	4.0–4.5.2 ^[29]
4.6.1	4	2015-11-17 ^[32]	Visual Studio 2015 Update 1	10 Version 1511	N/A	4.0–4.6 ^[29]

Table 1: Overview / history of .Net Framework

1. .NET Framework 1.0

The first version of .NET Framework.

2. .NET Framework 1.1 C# Version: 1.2(April 2003) CLR Version:1.1

Major Features:

- ASP.NET and ADO.NET updates
- Side by side execution
- .NET Compact framework
- Secure coding guideline
- Information about application deployment

3. .NET Framework 2.0

C# Version: 2.0(November 2003) CLR Version:2.0

Major Features:

- 64-bit platform support
- Access Control ListSupport
- Authenticated Stream
- COM Interop services enhancement
- Console Class addition
- Data Protection APIs
- Debugger display attributes
- Debugger edit and continuous support
- EventLog Enhancement
- FTPSupport
- Generics and Generic Collection
- Globalization
- I/O enhancement
- Manifest-based activation
- .NET Framework Remoting
- Ping
- Obtaining information about Local Computer
- Processing HTTP request within the application
- Programmatic control of caching
- Security Exception
- Serialization

- SMTP Support
- Strongly Type Support
- Threading Improvements
- Trace Data Filtering
- Transactions
- Web Services

4. .NET Framework 3.0

C# Version: 3.0(November 2007) CLR Version:2.0

Major Features:

- Windows Communication Foundation
- Windows Presentation Foundation
- Windows Forms
- Card Space

5. .NET Framework 3.5

C# Version: 3.0(November 2007) CLR Version:2.0

Major Features,

- ASP.NET Ajax-enabled Websites
- LINQ
- Dynamic Data
- Collections
- I/O and Pipes
- Latency Mode in Garbage Collection

- Better Reader and Writer Lock
- ThreadPool Performance Enhancement
- Time Zone Improvements
- TimeZone Info
- DateTime Offset
- Cryptography Enhancements
- Peer-to-peer networking
- Collaboration using Peer-to-peer networking
- Socket Performance Enhancements
- Durable Services
- WCF Syndications
- WCF and Partial Trusts
- Web Service interoperability

6. .NET Framework 4.0

C# Version: 4.0(April 2010) CLR Version:4

Major Features,

- Expanded BaseClass
- Cross-Platform Development with a portable class library
- Managed Extensibility Framework
- Dynamic Language Runtime
- Code Contracts

- Covariance and Contravariance
- Parallel Computing

7. .NET Framework 4.5

C# Version: 5.0(August 2012) CLR Version:4

Major Features,

- Support for Windows StoreApps
- Support for X509 certificates containing FIPS 186-3DSA
- Increased clarity for inputs to ECD iffie Hellman key derivation routines
- persisted-key symmetric encryption
- SHA-2 Hashing
- Soft Keyboard Support
- Per-monitor DPI
- WPF, WCF, WF, ASP.NET updates

8. .NET Framework4.5.1

C# Version: 5.0CLR Version:4.0

MajorFeatures,

- Support for Windows Phone Store Apps
- Automatic Binding Redirection
- Performance and Debugging Enhancements
- Async Programming
- Caller info Attributes
- Loop variable Closure

9. .NET Framework 4.5.2

C# Version: 5.0 CLR Version: 4.0

Major Features,

- New APIs for transactional System
- System DPI resizing in Windows Forms controls
- Profiling Improvements
- ETW and Stress logging improvements

10. .NET Framework 4.6

C# Version: 6.0 CLR Version: 4.0

Major Features,

- Compilation Using .NET Native
- ASP.NET Core 5
- Event Tracing Improvements
- Support for page endings
- Task-based API for Asynchronous Response Flushing
- Model binding supports task-returning methods
- HTTP/2 Support
- Support for the Token Binding Protocol
- Randomized string hash algorithms
- 64-bit JIT compiler for managed code
- Assembly loader improvements
- SIMD-enabled types

- Enhancements to garbage collection (GC)
- Compatibility switches
- Task-based asynchronous pattern(TAP)
- HDPI improvements
- SSL Support
- Sending messages using different HTTP connections

11. .NET Framework 4.6.1

C# Version: 6.0 CLR Version: 4.0

Major Features,

- Support for X509 certificates containing ECDSA
- Always Encrypted support for hardware protected keys in ADO.NET
- Spell checking improvements in WPF
- Native Image Generator (NGEN) PDBs

12. .NET Framework 4.6.2

C# Version: 6.0 CLR Version: 4.0

Major Features

- Cryptography enhancements
- Including support for X509 certificates containing FIPS 186-3 DSA
- Support for persisted-key symmetric encryption
- SignedXml support for SHA-2 hashing
- Increased clarity for inputs to ECDiffie Hellman key derivation routines.
- Support for converting Windows Forms and WPF apps to UWP apps.

- Click Once support for the TLS 1.1 and TLS 1.2 protocols.
- Using directives to import static members
- Exception Filter
- Indexed Members
- Element Initializers
- Await in catch and finally block
- Collection initializers

NET Framework is a technology that supports building and running Windows apps and web services. .NET Framework is designed to fulfil the following objectives:

- Provide a consistent, object-oriented programming environment whether object code is stored and executed locally, executed locally but web-distributed, or executed remotely.
- Provide a code-execution environment
- Minimizes software deployment and versioning conflicts.
- Promotes safe execution of code, including code created by an unknown or semi-trusted third party.
- Eliminates the performance problems of scripted or interpreted environments.
- Make the developer experience consistent across widely varying types of apps, such as Windows-based apps and Web-based apps.
- Build all communication on industry standards to ensure that code based on .NET Framework integrates with any other code.

1.6 Features of .Net framework

The Microsoft .NET framework provides a lot of features. Microsoft has designed the features of the .NET framework by using the technologies that are required by software developers to develop applications for modern as well as future business needs. The key features of .NET are:

1. Common Executive Environment:- All .NET applications run under a common execution environment, called the Common Language Runtime. The CLR facilitates the interoperability between different .NET languages such as C#, Visual Basic, Visual C++, etc. by providing a common environment for the execution of code written in any of these languages.

2. Common Type System:- The .NET framework follows types of systems to maintain data integrity across the code written in different .NET compliant programming languages. CTS ensures that objects of the programs that are written in different programming languages can communicate with each other to shared data.

CTS prevents data loss when a type in one language transfers data to its equivalent type in one language transfer data to its equivalent type in other languages. For example, CTS ensures that data is not lost while transferring an integer variable of visual basic code to an integer variable of C# code.

The common type system CTS defines a set of types and rules that are common to all languages targeted at the CLR. It supports both value and reference types. Value types are created in the stack and include all primitive types, structs, and enums. In contrast, reference types are created in the managed heap and include objects, arrays, collections, etc.

3. Multi-language support: - .NET provides multi-language support by managing the compilers that are used to convert the source to intermediate language (IL) and from IL to native code, and it enforces program safety and security.

The basis for multiple language support is the common type system and metadata. The basic datatypes used by the CLR are common to all languages. There are there for eno conversion issues with the basic integer, floating- point and string types.

All languages deal with all data types in the same way. There is also a mechanism for defining and managing new types.

4. tool Support:- The CLR works hand-in-hand with tools like visual studio, compilers, debuggers, and profilers to make the developer's job much simpler.

5. Security:- The CLR manages system security through user and code identity coupled with permission checks. The identity of the code can be known and permission for the use of resources granted accordingly. This type of security is a major feature of .NET. The .NET framework also provides support for role-based security using windows NT accounts and groups.

6. Automatic Resource Management:- The .NETCLR provides efficient and automatic resource management such as memory, screenspace, network connections, database, etc. CLR invokes various built-in functions of .NET framework to allocate and de-allocate the memory of .NETObjects.

Therefore, programmers need not write the code to explicitly allocate and de-allocate memory to the program.

7. Easy and rich debugging support:- The .NET IDE (integrated development environment) provides an easy and rich debugging support. Once an exception occurs at runtime, the program stops and the IDE marks the line which contains the error along with the details of that error and possible solutions. The runtime also provides built-in stack walking facilities making it much easier to locate bugs and error.

8. Simplified development:- With .NET installing or uninstalling, a window-based application is a matter of copying or deleting files. This is possible because .NET components are not referenced in the registry.

9. Framework class library:- The framework class library(FCL) of the .NET framework contains a rich collection of classes that are available for developers to use these classes in code Microsoft has developed these classes to fulfill various tasks of applications, such as working with files and other data storages, performing input-output operations, web services, data access, and drawing graphics. The classes in the FCL are logically grouped under various namespaces such as system, System. collections, system. Diagnostics, system. Globalization, system.IO, system.Text etc.

10. Portability: - The application developed in the .NET environment is portable. When the source code of a program written in a CLR compliant language compiles, it generates a machine-independent and intermediate code. This was originally called the Microsoft Intermediate Language (MSIL) and has now been renamed as the common Intermediate Language (CIL). CIL is the key to portability in .NET.

1.7 Asp .net Framework

ASP.NET is open source and a subset of the .NET Frame work and successor of the classic ASP (ActiveServerPages). ASP.NET is built on the CLR (Common Language Runtime) which allows the programmers to execute its code using any .NET language (C#,VBetc.). It is specially

designed to work with HTTP and for web developers to create dynamic web pages, web applications, web sites, and web services as it provides a good integration of HTML, CSS, and JavaScript.

.NET Framework is used to create a variety of applications and services like Console, Web, and Windows, etc. But ASP.NET is only used to create web applications and web services. That's why we termed ASP.NET as a subset of the .NET Framework.

These are some things that ASP.NET adds to the .NET platform:

- Base framework for processing web requests in C# or F#
- Web-page templating syntax, known as Razor, for building dynamic web pages using C#
- Libraries for common web patterns, such as Model View Controller (MVC)

Authentication system that includes libraries, a database, and template pages for handling logins, including multi-factor authentication and external authentication with Google, Twitter, and more. Editor extensions to provide syntax highlighting, code completion, and other functionality specifically for developing web pages.

Asp .net Core Services

The built-in container is represented by `IServiceProvider` implementation that supports constructor injection by default. The types (classes) managed by built-in IoC container are called services. There are basically two types of services in ASP.NET Core:

- Framework Services: Services which are a part of ASP.NET Core framework such as `IApplicationBuilder`, `IHostingEnvironment`, `ILoggerFactory` etc.
- Application Services: The services (custom types or classes) which you as a programmer create for your application.

In order to let the IoC container automatically inject our application services, we first need to register them with IoC container.

Registering Application Service

Consider the following example of simple `ILog` interface and its implementation class. We will see how to register it with built-in IoC container and use it in our application.

```

public interface ILog
{
    void info (string str);
}

class MyConsoleLogger : ILog
{
    public void info (string str)
    {
        Console.WriteLine(str);
    }
}

```

ASP.NET Core allows us to register our application services with IoC container, in the ConfigureServices method of the Startup class. The ConfigureServices method includes a parameter of IServiceCollection type which is used to register application services.

Let's register above ILog with IoC container in ConfigureServices() method as shown below.

Example: Register Service Copy public class Startup

```

{
    public void ConfigureServices(IServiceCollection services)
    {
        services.Add(new ServiceDescriptor(typeof(ILog), new MyConsoleLogger()));
    }

    // other code removed for clarity.
}

```

As you can see above, add () method of IServiceCollection instance is used to register a service with an IoC container. The ServiceDescriptor is used to specify a service type and its instance. We have specified ILog as service type and MyConsoleLogger as its instance. This will register ILog service as a singleton by default. Now, an IoC container will create a singleton object of MyConsoleLogger class and inject it in the constructor of classes wherever we include ILog as a constructor or method parameter throughout the application. Thus, we can register our custom application services with an IoC container in ASP.NET Core application. There are other extension methods available for quick and easy registration of services.

1.8 Understanding Service Lifetime

Built-in IoC container manages the lifetime of a registered service type. It automatically disposes a service instance based on the specified lifetime.

The built-in IoC container supports three kinds of lifetimes:

- Singleton: IoC container will create and share a single instance of a service throughout the application's lifetime.
- Transient: The IoC container will create a new instance of the specified service type every time you ask for it.
- Scoped: IoC container will create an instance of the specified service type once per request and will be shared in a single request.

The following example shows how to register a service with different lifetimes.

Example: Register a Service with Lifetime Copy

```
public void ConfigureServices(IServiceCollection services)
{
    services.Add(new ServiceDescriptor(typeof(ILog), new MyConsoleLogger())); // singleton
    services.Add(new ServiceDescriptor(typeof(ILog), typeof(MyConsoleLogger),
    ServiceLifetime.Transient)); // Transient
    services.Add(new ServiceDescriptor(typeof(ILog), typeof(MyConsoleLogger),
    ServiceLifetime.Scoped)); // Scoped
}
```

Extension Methods for Registration

ASP.NET Core framework includes extension methods for each type of lifetime; `AddSingleton()`, `AddTransient()` and `AddScoped()` methods for singleton, transient and scoped lifetime respectively.

The following example shows the ways of registering types (service) using extension methods.

Example:ExtensionMethods Copy

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddSingleton<ILog, MyConsoleLogger>(); services.AddSingleton(typeof(ILog),
    typeof(MyConsoleLogger)); services.AddTransient<ILog, MyConsoleLogger>();
    services.AddTransient(typeof(ILog), typeof(MyConsoleLogger)); services.AddScoped<ILog,
    MyConsoleLogger>(); services.AddScoped(typeof(ILog), typeof(MyConsoleLogger));
}
```

Constructor Injection

Once we register a service, the IoC container automatically performs constructor injection if a service type is included as a parameter in a constructor.

Example, we can use `ILog` service type in any MVC controller. Consider the following example.
Using Service Copy

```
public class HomeController : Controller
{
    ILog _log;

    public HomeController(ILog log)
    {
        _log = log;
    }
}
```



```

public IActionResult Index()
{
    _log.info("Executing /home/index");
    return View();
}
}

```

In the above example, an IoC container will automatically pass an instance of `MyConsoleLogger` to the constructor of `HomeController`. We don't need to do anything else. An IoC container will create and dispose an instance of `ILog` based on the registered lifetime.

Action Method Injection

Sometimes we may only need dependency service type in a single action method. For this, use `[FromServices]` attribute with the service type parameter in the method.

Example: Action Method Injection Copy

```

using Microsoft.AspNetCore.Mvc;

public class HomeController : Controller
{
    public HomeController()
    {
    }

    public IActionResult Index([FromServices] ILog log)
    {
        log.info("Index method executing"); return View();
    }
}

```

Property Injection

Built-in IoC container does not support property injection. You will have to use third party IoC container.

It is not required to include dependency services in the constructor. We can access dependent services configured with built-in IoC container manually using Request Services property of Http Context as shown below.

Example: Get Service Instance Manually Copy public class HomeController : Controller

```
{  
  
public HomeController()  
  
{  
  
}  
  
public IActionResult Index()  
  
{  
  
var services = this.HttpContext.RequestServices; var log = (ILog) services. GetService  
(typeof(ILog));  
  
log.info("Index method executing"); return View();  
  
}  
  
}
```

It is recommended to use constructor injection instead of getting it using Request Services.

1.9 ASP.Net Web Forms

Web Forms are web pages built on the ASP.NET Technology. It executes on the server and generates output to the browser. It is compatible to any browser to any language supported by .NET common language runtime. It is flexible and allows us to create and add custom controls.

We can use Visual Studio to create ASP.NET Web Forms. It is an IDE (Integrated Development Environment) that allows us to drag and drop server controls to the web forms. It also allows us to set properties, events and methods for the controls. To write business logic, we can choose any .NET language like: Visual Basic or VisualC#.

Web Forms are made up of two components: the visual portion (the ASPX file), and the code behind the form, which resides in a separate class file.

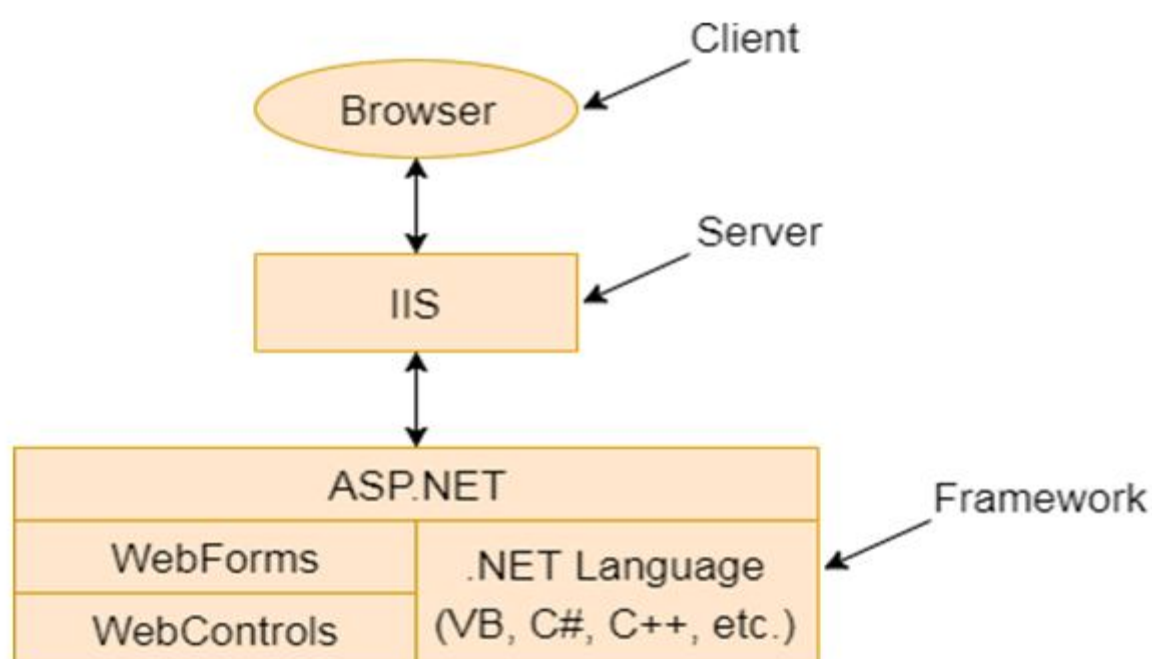


Figure 1: ASP web form

As you build your website, you'll need to add new web pages and other items. To add these ingredients, choose Website

Add New Item from the Visual Studio menu. When you do, the Add New Item dialog box will appear. You can add various types of files to your web application, including resources you want to use (such as bitmaps), ordinary HTML files, code files with class definitions, style sheets, data files, configuration files, and much more. Visual Studio even provides basic designers that allow you to edit most of these types of files directly in the IDE. However, the most common ingredients that you'll add to any website are web forms—ASP.NET web pages

that are fueled with C# code. To add a web form, choose WebForm in the Add New Item dialog box. You'll see two additional options in the bottom-right corner of the Add New Item dialog box

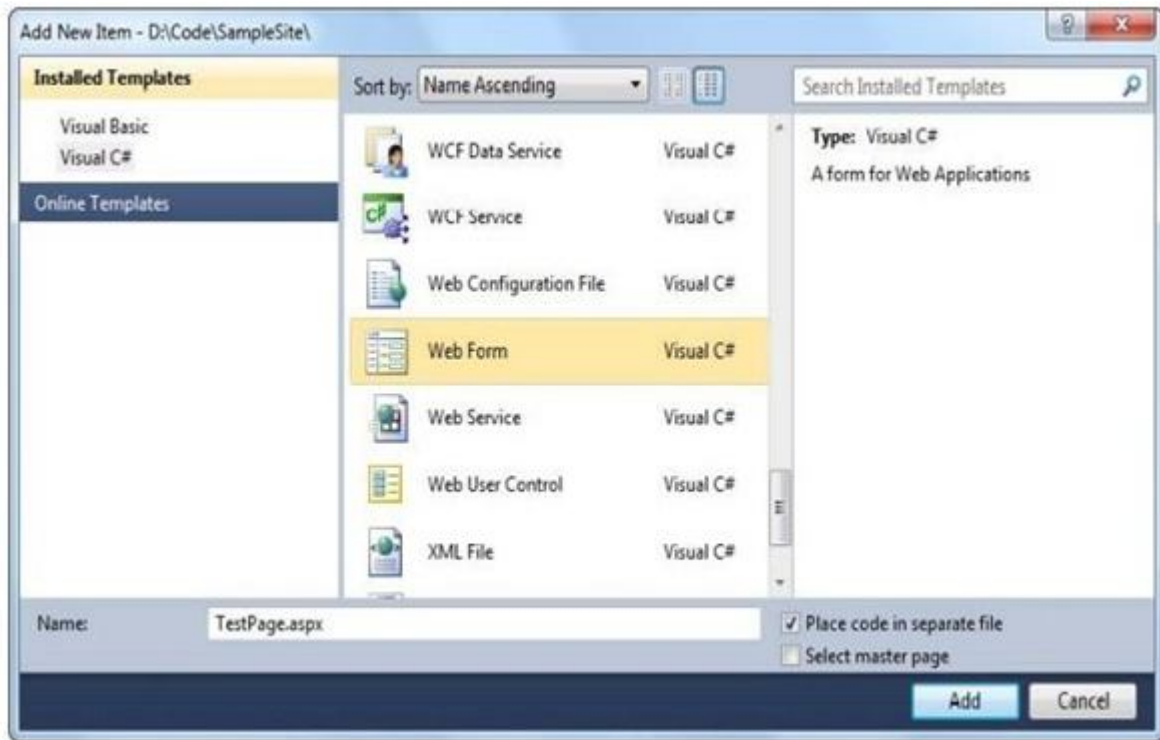


Figure 2: Adding an ASP.Net web form

The Place Code in a Separate File option allows you to choose the coding model for your web page. If you clear this check box, Visual Studio will create a single-file web page. You must then place all the C# code for the file in the same file that holds the HTML markup. If you select the Place Code in a Separate File option, Visual Studio will create two distinct files for the web page, one with the markup and the other for your C# code. This is the more structured approach that you'll use in this book. The key advantage of splitting the web page into separate files is that it's more manageable when you need to work with complex pages. However, both approaches give you the same performance and functionality. You'll also see another option named Select Master Page, which allows you to create a page that uses the layout you've standardized in a separate file.

Once you've chosen the coding model and typed in a suitable name for your webpage, click Add to create it. If you've chosen to use the Place Code in Separate File checkbox (which is

recommended), your project will end up with two files for each web page. One file includes the web page markup (and has the file extension .aspx). The other file stores the source code for the page (and uses the same file name, with the file extension.aspx.cs). To make the relationship clear, the Solution Explorer displays the code file underneath the .aspx file

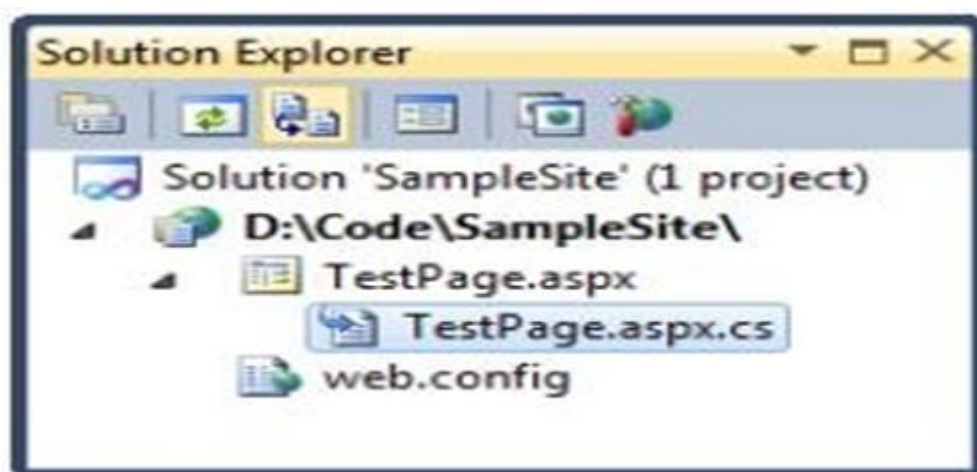


Figure 3: A code file for a web page

You can also add files that already exist by selecting Website Add Existing Item. You can use this technique to copy files from one website to another. Visual Studio leaves the original file alone and simply creates a copy in your web application directory. However, don't use this approach with a webpage that has been created in an older version of Visual Studio. Instead, you should open the entire old website in the Visual Studio 2010, which makes sure it's properly updated

Dynamic DataForm

ASP.NET Dynamic Data is a framework that lets you create data-driven ASP.NET Web applications easily. It does this by automatically discovering data-model metadata at run time and deriving UI behavior from it. A scaffolding framework provides a functional Website for viewing and editing data. You can easily customize the scaffolding frame work by changing elements or creating new ones to override the default behavior. Existing applications can easily integrate scaffolding elements with ASP.NET pages.

Dynamic Data offers the following features:

Web scaffolding that can run a Web application that is based on reading the underlying database schema. Dynamic Data scaffolding can generate a standard UI from the data model for Full

data access operations (create, update, remove, display), relational operations, and data validation. Automatic support for foreign-key relationships. Dynamic Data detects relationships between tables and creates UI that makes it easy for users to view data from related tables. For more information, see Walkthrough: Creating a New ASP.NET Dynamic Data Web Site Using Scaffolding. The ability to customize the UI that is rendered to display and edit specific data fields. For more information, see How to: Customize ASP.NET Dynamic Data Default Field Templates and How to: Customize Data Field Appearance and Behavior in the Data Model.

The ability to customize the UI that is rendered to display and edit data fields for a specific table. For more information, see How to: Customize the Layout of an Individual Table By Using a Custom Page Template.

The ability to customize data field validation. This lets you keep business logic in the data layer without involving the presentation layer. For more information, see How to: Customize Data Field Validation in the Data Model.

The dynamic nature of Dynamic Data is that it can infer the appearance and the behavior of data fields from the underlying database schema at run time. This mechanism, together with the availability of default page and field templates and many possibilities for customization, gives you a variety of design choices, including the following:

1. Building a Web site using scaffolding.
2. Adding dynamic data to an existing Website.
3. Adding data field validation business logic.
4. Customizing the UI that is rendered in order to display and edit specific data fields or a specific table.

1.10 DataModel

The data model represents the information that is in a database and how the items in the database are related to each other. Dynamic Data supports the LINQ-to-SQL data model and the ADO.NET Entity Framework data model. You can include multiple instances of data models in a Web application, but the models that are used in Dynamic Data must be of the same type.

You register the data model or models that you want to use with Dynamic Data in the Web application's Global.asax file. After a data model is registered with Dynamic Data, the data

model can perform automatic validation of data fields, and it lets you control appearance and behavior of data at the data layer level.

Scaffolding

Scaffolding is a mechanism that enhances the existing ASP.NET page framework by dynamically displaying pages based on the data model. Scaffolding provides the following capabilities: Minimal or no code to create a data-driven Web application.

- Quick development time.
- Built-in data validation based on the database schema.
- Automatic data selection created for each foreign key or Boolean field.

PageTemplates

Dynamic Data scaffolding uses page templates to provide a default view of data tables. Page templates are ASP.NET Web pages that are configured to display data from any table that is available to Dynamic Data. Dynamic Data includes page templates for different views of data, such as listing a table (List view), displaying master/detail tables (Details view), editing data (Edit view), and so on. By default, Dynamic Data is configured to use only a List view page template. You can change the default page template or change Dynamic Data to use different page templates for different purposes. For more information, see ASP.NET Dynamic Data Infrastructure.

Field Templates

Dynamic Data uses field templates to render the UI for displaying and editing individual data fields. It determines the appropriate field template from the data field types. Dynamic Data includes separate field templates for displaying and for editing data fields.

For example, for a DateTime data field Dynamic Data uses the following field templates: DateTime.ascx. This template displays DateTime data type as text (a string) and renders it as a Literal control.

DateTime_Edit.ascx. This template renders a TextBox control. If the field in the database cannot be null or if the data model has been customized to require an entry, this control also renders a RequiredFieldValidator control. The DateTime_Edit.ascx field template provides a

DynamicValidator control that handles any exceptions that are thrown from the data model. It also supports the Regex class.

1.11 ASP.Net Life Cycle

The application life cycle has the following stages:

- User makes a request for accessing application resource, a page. Browser sends this request to the web server.
- A unified pipeline receives the first request and the following events take place:
- An object of the class Application Manager is created.
- An object of the class Hosting Environment is created to provide information regarding the resources.
- Top level items in the application are compiled.
- Response objects are created. The application objects such as HttpContext, HttpRequest and HttpResponse are created and initialized.
- An instance of the HttpApplication object is created and assigned to the request.
- The request is processed by the HttpApplication class. Different events are raised by this class for processing the request.

ASP.NET Page Life Cycle

When a page is requested, it is loaded in to the server memory, processed, and sent to the browser. Then it is unloaded from the memory. At each of these steps, methods and events are available, which could be overridden according to the need of the application. In other words, you can write your own code to override the default code. The Page class creates a hierarchical tree of all the controls on the page. All the components on the page, except the directives, are part of this control tree. You can see the control tree by adding trace= "true" to the page directive. We will cover page directives and tracing under 'directives' and 'event handling'.

The page life cycle phases are:

- Initialization
- Instantiation of the controls on the page

- Restoration and maintenance of the state
- Execution of the event handler codes
- Page rendering

Understanding the page cycle helps in writing codes for making some specific thing happen at any stage of the page life cycle. It also helps in writing custom controls and initializing them at right time, populate their properties with view-state data and run control behavior code.

The different stages of an ASP.NET page:

- **Page request** - When ASP.NET gets a page request, it decides whether to parse and compile the page, or there would be a cached version of the page; accordingly, the response is sent.
- **Starting of page life cycle** - At this stage, the Request and Response objects are set. If the request is an old request or post back, the `IsPostBack` property of the page is set to true. The `UICulture` property of the page is also set.
- **Page initialization** - At this stage, the controls on the page are assigned unique ID by setting the `UniqueID` property and the themes are applied. For a new request, post back data is loaded and the control properties are restored to the view-state values.
- **Page load** - At this stage, control properties are set using the view state and control state values.
- **Validation** - `Validate` method of the validation control is called and on its successful execution, the `IsValid` property of the page is set to true.
- **Postback event handling** - If the request is a postback (old request), the related event handler is invoked.
- **Page rendering** - At this stage, view state for the page and all controls are saved. The page calls the `Render` method for each control and the output of rendering is written to the `Output Stream` class of the `Response` property of page.
- **Unload** - The rendered page is sent to the client and page properties, such as `Response` and `Request`, are unloaded and all clean up done.

1.12 ASP.Net WebServices

A web service is a web-based functionality accessed using the protocols of the web to be used by the web applications. There are three aspects of web service development:

- Creating the web service
- Creating a proxy
- Consuming the web service

Creating a Web Service

A web service is a web application which is basically a class consisting of methods that could be used by other applications. It also follows a code-behind architecture such as the ASP.NET web pages, although it does not have a user interface.

To understand the concept let us create a web service to provide stock price information. The clients can query about the name and price of a stock based on the stock symbol. To keep this example simple, the values are hardcoded in a two-dimensional array. This web service has three methods:

- A default HelloWorldmethod
- A GetNameMethod
- A GetPriceMethod

Take the following steps to create the web service:

Step (1): Select File -> New -> Web Site in Visual Studio, and then select ASP.NET Web Service.

Step (2): A web service file called Service.asmx and its code behind file, Service.cs is created in the App_Code directory of the project.

Step (3): Change the names of the files to StockService.asmx and StockService.cs.

Step (4): The .asmx file has simply a WebService directive on it

Step (5): Open the StockService.cs file, the code generated in it is the basic Hello World service.

Step (6): Change the code behind file to add the two-dimensional array of strings for stock symbol, name and price and two web methods for getting the stock information.

Step (7): Running the web service application gives a web service test page, which allows testing the service methods.

Step (8): Click on a method name, and check whether it runs properly.

Step (9): For testing the GetName method, provide one of the stock symbols, which are hard coded, it returns the name of the stock

1.13 Summary

.net framework works on tools and programming language of independent platforms. It works on most of the operating systems. The evolution of the .net framework is of various version and improvements. The benefits and features of .net framework made the platform to be easy in debugging a portable etc. ASP.NET is open source and a subset of the .NET Framework. Asp .Net is compatible to any browser and language support. The dynamic data form automatically discovers data-model metadata at run time and deriving UI behaviour from it. The page life cycle phases are: Initialization, Instantiation of the controls on the page, Restoration and maintenance of the state, Execution of the event handler codes, Page rendering

1.14 Keywords

Evolution, Benefits, Overview, Features of .Net framework, Core Services, Web Forms, Data Model, Life cycle, Web Services

1.15 Check your progress

1. What does UMDF stand for?

- A. User Mode Device Framework
- B. User Mode Driver Framework**
- C. User Modification Device Framework
- D. User Modify Driver Framework

34

2. Is .net framework language sensitive?

- A. YES
- B. **NO**

3. Which is the C# version of .net framework 4.6

- A. C# VERSION 5.0
- B. **C# VERSION 6.0**
- C. C# VERSION 5.0(August 2012)
- D. C# VERSION 4.0

4. How many phases are there in asp .net life cycle?

- A. 3
- B. **5**
- C. 6
- D. 4

5. Which built in lifetime creates new instance of the specified service when we ask for it

- A. Singleton
- B. **Transient**
- C. Scoped
- D. Web services

6. Which built in lifetime shares single instance throughout the application
- A. **Singleton**
 - B. Transient
 - C. Scoped
 - D. Web services

1.16 Model Questions

1. Write short notes on Server-side Programming?
2. What are the benefits of .net framework?
3. Write a short note on client-side programming?
4. Explain the overview of .net framework?
5. Explain in details the features of .net programming?
6. Write short notes on Asp Core services?
7. What are asp .net webforms?
8. Write a short note asp .net lifecycle?
9. Explain dynamic data form?
10. Write short note on asp .net web services?

LESSON - 2

C#

Structure

- 2.1 Introduction**
- 2.2 Objectives**
- 2.3 C# Features**
- 2.4 C# Data Types**
- 2.5 C# Variable**
- 2.6 Object Based Manipulation**
- 2.7 Conditional Statements**
- 2.8 Conditional Loops**
- 2.9 Methods**
- 2.10 Objects**
- 2.11 Namespaces**
- 2.12 Classes**
- 2.13 Static members of C# classes**
- 2.14 Delegates**
- 2.15 Summary**
- 2.16 Check your progress**
- 2.17 Model Questions**

2.1 Introduction

C# is pronounced as “C-Sharp”. It is an object-oriented programming language provided by Microsoft that runs on .Net Framework.

There are different types of secured and robust applications:

- Window applications
- Web applications
- Distributed applications
- Web service applications
- Database applicationsetc.

2.2 Learning objectives

- ✓ Learn the Key features of C#
- ✓ Explore the datatypes
- ✓ Understanding the variables and/scope
- ✓ Learn the object-based manipulation
- ✓ Learn the conditional structures, methods, object & namespace
- ✓ Learn the classes, reference types, using name spaces, delegates & events

2.3 C# FEATURES

1) Simple

C# is a simple language in the sense that it provides structured approach (to break the problem into parts), rich set of library functions, data types etc.

2) Modern Programming Language

C# programming is based upon the current trend and it is very powerful and simple for building scalable, interoperable and robust applications.

3) ObjectOriented

C# is object-oriented programming language. OOPs makes development and maintenance easier where as in Procedure- oriented programming language it is not easy to manage if code grows as project size grow.

4) TypeSafe

C# type safe code can only access the memory location that it has permission to execute. Therefore, it improves a security of the program.

5) Interoperability

Interoperability process enables the C# programs to do almost anything that a native C++ application can do.

6) Scalable and Updateable

C# is automatic scalable and updateable programming language. For updating our application, we delete the old files and update them with new ones.

7) Component Oriented

C# is component-oriented programming language. It is the predominant software development methodology used to develop more robust and highly scalable applications.

8) Structured ProgrammingLanguage

C# is a structured programming language in the sense that we can break the program into parts using functions. So, it is easy to understand and modify.

9) RichLibrary

C# provides a lot of inbuilt functions that makes the development fast.

10) Fast Speed

The compilation and execution time of C# language is fast.

2.4 C# DATA TYPES

A data type specifies the type of data that a variable can store such as integer, floating, character etc.

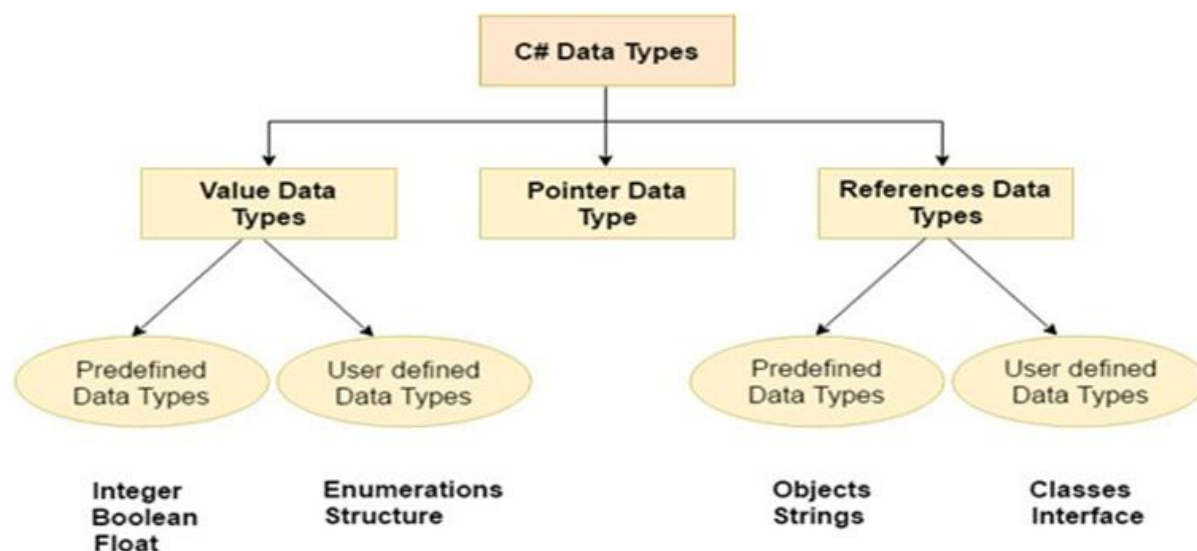


Figure 1: Datatype in C #

Table 1: 3 types of data types in C# language.

Types	Data Types
Value Data Type	short, int, char, float, double etc
Reference Data Type	String, Class, Object and Interface
Pointer Data Type	Pointers

Value Data Type

The value data types are integer-based and floating-point based. C# language supports both signed and unsigned literals.

There are 2 types of value data type in C# language.

1. Predefined Data Types - such as Integer, Boolean, Float, etc.

2. User defined Data Types - such as Structure, Enumerations, etc.

The memory size of data types may change according to 32- or 64-bit operating system.

Reference Data Type

The reference data types do not contain the actual data stored in a variable, but they contain a reference to the variables.

If the data is changed by one of the variables, the other variable automatically reflects this change in value. There are 2 types of reference data type in C# language.

1. Predefined Types - such as Objects, String.

2. User defined Types - such as Classes, Interface.

Pointer Data Type

The pointer in C# language is a variable, it is also known as locator or indicator that points to an address of a value.

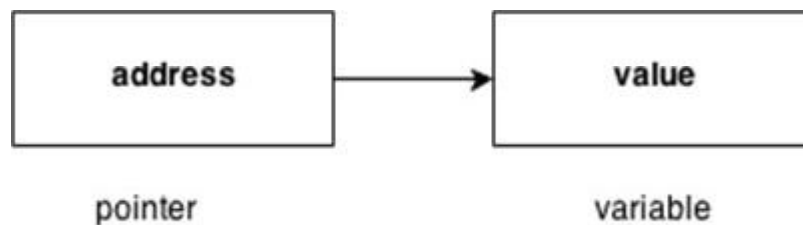


Figure 2: Pointer → variable

Table2: Symbols used in pointer

Symbol	Name	Description
& (ampersand sign)	Address operator	Determine the address of a variable.
* (asterisk sign)	Indirection operator	Access the value of an address.

Declaring a pointer

The pointer in C# language can be declared using * (asterisk symbol).

```
int * a; //pointer to int
```

```
char * c; //pointer to char
```

2.5 C# VARIABLE

A variable is a name of memory location. It is used to store data. Its value can be changed and it can be reused many times. It is a way to represent memory location through symbol so that it can be easily identified.

Table 3: type available in C#

Variable Type	Example
Decimal types	decimal
Boolean types	True or false value, as assigned
Integral types	int, char, byte, short, long
Floating point types	float and double
Nullable types	Nullable data types

syntax to declare a variable:

```
type variable_list;
```

The example of declaring variable is given below:

- `int i, j;`
- `double d;`
- `float f;`
- `char ch;`

Here, i, j, d, f, ch are variables and int, double, float, char are data types.

We can also provide values while declaring the variables as given below:

- `int i=2,j=4; //declaring 2 variable of integer type`
- `float f=40.2;`
- `char ch='B';`

Rules for defining variables

1. A variable can have alphabets, digits and underscore.
2. A variable name can start with alphabet and underscore only. It can't start with digit.
3. No white space is allowed within variable name.
4. A variable name must not be any reserved word or keyword e.g. char, float etc.

Valid variable names:

- `int x;`
- `int _x;`
- `int k20;`

Invalid variable names:

- `int 4;`
- `int x y;`
- `int double;`

SCOPE OF VARIABLES IN C#

The part of the program where a particular variable is accessible is termed as the Scope of that variable. A variable can be defined in a class, method, loop etc. In C/C++, all identifiers are lexically (or statically) scoped, i.e. Scope of a variable can be determined at compile time and independent of the function call stack. But the C# programs are organized in the form of classes.

So, C# scope rules of variables can be divided into three categories as follows:

- Class Level Scope
- Method Level Scope
- Block Level Scope

Class Level Scope

- Declaring the variables in a class but outside any method can be directly accessed anywhere in the class.
- These variables are also termed as the fields or class members.
- Class level scoped variable can be accessed by the non-static methods of the class in which it is declared.
- Access modifier of class level variables doesn't affect their scope within a class.
- Member variables can also be accessed outside the class by using the access modifiers.

Method Level Scope

- Variables that are declared inside a method have method level scope. These are not accessible outside the method.
- However, these variables can be accessed by the nested code blocks inside a method.
- These variables are termed as the local variables.
- There will be a compile-time error if these variables are declared twice with the same name in the same scope.
- These variables don't exist after method's execution is over.

Block Level Scope

- These variables are generally declared inside the for, while statement etc.
- These variables are also termed as the loop variables or statements variable as they have limited their scope up to the body of the statement in which it declared.

- Generally, a loop inside a method has three level of nested code blocks (i.e., class level, method level, loop level).
- The variable which is declared outside the loop is also accessible within the nested loops. It means a class level variable will be accessible to the methods and all loops. Method level variable will be accessible to loop and method inside that method.
- A variable which is declared inside a loop body will not be visible to the outside of loop body.

Use the access modifiers, public, protected, internal, or private, to specify one of the following declared accessibility levels for members.

Table 4: Accessibility levels

ACCESSIBILITY LEVELS (C# REFERENCE)

Declared	accessible Meaning
<u>public</u>	Access is not restricted.
<u>protected</u>	Access is limited to the containing class or types derived from the containing class.
<u>internal</u>	Access is limited to the current assembly.
<u>protected internal</u>	Access is limited to the current assembly or types derived from the containing class.
<u>private</u>	Access is limited to the containing type.
<u>private protected</u>	Access is limited to the containing class or types derived from the containing class within the assembly. Available since C#7.2.

2.6 Object Based Manipulation

Object-oriented programming (OOP) is the core ingredient of the .NET framework. The fundamental idea behind OOP is to combine into a single unit both data and the methods that operate on that data; such units are called an object. All OOP languages provide mechanisms that help you implement the object-oriented model. They are encapsulation, inheritance, polymorphism and reusability.

Encapsulation

Encapsulation binds together code and the data it manipulates and keeps them both safe from outside interference and misuse. Encapsulation is a protective container that prevents code and data from being accessed by other code defined outside the container.

Inheritance

Inheritance is the process by which one object acquires the properties of another object. A type derives from a base type, taking all the base type members fields and functions. Inheritance is most useful when you need to add functionality to an existing type. For example, all .NET classes inherit from the System.Object class, so a class can include new functionality as well as use the existing object's class functions and properties as well.

Polymorphism

Polymorphism is a feature that allows one interface to be used for a general class of action. This concept is often expressed as "one interface, multiple actions". The specific action is determined by the exact nature of circumstances.

Reusability

Once a class has been written, created and debugged, it can be distributed to other programmers for use in their own program. This is called reusability, or in .NET terminology this concept is called a component or a DLL. In OOP, however, inheritance provides an important extension to the idea of reusability. A programmer can use an existing class and without modifying it, add additional features to it.

2.7 Conditional Statement

A statement that can be executed based on a condition is known as a "Conditional Statement". The statement is often a block of code.

The following are the 2 types:

- Conditional Branching
- Conditional Looping

Conditional Branching

This statement allows you to branch your code depending on whether or not a certain condition is met. In C# are the following 2 conditional branching statements:

1. IF statement
2. Switch statement

IF Statement

The if statement allows you to test whether or not a specific condition is met.

Syntax

```
If(<Condition>)
```

```
<statements>;
```

```
Else if(<Condition>)
```

```
<statements>;
```

```
Else
```

```
<statements>;
```

Example

To find the greatest number using an if statement: using System;

```
class ifdemo
```

```
{
```

```
public static void Main()
```

```
{
```

```
inta,b;
```

```
Console.WriteLine("enter 2 no "); a=Int.Parse (Console.ReadLine()); b=Int.Parse  
(Console.ReadLine()); if(a>b)
```



```
{  
    Console.WriteLine("a is greather");  
}  
else If(a< b)  
{  
    Console.WriteLine("b is greather");  
}  
else  
{  
    Console.WriteLine("both are Equals");  
}  
Console.ReadLine();  
}
```

Switch Statement

The switch statement compares two logical expressions.

Syntax

```
Switch(<Expression>  
{  
    Case <Value> :  
        <stmts> Break;  
    Default :
```

```
<stmts> Break;

}
```

Example

```
To choose a color using a switch case. using System;

using System.Collections.Generic; using System.Linq;

using System.Text;

using System.Threading.Tasks;

namespace ConditionalStatementDemo

{

class Switchdemo

{

int ch;

public void getdata()

{

Console.WriteLine("choose the following color"); ch = int.Parse(Console.ReadLine());

switch (ch)

{

case 1:

Console.WriteLine("you choose Red"); break;

case 2:

Console.WriteLine("you choose Green");
```

```
break;

case 3:

Console.WriteLine("you choose Pink"); break;

default:

Console.WriteLine("you can't choose correct color"); break;

}

}

public static void Main ()

{

Switchdemoobj = new Switchdemo(); obj.getdata();

Console.ReadLine();

}

}

}
```

2.8 Conditional Loops

C# provides 4 loops that allow you to execute a block of code repeatedly until a certain condition is met; they are:

1. For Loop
2. While loop
3. Do ... While Loop
4. Foreach Loop

Each and every loop requires the following 3 things in common.

1. Initialization: that sets a starting point of the loop
2. Condition: that sets an ending point of the loop
3. Iteration: that provides each level, either in the forward or backward direction

Syntax

```
For(initializer; Condition; iterator)
```

```
{
< statement >
}
```

Example

```
using System;

using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Conditional Statement Demo
{
    class ForLoop
    {
        public void getdata()
        {
```

```
for (int i = 0; i <= 50; i++)  
  
{  
  
    Console.WriteLine(i);  
  
}  
  
}  
  
public static void Main()  
  
{  
  
    ForLoop f = new ForLoop(); f.getdata(); Console.ReadLine();  
  
}  
  
}  
  
}
```

While loop

Syntax

While(Condition)

```
{  
  
    < statement >  
  
}
```

Example

```
using System;  
  
using System.Collections.Generic;
```

52

```
using System.Linq;

using System.Text;

using System.Threading.Tasks;

namespace ConditionalStatementDemo
{
    class WhileDemo
    {
        int x;

        public void whiledemo()
        {
            while (x <= 50)
            {
                Console.WriteLine(x); x++;
            }
        }

        public static void Main()
        {
            WhileDemo obj = new WhileDemo(); obj.whiledemo(); Console.ReadLine();
        }
    }
}
```

Do...WhileLoop

Syntax

Do

{

< statement >

}

While(Condition)

Example

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.Linq;
```

```
using System.Text;
```

```
using System.Threading.Tasks;
```

```
namespace ConditionalStatementDemo
```

```
{
```

```
class DoWhileDemo
```

```
{
```

```
int x;
```

```
public void does()
```

```
{
```

```
do
```

54

```
{  
  
    Console.WriteLine(x); x++;  
  
}  
  
while (x <= 50);  
  
}  
  
public static void Main()  
  
{  
  
    DoWhileDemo obj = new DoWhileDemo(); obj.does();  
  
    Console.ReadLine();  
  
}  
  
}  
  
}
```

In the case of a for and a while loop from the first execution there will be condition verification. But in the case of a do .. while, the condition is verified only after the first execution, so a minimum number of executions occur. In the case of for and while the minimum number of executions will be zero whereas it is 1 in the case of a do-while loop.

For Each Loop

It is specially designed for accessing the values of an array and collection.

Syntax

Foreach (type var in coll/Arr)

```
{  
  
    < statement >;  
  
}
```


2.9 Methods

A method is a group of statements that together perform a task. Every C# program has at least one class with a method named Main.

To use a method, you need to “

- Define the method
- Call the method

Defining Methods in C#

When you define a method, you basically declare the elements of its structure. The syntax for defining a method in C# is as follows “

```
<Access Specifier><Return Type><Method Name>(Parameter List) { Method Body
}
```

Following are the various elements of a method “

- **Access Specifier** “ This determines the visibility of a variable or a method from another class.
- **Return type** “ A method may return a value. The return type is the data type of the value the method returns. If the method is not returning any values, then the return type is **void**.
- **Method name** “ Method name is a unique identifier and it is case sensitive. It cannot be same as any other identifier declared in the class.
- **Parameter list** “ Enclosed between parentheses, the parameters are used to pass and receive data from a method. The parameter list refers to the type, order, and number of the parameters of a method. Parameters are optional; that is, a method may contain no parameters.
- **Method body** “ This contains the set of instructions needed to complete the required activity.

```

class NumberManipulator{

public int FindMax(int num1, int num2)

{

/* local variable declaration */ int result;

if (num1 > num2) result = num1;

else

result = num2; return result;

}

...

}

```

FindMax that takes two integer values and returns the larger of the two. It has public access specifier, so it can be accessed from outside the class using an instance of the class.

Calling Methods in C#

You can call a method using the name of the method. The following example illustrates this “using System;”

using system

```

namespace CalculatorApplication{ classNumberManipulator {

public int FindMax(int num1, int num2) {

/* local variable declaration */ int result;

if (num1 > num2) result = num1;

else

result = num2; return result;

```

```

}

static void Main(string[] args)

{

/* local variable definition */

int a = 100;

int b = 200; int ret;

NumberManipulator n = new NumberManipulator();

//calling the FindMax method ret = n.FindMax(a, b);

Console.WriteLine("Max value is : {0}", ret ); Console.ReadLine();

}}}

```

When the above code is compiled and executed, it produces the following result “ Max value is: 200

Recursive Method Call

A method can call itself. This is known as recursion. Following is an example that calculates factorial for a given number using a recursive function “

```

using System;

namespace CalculatorApplication{ classNumberManipulator {

public int factorial(int num) {

/* local variable declaration */ int result;

if (num == 1) { return 1;

} else {

result = factorial(num - 1) * num; return result;

```

58

}

}

```
static void Main(string[] args) {
```

```
    NumberManipulator n = new NumberManipulator();
```

```
    //calling the factorial method {0}", n.factorial(6)); Console.WriteLine("Factorial of 7 is : {0}",  
    n.factorial(7)); Console.WriteLine("Factorial of 8 is : {0}", n.factorial(8)); Console.ReadLine();
```

```
}
```

```
}
```

```
}
```

When the above code is compiled and executed, it produces the following result “ Factorial of 6 is: 720

Factorial of 7 is: 5040

Factorial of 8 is: 40320

Passing Parameters to a Method

When method with parameters is called, you need to pass the parameters to the method. There are three ways that parameters can be passed to a method”

Value Parameters

This method copies the actual value of an argument into the formal parameter of the function. In this case, changes made to the parameter inside the function have no effect on the argument.

Reference Parameters

This method copies the reference to the memory location of an argument into the formal parameter. This means that changes made to the parameter affect the argument.

Output Parameters

This method helps in returning more than one value.

2.10 Object

The Object class is the base class for all the classes in .Net Framework. It is present in the System namespace. In C#, the .NET Base Class Library (BCL) has a language-specific alias which is Object class with the fully qualified name as System.Object. Every class in C# is directly or indirectly derived from the Object class. If a Class does not extend any other class then it is the direct child class of Object class and if extends other class then it is an indirectly derived. Therefore, the Object class methods are available to all C# classes. Hence Object class acts as a root of the inheritance hierarchy in any C# Program. The main purpose of the Object class is to provide the low-level services to derived classes.

There are two types in C# i.e. Reference types and Value types. By using System.ValueType class, the value types inherit the object class implicitly. System.ValueType class overrides the virtual methods from Object Class with more appropriate implementations for value types. In other programming languages, the built-in types like int, double, float etc. does not have any object-oriented properties. To simulate the object-oriented behavior for built-in types, they must be explicitly wrapped into the objects. But in C#, we have no need of such wrapping due to the presence of value types which are inherited from System.ValueType that are further inherited from System.Object. So in C#, value types also work similar to reference types. Reference types directly or indirectly inherit the object class by using other reference types.

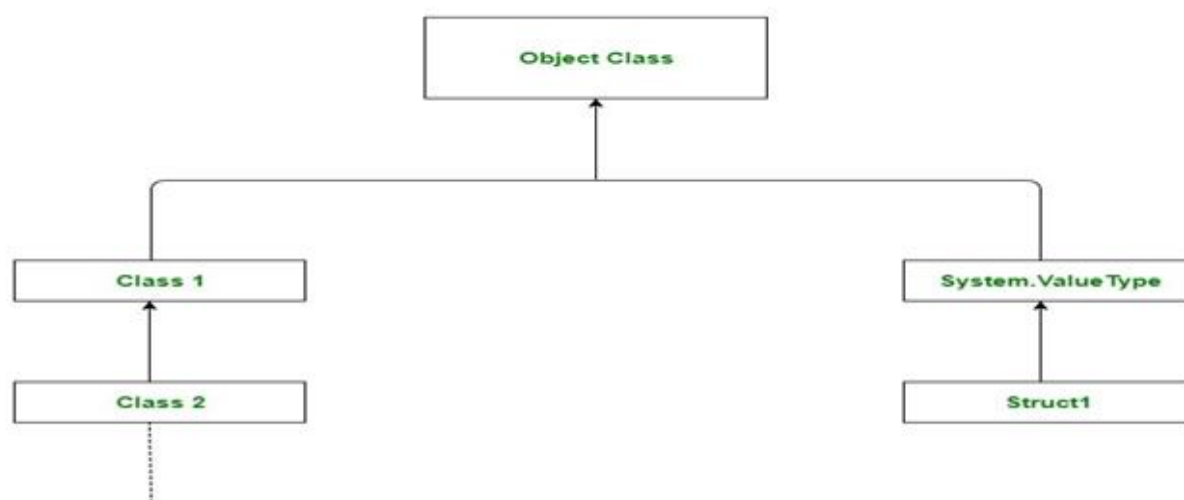


Figure 3: Object Declaration

Object class at the top of the type hierarchy . *Class 1* and *Class2* are the reference types. *Class1* is directly inheriting the Object class while *Class 2* is Indirectly inheriting by using *Class*

1. Struct1 is value type that implicitly inheriting the Object class through the *System.ValueType*.

```
using System;

using System.Text;

class Geeks {

// Main Method

static void Main(string[] args)

{

// taking object type

Object obj1 = new Object();

// taking integer int i = 10;

// taking Type type and assigning

// the value as type of above

// defined types using GetType

// method

Type t1 = obj1.GetType(); Type t2 = i.GetType();

// Displaying result

Console.WriteLine("For Object obj1 = new Object();");

// BaseType is used to display

// the base class of current type

// it will return nothing as Object

// class is on top of hierarchy Console.WriteLine(t1.BaseType);
```

```

// It will return the name class Console.WriteLine(t1.Name);

// It will return the

// fully qualified name Console.WriteLine(t1.FullName);

// It will return the Namespace

// By default Namespace is System Console.WriteLine(t1.Namespace);

Console.WriteLine(); Console.WriteLine("For String str");

// BaseType is used to display

// the base class of current type

// it will return System.Object

// as Object class is on top

// of hierarchy Console.WriteLine(t2.BaseType);

// It will return the name class Console.WriteLine(t2.Name);

// It will return the

// fully qualified name Console.WriteLine(t2.FullName);

// It will return the Namespace

// By default Namespace is System Console.WriteLine(t2.Namespace);

}

}

```

Output:

For Object obj1 = new Object();

Object

System.Object

System

For String str

System.ValueType

Int32 System.Int32

System

Constructor

CONSTRUCTOR	DESCRIPTION
Object()	Initializes a new instance of the Object class. This constructor is called by constructors in derived classes, but it can also be used to directly create an instance of the Object class.

Methods

Table 5: 8 methods present in the C# Object class

METHODS	DESCRIPTION
Equals(Object)	Determines whether the specified object is equal to the current object.
Equals(Object, Object)	Determines whether the specified object instances are considered equal.
Finalize()	Allows an object to try to free resources and perform other cleanup operations before it is reclaimed by garbage collection.
GetHashCode()	Serves as the default hash function.
GetType()	Gets the Type of the current instance.
MemberwiseClone()	Creates a shallow copy of the current Object.

ReferenceEquals

(Object, Object) Determines whether the specified Object instances are the same instance.

ToString() Returns a string that represents the current object.

Important Points:

- C# classes don't require to declare the inheritance from Object class as the inheritance is implicit.
- Every method defined in the Object class is available in all objects in the system as all classes in the *.NET Framework* are derived from Objectclass.
- Derived classes can and do override Equals, Finalize, GetHashCode and ToString methods of Objectclass.
- The process of boxing and unboxing a type internally causes a performance cost. Using the type-specific class to handle the frequently used types can improve the performancecost.

2.11 Namespaces

Defining a namespace

A namespace definition begins with the keyword namespace followed by the namespace name as follows –

```
namespace namespace_name {

// code declarations

}
```

To call the namespace-enabled version of either function or variable, prepend the namespace name as follows “ namespace_name.item_name;

The following program demonstrates use of namespaces “

```
using System;
```

```

namespace first_space{ classnamespace_cl {

public void func() { Console.WriteLine("Inside first_space");

}

}

}

namespace second_space{ classnamespace_cl {

public void func() { Console.WriteLine("Inside second_space");

}

}

}

class TestClass {

static void Main(string[] args) {

first_space.namespace_cl fc = new first_space.namespace_cl();

second_space.namespace_clsc = new second_space.namespace_cl(); fc.func();

sc.func(); Console.ReadKey();

}

}

```

When the above code is compiled and executed, it produces the following result “

```
Inside first_space Inside second_space
```

The using Keyword

The using keyword states that the program is using the names in the given namespace. For example, we are using the System namespace in our programs. The class Console is defined there. We just write”

```
Console.WriteLine ("Hello there");
```

We could have written the fully qualified name as –

```
System.Console.WriteLine("Hello there");
```

You can also avoid prepending of namespaces with the using namespace directive. This directive tells the compiler that the subsequent code is making use of names in the specified namespace. The namespace is thus implied for the following code

Let us rewrite our preceding example, with using directive –

```
using System; using first_space;
```

```
using second_space;
```

```
namespace first_space{ classabc {
```

```
public void func() {
```

```
Console.WriteLine("Inside first_space");
```

```
}
```

```
}
```

```
}
```

```
namespace second_space{ classefg {
```

```
public void func() { Console.WriteLine("Inside second_space");
```

```
}
```

```
}
```

```
}
```

```
class TestClass {
```

```
static void Main(string[] args) { abc fc = new abc();
```

```
efgsc = new efg(); fc.func();

sc.func(); Console.ReadKey();

}

}
```

When the above code is compiled and executed, it produces the following result “

```
Inside first_space Inside second_space
```

Nested Namespaces

You can define one namespace inside another namespace as follows “

```
namespace namespace_name1 {

// code declarations

namespace namespace_name2 {

// code declarations

}

}
```

2.12 Classes

Classes are special kinds of templates from which you can create objects. Each object contains data and methods to manipulate and access that data. The class defines the data and the functionality that each object of that class can contain.

A class declaration consists of a class header and body. The class header includes attributes ,modifiers, and the class keyword. The class body encapsulates the members of the class, that are the data members and member functions. The syntax of a class declaration is as follows:

```
Attributes accessibility modifiers class identifier: base list {body}
```

Attributes provide additional context to a class, like adjectives; for example, the Serializable attribute. Accessibility is the visibility of the class. The default accessibility of a class is internal. Private is the default accessibility of class members. The following table lists the accessibility keywords;

Table 6: Keyword & Description

Keyword	Description
public	Public class is visible in the current and referencing assembly.
private	Visible inside current class.
protected	Visible inside current and derived class.
Internal	Visible inside containing assembly.
Internal protect	Visible inside containing assembly and descendent of the current class.

Modifiers refine the declaration of a class. The list of all modifiers defined in the table are as follows;

Modifier	Description
sealed	Class can't be inherited by a derived class.
static	Class contains only static members.
unsafe	The class that has some unsafe construct like pointers.
Abstract	The instance of the class is not created if the Class is abstract.

The base list is the inherited class. By default, classes inherit from the System.Object type. A class can inherit and implement multiple interfaces but doesn't support multiple inheritances. Objects are instances of a class. The methods and variables that constitute a class are called members of the class.

Defining a Class

A class definition starts with the keyword class followed by the class name; and the class body enclosed by a pair of curly braces. Following is the general form of a class definition “

```

<access specifier> class class_name {

// member variables

<access specifier><data type>variable1;

<access specifier><data type>variable2;

...

<access specifier><data type>variableN;

// member methods

<access specifier><return type> method1(parameter_list) {

// method body

}

<access specifier><return type> method2(parameter_list) {

// method body

}

...

<access specifier><return type>methodN(parameter_list) {

// method body

}

}

```

- Access specifiers specify the access rules for the members as well as the class itself. If not mentioned, then the default access specifier for a class type is **internal**. Default access for the members is **private**.
- Data type specifies the type of variable, and return type specifies the data type of the data the method returns, if any.

- To access the class members, you use the dot (.)operator.
- The dot operator links the name of an object with the name of a member.

```
using System;
```

```
namespace BoxApplication{ class Box {
```

```
public double length; // Length of a box public double breadth; // Breadth of a box public double
height; // Height of a box
```

```
}
```

```
class Boxtester {
```

```
static void Main(string[] args) {
```

```
Box Box1 = new Box(); // Declare Box1 of type Box Box Box2 = new Box(); // Declare Box2 of
type Box double volume = 0.0; // Store the volume of a box here
```

```
// box 1specification Box1.height =5.0;
```

```
Box1.length =6.0;
```

```
Box1.breadth =7.0;
```

```
// box 2specification Box2.height =10.0;
```

```
Box2.length =12.0;
```

```
Box2.breadth = 13.0;
```

```
// volume of box 1
```

```
volume = Box1.height * Box1.length * Box1.breadth; Console.WriteLine("Volume of Box1 :
{0}", volume);
```

```
// volume of box 2
```

```
volume = Box2.height * Box2.length * Box2.breadth; Console.WriteLine("Volume of Box2 :
{0}", volume); Console.ReadKey();
```

70

```
}
```

```
}
```

```
}
```

The following example illustrates the concepts discussed so far”

When the above code is compiled and executed, it produces the following result “

Volume of Box1 : 210

Volume of Box2 : 1560

Member Functions and Encapsulation

A member function of a class is a function that has its definition or its prototype within the class definition similar to any other variable. It operates on any object of the class of which it is a member, and has access to all the members of a class for that object.

Member variables are the attributes of an object (from design perspective) and they are kept private to implement encapsulation. These variables can only be accessed using the public member functions.

Let us put above concepts to set and get the value of different class members in a class “

using System;

```
namespace BoxApplication{ class Box {
```

```
private double length; // Length of a box private double breadth; // Breadth of a box
```

```
private double height; // Height of a box
```

```
public void setLength( doublelen ) { length = len;
```

```
}
```

```
public void setBreadth( doublebre ) { breadth = bre;
```

```
}
```



```

public void setHeight( double hei ) { height = hei;

}

public double getVolume() { return length * breadth * height;

}

}

class Boxtester {

static void Main(string[] args) {

Box Box1 = new Box(); // Declare Box1 of type Box
Box Box2 = new Box();

double volume;

// Declare Box2 of type Box

// box 1 specification Box1.setLength(6.0); Box1.setBreadth(7.0); Box1.setHeight(5.0);

// box 2 specification Box2.setLength(12.0); Box2.setBreadth(13.0); Box2.setHeight(10.0);

// volume of box 1

volume = Box1.getVolume(); Console.WriteLine("Volume of Box1 : {0}", volume);

// volume of box 2

volume = Box2.getVolume(); Console.WriteLine("Volume of Box2 : {0}", volume);

Console.ReadKey();

}

}

}

```

When the above code is compiled and executed, it produces the following result “

Volume of Box1 : 210

Volume of Box2 : 1560

C# Constructors

A class **constructor** is a special member function of a class that is executed whenever we create new objects of that class. A constructor has exactly the same name as that of class and it does not have any return type. Following example explains the concept of constructor “

```
using System;

namespace LineApplication { class Line {

    private double length; // Length of a line

    public Line() {

        Console.WriteLine("Object is being created");

    }

    public void setLength( double len ) { length = len;

    }

    public double getLength() { return length;

    }

    static void Main(string[] args) { Line line = new Line();

    // set line length line.setLength(6.0);

    Console.WriteLine("Length of line : {0}", line.getLength());

    Console.ReadKey();

    }

}
```

```
}
```

When the above code is compiled and executed, it produces the following result “

Object is being created Length of line : 6

A **default constructor** does not have any parameter but if you need, a constructor can have parameters. Such constructors are called **parameterized constructors**. This technique helps you to assign initial value to an object at the time of its creation as shown in the following example—

C# Destructors

A **destructor** is a special member function of a class that is executed whenever an object of its class goes out of scope. A **destructor** has exactly the same name as that of the class with a prefixed tilde (~) and it can neither return a value nor can it take any parameters.

Destructor can be very useful for releasing memory resources before exiting the program. Destructor can not be inherited or overloaded.

Following example explains the concept of destructor “

```
using System;
```

```
namespace LineApplication{ class Line{
```

```
private double length; // Length of a line
```

```
public Line() { // constructor Console.WriteLine("Object is being created");
```

```
}
```

```
~Line() { //destructor Console.WriteLine("Object is being deleted");
```

```
}
```

```
public void setLength( double len ) { length = len;
```

```
}
```

```

public double getLength() { return length;

}

static void Main(string[] args) { Line line = new Line();

// set line length line.setLength(6.0);

Console.WriteLine("Length of line : {0}", line.getLength());

}

}

}

```

When the above code is compiled and executed, it produces the following result “

Object is being created Length of line : 6

Object is being deleted

2.13 Static Members of a C# Class

We can define class members as static using the **static** keyword. When we declare a member of a class as static, it means no matter how many objects of the class are created, there is only one copy of the static member.

The keyword **static** implies that only one instance of the member exists for a class. Static variables are used for defining constants because their values can be retrieved by invoking the class without creating an instance of it. Static variables can be initialized outside the member function or class definition. You can also initialize static variables inside the class definition.

The following example demonstrates the use of **static variables** “

```

using System;

namespace StaticVarApplication{ classStaticVar {

public static int num;

```

```
public void count() { num++;  
  
}  
  
public int getNum() { return num;  
  
}  
  
}  
  
class StaticTester {  
  
    static void Main(string[] args) { StaticVar s1 = new StaticVar(); StaticVar s2 = new StaticVar();  
  
    s1.count();  
  
    s1.count();  
  
    s1.count();  
  
    s2.count();  
  
    s2.count();  
  
    s2.count();  
  
    Console.WriteLine("Variable num for s1: {0}", s1.getNum()); Console.WriteLine("Variable num  
for s2: {0}", s2.getNum()); Console.ReadKey();  
  
}  
  
}  
  
}
```

When the above code is compiled and executed, it produces the following result “

Variable num for s1: 6

Variable num for s2: 6

Base and Derived Classes

A class can be derived from more than one class or interface, which means that it can inherit data and functions from multiple base classes or interfaces.

The syntax used in C# for creating derived classes is as follows –

```
<access-specifier> class <base_class> {
...
}

class <derived_class> :<base_class> {
...
}
```

Initializing Base Class

The derived class inherits the base class member variables and member methods. Therefore the super class object should be created before the subclass is created. You can give instructions for superclass initialization in the member initialization list.

```
using System;

namespace RectangleApplication{ class Rectangle {

//member variables protected double length; protected double width;

public Rectangle(double l, double w) { length =l;

width =w;

}

public double GetArea() { return length * width;

}
```

```

public void Display() { Console.WriteLine("Length: {0}", length);

Console.WriteLine("Width: {0}", width);

Console.WriteLine("Area: {0}", GetArea());

}

} //end class Rectangle

class Tabletop : Rectangle { private double cost;

public Tabletop(double l, double w) : base(l, w) { }

public double GetCost() { double cost;

cost = GetArea() * 70; return cost;

}

public void Display() { base.Display();

Console.WriteLine("Cost: {0}", GetCost());

}

}

class ExecuteRectangle {

static void Main(string[] args) { Tabletop t = new Tabletop(4.5, 7.5); t.Display();

Console.ReadLine();

}

}

}

```

When the above code is compiled and executed, it produces the following result “

Length: 4.5

Width: 7.5

Area: 33.75 Cost: 2362.5

Reference Type

Reference type doesn't store its value directly. Instead, it stores the address where the value is being stored. In other words, a reference type contains a pointer to another memory location that holds the data.

For example, consider the following string variable:

```
string s = "Hello World!!";
```

The following image shows how the system allocates the memory for the above string variable.

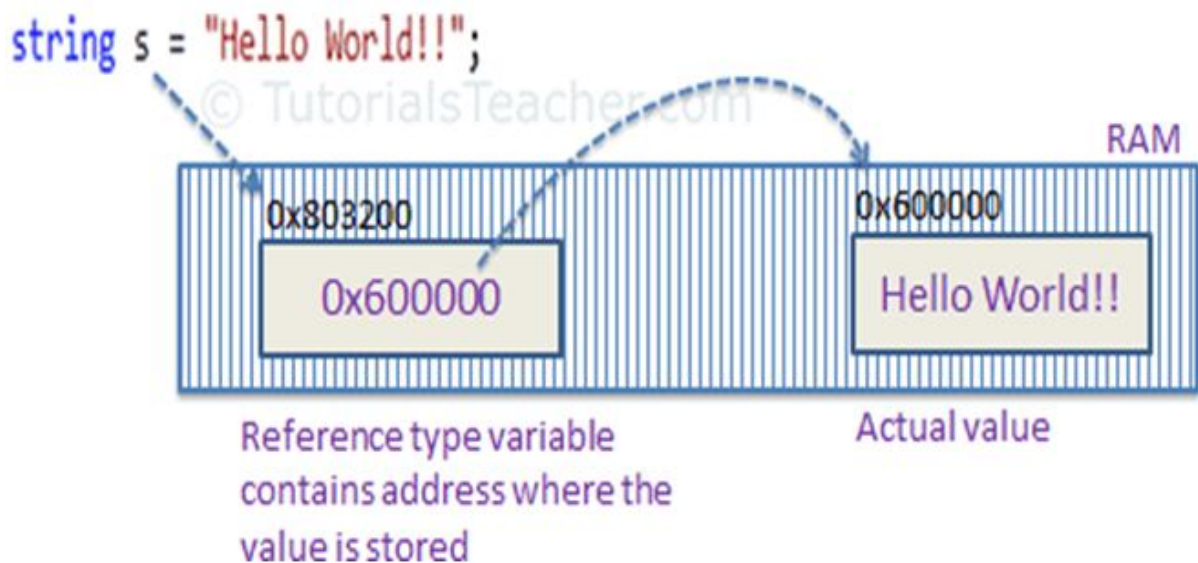


Figure 4: Reference allocation

Memory Allocation of Reference Type Variable

As you can see in the above image, the system selects a random location in memory (0x803200) for the variable `s`. The value of a variable `s` is 0x600000, which is the memory address of the actual data value. Thus, reference type stores the address of the location where the actual value is stored instead of the value itself.

The followings are reference type data types:

- String
- Arrays (even if their elements are valuetypes)
- Class
- Delegate

Passing Reference Type Variables

When you pass a reference type variable from one method to another, it doesn't create a new copy; instead, it passes the variable's address. So, If we change the value of a variable in a method, it will also be reflected in the calling method.

Example: Passing Reference TypeVariable

```
static void ChangeReferenceType(Student std2)
{
    std2.StudentName = "Steve";
}

static void Main(string[] args)
{
    Student std1 = new Student(); std1.StudentName ="Bill";

    ChangeReferenceType(std1);

    Console.WriteLine(std1.StudentName);
}
```

OUTPUT

In the above example, we pass the Studentobject std1to the ChangeReferenceType()method. Here, it actually pass the memory address of std1. Thus, when the Change Reference Type()

method changes Student Name, it is actually changing Student Name of std1 object, because std1 and std2 are both pointing to the same address in memory.

Null

The default value of a reference type variable is null when they are not initialized. Null means not referring to any object.

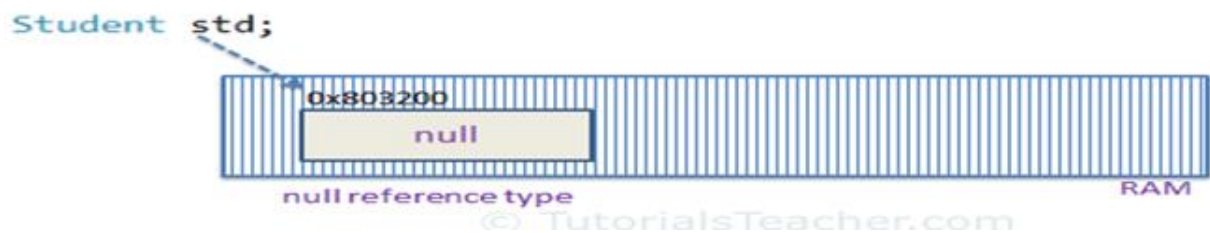


Figure 5: Null reference

Null Reference Type

A value type variable cannot be null because it holds value, not a memory address. C# 2.0 introduced nullable types, using which you can assign null to a value type variable or declare a value type variable without assigning a value to it.

2.14 Delegate

A Delegate can be defined as a delegate type. Its definition must be similar to the function signature. A delegate can be defined in a namespace and within a class. A delegate cannot be used as a data member of a class or local variable within a method. The prototype for defining delegate type is as the following;

accessibility delegate

return

type delegatename(parameterlist);

Delegate declarations look almost exactly like abstract method declarations, you just replace the abstract keyword with the delegate keyword. The following is a valid delegate declaration:

```
public delegate int operation(int x, int y);
```

When the C# compiler encounters this line, it defines a type derived from `MulticastDelegate`, that also implements a method named `Invoke` that has exactly the same signature as the method described in the delegate declaration. For all practical purposes, that class looks like the following:

```
public class operation : System.MulticastDelegate
{
    public double Invoke(int x, int y);

    // Other code
}
```

As you can see using ILDASM.exe, the compiler generated operation class defines three public methods as in the following:

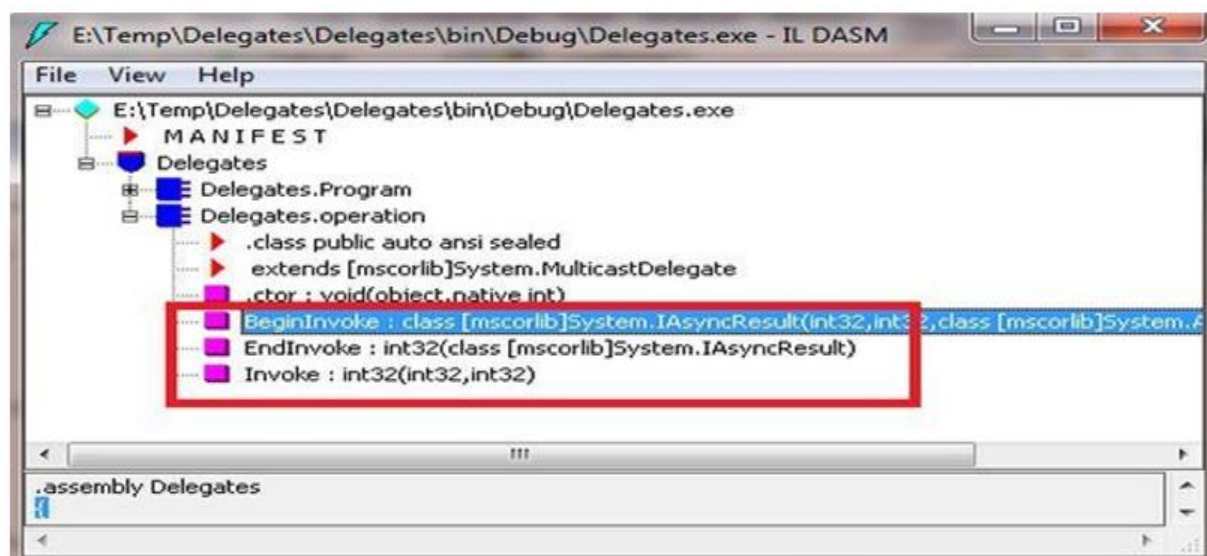


Figure 6: Delegate Selection

After you have defined a delegate, you can create an instance of it so that you can use it to store the details of a particular method.

Delegates Sample Program

The delegate implementation can cause a great deal of confusion when encountered the first time. Thus, it is a great idea to get an understanding by creating this sample program as:

```
using System;

namespace Delegates
{
    // Delegate Definition

    public delegate int operation(int x, int y);

    class Program
    {
        // Method that is passes as an Argument

        // It has same signature as Delegates

        static int Addition(int a, int b)
        {
            return a + b;
        }

        static void Main(string[] args)
        {
            // Delegate instantiation

            operation obj = new operation(Program.Addition);

            // output

            Console.WriteLine("Addition is={0}",obj(23,27)); Console.ReadLine();
        }
    }
}
```

```
}
}
}
```

Here, we are defining the delegate as a delegate type. The important point to remember is that the signature of the function reference by the delegate must match the delegate signature as:

// Delegate Definition

```
public delegate int operation(int x, int y);
```

The format of the operation delegate type declaration; it specifies that the operation object can permit any method taking two integers and returning an integer. When you want to insert the target methods to a given delegate object, simply pass in the name of the method to the delegate constructor as:

// Delegate instantiation

```
operation obj = new operation(Program.Addition);
```

At this point, you are able to invoke the member pointed to using a direct function invocation as:

```
Console.WriteLine("Addition is={0}",obj(23,27));
```

Recall that .NET delegates are type-safe. Therefore, if you attempt to pass a delegate a method that does not match the signature pattern, the .NET will report a compile-time error.

Array of Delegates

Creating an array of delegates is very similar to declaring an array of any type. The following example has a couple of static methods to perform specific math related operations. Then you use delegates to call these methods.

```
using System;
```

```
namespace Delegates
```

```
{
```

84

```
public class Operation

{

    public static void Add(int a, int b)

    {

        Console.WriteLine("Addition={0}", a + b);

    }

    public static void Multiple(int a, int b)

    {

        Console.WriteLine("Multiply={0}", a * b);

    }

}

class Program

{

    delegate void DelOp(int x, int y);

    static void Main(string[] args)

    {

        // Delegate instantiation DelOp[] obj =

        {

            new DelOp(Operation.Add),

            new DelOp(Operation.Multiple)

        };

    }

}
```

```
for (int i = 0; i<obj.Length; i++)  
{  
    obj[i](2,5);  
    obj[i](8,5);  
    obj[i](4,6);  
}  
Console.ReadLine();  
}  
}  
}
```

In this code, you instantiate an array of `Delop` delegates. Each element of the array is initialized to refer to a different operation implemented by the operation class. Then, you loop through the array, apply each operation to three different values. After compiling this code the output



Figure 7 Output image

MulticastDelegate

So far, you have seen a single method invocation/call with delegates. If you want to invoke/call more than one method, you need to make an explicit call through a delegate more than once. However, it is possible for a delegate to do that via multicast delegates.

A multi cast delegate is similar to a virtual container where multiple functions reference a stored invocation list. It successively calls each method in FIFO (first in, first out) order. When you wish to add multiple methods to a delegate object, you simply make use of an overloaded += operator, rather than a direct assignment.

```
using System;

namespace Delegates
{
    public class Operation
    {
        public static void Add(int a)
        {
            Console.WriteLine("Addition={0}", a + 10);
        }

        public static void Square(int a)
        {
            Console.WriteLine("Multiple={0}", a * a);
        }
    }

    class Program
    {
        delegate void DelOp(int x);

        static void Main(string[] args)
```



```

{
// Delegate instantiation DelOpobj = Operation.Add; obj += Operation.Square;

obj(2);

obj(8);

Console.ReadLine();

}

}

}

```

The code above combines two delegates that hold the functions Add() and Square(). Here the Add() method executes first and Square() later. This is the order in which the function pointers are added to the multicast delegates.

To remove function references from a multicast delegate, use the overloaded -= operator that allows a caller to dynamically remove a method from the delegate object invocation list.

```
//Delegate instantiation DelOp obj = Operation.Add; obj -= Operation.Square;
```

Here when the delegate is invoked, the Add() method is executed but Square() is not because we unsubscribed it with the -= operator from a given runtime notification.

Events

The applications and windows communicate via predefined messages. These messages contain various pieces of information to determine both window and application actions. The .NET considers these messages as an event. If you need to react to a specific incoming message then you would handle the corresponding event. For instance, when a button is clicked on a form, Windows sends a WM_MOUSECLICK message to the button message handler. You don't need to build custom methods to add or remove methods to a delegate invocation list. C# provides the event keyword. When the compiler processes the event keyword, you can subscribe and unsubscribe methods as well as any necessary member variables for your delegate types. The syntax for the event definition should be as in the following:

accessibility

Event

Delegate name

Event name

Defining an event is a two-step process. First, you need to define a delegate type that will hold the list of methods to be called when the event is fired. Next, you declare an event using the event keyword. To illustrate the event, we are creating a console application. In this iteration, we will define an event to add that is associated to a single delegate `DelEventHandler`.

```
using System;
```

```
namespace Delegates
```

```
{
```

```
    public delegate void DelEventHandler();
```

```
    class Program
```

```
    {
```

```
        public static event DelEventHandler add;
```

```
        static void Main(string[] args)
```

```
        {
```

```
            add += new DelEventHandler(USA); add += new DelEventHandler(India); add += new  
            DelEventHandler(England); add.Invoke();
```

```
            Console.ReadLine();
```

```
        }
```

```
        static void USA()
```

```
        {
```

```

Console.WriteLine("USA");

}

static void India()

{

Console.WriteLine("India");

}


static void England()

{

Console.WriteLine("England");

}

}

}

```

In the main method, we associate the event with its corresponding event handler with a function reference. Here, we are also filling the delegate invocation lists with a couple of defined methods using the += operator as well. Finally, we invoke the event via the Invoke method. Event Handlers can't return a value. They are always void.

Later in the Program class, we are defining a CatchEvent method that would be referenced in the delegate invocation list. Finally, in the main method, we instantiated the operation class and put in the event firing implementation.

Finally we are elaborating on the events with a realistic example of creating a custom button control over the Windows form that would be called from a console application. First, we inherit the program class from the Form class and define its associated namespace at the top. Thereafter, in the program class constructor, we are crafting a custom button control.

90

```
using System;

using System.Drawing;

using System.Windows.Forms;

namespace Delegates
{
    //custom delegate

    public delegate void DelEventHandler();

    class Program :Form
    {
        //custom event

        public event DelEventHandler add;

        public Program()
        {
            // desing a button over form Button btn = new Button(); btn.Parent = this;

            btn.Text = "Hit Me";

            btn.Location = new Point(100,100);

            //Event handler is assigned to

            // the button click event

            btn.Click += new EventHandler(onClcik); add += new DelEventHandler(Initiate);

            //invoke the event

            add();
        }
    }
}
```

```

}

//call when event is fired

public void Initiate()
{
    Console.WriteLine("Event Initiated");
}

//call when button clicked

public void onClcik(object sender, EventArgs e)
{
    MessageBox.Show("You clicked me");
}

static void Main(string[] args)
{
    Application.Run(new Program());

    Console.ReadLine();
}

}}

```

We are calling the Windows Form by using the Run method of the Application class. Finally when the user runs this application, first the Event fired message will be displayed on the screen and a Windows Form with the custom button control. When the button is clicked, a message box is shown.

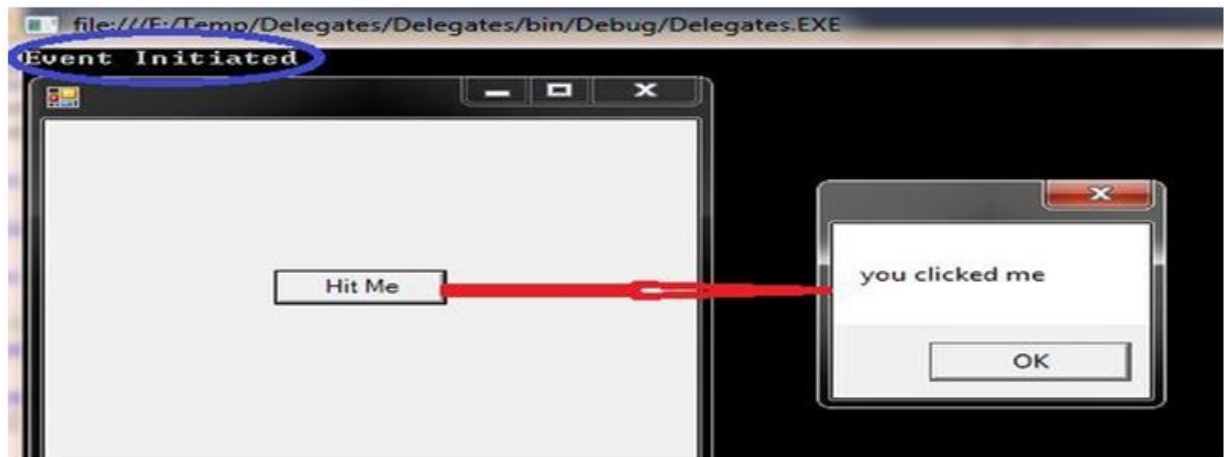


Figure 8: Event Initiated

2.15 Summary

It is an object-oriented programming language provided by Microsoft that runs on .Net Framework. A data type specifies the type of data that a variable can store such as integer, float, character etc. A variable can be defined in a class, method, loop etc. All OOP languages provide mechanisms that help you implement the object-oriented model. They are encapsulation, inheritance, polymorphism and reusability. The conditional statements are two types which help in branching and looping. The Object class is the base class for all the classes in .NetFramework. Reference type doesn't store its value directly. Instead, it stores the address where the value is being stored.

Keywords

Features, Object, Variable, Scope, Classes, Namespace, Reference types

2.16 Check your answers

1. which of this is odd reference data type?

- A. string
- B. **char**
- C. object
- D. classes

2. which is not valid syntax declaration

- A. int x;
- B. int k20;
- C. **int 4;**
- D. int l,j;

3. In which access is limited to current assembly

- A. public
- B. **internal**
- C. private
- D. protected

2.17 Model Questions:

1. Write short notes on c# data types?
2. Explain briefly the conditional statements?
3. Write a short note on classes and objects?
4. Explain delegate and events?
5. Explain shortly the scope of variable in c#?

LESSON - 3

ASP.NET APPLICATION ESSENTIALS

Structure

- 3.1 Introduction
- 3.2 Objectives
- 3.3 Creating Web sites
- 3.4 Designing Web page
- 3.6 The Essentials of XHTML
- 3.7 Writing Code
- 3.8 The Visual Studio Web Server
- 3.9 The Anatomy of an ASP.NET Application
- 3.10 Introducing Server Controls
- 3.11 Summary
- 3.12 Check your progress
- 3.13 Model Questions

3.1 Introduction

Visual Studio includes a built-in test web server, which allows you to create and test a complete ASP.NET website without worrying about setting up a real web server. In this chapter, you'll learn how to create a web application using Visual Studio. Along the way, you'll take a look at the anatomy of an ASP.NET web form and review the essentials of XHTML. Visual Studio is an indispensable tool for developers on any platform. It provides several impressive benefits:

Integrated error checking

The web form designer

An integrated web server

Developer productivity enhancements

Fine-grained debugging

Complete extensibility

3.2 Learning Objectives

- ✓ Create a web application using Visual Studio
- ✓ Understand the anatomy of an ASP.NET web form
- ✓ Debug the code
- ✓ Understand anatomy of ASP.net applications
- ✓ Explore Server Controls

3.3 Creating Websites

You start Visual Studio by selecting Start → All Programs → Microsoft Visual Studio 2010 → Microsoft Visual Studio 2010. To do anything practical with Visual Studio, you need to create a web application

Creating an Empty Web Application

To create your first Visual Studio application, follow these steps:

1. Select File → New → Web Site from the Visual Studio menu.

The New Web Site dialog box (shown in Figure) will appear.

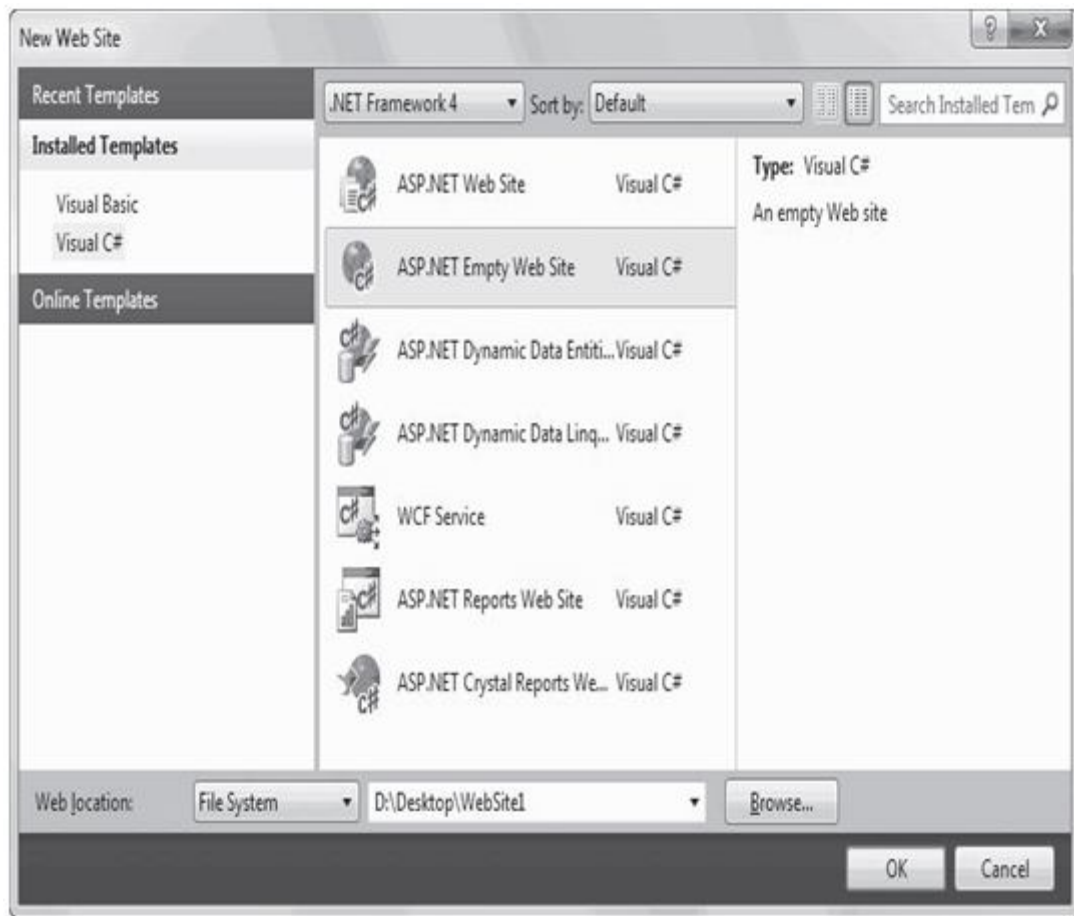


Figure 1. The New Web Site dialog box

2. In the list on the left, choose the language you want to use to write your code.
3. In the drop-down list at top of the window, choose the version of .NET that you want to use. Usually, you'll pick .NET Framework 4.
4. In the list of templates in the middle of the window, choose the type of application that you want to create.
5. Choose a location for the website. The location specifies where the website files will be stored.

Websites and Web Projects

Visual Studio uses project files to store information about the applications you create. Web projects are the original way of creating ASP.NET web applications, and they're still supported in Visual Studio 2010.

You can create a web project by choosing File ► New ► Project and then choosing the ASP.NET Web Application template or by clicking the New Project link on the Start Page. Web projects support all the same features as projectless websites, but they use an extra project file (with the extension .csproj). The web project file keeps track of the web pages, configuration files, and other resources that are considered part of your web application. It's stored in the same directory as all your web pages and code files.

There are a few reasons why you would consider using web projects:

- You have an old web project that was created in a version of Visual Studio, it will be migrated as a web project automatically to avoid strange compatibility
- You want to place two (or more) web projects in the same website folder.
- You have a really huge website that has lots of resources files
- You want to use the Visual Studio web package feature or the MSBuild utility to automate the way your web application is deployed to a web server.

The Hidden Solution Files

Visual Studio still creates one type of resource file, called a *solution* file. Solutions are a similar concept to projects—the difference is that a single solution can hold one or more projects. Visual Studio places solution files into the user specific document directory that's named in this form:

c:\Users\[UserName]\Documents\Visual Studio 2010\Projects\[WebsiteFolderName]

Visual Studio locates the matching solution file automatically and uses the settings in that solution. The solution files store some Visual Studio–specific details that aren't directly related to ASP.NET, such as debugging and view settings. For example, Visual Studio tracks the files that are currently open so, it can reopen them when you resume your website development.

The Solution Explorer

To take a high-level look at your website, you can use the Solution Explorer—the window at the top-right corner of the design environment that lists all the files in your web application directory (see Figure 2).

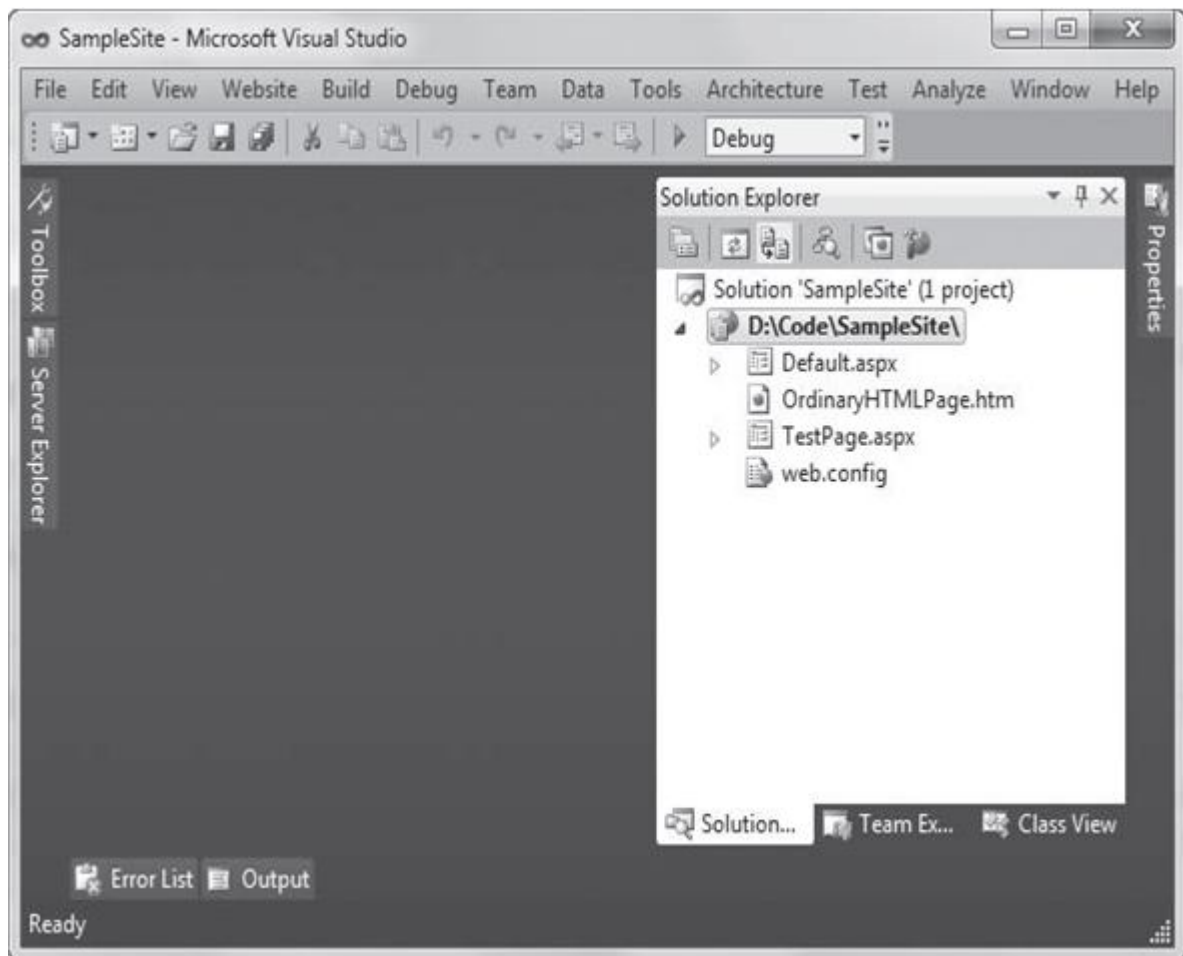


Figure 2. The Solution Explorer

The Solution Explorer reflects *everything* that's in the application directory of a projectless website. When you create a new website using the ASP.NET Empty Web Site template, the Solution Explorer starts out as a lonely place, with nothing more than a basic configuration file. To start creating your website, you need to add web forms.

Adding Web Forms

As you build your website, you'll need to add new web pages and other items. To add choose Website > Add New Item from the Visual Studio menu. When you do, the Add New Item dialogbox will appear. You can add various types of files to your web application, including resources you want to use (such as bitmaps), ordinary HTML files, code files with class definitions, style sheets, data files, configuration files, and much more.

To add a web form, choose Web Form in the Add New Item dialog box. You'll see two additional options in the bottom-right corner of the Add New Item dialog box (as shown in Figure 3).

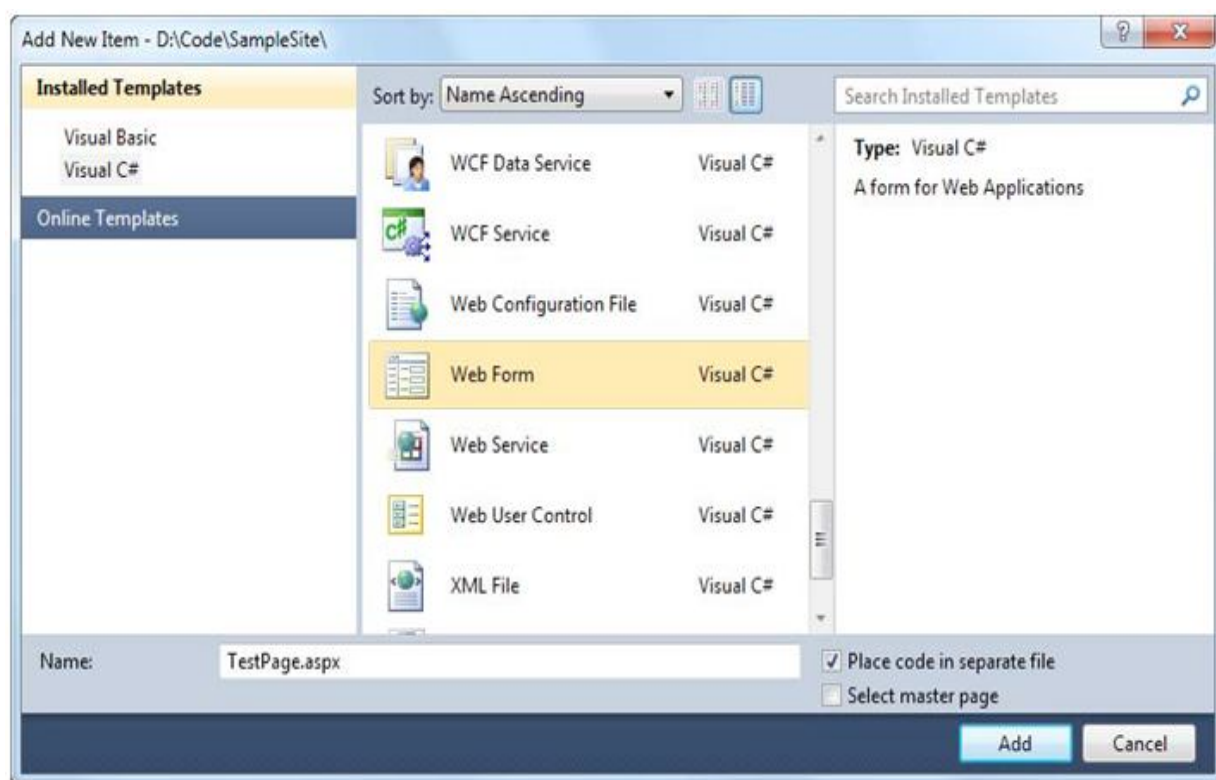


Figure 3. Adding an ASP.NET web form

The Place Code in a Separate File option allows you to choose the coding model for your webpage. If you clear this check box, Visual Studio will create a single-file web page. You must then place all the C# code for the file in the same file that holds the HTML markup. If you select the Place Code in a Separate File option, Visual Studio will create two distinct files for the web

page, one with the markup and the other for your C# code. This is the more structured approach that you'll use in this book. The key advantage of splitting the web page into separate files is that it's more manageable when you need to work with complex pages. However, both approaches give you the same performance and functionality.

You'll also see another option named *Select Master Page*, which allows you to create a page that uses the layout you've standardized in a separate file.

Migrating a Website from a Previous Version of Visual Studio

If you have an existing ASP.NET web application created with an earlier version of Visual Studio, you can migrate it to the ASP.NET world with ease.

If you created a projectless website with an earlier version of Visual Studio, you use the *File* *Open* *Web Site* command, just as you would with a website created in Visual Studio 2010.

If you created a web project with an earlier version of Visual Studio, you need to use the *File* *Open* *Project/Solution* command. When you do, Visual Studio begins the Conversion Wizard.

3.4 Designing a Web Page

To start, in the Solution Explorer, double-click the web page you want to design. (Start with *Default.aspx* if you haven't added any additional pages.)

Visual Studio gives you three ways to look at an .aspx page:

Design view: Here you'll see a graphical representation of what your page looks like.

Source view: Here you'll see the underlying markup, with the HTML for the page and the ASP.NET control tags.

Split view: This combined view allows you to see both the design view and source view at once, stacked one on top of the other.

Adding Web Controls

To add an ASP.NET web control, drag the control you want from the Toolbox on the left, and drop it onto your web page.

The controls in the Toolbox are grouped in numerous categories based on their functions.

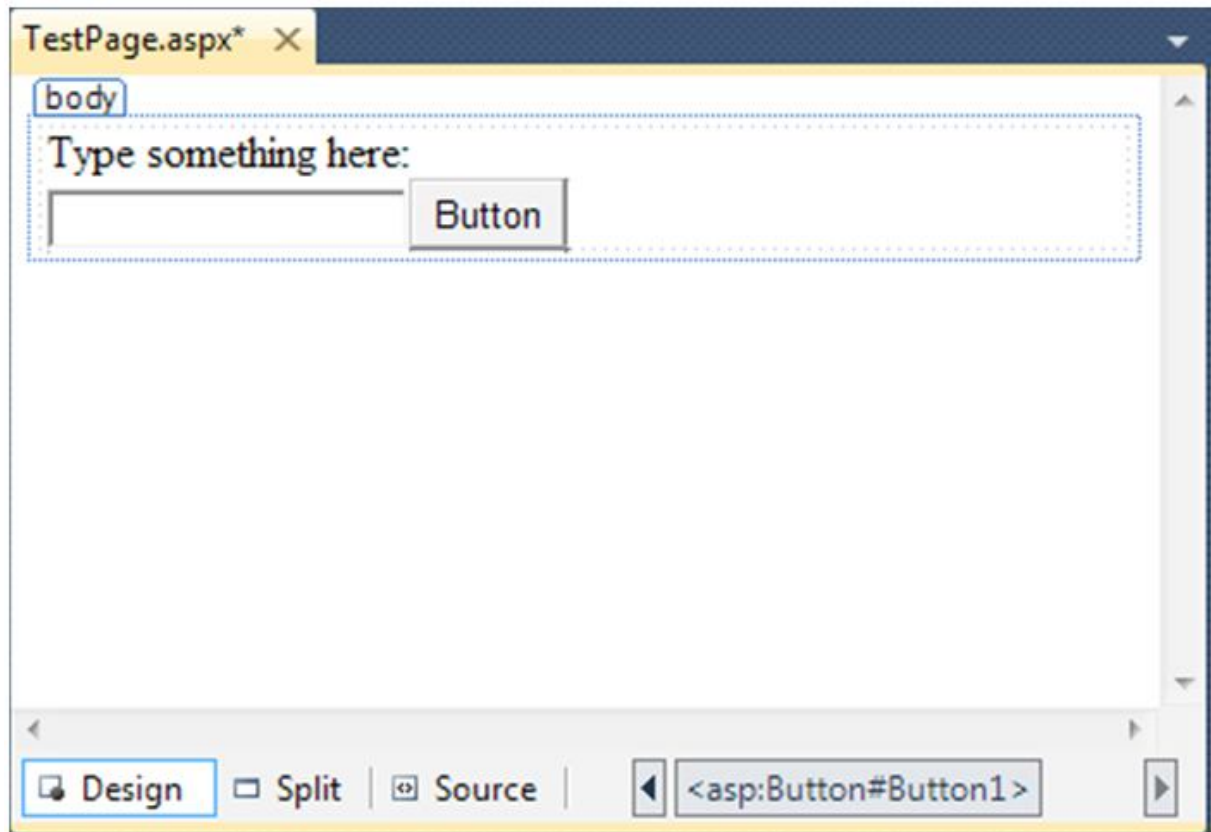


Figure 4. The design view for a page

As you add web controls to the design surface, Visual Studio automatically adds the corresponding control tags to your .aspx file. To look at the markup it's generated, you can click the Source button to switch to source view.

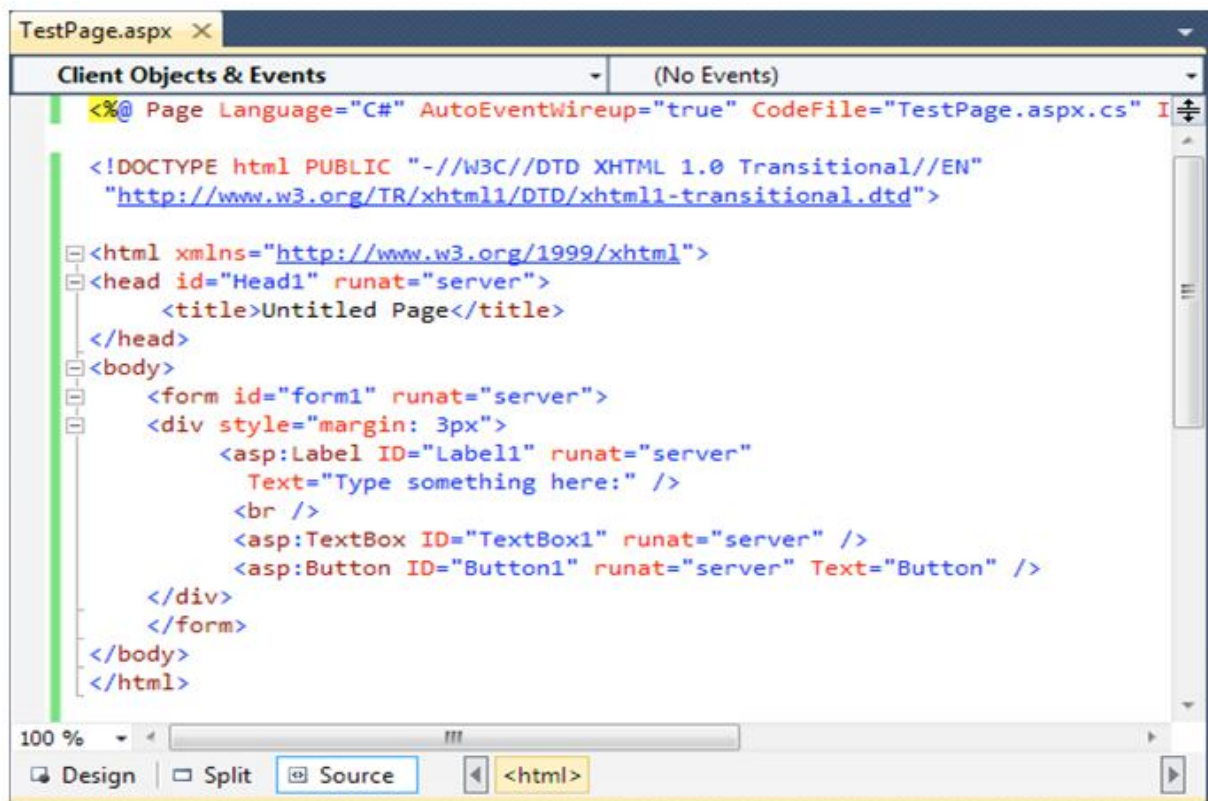


Figure 5. The source view for a page

The Properties Window

Properties window shows you the properties of the currently selected web control—and lets you tweak them. To configure a control in design view, you must first select it on the page. Once you've selected the control you want, you can modify any of its properties. If the Properties window isn't visible, you can pop it into view by choosing **View > Properties Window**. Every time you make a selection in the Properties window, Visual Studio adjusts the web page markup accordingly.

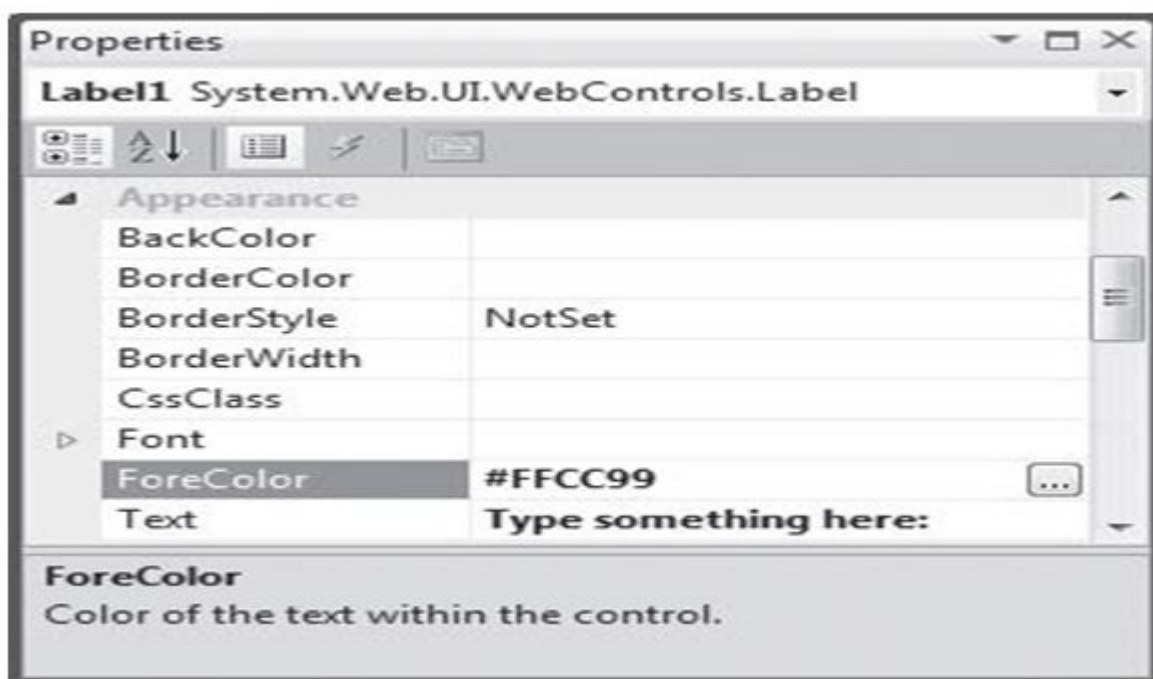


Figure 6. The ForeColor property in the Properties window

you can select two objects in the Properties window explanation—<PAGE> and DOCUMENT. <PAGE> includes a number of ASP.NET options that are tied to the page directive—the essential one- or two-line statement that sits at the top of your .aspx files. The DOCUMENT item consists of a smaller set of properties, including a subset of the <PAGE> options and a few properties that let you alter the <title> and <body> elements.

3.5 The Anatomy of a Web Form

Some types of changes are easier to make when you are working directly with your markup.

The Web Form Markup

The source for an ASP.NET web form isn't 100 percent HTML. Instead, it's an HTML document with an extra ingredient: ASP.NET web controls. Every ASP.NET web form includes the standard HTML tags, such as <html>, <head>, and <body> and ASP.NET-only elements <asp:Button> represents a clickable button that triggers a task on the web server. When you add an ASP.NET web control, you create an object that you can interact with in your web page code, which is tremendously useful.

Here's a look at the web page shown in Figure. The details that are not part of ordinary HTML are highlighted.

```
1 <%@ Page Language="C#" AutoEventWireup="true"
2 CodeFile="Default.aspx.cs" Inherits="_Default" %>
3 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
4 "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
5 <html xmlns="http://www.w3.org/1999/xhtml">
6 <head runat="server">
7 <title>Untitled Page</title>
8 </head>
9 <body>
10 <form ID="form1" runat="server">
11 <div>
12 <asp:Label ID="Label1" runat="server"
13 Text="Type something here:" />
14 <br />
15 <asp:TextBox ID="TextBox1" runat="server" />
16 <asp:Button ID="Button1" runat="server" Text="Button" />
17 </div>
18 </form>
19 </body>
20 </html>
```

Obviously, the ASP.NET-specific details (the highlighted bits) don't mean anything to a web browser because they aren't valid HTML. This isn't a problem, because the web browser never sees these details. Instead, the ASP.NET engine creates an HTML "snapshot" of your page after all your code has finished processing on the server. At this point, details like the `<asp:Button>` are replaced with HTML tags that have the same appearance. The ASP.NET engine sends this HTML snapshot to the browser.

The Page Directive

The `Default.aspx` page, like all ASP.NET web forms, consists of three sections. The first section is the *page directive*:

```
<%@ Page Language="C#" AutoEventWireup="true" CodeFile="Default.aspx.cs"
Inherits="_Default" %>
```

The page directive gives ASP.NET basic information about how to compile the page. It indicates the language you're using for your code and the way you connect your event handlers. If you're using the code-behind approach (which is recommended), the page directive also indicates where the code file is located and the name of your custom page class.

The Doctype

The doctype indicates the type of markup (for example, HTML or XHTML) that you're using to create your web page. The doctype is also important because it influences how a browser interprets your web page. There are a small set of allowable doctypes that you can use. By default, newly created web pages in Visual Studio use the following doctype:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
```

```
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

XHTML transitional enforces all the structural rules of XHTML but allows some HTML formatting features that have been replaced by Cascading Style Sheets (CSS) and are considered obsolete.

XHTML strict using this doctype:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
```

```
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

These are the two most common doctypes in use today. If you're working with an existing website that's based on the somewhat out-of-date HTML standard, this is the doctype you need:

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
    "http://www.w3.org/TR/html4/strict.dtd">
```

And if you want to use the slightly tweaked XHTML 1.1 standard (rather than XHTML 1.0), you need the following doctype:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
    "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
```

3.6 The Essentials of XHTML

Part of the goal of ASP.NET is to allow you to build rich web pages without forcing you to slog through the tedious details of XHTML (or HTML). However, ASP.NET doesn't isolate you from XHTML altogether. In fact, a typical ASP.NET web page mingles ASP.NET web controls with ordinary XHTML content. When that page is processed by the web server, the ASP.NET web controls are converted to XHTML markup (a process known as *rendering*) and inserted into the page. The final result is a standard XHTML document that's sent back to the browser.

Elements

The most important concept in the XHTML (and HTML) standard is the idea of *elements*. Elements are containers that contain bits of your web page content. A typical web page is actually composed of dozens (or hundreds) of elements. A typical element consists of three pieces: a start tag, some content, and an end tag. Here's an example:

```
<p>This is a sentence in a paragraph.</p>
```

Browsers have built-in rules about how to process and display different elements. Many XHTML elements can contain other elements. For example, you can use the `<b>` element inside the `<p>` element to apply bold formatting to a portion of a paragraph:

```
<p>This is a <b>sentence</b> in a paragraph.</p>
```

Table 4-1 lists some of the most commonly used XHTML elements. The Type column distinguishes between two types of XHTML—those that typically hold content or other nested elements (containers) and those that can be used on their own with the empty tag syntax you just considered (stand-alone).

Table. Basic XHTML Elements

Tag	Name	Type	Description
<code></code> , <code><i></code> , <code><u></code>	Bold, Italic, Underline	Container	These elements are used to apply basic formatting and make text bold, italic, or underlined.
<code><p></code>	Paragraph	Container	The paragraph groups a block of free-flowing text together
<code><h1></code> , <code><h2></code> , <code><h3></code> , <code><h4></code> , <code><h5></code> , <code><h6></code>	Heading	Container	These elements are headings, which give text bold formatting and a large font size.
<code></code>	Image	Stand-alone	The image element shows an external image file (specified by the <code>src</code> attribute) in a web page.
<code>
</code>	Line Break	Stand-alone	This element adds a single line break, with no extra space.
<code><hr></code>	Horizontal Line	Stand-alone	This element adds a horizontal line
<code><a></code>	Anchor	Container	The anchor element wraps a piece of text and turns it into a link. You set the link target using the <code>href</code> attribute.

<code></code> , <code></code>	Unordered List, List Item	Container	These elements allow you to build bulleted lists. The <code></code> element defines the list, while the <code></code> element defines an item in the list
<code></code> , <code></code>	Ordered List, List Item	Container	These elements allow you to build numbered lists. The <code></code> element defines the list, while the <code></code> element defines an item in the list
<code><table></code> , <code><tr></code> , <code><td></code> , <code><th></code>	Table	Container	The <code><table></code> element allows you to create a multicolumn, multirow table. Each row is represented by a <code><tr></code> element inside the <code><table></code> . Each cell in a row is represented by a <code><td></code> element inside a <code><tr></code> . You place the actual content for the cell in the individual <code><td></code> elements
<code><div></code>	Division	Container	This element is an all-purpose container for other elements. It's used to separate different regions on the page, so you can format them or position them separately. For example, you can use a <code><div></code> to create a shaded box around a group of elements.
<code></code>	Span	Container	This element is an all-purpose container for bits of text content inside other elements (such as headings or paragraphs). It's most commonly used to format those bits of text. For example, you can use a <code></code> to change the color of a few words in a sentence.
<code><form></code>	Form	Container	This element is used to hold all the controls on a webpage. Controls are HTML elements that can send information back to the web server when the page is submitted. For example, text boxes submit their text, list boxes submit the currently selected item in the list, and so on.

Attributes

Attributes are individual pieces of information that you attach to an element, inside the start tag. Attributes have many uses for example, they allow you to explicitly attach a style to an element so that it gets the right formatting. Some elements require attributes.

The `<img>` tag requires two pieces of information—the image URL (the source) and the alternate text that describes the picture (which is used for accessibility purposes, as with screen-reading software). These two pieces of information are specified using two attributes, named *src* and *alt*:

```

```

Formatting

XHTML elements are intended to indicate the *structure* of a document, not its formatting. Although you can adjust colors, fonts, and some formatting characteristics using XHTML elements, a better approach is to define formatting using a CSS style sheet.

In an ASP.NET web page, there are two ways you can use CSS. You can use it directly to format elements. Or, you can configure the properties of the ASP.NET controls you're using, and they'll generate the styles they need automatically.

A Complete Web Page

You now know enough to put together a complete XHTML page. Every XHTML document starts out with this basic structure (right after the doctype):

```
<html xmlns="http://www.w3.org/1999/xhtml">
```

```
<head runat="server">
```

```
<title>Untitled Page</title>
```

```
</head>
```

```
<body>
```

```
</body>
```

```
</html>
```

When you create a new web form in Visual Studio, this is the structure you start with. Here's what you get:

- XHTML documents start with the `<html>` tag and end with the `</html>` tag. This `<html>` element contains the complete content of the web page.
- Inside the `<html>` element, the web page is divided into two portions. The first portion is the `<head>` element, which stores some information about the webpage. You'll use this to store the title of your web page, which will appear in the title bar in your web browser. (You can also add other details here like search keywords, although these are mostly ignored by web browsers these days.) When you generate a web page in Visual Studio, the `<head>` section has a `runat="server"` attribute. This gives you the ability to manipulate it in your code (a topic you'll explore in the next chapter)
- The second portion is the `<body>` element, which contains the actual page content that appears in the web browser window.

In an ASP.NET web page, there's at least one more element. Inside the `<body>` element is a `<form>` element. The `<form>` element is required because it defines a portion of the page that can send information back to the web server. This becomes important when you start adding text boxes, lists, and other controls. As long as they're in a form, information like the current text in the text box and the current selection in the list will be sent to the web server using a process known as a post back.

When you create a new web page in Visual Studio, there's one more detail—the `<div>` element inside the `<form>` element:

```
<html xmlns="http://www.w3.org/1999/xhtml">
```

```
<head runat="server">
```

```
<title>Untitled Page</title>
```

```
</head>
```

```
<body>
```

```
<form ID="form1" runat="server">
```

```
<div>
```

```
</div>
```



```
</form>
```

```
</body>
```

```
</html>
```

3.7 Writing Code

To start coding, you need to switch to the code-behind view. To switchback and forth, you can use two View Code or View Designer buttons

The Code-Behind Class

When you switch to code view, you'll see the page class for your web page. For example, if you've created a web page named SimplePage.aspx, you'll see a code-behind class that looks like this:

```
using System;

using System;

using System.Collections.Generic;

using System.Linq;

using System.Web;

using System.Web.UI;

using System.Web.UI.WebControls;

public partial class SimplePage: System.Web.UI.Page
{

protected void Page_Load(object sender, EventArgs e)

{

}

}
```

Inside your page class you can place methods, which will respond to control events.

Adding Event Handlers

Most of the code in an ASP.NET web page is placed inside event handlers that react to web control events. Using Visual Studio, you have three easy ways to add an event handler to your code:

- *Type it in manually*
- *Double-click a control in design view*
- *Choose the event from the Properties window*

Type it in manually: In this case, you add the subroutine directly to the page class in your C# codefile. You must specify the appropriate parameters.

Double-click a control in design view: In this case, Visual Studio will create an event handler for that control's default event, if it doesn't already exist.

Choose the event from the Properties window: Just select the control, and click the lightning bolt in the Properties window. You'll see a list of all the events provided by that control. Double-click next to the event you want to handle, and Visual Studio will automatically generate the event handler in your page class.

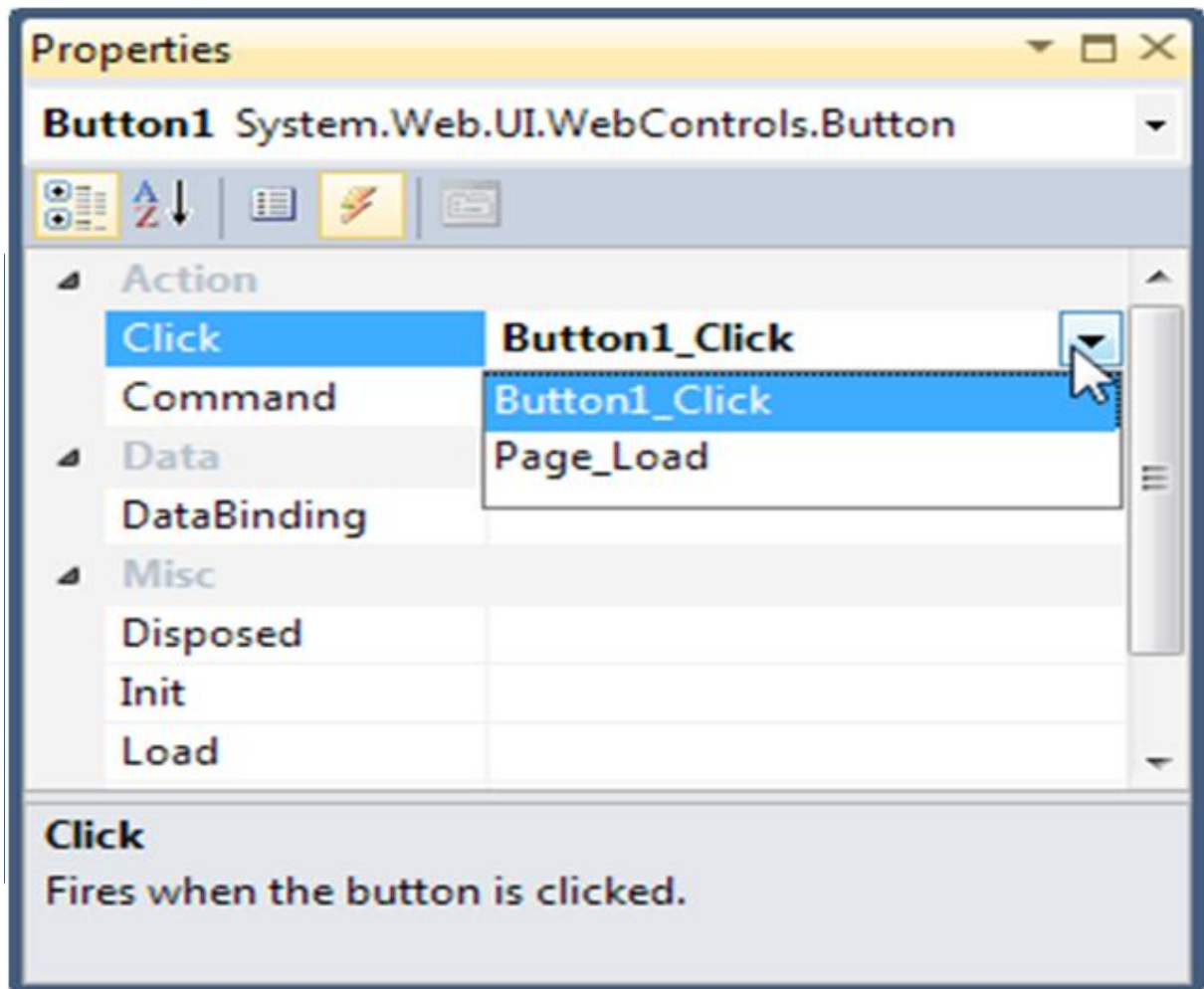


Figure 7. Creating or attaching an event handler

Inside your event handler method, you can interact with any of the control objects on your webpage using their IDs. For example, if you've created a TextBox control named TextBox1, you can set the text using the following line of code:

```
protected void Button1_Click (object sender, EventArgs e)
{
    TextBox1.Text = "Here is some sample text.";
}
```

Outlining

Outlining allows Visual Studio to “collapse” a method, class, structure, namespace, or region to a singleline. It allows you to see the code that interests you while hiding unimportant code. To collapse a portion of code, click the minus (–) symbol next to the first line. To expand it, click the box again, which will now have a plus (+) symbol (see Figure 8).

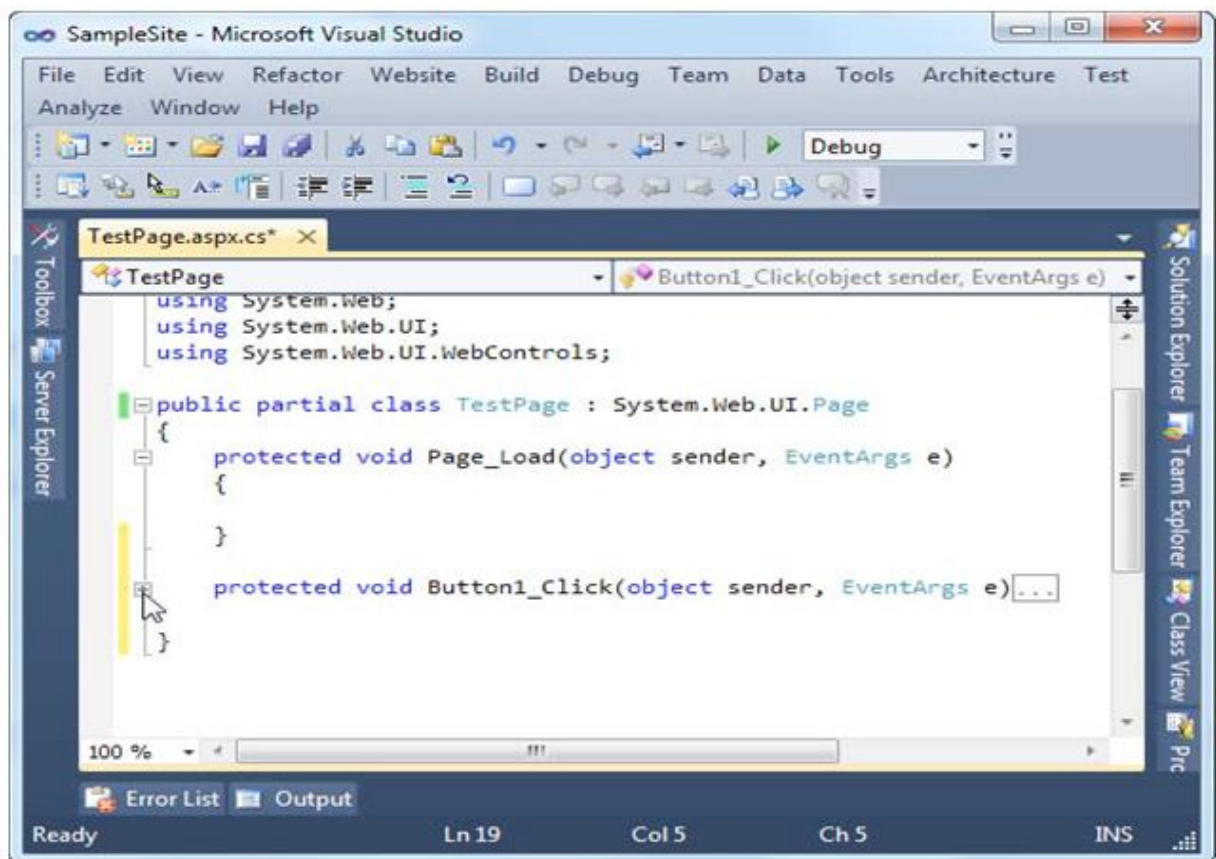


Figure 8. Collapsing code

You can hide every method at once by right-clicking anywhere in the code window and choosing **Outlining > Collapse to Definitions**.

IntelliSense

IntelliSense—a group of features that prompt you with valuable code suggestions (and catch mistakes) as you type.

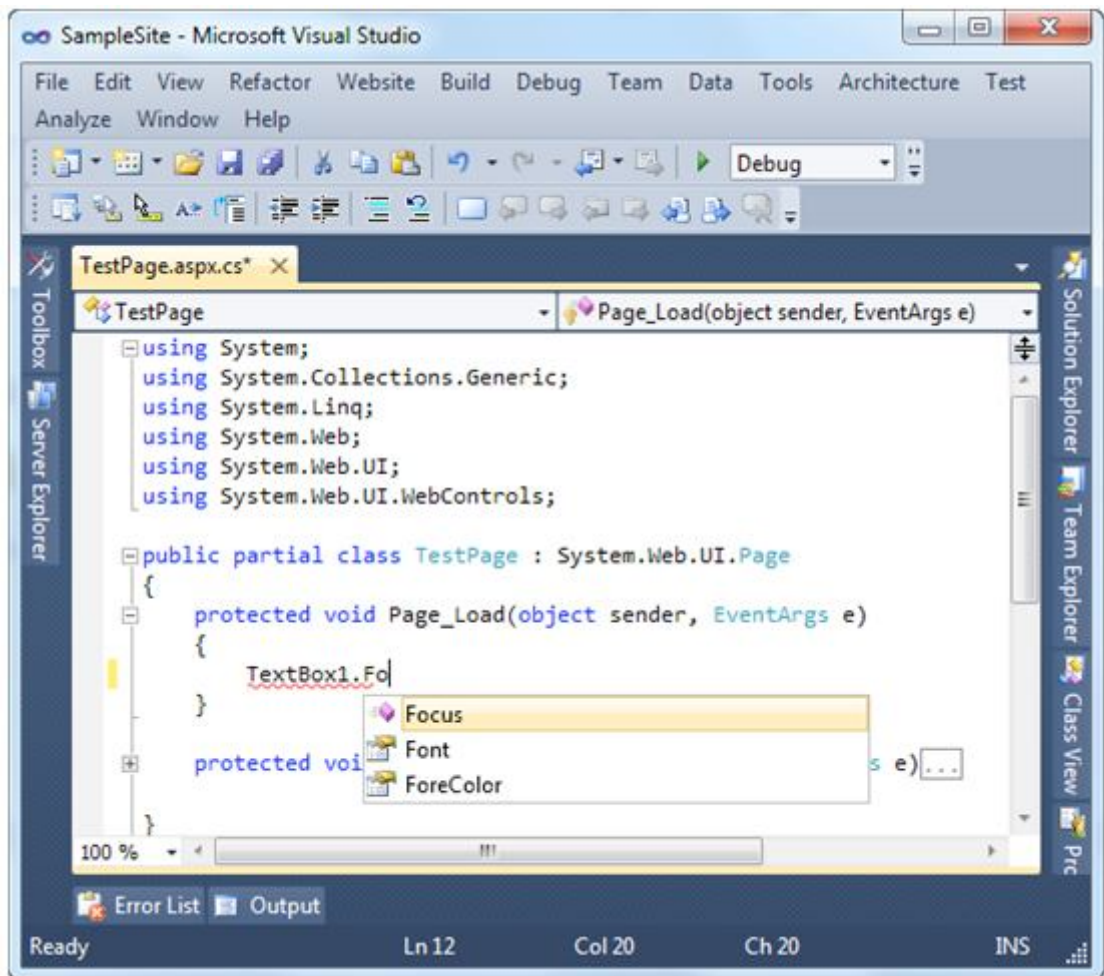


Figure 9. IntelliSense at work

Member List

When you type a class or object name, it pops up a list of available properties and methods that match what you've typed so far. Visual Studio also provides a list of parameters and their data types when you call a method or invoke a constructor. This information is presented in a tooltip below the code and appears as you type.

Error Underlining

One of the code editor's most useful features is error underlining. Visual Studio is able to detect a variety of error conditions, such as undefined variables, properties, or methods; invalid

data type conversions; and missing code elements. Rather than stopping you to alert you that a problem exists, the Visual Studio editor underlines the offending code

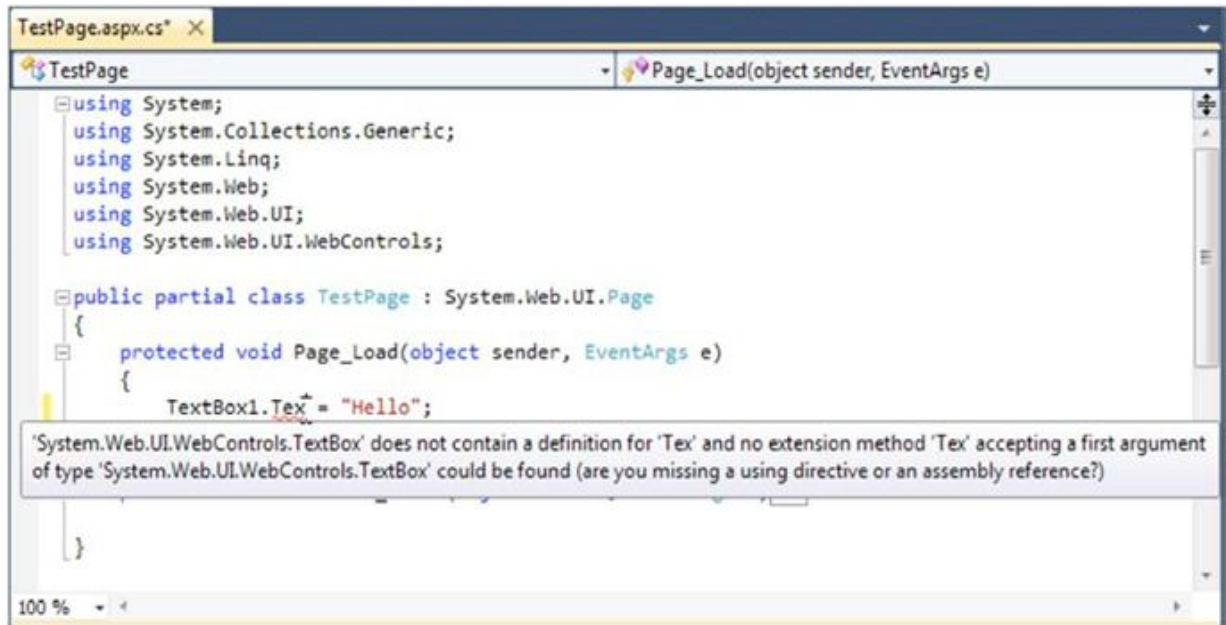


Figure 10. Highlighting errors at design time

Whenever you attempt to build an application that has errors, Visual Studio will display the ErrorList window with a list of all the problems it detected, as shown in Figure 11. You can then jump quickly to a problem by double-clicking it in the list.

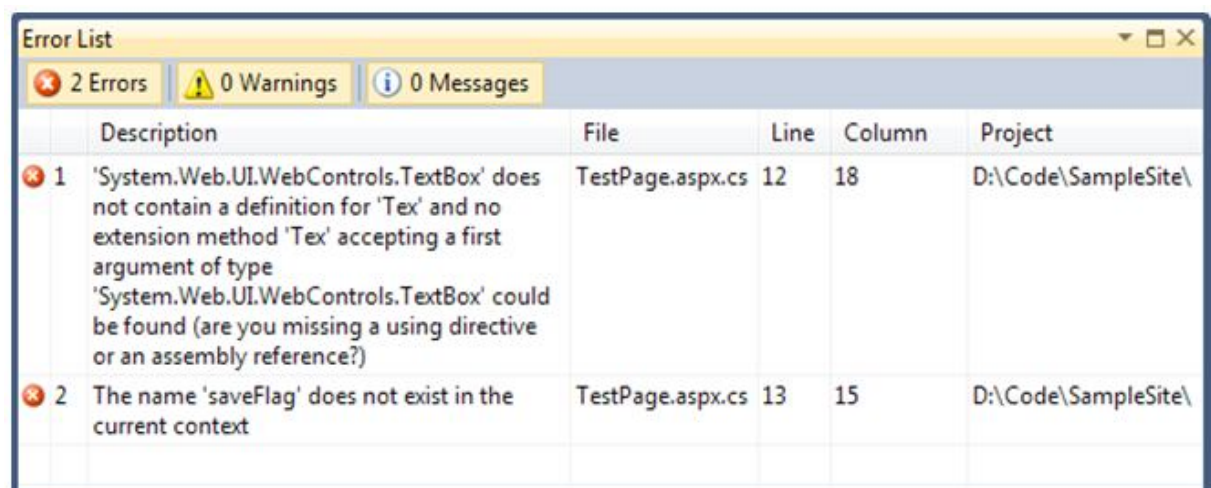


Figure 11. The Error List window

Visual Studio doesn't check for all types of errors at once.

Automatically Importing Namespaces

Sometimes, you'll run into an error because you haven't imported a namespace that you need. Forexample, imagine you type a line of code like this:

```
FileStream fs = new FileStream("newfile.txt", FileMode.Create);
```

This line creates an instance of the `FileStream` class, which resides in the `System.IO` namespace. However, if you haven't imported the `System.IO` namespace, you'll run into a compile-time error.

Code Formatting and Coloring

It automatically colors your code, making comments green, keywords blue, and normal code black. The result is much more readable code.

Visual Studio Debugging

Once you've created an application, you can compile and run it by choosing **Debug > Start Debugging** from the menu, by clicking the **Start Debugging** button on the toolbar. The first time you launch a web application, Visual Studio will ask you whether you want to configure your web application to allow debugging by adjusting its configuration file. (Figure 12 shows

the message you'll see.) Choose "Modify the Web.config file to enable debugging," and click **OK**.

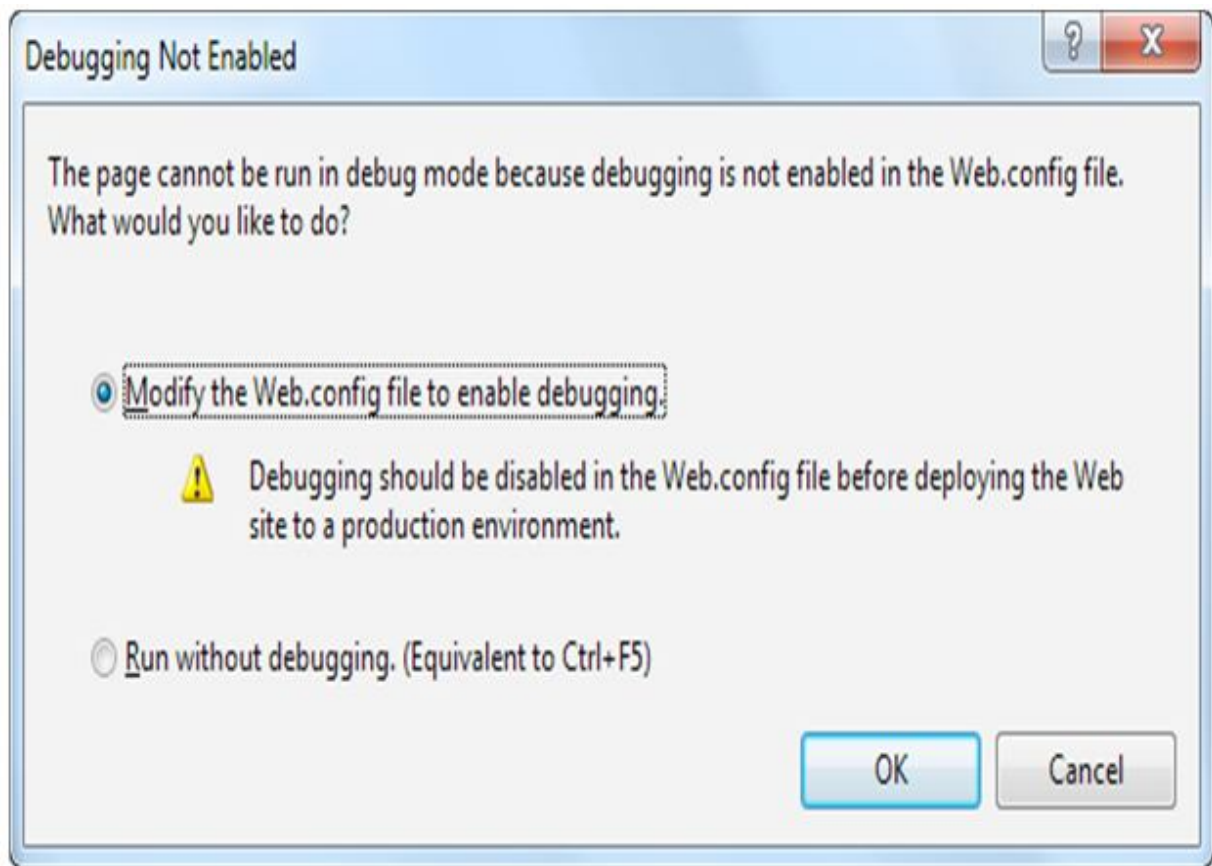


Figure 12. Enabling debugging

3.8 The Visual Studio Web Server

When you run a web application, Visual Studio starts its integrated web server. Behind the scenes, ASP.NET compiles the code for your web application, runs your web page, and then returns the final HTML to the browser. The first time you run a web page, you'll see a new icon appear in the system tray at the bottom-right corner of the taskbar. This icon is Visual Studio's test web server, which runs in the background hosting your website. The test server only runs while Visual Studio is running, and it only accepts requests from your computer (so other users can't connect to it over a network).

Visual Studio's built-in web server also allows you to retrieve a listing of all the files in your website. This means if you create a web application named SampleSite, you can request it in the form `http://localhost:port/SampleSite` (omitting the page name) to see a list of all the files in your web application folder. Then, just click the page you want to test.

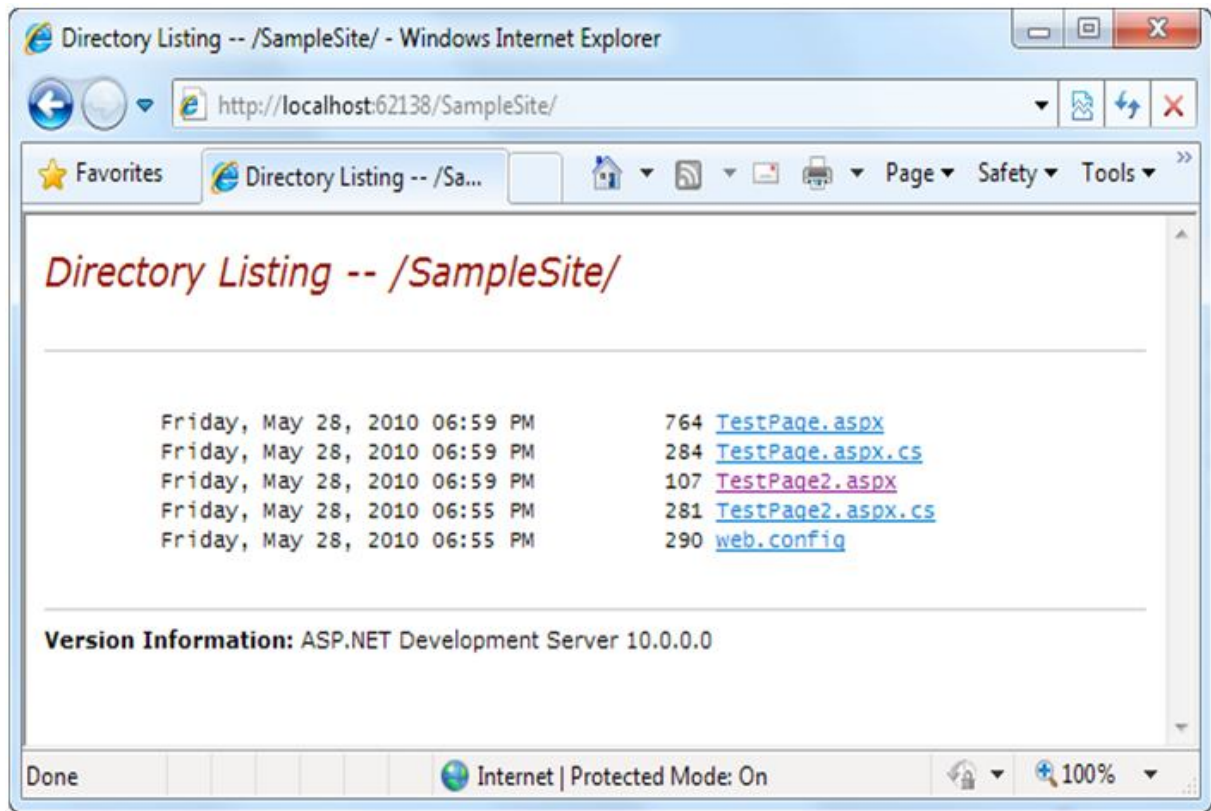


Figure 13. Choosing from a list of pages

This trick won't work if you have a Default.aspx page.

Single-Step Debugging

Single-step debugging allows you to test your assumptions about how your code works and see what's really happening under the hood of your application

1. Find a location in your code where you want to pause execution. Click in the margin next to the line of code (or press F9), and a red breakpoint will appear.
2. Now start your program as you would ordinarily (by pressing the F5 key or using the Start button on the toolbar). When the program reaches your breakpoint, execution will pause, and you'll be switched to the Visual Studio code window.
3. At this point, you have several options. You can execute the current line by pressing F11. The following line in your code will be highlighted with a yellow arrow, indicating that this is the next line that will be executed.

4. Whenever the code is in break mode, you can hover over variables to see their current contents (see Figure). This allows you to verify that variables contain the values you expect.
5. You can also use any of the commands listed in Table while in break mode.

Table. *Commands Available in Break Mode*

Command (Hot Key)	Description
Step Into (F11)	Executes the currently highlighted line and then pauses. If the currently highlighted line calls a method, execution will pause at the first executable line inside the method
Step Over (F10)	The same as Step Into, except it runs methods as though they are a single line. If you select Step Over while a method call is highlighted, the entire method will be executed. Execution will pause at the next executable statement in the current method
Step Out (Shift+F11)	Executes all the code in the current procedure and then pauses at the statement that immediately follows the one that called this method or function.
Continue (F5)	Resumes the program and continues to run it normally, without pausing until another breakpoint is reached.
Run to Cursor	Allows you to run all the code up to a specific line (where your cursor is currently positioned). You can use this technique to skip a time-consuming loop.
Set Next Statement	Allows you to change the path of execution of your program while debugging. This command causes your program to mark the current line (where your cursor is positioned) as the current line for execution. When you resume execution, this line will be executed, and the program will continue from that point.

Show Next Statement	Brings you to the line of code where Visual Studio is currently halted. (This is the line of code that will be executed next when you continue.) This line is marked by a yellow arrow.
---------------------	---

These commands are available from the context menu by right-clicking the code window or by using the associated hot key. You can switch your program into break mode at any point by clicking the Pause button in the toolbar or selecting Debug > Break

When debugging a large website, you might place breakpoints in different places in your code and in multiple web pages. To get an at-a-glance look at all the breakpoints in your web application, choose Debug > Windows > Breakpoints. You'll see a list of all your breakpoints, as shown in Figure 14).

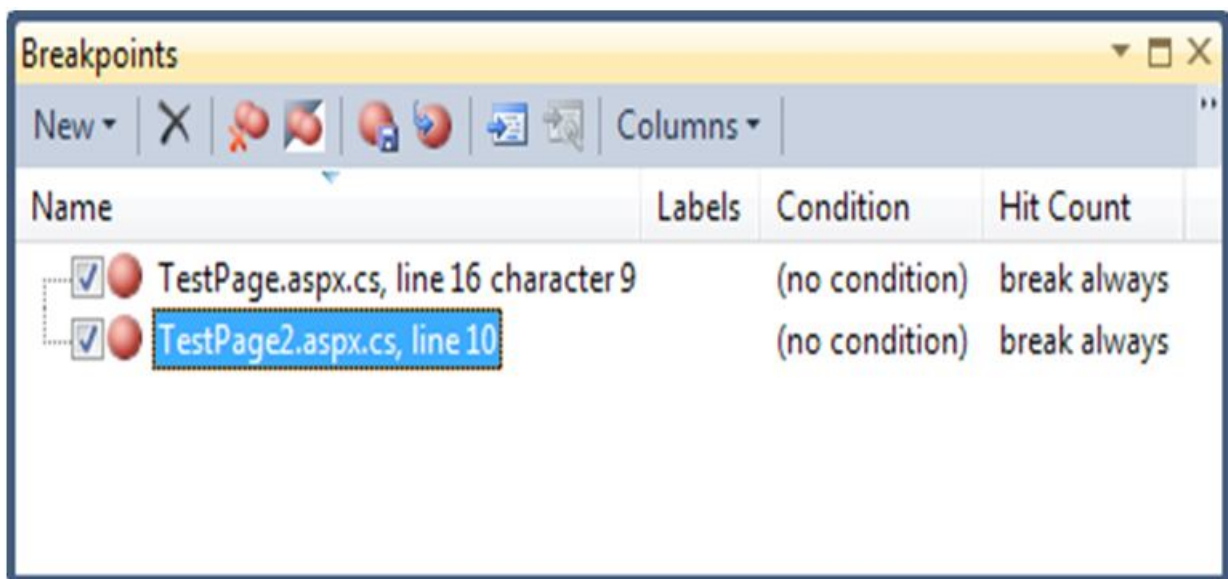


Figure 14. The Breakpoints window

Variable Watches

In some cases, you might want to track the status of a variable without switching into break mode repeatedly. In this case, it's more useful to use the Autos, Locals, and Watch windows, which allow you to track variables across an entire application.

Table. *Variable Watch Windows*

Window	Description
Autos	Automatically displays variables that Visual Studio determines are important for the current code statement. For example, this might include variables that are accessed or changed in the previous line.
Locals	Automatically displays all the variables that are in scope in the current method. This offers a quick summary of important variables.
Watch	Displays variables you have added. Watches are saved with your solution, so you can continue tracking a variable later. To add a watch, right-click a variable in your code while in break mode, and select Add Watch. Alternatively, double-click the last row in the Watch window and type in the variable name.

If you are missing one of the Watch windows, you can show it manually by selecting it from the **Debug > Windows** submenu.

3.9 The Anatomy of an ASP.NET Application

ASP.NET applications are almost always divided into multiple web pages. This division means a user can enter an ASP.NET application at several different points or follow a link from the application to another part of the website or another web server.

Every ASP.NET application shares a common set of resources and configuration settings. Web pages from other ASP.NET applications don't share these resources, even if they're on the same web server.

The standard definition of an ASP.NET application describes it as a combination of files, pages, handlers, modules, and executable code that can be invoked from a virtual directory (and, optionally, its subdirectories) on a web server. In other words, the virtual directory is the basic grouping structure that delimits an application. Figure 5-1 shows a web server that hosts four separate web applications.

ASP.NET File Types

ASP.NET applications can include many types of files.

Table. ASP.NET File Types

File Name	Description
Ends with .aspx	These are ASP.NET web pages (the .NET equivalent of the .asp file in an ASP application). They contain the user interface and, optionally, the underlying application code
Ends with .ascx	These are ASP.NET user controls. User controls are similar to web pages, except that the user can't access these files directly. User controls allow you to develop a small piece of user interface and reuse it in as many web forms as you want without repetitive code.
web.config	This is the XML-based configuration file for your ASP.NET application. It includes settings for customizing security, state management, memory management
global.asax	This is the global application file. You can use this file to define global variables (variables that can be accessed from any web page in the web application) and react to global events (such as when a web application first starts)
Ends with .cs	These are code-behind files that contain C# code. They allow you to separate the application logic from the user interface of a web page.

In addition, your web application can contain other resources that aren't special ASP.NET files. For example, your virtual directory can hold image files, HTML files, or CSS style sheets.

ASP.NET Application Directories

Every web application should have a well-planned directory structure. For example, you'll probably want to store images in a separate folder from where you store your web pages. Or you might want to put public ASP.NET pages in one folder and restricted ones in another so you can apply different security settings based on the directory.

Along with the directories you create, ASP.NET also uses a few specialized subfolders, which it recognizes by name. Keep in mind that you won't see all these subfolders in a typical application. Visual Studio will prompt you to create them as needed.

Table. *ASP.NET Directories*

Directory	Description
App_Browsers	Contains .browser files that ASP.NET uses to identify the browsers that are using your application and determine their capabilities.
App_Code	Contains source code files that are dynamically compiled for use in your application.
App_GlobalResources	Stores global resources that are accessible to every page in the web application. This directory is used in localization scenarios, when you need to have a website in more than one language
App_LocalResources	Serves the same purpose as App_GlobalResources, except these resources are accessible to a specific page only
App_WebReferences	Stores references to web services, which are remote code routines that a web application can call over a network or the Internet
App_Data	Stores data, including SQL Server Express database files
App_Themes	Stores the themes that are used to standardize and reuse formatting in your web application.
Bin	Contains all the compiled .NET components (DLLs) that the ASP.NET web application uses. For example, if you develop a custom component for accessing a database you'll place the component there.

3.10 Introducing Server Controls

ASP.NET actually provides *two* sets of server-side controls that you can incorporate into your web forms. These two different types of controls play subtly different roles:

HTML server controls: These are server-based equivalents for standard HTML elements.

Web controls: These are similar to the HTML server controls, but they provide a richer object model with a variety of properties for style and formatting details.

HTML Server Controls

HTML server controls provide an object interface for standard HTML elements. They provide three key features:

They generate their own interface: You set properties in code, and the underlying HTML tag is created automatically when the page is rendered and sent to the client.

They retain their state: Because the Web is stateless, ordinary web pages need to do a lot of work to store information between requests. HTML server controls handle this task automatically.

They fire server-side events: For example, buttons fire an event when clicked, text boxes fire an event when the text they contain is modified, and so on.

Converting an HTML Page to an ASP.NET Page

Figure 15 shows a currency converter web page. It allows the user to convert a number of U.S. dollars to the equivalent amount of euros. It's just a plain, inert HTML page. Nothing happens when the button is clicked.

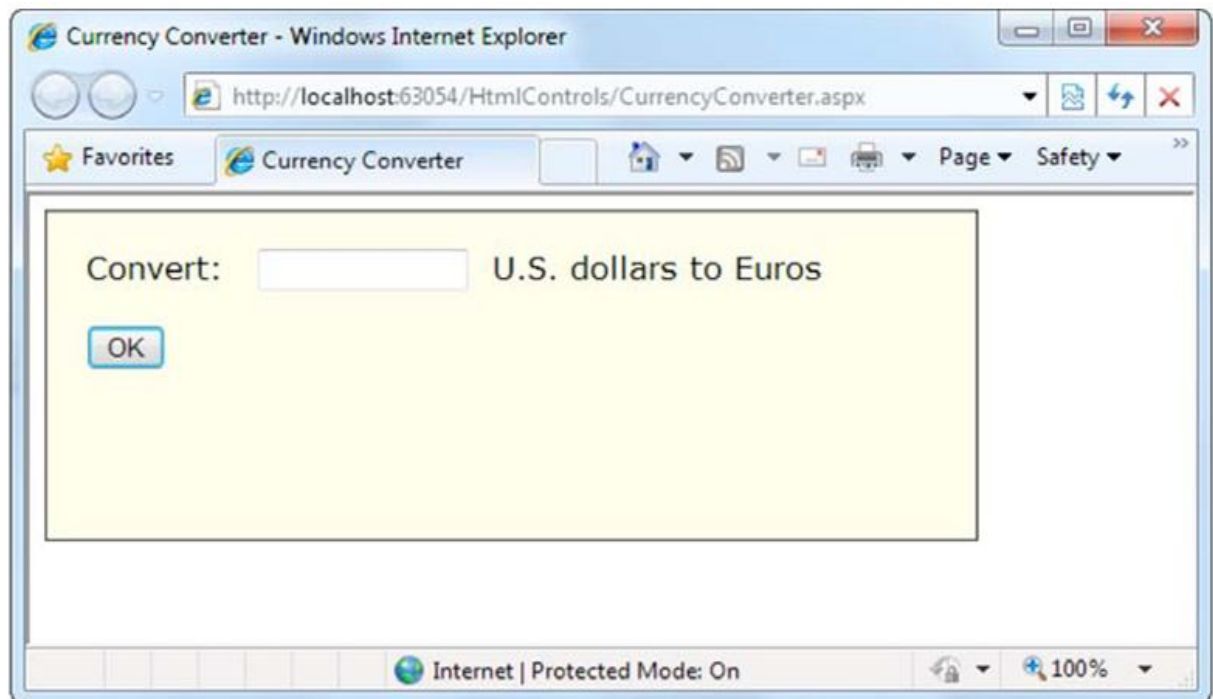


Figure 15. A simple currency converter

The following listing shows the markup for this page.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
"http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title>Currency Converter</title>
</head>
<body>
<form method="post">
<div>
Convert:&nbsp;
```



```

<input type="text" />

 U.S. dollars to Euros.

<br /><br />

<input type="submit" value="OK" />

</div>

</form>

</body>

</html>

```

As it stands, this page looks nice but provides no functionality. It consists entirely of the userinterface (HTML elements) and contains no code. It's an ordinary HTML page—not a web form.

The easiest way to convert the currency converter to ASP.NET is to start by generating a new webform in Visual Studio.

Now you need to add the attribute `runat="server"` to each tag that you want to transform into a server control.

In the currency converter application, it makes sense to change the input text box and the submit button into HTML server controls. In addition, the `<form>` element must be processed as a server control to allow ASP.NET to access the controls it contains. Here's the complete, correctly modified page:

```

<%@ Page Language="C#" AutoEventWireup="true"

CodeFile="CurrencyConverter.aspx.cs" Inherits="CurrencyConverter" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"

"http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">

```

```

<head>

<title>Currency Converter</title>

</head>

<body>

<form runat="server">

<div>

Convert: &nbsp;

<input type="text" ID="US" runat="server" />

&nbsp; U.S. dollars to Euros.

<br /><br />

<input type="submit" value="OK" ID="Convert" runat="server" />

</div>

</form>

</body>

</html>

</html>

```

The web page still won't do anything when you run it, because you haven't written any code. However, now that you've converted the static HTML elements to HTML server controls, you're ready to work with them.

View State

To try this page, launch it in Visual Studio by pressing F5. Remember, the first time you run your web application you'll be prompted to let Visual Studio modify your web.config file to allow debugging. Click OK to accept its recommendation and launch your web page in the browser.

Then, select View $\mathcal{L}$ Source in your browser to look at the HTML that ASP.NET sent your way.

The first thing you'll notice is that the HTML that was sent to the browser is slightly different from the information in the .aspx file. First, the `runat="server"` attributes are stripped out (because they have no meaning to the client browser, which can't interpret them). Second, and more important, an additional hidden field has been added to the form. This hidden field stores information, in a compressed format, about the state of every control in the page. It allows you to manipulate control properties in code and have the changes automatically persisted across multiple trips from the browser to the web server. This is a key part of the web forms programming model. Thanks to view state, you can often forget about the stateless nature of the Internet and treat your page like a continuously running application.

The HTML Control Classes

All the HTML server controls are defined in the `System.Web.UI.HtmlControls` namespace. Each kind of control has a separate class. Table 5-3 describes the basic HTML server controls and shows you the related HTML element.

Table. *The HTML Server Control Classes*

Class Name	HTML Element	Description
<code>HtmlForm</code>	<code><form></code>	Wraps all the controls on a web page. All <code>ASP.NET</code> server controls must be placed inside an <code>HtmlForm</code> control so that they can send their data to the server when the page is submitted.
<code>HtmlAnchor</code>	<code><a></code>	A hyperlink that the user clicks to jump to another page.
<code>HtmlImage</code>	<code></code>	A link that points to an image, which will be inserted into the web page at the current location.
<code>HtmlTable</code> , <code>HtmlTableRow</code> , and <code>HtmlTableCell</code>	<code><table></code> , <code><tr></code> , <code><th></code> , and <code><td></code>	A table that displays multiple rows and columns of static text.

<u>HtmlInputButton</u> , <u>HtmlInputSubmit</u> , and <u>HtmlInputReset</u>	<code><input type="button"></code> , <code><input type="submit"></code> , and <code><input type="reset"></code>	A button that the user clicks to perform an action(<u>HtmlInputButton</u>), submit the page(<u>HtmlInputSubmit</u>), or clear all the user-supplied values in all the controls (<u>HtmlInputReset</u>).
<u>HtmlButton</u>	<code><button></code>	A button that the user clicks to perform an action
<u>HtmlInputCheckBox</u>	<code><input type="checkbox"></code>	A check box that the user can check or clear. Doesn't include any text of its own.
<u>HtmlInputRadioButton</u>	<code><input type="radio"></code>	A radio button that can be selected in a group. Doesn't include any text of its own.
<u>HtmlInputText</u> and <u>HtmlInputPassword</u>	<code><input type="text"></code> and <code><input type="password"></code>	A single-line text box where the user can enter information. Can also be displayed as a password field (which displays bullets instead of characters to hide the user input).
<u>HtmlTextArea</u>	<code><textarea></code>	A large text box where the user can type multiple lines of text.
<u>HtmlInputImage</u>	<code><input type="image"></code>	Similar to the <code></code> tag, but inserts a "clickable" image that submits the page.
<u>HtmlInputFile</u>	<code><input type="file"></code>	A Browse button and text box that can be used to upload a file to your web server
<u>HtmlInputHidden</u>	<code><input type="hidden"></code>	Contains text information that will be sent to the server when the page is posted back but won't be visible in the browser.
<u>HtmlSelect</u>	<code><select></code>	A drop-down or regular list box where the user can select an item.
<u>HtmlHead</u> and <u>HtmlTitle</u>	<code><head></code> and <code><title></code>	Represents the header information for the page which includes information about the page that isn't actually displayed in the page
<u>HtmlGenericControl</u>	Any other HTML element	This control can represent a variety of HTML elements that don't have dedicated control classes.

there are two ways to add any HTML server control. You can add it by hand to the markup in the .aspx file (simply insert the ordinary HTML element, and add the `runat="server"` attribute). Alternatively, you can drag the control from the HTML tab of the Toolbox, and drop it onto the design surface of a web page in Visual Studio.

Adding the Currency Converter Code

To actually add some functionality to the currency converter, you need to add some ASP.NET code. Webforms are event-driven. It makes sense to add another control that can display the result of the calculation. In this case, you can use a `<p>` element named `Result`.

```
<p style="font-weight: bold" ID="Result" runat="server">/p>
```

The example now has the following four server controls:

- A form (HtmlForm object). This is the only control you do not need to access in your code-behind class.
- An input text box named `US` (HtmlInputText object).
- A submit button named `Convert` (HtmlInputButton object).
- A `<p>` element named `Result` (HtmlGenericControl object).

Listing 1 shows the revised web page (`CurrencyConverter.aspx`), but leaves out the doctype to save space. Listing 2 shows the code-behind class (`CurrencyConverter.aspx.cs`). The code-behind class includes an event handler that reacts when the convert button is clicked. It calculates the currency conversion and displays the result.

Listing 1. CurrencyConverter.aspx

```
<%@ Page Language="C#" AutoEventWireup="true"
CodeFile="CurrencyConverter.aspx.cs" Inherits="CurrencyConverter" %>

<html xmlns="http://www.w3.org/1999/xhtml">

<head>

<title>Currency Converter</title>
```

```

</head>

<body>

<form runat="server">

<div>

Convert: &nbsp;

<input type="text" ID="US" runat="server" />

&nbsp; U.S. dollars to Euros.

<br /><br />

<input type="submit" value="OK" ID="Convert" runat="server"
OnServerClick="Convert_ServerClick" />

<br /><br />

<p style="font-weight: bold" ID="Result" runat="server"></p>

</div>

</form>

</body>

</html>

```

Listing 2. CurrencyConverter.aspx.cs

```

using System;

using System.Collections.Generic;

using System.Linq;

using System.Web;

using System.Web.UI;

using System.Web.UI.WebControls;

public partial class CurrencyConverter : System.Web.UI.Page

```

```

{
    protected void Convert_ServerClick(object sender, EventArgs e)
    {
        decimal USAmount = Decimal.Parse(US.Value);
        decimal euroAmount = USAmount * 0.85M;
        Result.InnerText = USAmount.ToString() + " U.S. dollars = ";
        Result.InnerText += euroAmount.ToString() + " Euros.";
    }
}

```

The code-behind class is a typical example of an ASP.NET page. You'll notice the following conventions:

- It starts with several using statements. This provides access to all the important namespaces. This is a typical first step in any code-behind file.
- The page class is defined with the partial keyword. That's because your class code is merged with another code file that you never see. This extra code, which ASP.NET generates automatically, defines all the server controls that are used on the page. This allows you to access them by name in your code.
- The page defines a single event handler. This event handler retrieves the value from the text box, converts it to a numeric value, multiplies it by a present conversion ratio (which would typically be stored in another file or a database), and sets the text of the <p> element. You'll notice that the event handler accepts two parameters (sender and e). This is the .NET standard for all control events. It allows your code to identify the control that sent the event (through the sender parameter) and retrieve any other information that may be associated with the event (through the e parameter). You'll see examples of these advanced techniques in the next chapter, but for now, it's important to realize that you won't be allowed to handle an event unless your event handler has the correct, matching signature.
- The event handler is connected to the control event using the OnServerClick attribute in the <input> tag for the button. You'll learn more about how this hookup works in the next section.

- The += operator is used to quickly add information to the end of the label, without replacing the existing text.
- The event handler uses ToString() to convert the decimal value to text so it can be added to the InnerText property. In this particular statement, you don't need ToString(), because C# is intelligent enough to realize you're joining together pieces of text. However, this isn't always the case, so it's best to be explicit about data type conversions.

You can launch this page to test your code. When you enter a value and click the OK button, the page is resubmitted, the event handling code runs, and the page is returned to you with the conversion details.

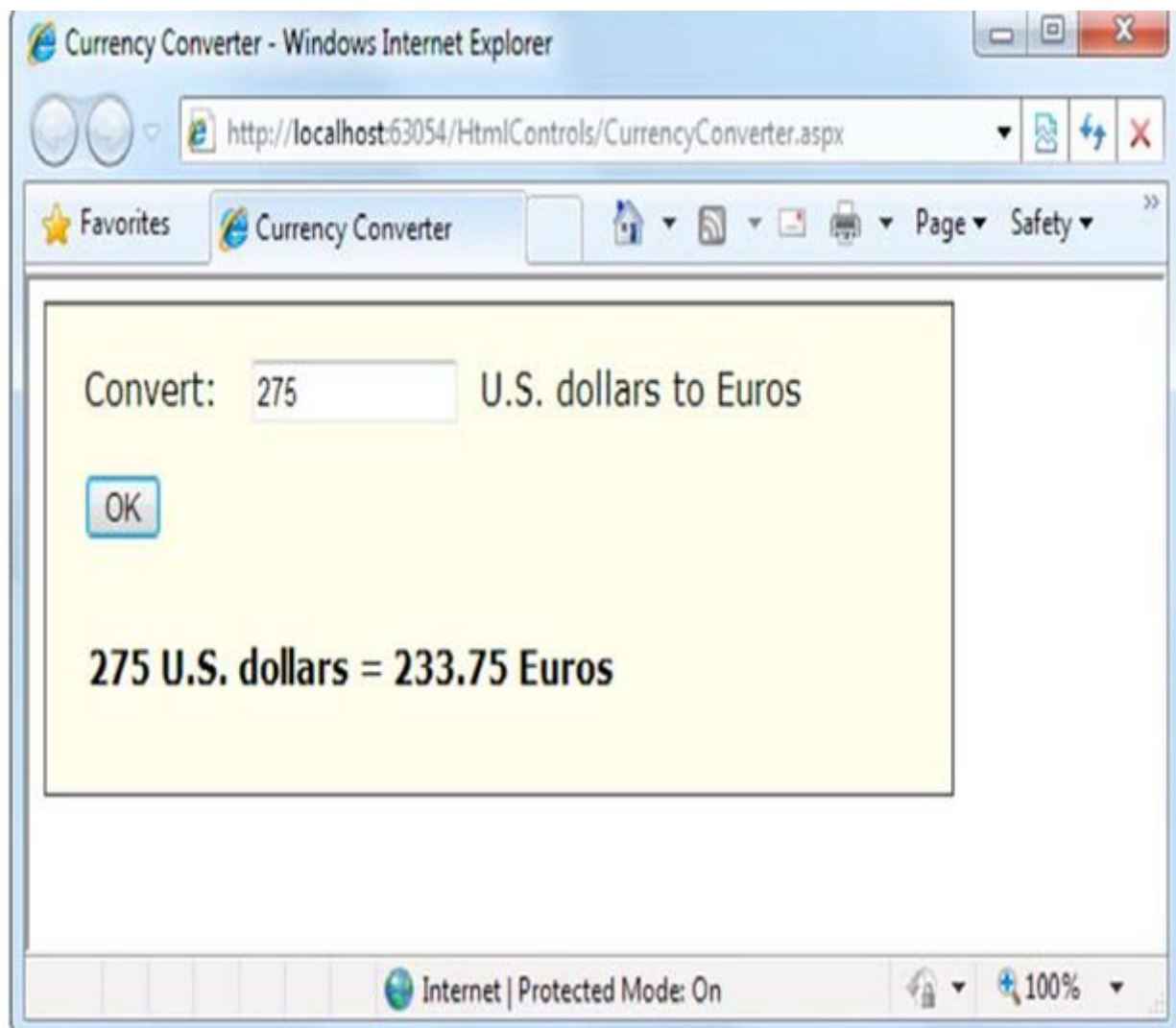


Figure 16. The ASP.NET currency converter

Event Handling

When the user clicks the Convert button and the page is sent back to the web server, ASP.NET needs to know exactly what code you want to run. To create this relationship and connect an event to an eventhandling method, you need to add an attribute in the control tag.

For example, if you want to handle the ServerClick method of the Convert button, you simply need to set the OnServerClick attribute in the control tag with the name of the event handler you want to use:

```
<input type="submit" value="OK" ID="Convert"
```

```
OnServerClick="Convert_ServerClick" runat="server">
```

ASP.NET allows you to use another technique to connect events—you can do it using code. For example, here's the code that's required to hook up the ServerClick event of the Convert button using manual event wireup:

```
Convert.ServerClick += this.Convert_ServerClick;
```

Behind the Scenes with the Currency Converter

So, what really happens when ASP.NET receives a request for the CurrencyConverter.aspx page? The process actually unfolds over several steps:

1. First, the request for the page is sent to the web server. If you're running a livesite, the web server is almost certainly IIS. If you're running the page in Visual Studio, the request is sent to the built-in test server.
2. The web server determines that the .aspx file extension is registered with ASP.NET. If the file extension belonged to another service (as it would for .asp or .html files), ASP.NET would never get involved.
3. If this is the first time a page in this application has been requested, ASP.NET automatically creates the application domain. It also compiles all the web page code for optimum performance, and caches the compiled files. If this task has already been performed, ASP.NET will reuse the compiled version of the page.

4. The compiled `CurrencyConverter.aspx` page acts like a miniature program. It starts firing events (most notably, the `Page.Load` event). However, you haven't created an event handler for that event, so no code runs. At this stage, everything is working together as a set of in-memory .NET objects.

5. When the code is finished, ASP.NET asks every control in the web page to render itself into the corresponding HTML markup.

6. The final page is sent to the user, and the application ends.

Error Handling

The currency converter expects that the user will enter a number before clicking the Convert button. If the user enters something else—for example, a sequence of text or special characters that can't be interpreted as a number—an error will occur when the code attempts to use the `Decimal.Parse()` method. We can use `Decimal.TryParse()` method instead of `Decimal.Parse()`. Unlike `Parse()`, `TryParse()` does not generate an error if the conversion fails—it simply informs you of the problem. `TryParse()` accepts two parameters. The first parameter is the value you want to convert (in this example, `US.Value`). The second parameter is an output parameter that will receive the converted value (in this case, the variable named `USAmount`). What's special about `TryParse()` is its Boolean return value, which indicates if the conversion was successful (true) or not (false).

Here's a revised version of the `ServerClick` event handler that uses `TryParse()` to check for conversion problems and inform the user:

```
protected void Convert_ServerClick(object sender, EventArgs e)

{

    decimal USAmount;

    // Attempt the conversion.

    bool success = Decimal.TryParse(US.Value, out USAmount);

    // Check if it succeeded.

    if (success)
```

```

{

// The conversion succeeded.

decimal euroAmount = USAmount * 0.85M;

Result.InnerText = USAmount.ToString() + " U.S. dollars = ";

Result.InnerText += euroAmount.ToString() + " Euros.";

}

else

{

// The conversion failed.

Result.InnerText = "The number you typed in was not in the " +

"correct format. Use only numbers.";

}

}

```

Dealing with error conditions like these is an essential technique in a real-world web page.

3.11 Summary

In this chapter, you took a quick look at Visual Studio. First, you saw how to create a new web application using the clean project less website model. Next, you considered how to design basic web pages, complete with controls and code. You should now understand how to create an ASP.NET web page and use HTML server controls. With HTML controls, the final HTML page that is sent to the client closely resembles the original .aspx page.

Keywords

Solution Explorer, project less website, Web form, Properties Window, Page Directive, ASP.NET File Types, Server Controls

3.12 Check your understanding here

1. In single step debugging which command Resumes the program and continues to run it normally, without pausing until another breakpoint is reached.
 - A. **Continue (F5)**
 - B. Run to Cursor
 - C. Step Over (F10)
 - D. Step Into (F11)
2. In Html <a> anchor tag is belonging to which type?
 - A. Stand-alone
 - B. Neither Stand-alone nor container
 - C. Group
 - D. **Container**
3. In which Asp.net file global variables are defined?
 - A. web.config
 - B. **global.asax**
 - C. File name Ends with .aspx
 - D. File name Ends with .ascx
4. Which Asp.net directory Stores the themes that are used to standardize and reuse formatting in your web application?
 - A. App_Data
 - B. App_WebReferences
 - C. **App_Themes**
 - D. App_Code

3.13 Model Questions

1. Write short note on The Anatomy of a Web Form.
1. What are the ways to add event handler to web page?
2. Write notes on different Asp.NET file types.
3. What is server control?
4. Describe the HTML Control Classes and elements.

Reference Books:

1. **Mathew Mac Donald, "Beginning ASP.NET 4 in C# 2010" Apress 2010**

LESSON - 4

WEB PAGE DEVELOPMENT

Structure

- 4.1 Introduction**
- 4.2 Objectives**
- 4.3 HTML Control Classes**
- 4.4 The Page class**
- 4.5 Application event**
- 4.6 ASP.NET Configuration**
- 4.7 The Website Administration Tool (WAT)**
- 4.8 Web controls**
- 4.9 Web Control Events and AutoPostBack**
- 4.10 A Simple Web Page**
- 4.11 Summary**
- 4.12 Check your progress**
- 4.13 Model Questions**

4.1 Introduction

ASP.NET applications are almost always divided into multiple web pages. This division means a user can enter an ASP.NET application at several different points or follow a link from the application to another part of the website or another web server. Application domains restrict a web page in one application from accessing the in-memory information of another application. Each web application is maintained separately and has its own set of cached, application, and session data using application events, you can write logging code that runs every time a request is received, no matter what page is being requested.

4.2 Learning objectives

- ✓ Understand HTML control Classes
- ✓ Explore Page Class
- ✓ Handle application events
- ✓ Write to code in Asp.net Configuration
- ✓ Understand Web control classes
- ✓ Handle web control events
- ✓ Create Sample web page

4.3 HTML Control Classes

Related classes in the .NET Framework use inheritance to share functionality. For example, every HTMLcontrol inherits from the base class HtmlControl. The HtmlControl class provides essential features every HTML server control uses.

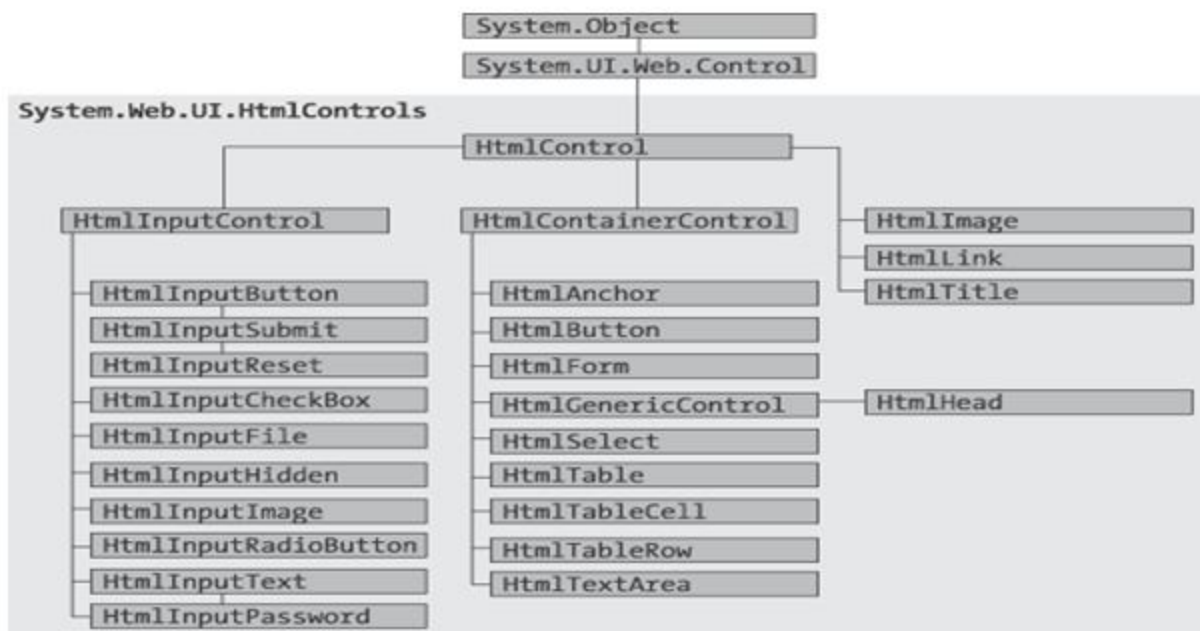


Figure 1 HTML control inheritance

HTML server controls generally provide properties that closely match their tag attributes.

HTML Control Events

HTML server controls also provide one of two possible events: `ServerClick` or `ServerChange`. The `ServerClick` is simply a click that's processed on the server side. The following table shows which controls provide a `ServerClick` event and which ones provide a `ServerChange` event.

Table:HTML Control Events

Event	Controls That Provide It
<code>ServerClick</code>	<code>HtmlAnchor</code> , <code>HtmlButton</code> , <code>HtmlInputButton</code> , <code>HtmlInputImage</code> , <code>HtmlInputReset</code>
<code>ServerChange</code>	<code>HtmlInputText</code> , <code>HtmlInputCheckBox</code> , <code>HtmlInputRadioButton</code> , <code>HtmlInputHidden</code> , <code>HtmlSelect</code> , <code>HtmlTextArea</code>

The `HtmlControl` Base Class

Every HTML control inherits from the base class `HtmlControl`. This relationship means that every HTML control will support a basic set of properties and features. The following table shows these properties.

Table. `HtmlControl` Properties

Property	Description
<code>Attributes</code>	Provides a collection of all the attributes that are set in the control tag, and their values.
<code>Controls</code>	Provides a collection of all the controls contained inside the current control.
<code>Disabled</code>	Disables the control when set to true, thereby ensuring that the user cannot interact with it, and its events will not be fired.
<code>EnableViewState</code>	Disables the automatic state management for this control when set to false
<code>Page</code>	Provides a reference to the web page that contains this control as a <code>System.Web.UI.Page</code> object.

Parent	Provides a reference to the control that contains this control. If the control is placed directly on the page (rather than inside another control), it will return a reference to the page object.
Style	Provides a collection of CSS style properties that can be used to format the control.
TagName	Indicates the name of the underlying HTML element
Visible	Hides the control when set to false and will not be rendered to the final HTML page that is sent to the client.

The `HtmlControl` class also provides built-in support for data binding

The `HtmlContainerControl` Class

Any HTML control that requires a closing tag inherits from the `HtmlContainer` class. For example, elements such as `<a>`, `<form>`, and `<div>` always use a closing tag, because they can contain other HTML elements. On the other hand, elements such as `<img>` and `<input>` are used only as stand-alone tags. The `HtmlContainer` control adds two properties to those defined in `HtmlControl`

Table .`HtmlContainerControl` Properties

Property	Description
Inner HTML	The HTML content between the opening and closing tags of the control. Special characters that are set through this property will not be converted to the equivalent HTML entities.
Inner Text	The text content between the opening and closing tags of the control. Special characters will be automatically converted to HTML entities and displayed like text (for example, the less-than character (<code><</code>) will be converted to <code>&lt;</code> and will be displayed as <code><</code> in the web page).

The `HtmlInputControl` Class

The `HtmlInputControl` class inherits from `HtmlControl` and adds some properties (shown in Table) that are used for the `<input>` element.

Table .HtmlInputControl Properties

Property	Description
Type	Provides the type of input control. For example, a control based on <input type="file">would return <i>file</i> for the type property.
Value	Returns the contents of the control as a string. In the simple currency converter, this property allowed the code to retrieve the information entered in the text input control.

4.4 The Page Class

Every web page is a custom class that inherits from `System.Web.UI.Page`. By inheriting from this class, your web page class acquires a number of properties and methods that your code can use. These include properties for enabling caching, validation, and tracing, which are discussed throughout this book. The following table provides an overview of some of the more fundamental properties, which you'll use throughout this book.

Table .Basic Page Properties

Property	Description
IsPostBack	This Boolean property indicates whether this is the first time the page is being run (false) or whether the page is being resubmitted in response to a control event, typically with stored view state information (true).
EnableViewState	When set to false, this overrides the <code>EnableViewState</code> property of the contained controls, thereby ensuring that no controls will maintain state information.
Application	This collection holds information that's shared between all users in your website. For example, you can use the <code>Application</code> collection to count the number of times a page has been visited.
Session	This collection holds information for a single user, so it can be used in different pages. For example, you can use the <code>Session</code> collection to store the items in the current user's shopping basket on an e-commerce website.

Cache	This collection allows you to store objects that are time-consuming to create so they can be reused in other pages or for other clients. This technique, when implemented properly, can improve performance of your web pages.
Request	This refers to an <code>HttpRequest</code> object that contains information about the current web request. You can use the <code>HttpRequest</code> object to get information about the user's browser.
Response	This refers to an <code>HttpResponse</code> object that represents the response ASP.NET will send to the user's browser. You'll use the <code>HttpResponse</code> object to create cookies.
Server	This refers to an <code>HttpServerUtility</code> object that allows you to perform a few miscellaneous tasks. For example, it allows you to encode text so that it's safe to place it in a URL or in the HTML markup of your page.
User	If the user has been authenticated, this property will be initialized with user information.

Sending the User to a New Page

There are several ways to transfer a user from one page to another. One of the simplest is to use an ordinary `<a>` anchor element, which turns a portion of text into a hyperlink. In this example, the word *here* is a link to another page: Click

```
<a href="newpage.aspx">here</a>
```

to go to `newpage.aspx`. Another option is to send the user to a new page using code. This approach is useful if you want to use your code to perform some other work before you redirect the user.

To perform redirection in code, you first need a control that causes the page to be posted back. In other words, you need an event handler that reacts to the `ServerClick` event of a control such as `HtmlInputButton` or `HtmlAnchor`.

You can get access to the current `HttpResponse` object through the `Page.Response` property. Here's an example that sends the user to a different page in the same website directory:

```
Response.Redirect("newpage.aspx");
```

You can use the `Redirect()` method to send the user to any type of page. You can even send the user to another website using an absolute URL (a URL that starts with `http://`), as shown here:

```
Response.Redirect("http://www.prosetech.com");
```

ASP.NET gives you one other option for sending the user to a new page. You can use the `HttpServerUtility.Transfer()` method instead of `Response.Redirect()`. An `HttpServerUtility` object is provided through the `Page.Server` property, so your redirection code would look like this:

```
Server.Transfer("newpage.aspx");
```

HTML Encoding

There are certain characters that have a special meaning in HTML. For example, the angle brackets (`<>`) are always used to create tags. This can cause problems if you actually want to use these characters as part of the content of your web page.

Table. *Common HTML Special Characters.*

Result	Description	Encoded Entity
	Nonbreaking space	<code>&nbsp;</code>
<code><</code>	Less-than symbol	<code>&lt;</code>
<code>></code>	Greater-than symbol	<code>&gt;</code>
<code>&</code>	Ampersand	<code>&amp;</code>
<code>"</code>	Quotation mark	<code>&quot;</code>

4.5 Application Events

Using application events you can write logging code that runs every time a request is received, no matter what page is being requested. Basic ASP.NET features like session state and authentication use application events to plug into the ASP.NET processing pipeline.

You can't handle application events in the code behind for a web form. Instead, you need the help of another ingredient: the global.asax file.

The global.asax File

The global.asax file allows you to write code that responds to global application events.

To add a global.asax file to an application in Visual Studio, choose Website > Add New Item, and select the Global Application Class file type. Then, click OK.

The global.asax file looks similar to a normal .aspx file, except that it can't contain any HTML or ASP.NET tags. Instead, it contains event handlers that respond to application events. When you add the global.asax file the following global.asax file reacts to the EndRequest event, which happens just before the page is sent to the user:

```
<%@ Application Language="C#" %>
<script language="c#" runat="server">
protected void Application_EndRequest(object sender, EventArgs e)
{
    Response.Write("<hr />This page was served at " +
        DateTime.Now.ToString());
}
</script>
```

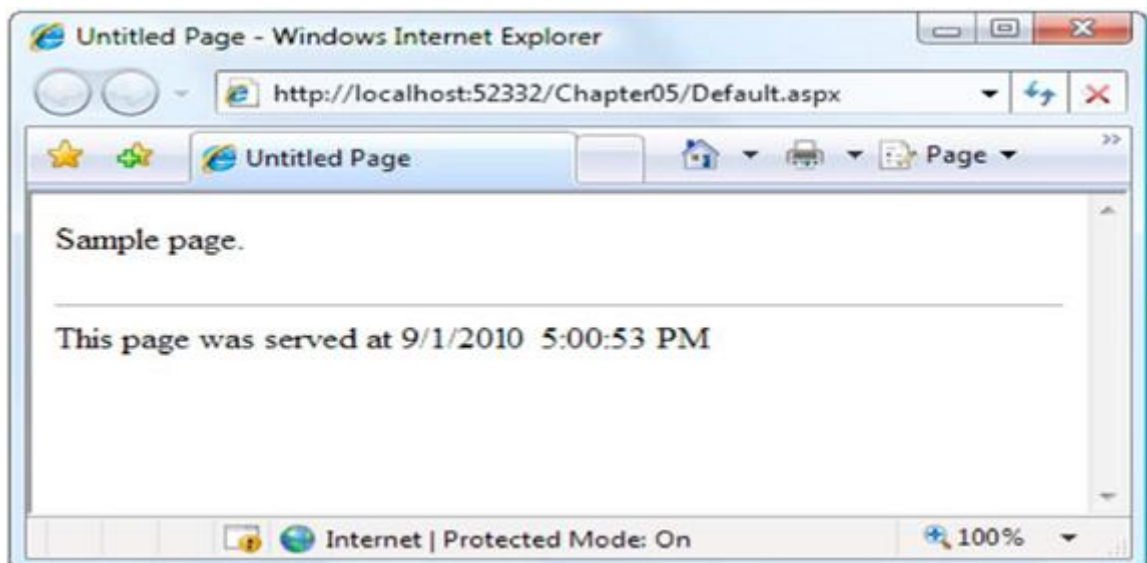


Figure 2. HelloWorld.aspx with an automatic footer

Each ASP.NET application can have one global.asax file. Once you place it in the appropriate website directory, ASP.NET recognizes it and uses it automatically.

Additional Application Events

ApplicationEndRequest is only one of more than a dozen events you can respond to in your code. To create a different event handler, you simply need to create a subroutine with the defined name. The following table lists some of the most common application events that you'll use

Table. *Basic Application Events*

Event Handling Method	Description
Application_Start()	Occurs when the application starts, which is the first time it receives a request from any user.
Application_End()	Occurs when the application is shutting down, generally because the web server is being restarted
Application_BeginRequest()	Occurs with each request the application receives, just before the page code is executed
Application_EndRequest()	Occurs with each request the application receives, just after the page code is executed.
Session_Start()	Occurs whenever a new user request is received and a session is started.
Session_End()	Occurs when a session times out or is programmatically ended. This event is only raised if you are using in-process session state storage
Application_Error()	Occurs in response to an unhandled error.

4.6 ASP.NET Configuration

Every web application includes a web.config file that configures fundamental settings—everything from the way error messages are shown to the security settings that lock out unwanted visitors. The ASP.NET configuration files have several key advantages:

They are never locked: You can update web.config settings at any point, even while your application is running.

They are easily accessed and replicated: Provided you have the appropriate network rights, you can change a web.config file from a remote computer.

The settings are easy to edit and understand: The settings in the web.config file are human-readable.

The web.config File

The web.config file uses a predefined XML format. The entire content of the file is nested in a root `<configuration>` element. Inside this element are several more subsections.

```
<?xml version="1.0" ?>

<configuration>

  <appSettings>...</appSettings>

  <connectionStrings>...</connectionStrings>

  <system.web>...</system.web>

</configuration>
```

the web.config file is case-sensitive, like all XML documents, and starts every setting with a lowercase letter. As a web developer, there are three sections in the web.config file that you'll work with. The `<appSettings>` section allows you to add your own miscellaneous pieces of information.

The `<connectionStrings>` section allows you to define the connection information for accessing a database. Finally, the `<system.web>` section holds every ASP.NET setting you'll need to configure.

Inside the `<system.web>` element are separate elements for each aspect of website configuration. You can include as few or as many of these as you want. For example, if you want to control how ASP.NET's security works, you'd add the `<authentication>` and `<authorization>` sections. When you create a new website, there's just one element in the `<system.web>` section, named

`<compilation>:`

`<?xml version="1.0" ?>`

`<configuration>`

`<system.web>`

`<compilation debug="true" targetFramework="4.0">`

`</compilation>`

`</system.web>`

`</configuration>`

The `<compilation>` element specifies two settings using two attributes: *debug*: This attribute tells ASP.NET whether to compile the application in debug mode so you can use Visual Studio's debugging tools. *targetFramework*: This attribute tells Visual Studio what version of .NET you're planning to use on your web server (which determines what features you'll have access to in your web application).

Nested Configuration

ASP.NET uses a multilayered configuration system that allows you to set settings at different levels. Every web server starts with some basic settings that are defined in a directory such as `asc:\Windows\Microsoft.NET\Framework\[Version]\Config`. The `[Version]` part of the directory name depends on the version of .NET that you have installed.

The Config folder contains two files, named `machine.config` and `web.config`. Generally, you won't edit either of these files by hand, because they affect the entire computer. Instead, you'll configure the `web.config` file in your web application folder.

Using that file, you can set additional settings or override the defaults that are configured in the two system files. You can use different settings for different parts of your application. To use this technique, you need to create additional subdirectories inside your virtual directory. These subdirectories can contain their own `web.config` files with additional settings. Subdirectories inherit `web.config` settings from the parent directory.

Storing Custom Settings in the web.config File

ASP.NET also allows you to store your own settings in the web.config file, in an element called `<appSettings>`. Note that the `<appSettings>` element is nested in the root `<configuration>` element.

Here's the basic structure:

```
<?xml version="1.0" ?>

<configuration>

  <appSettings>

    <!-- Custom application settings go here. -->

  </appSettings>

  <system.web>

    <!-- ASP.NET Configuration sections go here. -->

  </system.web>

</configuration>
```

The custom settings that you add are written as simple string variables. You might want to use a special web.config setting for several reasons:

To centralize an important setting that needs to be used in many different pages: For example, you could create a variable that stores a database query. Any page that needs to use this query can then retrieve this value and use it.

To make it easy to quickly switch between different modes of operation: For example, you might create a special debugging variable. Your web pages could check for this variable and, if it's set to a specified value, output additional information to help you test the application.

To set some initial values: Depending on the operation, the user might be able to modify these values, but the web.config file could supply the defaults.

You can enter custom settings using an `<add>` element that identifies a unique variable name (key) and the variable contents (value). The following example adds a variable that defines a file path where important information is stored:

```
<appSettings>
```

```
<add key="DataFilePath"
```

```
value="e:\NetworkShare\Documents\WebApp\Shared" />
```

```
</appSettings>
```

You can add as many application settings as you want.

4.7 The Website Administration Tool (WAT)

Website Administration Tool (WAT), which lets you configure various parts of the web.config file using a webpage interface. To run the WAT to configure the current web project in Visual Studio, select Website > ASP.NET Configuration. Internet Explorer will automatically log you on under the current Windows user account, allowing you to make changes.

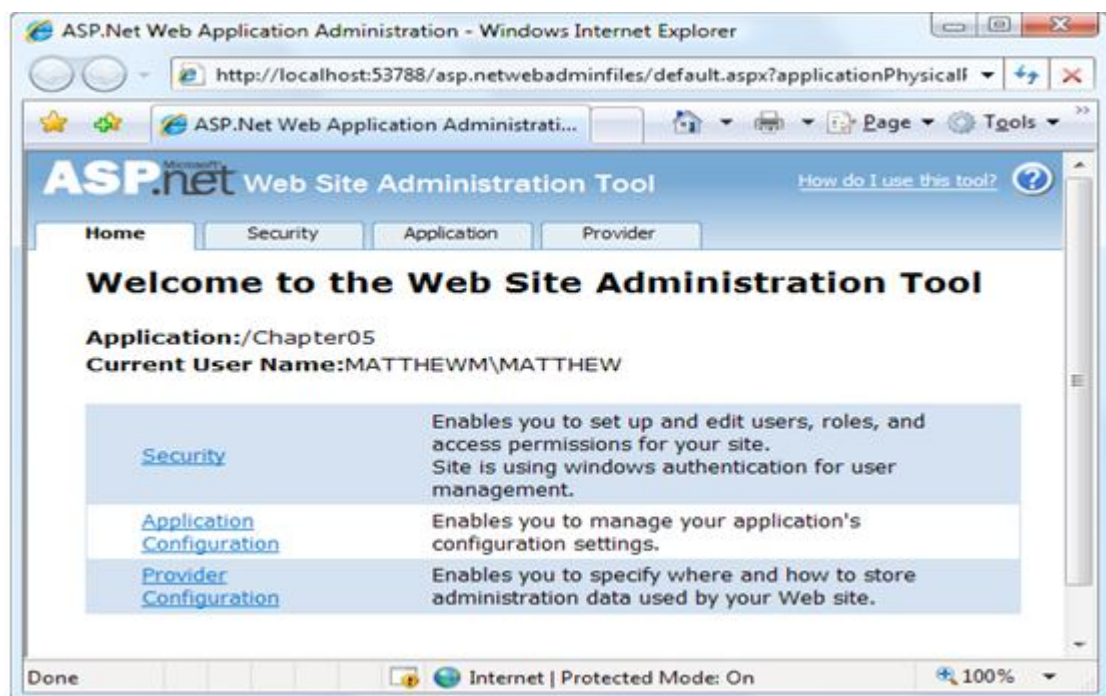


Figure 3. Running the WAT

To try this, click the Application tab. Using this tab, you can create a new setting

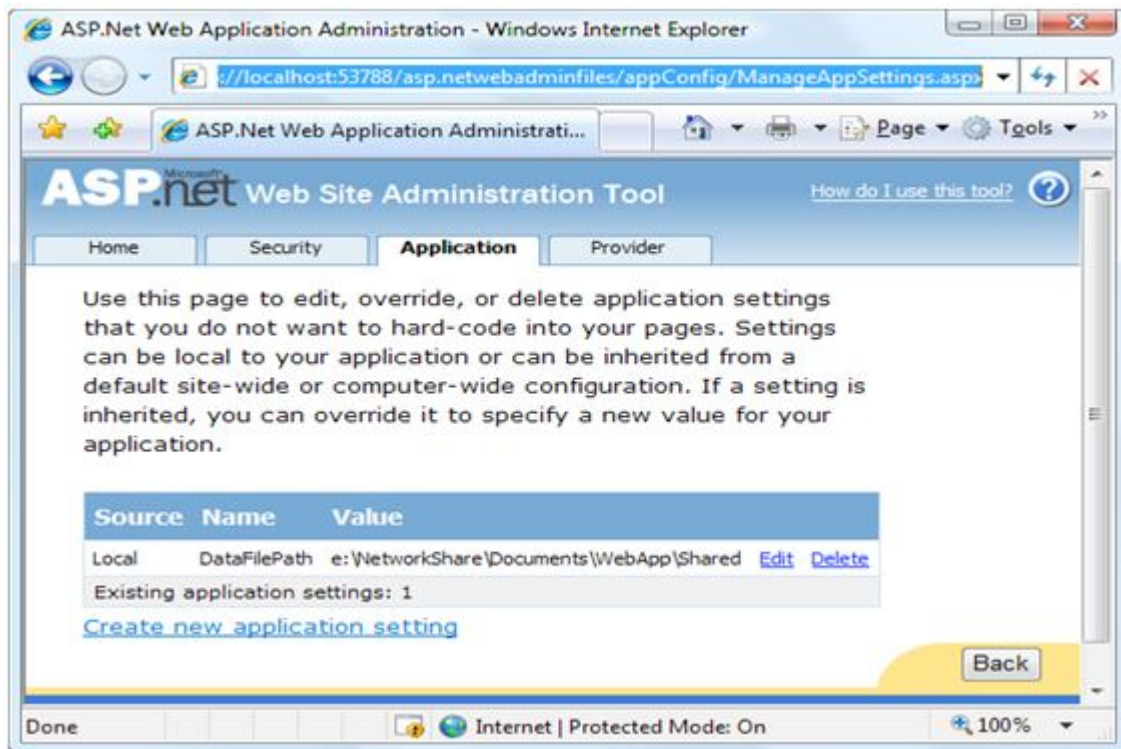


Figure 4. Editing an application setting with the WAT

This is the essential idea behind the WAT. You make your changes using a graphical interface (a webpage), and the WAT generates the settings you need and adds them to the web.config file for your application behind the scenes.

4.8 Web Controls

But in fact, HTML controls are much more limited than server controls need to be. so we need additional webControls. These are some of the reasons you should switch to web controls:

They provide a rich user interface:

They provide a consistent object model:

They tailor their output automatically:

They provide high-level features:

Basic Web Control Classes

Table lists the basic control classes and the HTML elements they generate.

Table:*Basic Web Controls*

Control Class	Underlying HTML Element
Label	
Button	<input type="submit"> or <input type="button">
TextBox	<input type="text">, <input type="password">, or <textarea>
CheckBox	<input type="checkbox">
RadioButton	<input type="radio">
Hyperlink	<a>
LinkButton	<a> with a contained tag
ImageButton	<input type="image">
Image	
ListBox	<select size="X"> where X is the number of rows that are visible at once
DropDownList	<select>
CheckBoxList	A list or <table> with multiple <input type="checkbox"> tags
RadioButtonList	A list or <table> with multiple <input type="radio"> tags
BulletedList	An ordered list (numbered) or unordered list (bulleted)
Panel	<div>
Table, TableRow, and TableCell	<table>, <tr>, and <td> or <th>

The Web Control Tags

ASP.NET tags have a special format. They always begin with the prefix `asp:` followed by the class name. If there is no closing tag, the tag must end with `/>`. Each attribute in the tag corresponds to a control property, except for the `runat="server"` attribute, which signals that the control should be processed on the server.

The following, for example, is an ASP.NET TextBox:

```
<asp:TextBox ID="txt" runat="server" />
```

When a client requests this .aspx page, the following HTML is returned. The name is a special attribute that ASP.NET uses to track the control.

```
<input type="text" ID="txt" name="txt" />
```

Here's one example of a customized TextBox:

```
<asp:TextBox ID="txt" BackColor="Yellow" Text="Hello World"
```

```
ReadOnly="True" TextMode="MultiLine" Rows="5" runat="server" />
```

The resulting HTML uses the `<textarea>` element and sets all the required attributes (such as rows and readonly) and the style attribute (with the background color). It also sets the cols attribute with the default width of 20 columns, even though you didn't explicitly set the `TextBox.Columns` property:

```
<textarea name="txt" rows="5" cols="20" readonly="readonly" id="txt"
```

```
style="background-color:Yellow;">Hello World</textarea>
```

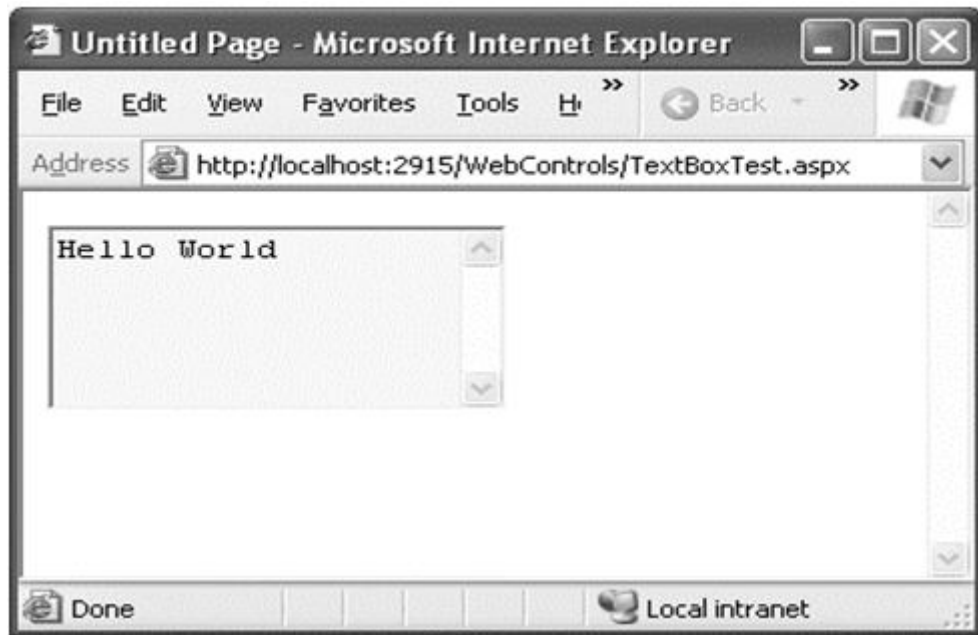


Figure 5. A customized text box

Web Control Classes

Web control classes are defined in the `System.Web.UI.WebControls` namespace. Figure shows most, but not all, of the webcontrols that ASP.NET provides.

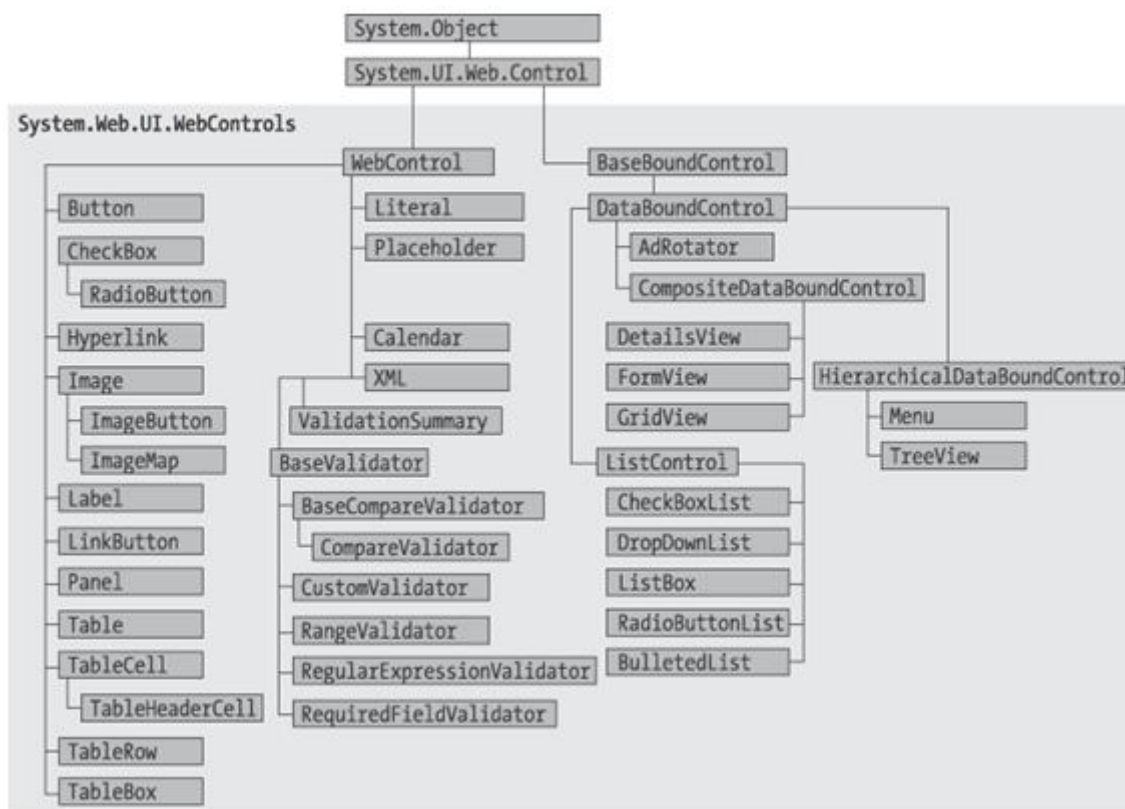


Figure 6. The web control hierarchy

The WebControl Base Class

Most web controls begin by inheriting from the WebControl base class. This class defines the essential functionality for tasks such as data binding and includes some basic properties that you can use with almost any web control, as described in Table.

Table. WebControl Properties

Property	Description
AccessKey	Specifies the keyboard shortcut as one letter. For example, if you set this to Y, the Alt+Y keyboard combination will automatically change focus to this web control.
BackColor, ForeColor, and BorderColor	Sets the colors used for the background, foreground, and border of the control

BorderWidth	Specifies the size of the control border.
BorderStyle	One of the values from the BorderStyle enumeration, including Dashed,Dotted, Double, Groove, Ridge, Inset, Outset, Solid, and None.
Controls	Provides a collection of all the controls contained inside the current control.Each object is provided as a generic System.Web.UI.Control object, so youwill need to cast the reference to access control-specific properties.
Enabled	When set to false, the control will be visible, but it will not be able to receiveuser input or focus.
EnableViewState	Set this to false to disable the automatic state management for this control. Inthis case, the control will be reset to the properties and formatting specifiedin the control tag (in the .aspx page) every time the page is posted back. If thisis set to true (the default), the control uses the hidden input field to storeinformation about its properties, ensuring that any changes you make in codeare remembered.
Font	Specifies the font used to render any text in the control as aSystem.Web.UI.WebControls.FontInfo object
Height and Width	Specifies the width and height of the control
ID	Specifies the name that you use to interact with the control in your code
Page	Provides a reference to the web page that contains this control as aSystem.Web.UI.Page object.
Parent	Provides a reference to the control that contains this control. If the control isplaced directly on the page (rather than inside another control), it will returna reference to the page object.
TabIndex	A number that allows you to control the tab order. The control with aTabIndex of 0 has the focus when the page first loads

ToolTip	Displays a text message when the user hovers the mouse above the control. Many older browsers don't support this property.
Visible	When set to false, the control will be hidden and will not be rendered to the final HTML page that is sent to the client.

Units

All the properties that use measurements, including `BorderWidth`, `Height`, and `Width`, require the `Unit` structure, which combines a numeric value with a type of measurement (pixels, percentage, and so on). This means when you set these properties in a control tag, you must make sure to append `px` (pixel) or `%` (for percentage) to the number to indicate the type of unit.

Here's an example with a `Panel` control that is 300 pixels high and has a width equal to 50 percent of the current browser window:

```
<asp:Panel Height="300px" Width="50%" ID="pnl" runat="server" />
```

If you're assigning a unit-based property through code, you need to use one of the static methods of the `Unit` type. Use `Pixel()` to supply a value in pixels, and use `Percentage()` to supply a percentage value:

```
// Convert the number 300 to a Unit object
```

```
// representing pixels, and assign it.
```

```
pnl.Height = Unit.Pixel(300);
```

```
// Convert the number 50 to a Unit object
```

```
// representing percent, and assign it.
```

```
pnl.Width = Unit.Percentage(50);
```

You could also manually create a `Unit` object and initialize it using one of the supplied constructors and the `UnitType` enumeration. This requires a few more steps but allows you to easily assign the same unit to several controls:

```
// Create a Unit object.
```

```
Unit myUnit = new Unit(300, UnitType.Pixel);
```

```
// Assign the Unit object to several controls or properties.
```

```
pnl.Height = myUnit;
```

```
pnl.Width = myUnit;
```

Enumerations

Enumerations are used heavily in the .NET class library to group a set of related constants. Foreexample, when you set a control's `BorderStyle` property, you can choose one of several predefined values from the `BorderStyle` enumeration. In code, you set an enumeration using the dot syntax:

```
ctrl.BorderStyle = BorderStyle.Dashed;
```

In the .aspx file, you set an enumeration by specifying one of the allowed values as a string. You don't include the name of the enumeration type, which is assumed automatically.

```
<asp:Label BorderStyle="Dashed" Text="Border Test" ID="lbl" runat="server" />
```

Figure shows the label with the altered border.

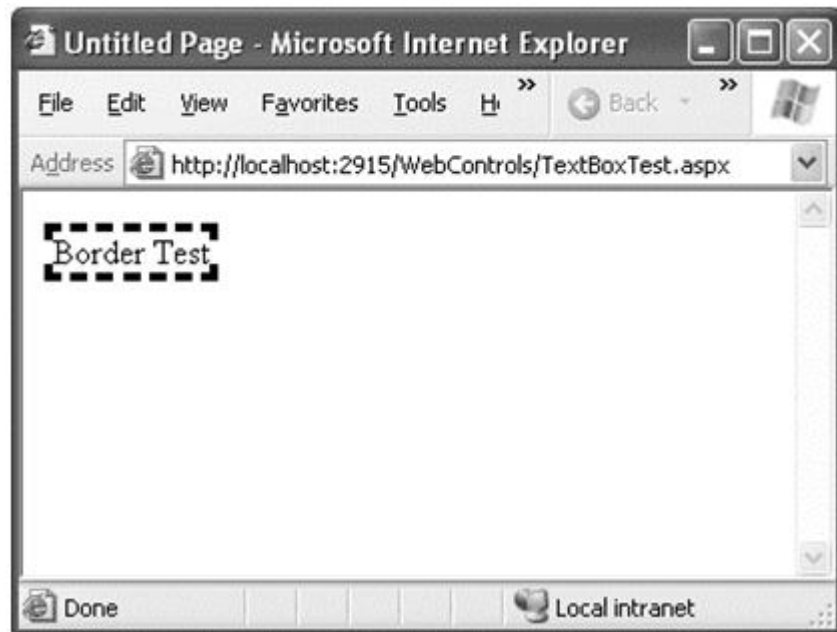


Figure 7. Modifying the border style

Colors

The Color property refers to a Color object from the System.Drawing namespace. You can create color objects in several ways:

Using an ARGB (alpha, red, green, blue) color value: You specify each value as an integer from 0 to 255. The alpha component represents the transparency of a color, and usually you'll use 255 to make the color completely opaque.

Using a predefined .NET color name: You choose the correspondingly named read-only property from the Color structure. These properties include the 140 HTML color names.

Using an HTML color name: You specify this value as a string using the ColorTranslator class. To use any of these techniques, you'll probably want to start by importing the System. Drawing namespace, as follows:

using System.Drawing;

The following code shows several ways to specify a color in code:

// Create a color from an ARGB value

```
int alpha = 255, red = 0, green = 255, blue = 0;
```

```
ctrl.ForeColor = Color.FromArgb(alpha, red, green, blue);
```

// Create a color using a .NET name

```
ctrl.ForeColor = Color.Crimson;
```

// Create a color from an HTML code

```
ctrl.ForeColor = ColorTranslator.FromHtml("Blue");
```

When defining a color in the .aspx file, you can use any one of the known color names:

```
<asp:TextBox ForeColor="Red" Text="Test" ID="txt" runat="server" />
```

The HTML color names that you can use are listed in the Visual Studio Help. Alternatively, you can use a hexadecimal color number (in the format #<red><green><blue>) as shown here:

```
<asp:TextBoxForeColor="#ff50ff" Text="Test"ID="txt" runat="server" />
```

Fonts

The Font property actually references a full FontInfo object, which is defined in the System.Web.UI.WebControls namespace. Every FontInfo object has several properties that define its name, size, and style

Table. FontInfo Properties

Property	Description
Name	A string indicating the font name (such as Verdana).
Names	An array of strings with font names, in the order of preference. The browser will use the first matching font that's installed on the user's computer.
Size	The size of the font as a FontUnit object. This can represent an absolute or relative size.
Bold, Italic, Strikeout, Underline, and Overline	Boolean properties that apply the given style attribute.

In code, you can assign a font by setting the various font properties using the familiar dot syntax:

```
ctrl.Font.Name = "Verdana";
```

```
ctrl.Font.Bold = true;
```

You can also set the size using the FontUnit type:

```
// Specifies a relative size.
```

```
ctrl.Font.Size = FontUnit.Small;
```

```
// Specifies an absolute size of 14 pixels.
```

```
ctrl.Font.Size = FontUnit.Point(14);
```

In the .aspx file, you need to use a special “object walker” syntax to specify object properties such as Font. The object walker syntax uses a hyphen (-) to separate properties. For example, you could set a control with a specific font (Tahoma) and font size (40 point) like this:

```
<asp:TextBoxFont-Name="Tahoma" Font-Size="40" Text="Size Test"
ID="txt"runat="server" />
```

Or you could set a relative size like this:

```
<asp:TextBox Font-Name="Tahoma" Font-Size="Large" Text="Size Test"ID="txt"
runat="server" />
```

Figure shows the altered TextBox in this example.



Figure 8. *Modifying a control's font*

A font setting is really just a recommendation. If the client computer doesn't have the font you request, it reverts to a standard font. To deal with this problem, it's common to specify a list of fonts, in order of preference. To do so, you use the Font.Names property instead of Font.Name, as shown here:

```
<asp:TextBoxFont-Names="Verdana,Tahoma,Arial"
```

```
Text="Size Test" ID="txt" runat="server" />
```

Here, the browser will use the Verdana font (if it has it). If not, it will fall back on Tahoma or, if that isn't present, Arial. When specifying fonts, it's a good idea to end with one of the following fonts, which are supported on all browsers:

- Times
- Arial and Helvetica
- Courier

The following fonts are found on almost all Windows and Mac computers, but not necessarily on other operating systems like Unix:

- Verdana
- Georgia
- Tahoma
- Comic Sans
- Arial Black
- Impact

Focus

Every web control provides a `Focus()` method. The `Focus()` method affects only input controls (controls that can accept keystrokes from the user). When the page is rendered in the client browser, the user starts in the focused control.

For example, if you have a form that allows the user to edit customer information, you might call the `Focus()` method on the first text box in that form. That way, the cursor appears in this text box immediately when the page first loads in the browser.

Rather than call the `Focus()` method programmatically, you can set a control that should always be focused by setting the `DefaultFocus` property of the `<form>` tag:

```
<form DefaultFocus="TextBox2" runat="server">
```

Another way to manage focus is using access keys. For example, if you set the `AccessKey` property of a `TextBox` to `A`, pressing `Alt+A` focus will switch to the `TextBox`.

For example, the following label gives focus to `TextBox2` when the keyboard combination `Alt+2` is pressed:

```

<asp:LabelAccessKey="2" AssociatedControlID="TextBox2"
    runat="server"Text="TextBox2:" />
<asp:TextBoxrunat="server" ID="TextBox2" />

```

The Default Button

Along with control focusing, ASP.NET also allows you to designate a default button on a web page. The default button is the button that is “clicked” when the user presses the Enter key. For example, if your web page includes a form, you might want to make the submit button into a default button. That way, if the user hits Enter at any time, the page is posted back and the Button.Click event is fired for that button.

To designate a default button, you must set the `HtmlForm.DefaultButton` property with the ID of the respective control, as shown here:

```

<form DefaultButton="cmdSubmit" runat="server">

```

The default button must be a control that implements the `IButtonControl` interface. The interface is implemented by the `Button`, `LinkButton`, and `ImageButton` web controls but not by any of the `HTMLServer` controls.

4.9 Web Control Events and AutoPostBack

The previous paragraphs explained that one of the main limitations of HTML server controls is their limited set of useful events—they have exactly two. HTML controls that trigger a postback, such as buttons, raise a `ServerClick` event. Input controls provide a `ServerChange` event that doesn’t actually fire until the page is posted back.

ASP.NET server controls are really an ingenious illusion. You’ll recall that the code in an ASP.NET page is processed on the server. It’s then sent to the user as ordinary HTML. Figure 6-11 illustrates the order of events in page processing.

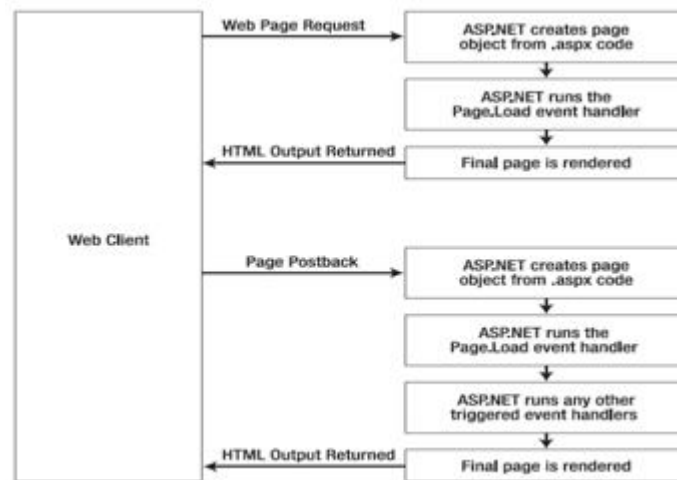


Figure 9. The page processing sequence

This is the same in ASP.NET as it was in traditional ASP programming. The question is, how can you write server code that will react *immediately* to an event that occurs on the client? Some events, such as the Click event of a button, do occur immediately. That's because when clicked, the button posts back the page. This is a basic convention of HTML forms. However, other actions *do* cause events but *don't* trigger a postback.

An example is when the user changes the text in a text box (which triggers the TextChanged event) or chooses a new item in a list (the SelectedIndexChanged event). You might want to respond to these events, but without a postback your code has no way to run. ASP.NET handles this by giving you two options:

- You can wait until the next postback to react to the event. For example, imagine you want to react to the SelectedIndexChanged event in a list. If the user selects an item in a list, nothing happens immediately. However, if the user then clicks a button to post back the page, *two* events fire: Button.Click followed by ListBox.SelectedIndexChanged
- You can use the *automatic postback* feature to force a control to post back the page immediately when it detects a specific user action. In this scenario, when the user clicks a new item in the list, the page is posted back, your code executes, and a new version of the page is returned.

All input web controls support automatic postbacks. Table 6-5 provides a basic list of web controls and their events.

Table. *Web Control Events*

Event	Web Controls That Provide It	Always Posts Back
Click	Button, ImageButton	True
TextChanged	TextBox (fires only after the user changes the focus to another control)	False
CheckedChanged	CheckBox, RadioButton	False
SelectedIndexChanged	DropDownList, ListBox, CheckBoxList, RadioButtonList	False

If you want to capture a change event (such as TextChanged, CheckedChanged, or Selected Index Changed) immediately, you need to set the control's AutoPostBack property to true.

When the page is posted back, ASP.NET will examine the page, load all the current information, and then allow your code to perform some extra processing before returning the page back to the user. Depending on the result you want, you could have a page that has some controls that postback automatically and others that don't.

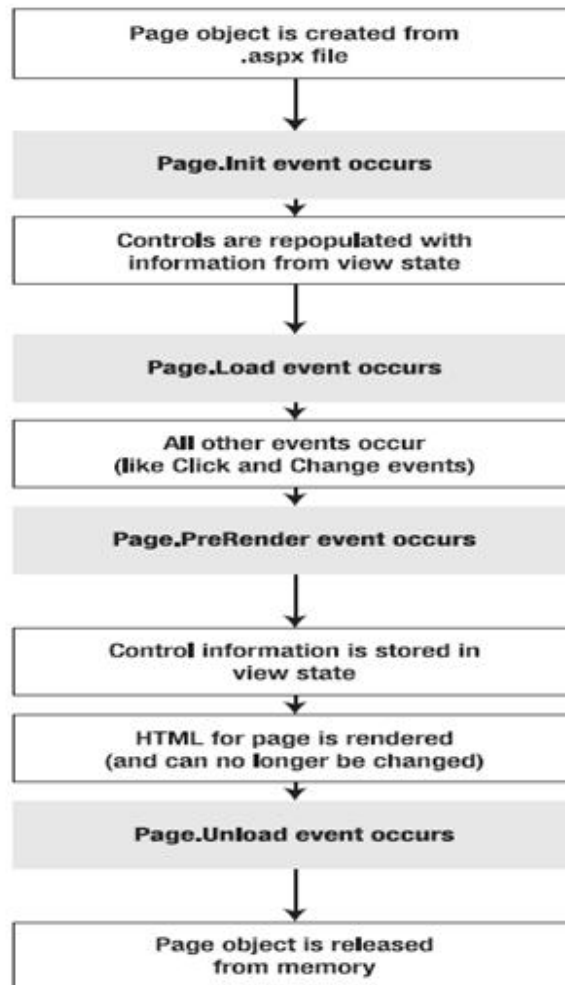


Figure 10. *The postback processing sequence*

This postback system isn't ideal for all events. For example, some events that you may be familiar with from Windows programs, such as mouse movement events or key press events, aren't practical in an ASP.NET application.

How Postback Events Work

If you create a web page that includes one or more web controls that are configured to use `AutoPostBack`, ASP.NET adds a special JavaScript function to the rendered HTML page. This function is named `__doPostBack()`. When called, it triggers a postback, sending data back to the web server.

ASP.NET also adds two additional hidden input fields that are used to pass information back to the server. This information consists of the ID of the control that raised the event and any additional information that might be relevant. These fields are initially empty, as shown here:

```
<input type="hidden" name="__EVENTTARGET" id="__EVENTTARGET" value="" />

<input type="hidden" name="__EVENTARGUMENT" id="__EVENTARGUMENT" value="" />
```

The `__doPostBack()` function has the responsibility for setting these values with the appropriate information about the event and then submitting the form. The `__doPostBack()` function is shown here:

```
<script language="text/javascript">

function __doPostBack(eventTarget, eventArgument) {
    if (!theForm.onsubmit || (theForm.onsubmit() != false)) {
        theForm.__EVENTTARGET.value = eventTarget;
        theForm.__EVENTARGUMENT.value = eventArgument;

        theForm.submit();
    }

    ...
}

</script>
```

ASP.NET generates the `__doPostBack()` function automatically, provided at least one control on the page uses automatic postbacks. Finally, any control that has its `AutoPostBack` property set to true is connected to the `__doPostBack()` function using the `onclick` or `onchange` attributes.

ASP.NET automatically changes a client-side JavaScript event into a server-side ASP.NET event, using the `__doPostBack()` function as an intermediary. Figure shows this process.

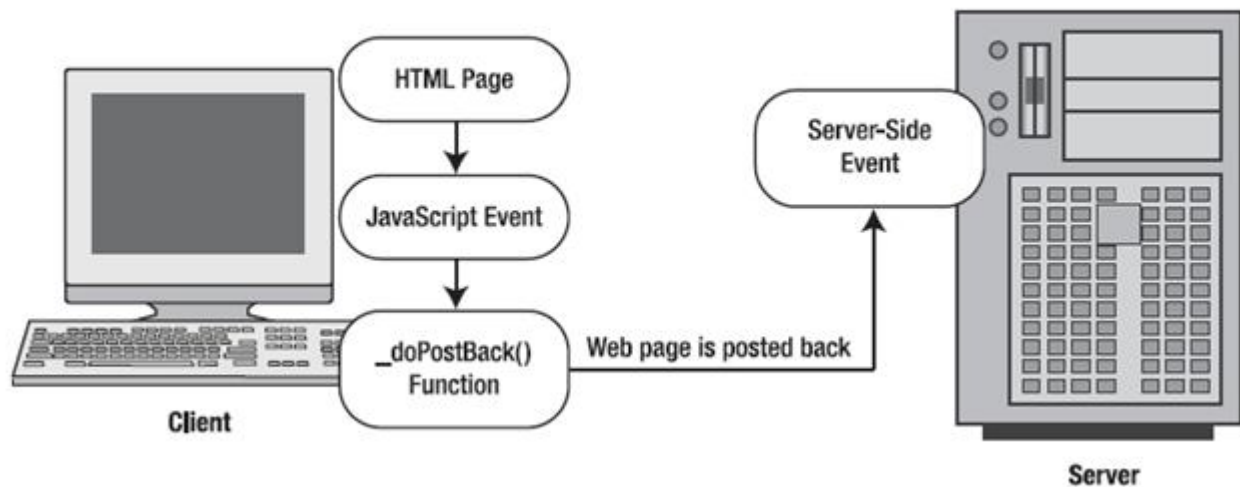


Figure 11. *An automatic postback*

The Page Life Cycle

To understand how web control events work, you need to have a solid understanding of the page life cycle. Consider what happens when a user changes a control that has the `AutoPostBack` property set to true:

1. On the client side, the JavaScript `__doPostBack` function is invoked, and the page is resubmitted to the server.
2. ASP.NET re-creates the Page object using the .aspx file.
3. ASP.NET retrieves state information from the hidden view state field and updates the controls accordingly.
4. The `Page.Load` event is fired.
5. The appropriate change event is fired for the control. (If more than one control has been changed, the order of change events is undetermined.)
6. The `Page.PreRender` event fires, and the page is rendered (transformed from a set of objects to an HTML page).
7. Finally, the `Page.Unload` event is fired.
8. The new page is sent to the client.

4.10 A Simple Web Page

This example demonstrates basic control manipulation with ASP.NET.

The web page is divided into two regions. On the left is an ordinary <div> tag containing a set of web controls for specifying card options. On the right is a Panel control (named pnlCard), which contains two other controls (lblGreeting and imgDefault) that are used to display user-configurable text and a picture. This text and picture represents the greeting card. When the page first loads, the card hasn't yet been generated, and the right portion is blank

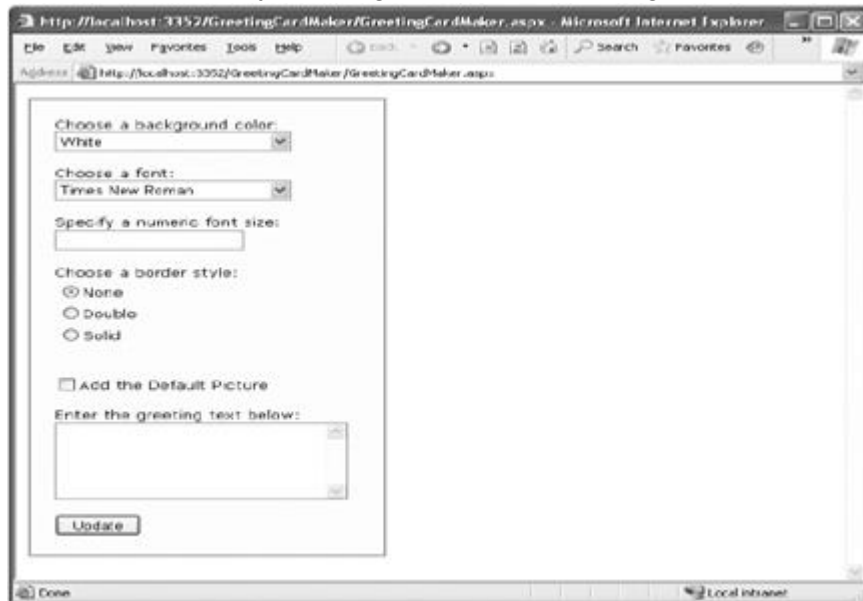


Figure 12. The e-card generator

Whenever the user clicks the Update button, the page is posted back and the "card" is updated (see Figure 6-16).



Figure 13. A user-configured greeting card

The .aspx layout code is straightforward. Of course, the sheer length of it makes it difficult to work with efficiently. Here's the markup, with the formatting details trimmed down to the bare essentials:

```
<%@ Page Language="C#" AutoEventWireup="true"
CodeFile="GreetingCardMaker.aspx.cs" Inherits="GreetingCardMaker" %>
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
<title>Greeting Card Maker</title>
</head>
<body>
<form runat="server">
<div>
<!-- Here are the controls: -->
```

Choose a background color:


```
<asp:DropDownList ID="lstBackColor" runat="server" Width="194px"
Height="22px"/><br /><br />
```

Choose a font:


```
<asp:DropDownList ID="lstFontName" runat="server" Width="194px"
Height="22px" /><br /><br />
```

Specify a numeric font size:


```
<asp:TextBox ID="txtFontSize" runat="server" /><br /><br />
```

Choose a border style:


```
<asp:RadioButtonList ID="lstBorder" runat="server" Width="177px"
Height="59px" /><br /><br />
```

```
<asp:CheckBox ID="chkPicture" runat="server"
```

```
Text="Add the Default Picture"></asp:CheckBox><br /><br />
```

Enter the greeting text below:


```
<asp:TextBox ID="txtGreeting" runat="server" Width="240px" Height="85px"
TextMode="MultiLine" /><br /><br />
```

```
<asp:Button ID="cmdUpdate" OnClick="cmdUpdate_Click"
runat="server" Width="71px" Height="24px" Text="Update" />
```

```
</div>
```

<!-- Here is the card: -->

```
<asp:Panel ID="pnlCard" runat="server"
```

```

Width="339px" Height="481px" HorizontalAlign="Center"
style="POSITION: absolute; TOP: 16px; LEFT: 313px;">
<br />&nbsp;
<asp:Label ID="lblGreeting" runat="server" Width="256px"
Height="150px" /><br /><br /><br />
<asp:Image ID="imgDefault" runat="server" Width="212px"
Height="160px" />
</asp:Panel>
</form>
</body>
</html>

```

The code follows the familiar pattern with an emphasis on two events: thePage.Load event, where initial values are set, and the Button.Click event, where the card is generated.

```

public partial class GreetingCardMaker : System.Web.UI.Page
{
    protected void Page_Load(object sender, EventArgs e)
    {
        if (!this.IsPostBack)
        {
            // Set color options.

            lstBackColor.Items.Add("White");

            lstBackColor.Items.Add("Red");

```



```
IstBackColor.Items.Add("Green");  
  
IstBackColor.Items.Add("Blue");  
  
IstBackColor.Items.Add("Yellow");  
  
// Set font options.  
  
IstFontName.Items.Add("Times New Roman");  
  
IstFontName.Items.Add("Arial");  
  
IstFontName.Items.Add("Verdana");  
  
IstFontName.Items.Add("Tahoma");  
  
// Set border style options by adding a series of  
  
// ListItem objects.  
  
ListItem item = new ListItem();  
  
// The item text indicates the name of the option.  
  
item.Text = BorderStyle.None.ToString();  
  
// The item value records the corresponding integer  
  
// from the enumeration. To obtain this value, you  
  
// must cast the enumeration value to an integer,  
  
// and then convert the number to a string so it  
  
// can be placed in the HTML page.  
  
item.Value = ((int)BorderStyle.None).ToString();  
  
// Add the item.  
  
IstBorder.Items.Add(item);  
  
// Now repeat the process for two other border styles.
```

```

item = new ListItem();

item.Text = BorderStyle.Double.ToString();

item.Value = ((int)BorderStyle.Double).ToString();

lstBorder.Items.Add(item);

item = new ListItem();

item.Text = BorderStyle.Solid.ToString();

item.Value = ((int)BorderStyle.Solid).ToString();

lstBorder.Items.Add(item);

lblGreeting.Font.Name = lstFontName.SelectedItem.Text;

if (Int32.Parse(txtFontSize.Text) > 0)
{
    lblGreeting.Font.Size =
        FontUnit.Point(Int32.Parse(txtFontSize.Text));
}

// Update the border style. This requires two conversion steps.

// First, the value of the list item is converted from a string

// into an integer. Next, the integer is converted to a value in

// the BorderStyle enumeration.

int borderValue = Int32.Parse(lstBorder.SelectedItem.Value);

pnlCard.BorderStyle = (BorderStyle)borderValue;

// Update the picture.

```

```
if (chkPicture.Checked)

{

imgDefault.Visible = true;

}

else

{

imgDefault.Visible = false;

}

// Set the text.

lblGreeting.Text = txtGreeting.Text;

}

lstBorder.SelectedIndex = 0;

// Set the picture.

imgDefault.ImageUrl = "defaultpic.png";

}

}

protected void cmdUpdate_Click(object sender, EventArgs e)

{

// Update the color.

pnlCard.BackColor = Color.FromName(lstBackColor.SelectedItem.Text);

// Update the font.
```

4.11 Summary

HTML controls are a compromise between ASP.NET web controls and traditional HTML. They use the familiar HTML elements but provide a limited object-oriented interface. Essentially, HTML controls are designed to be straightforward, predictable, and automatically compatible with existing programs. With HTML controls, the final HTML page that is sent to the client closely resembles the original .aspx page.

Keywords

The global.asax File, The web.config File, AutoPostBack, The Website Administration Tool (WAT), Page Life Cycle

4.12 Check your understanding here

1. Which Control class will create <select> html element?
 - A. **DropDownList**
 - B. Hyperlink
 - C. LinkButton
 - D. RadioButtonList
2. Which Control class will create <div> html element?
 - A. BulletedList
 - B. **Panel**
 - C. Hyperlink
 - D. DropDownList

3. Which property is used to Displays a text message when the user hovers the mouse above the control?
- A. TabIndex
 - B. Visible
 - C. **ToolTip**
 - D. Enabled
4. The default button for a page is set in which HTML tag?
- A. Page
 - B. Body
 - C. Head
 - D. **Form**

4.13 Model Questions

1. What is default button? How to define it?
2. What is WAT?
3. Explain about global.asax File.
4. Explain the page properties.
5. How post back event works?

LESSON - 5

ERROR HANDLING

Structure

- 5.1 Introduction
- 5.2 Objectives
- 5.3 Common errors
- 5.4 Exception handling
- 5.5 Mastering Exceptions
- 5.6 Logging Exceptions
- 5.7 A custom logging class
- 5.8 Page tracing
- 5.9 Control Tree
- 5.10 Session state and Application state
- 5.11 Query String collections
- 5.12 Summary
- 5.13 Check your progress
- 5.14 Model Questions

5.1 Introduction

In this chapter, you'll learn the error handling and debugging practices that you can use to defend your ASP.NET applications against common errors, track user problems, and solve mysterious issues. You'll learn how to use structured exception handling, how to keep a record of unrecoverable errors with logs, and how to set up web pages with custom error messages for common HTTP errors. You'll also learn how to use page tracing to see diagnostic information about ASP.NET pages.

5.2 Learning objectives

- ✓ Understand Log file for error handling
- ✓ Understand common errors
- ✓ Handling Exception
- ✓ Throw your own exception
- ✓ Trace page

5.3 Common Errors

Errors can occur in a variety of situations. Some of the most common causes of errors include attempts to divide by zero (usually caused by invalid input or missing information) and attempts to connect to a limited resource such as a file or a database (which can fail if the file doesn't exist, the database connection times out, or the code has insufficient security credentials).

One infamous type of error is the *null reference exception*, which usually occurs when a program attempts to use an uninitialized object. As a .NET programmer, you'll quickly learn to recognize and resolve this common but annoying mistake.

The following code example shows the problem in action, with two `SqlConnection` objects that represent database connections:

```
// Define a variable named conOne and create the object.
private SqlConnection conOne = new SqlConnection();

// Define a variable named conTwo, but don't create the object.
private SqlConnection conTwo;

public void cmdDoSomething_Click(object sender, EventArgs e)
{
    // This works, because the object has been created
    // with the new keyword.
    conOne.ConnectionString = "...";
    ..
}
```

```
// The following statement will fail and generate a
// null reference exception.
// You cannot modify a property (or use a method) of an
// object that doesn't exist!
conTwo.ConnectionString = "...";
...
}
```

When an error occurs in your code, .NET checks to see whether any *error handlers* appear in the current scope. If the error occurs inside a method, .NET searches for local error handlers and then checks for any active error handlers in the calling code. If no error handlers are found, the page processing is aborted, and Visual Studio enters debug mode. If you press the Play button to keep going, you'll see a detailed error page that explains the problem. These error pages are a development convenience—once you deploy your application, they are replaced by more general error pages, which you can configure in the IIS web server software.

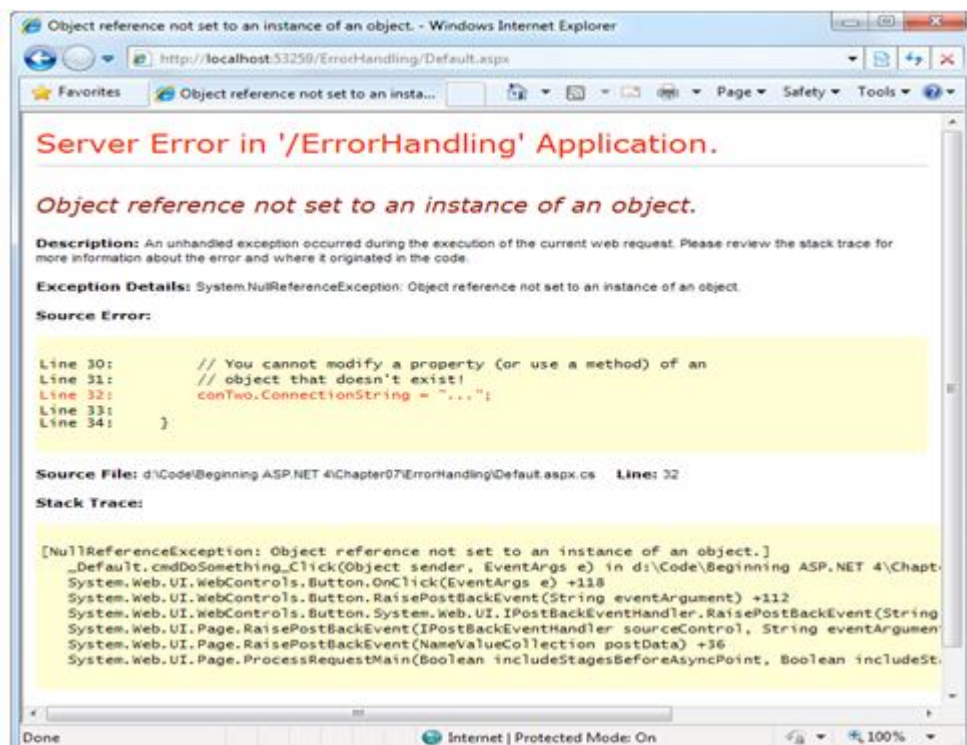


Figure 1.A sample error page

Even if an error is the result of invalid input or the failure of a third-party component, an error page can shatter the professional appearance of any application. The application users end up with a feeling that the application is unstable, insecure, or of poor quality—and they're at least partially correct.

If an ASP.NET application is carefully designed and constructed, an error page will almost never appear. Errors may still occur because of unforeseen circumstances, but they will be caught in the code and identified. If the error is a critical one that the application cannot solve on its own, it will report a more useful (and user-friendly) page of information that might include a link to a support e-mail or a phone number where the customer can receive additional assistance.

5.4 Exception Handling

Most .NET languages support *structured exception handling*. Essentially, when an error occurs in your application, the .NET Framework creates an exception object that represents the problem. You can catch this object using an exception handler. If you fail to use an exception handler, your code will be aborted, and the user will see an error page.

Structured exception handling provides several key features:

Exceptions are object-based: Each exception provides a significant amount of diagnostic information wrapped into a neat object, instead of a simple message and error code. These exception objects also support an `InnerException` property that allows you to wrap a generic error over the more specific error that caused it. You can even create and throw your own exception objects.

Exceptions are caught based on their type: This allows you to streamline error handling code without needing to sift through obscure error codes.

Exception handlers use a modern block structure: This makes it easy to activate and deactivate different error handlers for different sections of code and handle their errors individually.

Exception handlers are multilayered: You can easily layer exception handlers on top of other exception handlers, some of which may check only for a specialized set of errors.

Exceptions are a generic part of the .NET Framework: This means they're completely cross-language compatible. Thus, a .NET component written in C# can throw an exception that you can catch in a web page written in VB.

The Exception Class

Every exception class derives from the base class `System.Exception`. The .NET Framework is full of predefined exception classes, such as `NullReferenceException`, `IOException`, `SqlException`, and so on. The `Exception` class includes the essential functionality for identifying any type of error. Table lists its most important members.

Table. *Exception Properties*

Member	Description
<code>HelpLink</code>	A link to a help document, which can be a relative or fully qualified uniform resource locator (URL) or uniform resource name (URN), such as <code>file:///C:/ACME/MyApp/help.html#Err42</code> . The .NET Framework doesn't use this property, but you can set it in your custom exceptions if you want to use it in your web page code.
<code>InnerException</code>	A nested exception. For example, a method might catch a simple file input/output (IO) error and create a higher-level "operation failed" error. The details about the original error could be retained in the <code>InnerException</code> property of the higher-level error.
<code>Message</code>	A text description with a significant amount of information describing the problem.
<code>Source</code>	The name of the application or object where the exception was raised.

StackTrace	A string that contains a list of all the current method calls on the stack, in order of most to least recent. This is useful for determining where the problem occurred.
TargetSite	A reflection object (an instance of the <code>System.Reflection.MethodBase</code> class) that provides some information about the method where the error occurred. This information includes generic method details such as the method name and the data types for its parameter and return values. It doesn't contain any information about the actual parameter values that were used when the problem occurred.
GetBaseException()	A method useful for nested exceptions that may have more than one layer. It retrieves the original (deepest nested) exception by moving to the base of the <code>InnerException</code> chain.

When you catch an exception in an ASP.NET page, it won't be an instance of the generic `System.Exception` class. Instead, it will be an object that represents a specific type of error. This object will be based on one of the many classes that inherit from `System.Exception`. These include diverse classes such as `DivideByZeroException`, `ArithmeticException`, `IOException`, `SecurityException`, and many more. Some of these classes provide additional details about the error in additional properties.

Visual Studio provides a useful tool to browse through the exceptions in the .NET class library. Simply select `Debug > Exceptions` from the menu (you'll need to have a project open in order for this to work). The Exceptions dialog box will appear. Expand the `Common Language Runtime Exceptions` group, which shows a hierarchical tree of .NET exceptions arranged by namespace.

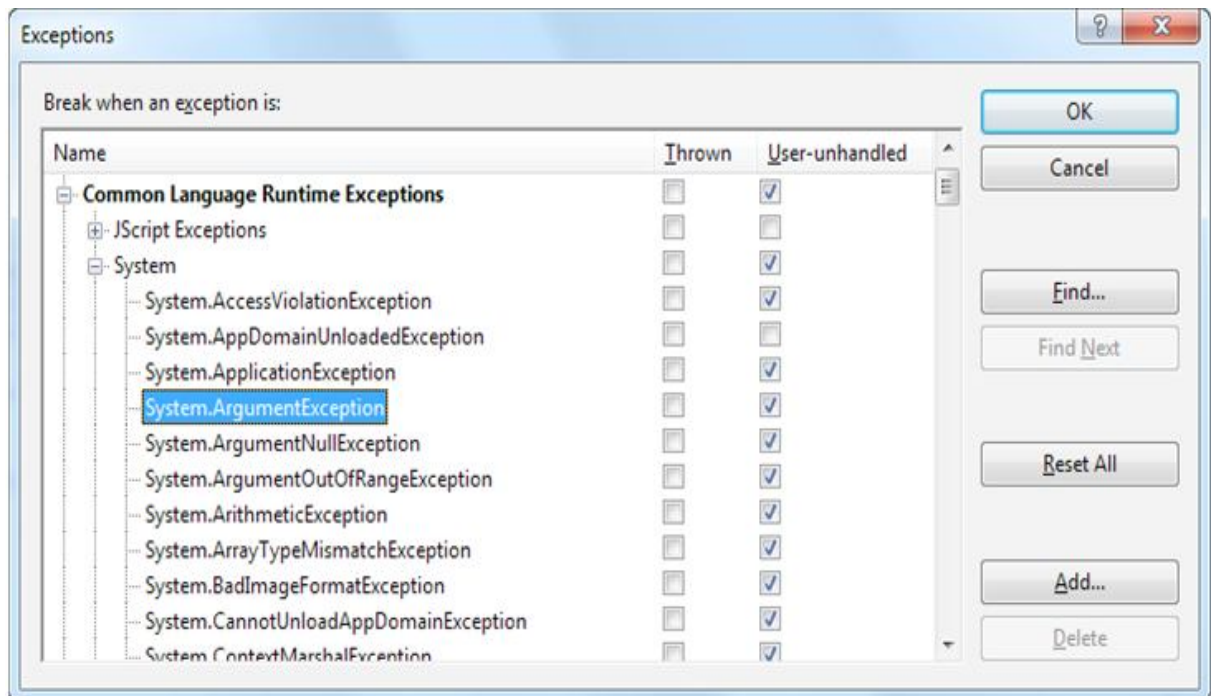


Figure 2. Visual Studio's exception viewer

The Exceptions dialog box allows you to specify what exceptions should be handled by your code when debugging and what exceptions will cause Visual Studio to enter break mode immediately. That means you don't need to disable your error handling code to troubleshoot a problem. For example, you could choose to allow your program to handle a common `FileNotFoundException` (which could be caused by an invalid user selection) but instruct Visual Studio to pause execution if an `unexpectedDivideByZero` exception occurs.

To set this up, add a check mark in the **Thrown** column next to the entry for the `System.DivideByZero` exception. This way, you'll be alerted as soon as the problem occurs. If you don't add a check mark to the **Thrown** column, your code will continue, run any exception handlers it has defined, and try to deal with the problem. You'll be notified only if an error occurs and no suitable exception handler is available.

The Exception Chain

Figure shows how the `InnerException` property works. In the specific scenario shown here, a `FileNotFoundException` led to a `NullReferenceException`, which led to a custom `UpdateFailed`

Exception. Using an exception handling block, the application can catch the `UpdateFailedException`. It can then get more information about the source of the problem by following the `InnerException` property to the `NullReferenceException`, which in turn references the original `FileNotFoundException`.

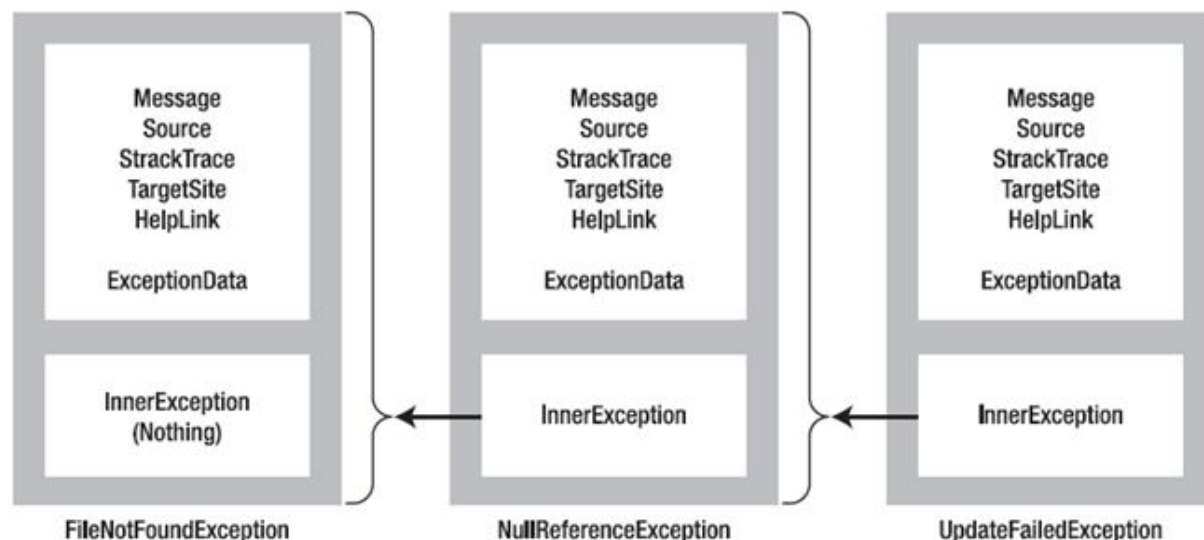


Figure 3 Exceptions can be chained together.

The `InnerException` property is an extremely useful tool for component-based programming.

Generally, it's not much help if a component reports a low-level problem such as a nullreference or adivide-by-zero error. Instead, it needs to communicate a more detailed message about which operationfailed and what input may have been invalid. The calling code can then often correct the problem andretry the operation.

Using an exception chain handles boththese scenarios: you receive as many linked exception objects as needed, which can specify informationfrom the least to the most specific error condition.

Handling Exceptions

The first line of defense in an application is to check for potential error conditions before performing anoperation. For example, a program can explicitly check whether the divisor is 0 before performing aacalculation or whether a file exists before attempting to open it:

```
if (divisor != 0)
```

```

{

// It's safe to divide some number by divisor.

}

if (System.IO.File.Exists("myfile.txt"))

{

// You can now open the myfile.txt file.

// However, you should still use exception handling because a variety of

// problems can intervene (insufficient rights, hardware failure, etc.).

}

```

Even if you perform this basic level of “quality assurance,” your application is still vulnerable. For example, you have no way to protect against all the possible file access problems that occur, including hardware failures or network problems that could arise spontaneously in the middle of an operation.

Similarly, you have no way to validate a user ID and password for a database before attempting to open a connection—and even if you did, that technique would be subject to its own set of potential errors. In some cases, it may not be practical to perform the full range of defensive checks, because they may impose a noticeable performance drag on your application. For all these reasons, you need a way to detect and deal with errors when they occur.

The solution is structured exception handling. To use structured exception handling, you wrap potentially problematic code in the special block structure shown here:

```

try

{

// Risky code goes here (opening a file, connecting to a database, and so on).

}

```

```

catch

{

// An error has been detected. You can deal with it here.

}

finally

{

// Time to clean up, regardless of whether or not there was an error.

}

```

The try statement enables error handling. Any exceptions that occur in the following lines can be “caught” automatically. The code in the catch block will be executed when an error is detected. And either way, whether a bug occurs or not, the finally block of the code will be executed last. This allows you to perform some basic cleanup, such as closing a database connection. The finally code is important because it will execute even if an error has occurred that will prevent the program from continuing. In other words, if an unrecoverable exception halts your application, you’ll still have the chance to release resources.

Catching Specific Exceptions

Structured exception handling is particularly flexible because it allows you to catch specific types of exceptions. To do so, you add multiple catch statements, each one identifying the type of exception (and providing a new variable to catch it in), as follows:

```

try

{

// Risky database code goes here.

}

catch (System.Data.SqlClient.SqlException err)

```

```

{

// Catches common problems like connection errors.

}

catch (System.NullReferenceException err)

{

// Catches problems resulting from an uninitialized object.

}

```

An exception will be caught as long as it's an instance of the indicated class or if it's derived from that class. In other words, if you use this statement:

```
catch (Exception err)
```

you will catch any exception, because every exception object is derived from the `System.Exception` base class.

Exception blocks work a little like conditional code. As soon as a matching exception handler is found, the appropriate catch code is invoked.

Therefore, you must organize your catch statements from most specific to least specific:

```

try

{

// Risky database code goes here.

}

catch (System.Data.SqlClient.SqlException err)

{

// Catches common problems like connection errors.

```



```

}

catch (System.NullReferenceException err)

{

    // Catches problems resulting from an uninitialized object.

}

catch (System.Exception err)

{

    // Catches any other errors.

}

```

Ending with a catch statement for the base Exception class is often a good idea to make sure no errors slip through. However, in component-based programming, you should make sure you intercept only those exceptions you can deal with or recover from.

Nested Exception Handlers

When an exception is thrown, .NET tries to find a matching catch statement in the current method. If the code isn't in a local structured exception block or if none of the catch statements matches the exception, .NET will move up the call stack one level at a time, searching for active exception handlers.

Consider the example shown here, where the Page.Load event handler calls a private

DivideNumbers() method:

```

protected void Page_Load(Object sender, EventArgs e)

{

    try

    {

```

```

DivideNumbers(5, 0);

}

catch (DivideByZeroException err)

{

// Report error here.

}

}

private decimal DivideNumbers(decimal number, decimal divisor)

{

return number/divisor;

}

```

In this example, the `DivideNumbers()` method lacks any sort of exception handler. However, the `DivideNumbers()` method call is made inside a try block, which means the problem will be caught further upstream in the calling code. This is a good approach because the `DivideNumbers()` routine could be used in a variety of circumstances

You can also overlap exception handlers in such a way that different exception handlers filter out different types of problems. Here's one such example:

```

protected void Page_Load(Object sender, EventArgs e)

{

try

{

decimal average = GetAverageCost(DateTime.Now);

}

catch (DivideByZeroException err)

{

```

```

// Report error here.

}

}

private decimal GetAverageCost(DateTime saleDate)
{
    try
    {
        // Use Database access code here to retrieve all the sale records
        // for this date, and calculate the average.
    }

    catch (System.Data.SqlClient.SqlException err)
    {
        // Handle a database related problem.
    }

    finally
    {
        // Close the database connection.
    }
}

```

5.4 Mastering Exceptions

Keep in mind these points when working with structured exception handling:

Break down your code into multiple try/catch blocks: If you put all your code into one exception handler, you'll have trouble determining where the problem occurred. The rule of thumb is to use one exception handler for one related task (such as opening a file and retrieving information).

Report all errors: During debugging, portions of your application's error handling code may mask easily correctable mistakes in your application. To prevent this from happening, make sure you report all errors, and consider leaving out some error handling logic in early builds.

Don't use exception handlers for every statement: Simple code statements (assigning a constant value to a variable, interacting with a control, and so on) may cause errors during development testing but will not cause any future problems once perfected. Error handling should be used when you're accessing an outside resource or dealing with supplied data that you have no control over (and thus may be invalid).

Throwing Your Own Exceptions

You can also define your own exception objects to represent custom error conditions. All you need to do is create an instance of the appropriate exception class and then use the throw statement. The next example introduces a modified `DivideNumbers()` method. It explicitly checks whether the specified divisor is 0 and then manually creates and throws an instance of the `DivideByZeroException` class to indicate the problem, rather than attempt the operation. Depending on the code, this pattern can save time by eliminating some unnecessary steps, or it can prevent a task from being initiated if it can't be completed successfully.

```
protected void Page_Load(Object sender, EventArgs e)
{
    try
    {
        DivideNumbers(5, 0);
    }
    catch (DivideByZeroException err)
    {
        // Report error here.
    }
}

private decimal DivideNumbers(decimal number, decimal divisor)
{
```

```

if (divisor == 0)
{
    DivideByZeroException err = new DivideByZeroException();
    throw err;
}
else
{
    return number/divisor;
}
}

```

In this case, any ordinary exception handler will still catch the `DivideByZeroException`. The only difference is that the error object has a modified `Message` property that contains the custom string. Figure shows the resulting exception.

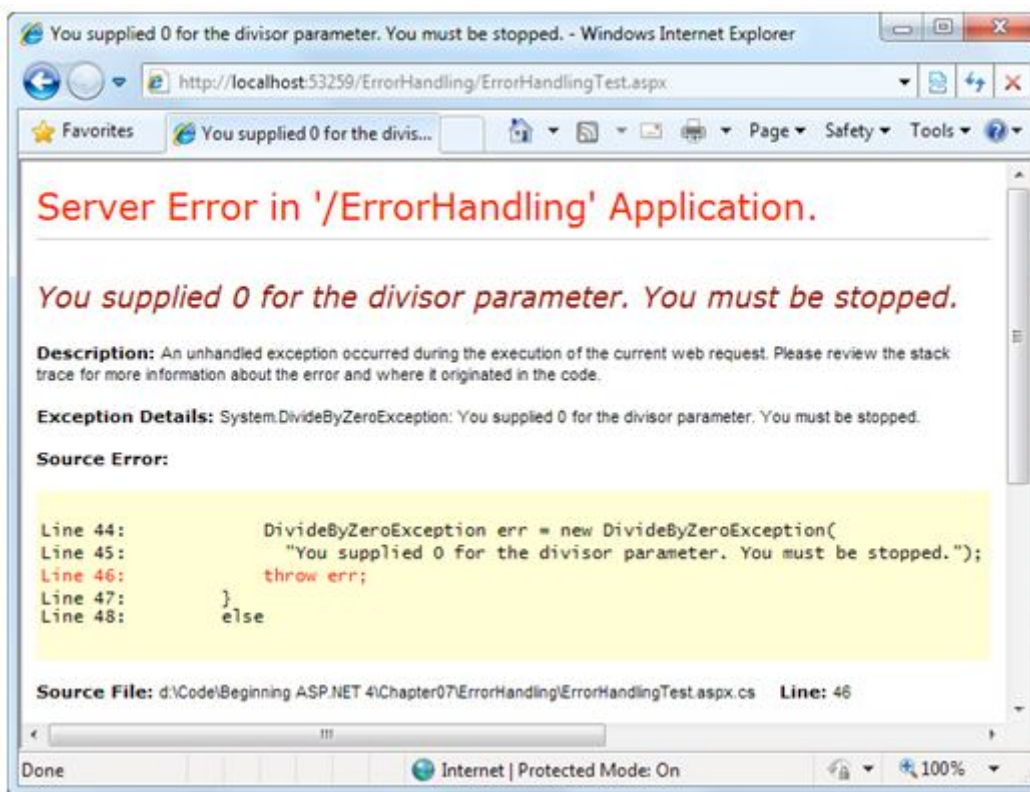


Figure 4. Standard exception, custom message

Throwing an exception is most useful in component-based programming. In component-based programming, your ASP.NET page is creating objects and calling methods from a class defined in a separately compiled assembly. In this case, the class in the component needs to be able to notify the calling code (the web application) of any errors. The component should handle recoverable errors quietly and not pass them up to the calling code. On the other hand, if an unrecoverable error occurs, it should always be indicated with an exception and never through another mechanism (such as a return code).

If you can find an exception in the class library that accurately reflects the problem that has occurred, you should throw it. If you need to return additional or specialized information, you can create your own custom exception class.

Custom exception classes should always inherit from `System.ApplicationException`, which itself derives from the base `Exception` class. This allows .NET to distinguish between two broad classes of exceptions—those you create and those that are native to the .NET Framework. When you create an exception class, you can add properties to record additional information. For example, here is a special class that records information about the failed attempt to divide by zero:

```
public class CustomDivideByZeroException : ApplicationException
{
    // Add a variable to specify the "other" number.

    // This might help diagnose the problem.

    public decimal DividingNumber;
}
```

You can throw this custom exception like this:

```
private decimal DivideNumbers(decimal number, decimal divisor)
{
    if (divisor == 0)
    {
```

```

CustomDivideByZeroException err = new CustomDivideByZeroException();

err.DividingNumber = number;

throw err;

}

else

{

return number/divisor;

}

}

```

To perfect the custom exception, you need to supply it with the three standard constructors. This allows your exception class to be created in the standard ways that every exception supports:

- On its own, with no arguments
- With a custom message
- With a custom message and an exception object to use as the inner exception

These constructors don't actually need to contain any code. All these constructors need to do is forward the parameters to the base class (the constructors in the inherited `ApplicationException` class)

using the `base` keyword, as shown here:

```

public class CustomDivideByZeroException : ApplicationException

{

// Add a variable to specify the "other" number.

private decimal dividingNumber;

public decimal DividingNumber

```

```

{
    get { return dividingNumber; }
    set { dividingNumber = value; }
}

public CustomDivideByZeroException() : base()
{
}

public CustomDivideByZeroException(string message) : base(message)
{
}

public CustomDivideByZeroException(string message, Exception inner) :
    base(message, inner)
{
}
}

```

The third constructor is particularly useful for component programming. It allows you to set the `InnerException` property with the exception object that caused the original problem. The next example shows how you could use this constructor with a component class called `ArithmeticUtility`:

```

public class ArithmeticUtilityException : ApplicationException
{
    public ArithmeticUtilityException() : base()
    {
    }

    public ArithmeticUtilityException(string message) : base(message)
    {
    }

    public ArithmeticUtilityException(string message, Exception inner) :
        base(message, inner)
    {
    }
}

```



```

}

public class ArithmeticUtility
{
    private decimal Divide(decimal number, decimal divisor)
    {
        try
        {
            return number/divisor;
        }
        catch (Exception err)
        {
            // Create an instance of the specialized exception class,
            // and place the original exception object (for example, a
            // DivideByZeroException) in the InnerException property.
            ArithmeticUtilityException errNew =
            new ArithmeticUtilityException("Calculation error", err);
            // Now throw the new exception.

            throw errNew;
        }
    }
}

```

Remember, custom exception classes are really just a standardized way for one class to communicate an error to a different portion of code. If you aren't using components or your own utility classes, you probably don't need to create custom exception classes.

5.6 Logging Exceptions

To diagnose errors and build a larger picture of site problems, you need to log exceptions so they can be reviewed later. The .NET Framework provides a wide range of logging tools. When certain errors occur, you can send an e-mail, add a database record, or write to a file.

One of the most fail-safe logging tools is the Windows *event logging* system, which is built into the Windows operating system and available to any application. Using the Windows event logs, your website can write text messages that record errors or unusual events. The Windows event logs store your messages as well as various other details, such as the message type (information, error, and so on) and the time the message was left.

Viewing the Windows Event Logs

To view the Windows event logs, you use the Event Viewer tool that's included with Windows. To launch it, begin by selecting Start > Control Panel. Open the Administrative Tools group, and then choose Event Viewer. Under the Windows Logs section, you'll see the four logs that are described in Table.

Table. *Windows Event Logs*

Log Name	Description
Application	Used to track errors or notifications from any application. Generally, you'll use this log when you're performing event logging, or you'll create your own custom log.
Security	Used to track security-related problems but generally used exclusively by the operating system.
System	Used to track operating system events.
Setup	Used to track issues that occur when installing Windows updates or other software.

Using the Event Viewer, you can perform a variety of management tasks with the logs. For example, if you right-click one of the logs in the Event Viewer list you'll see options that allow you to clear the events in the log, save the log entries to another file, and import an external log file.

Each event record in an event log identifies the source (generally, the application or service that created the record), the type of notification (error, information, warning), and the time the log entry was inserted. To see this information, you simply need to select a log entry, and the details will appear in a display area underneath the list of entries.

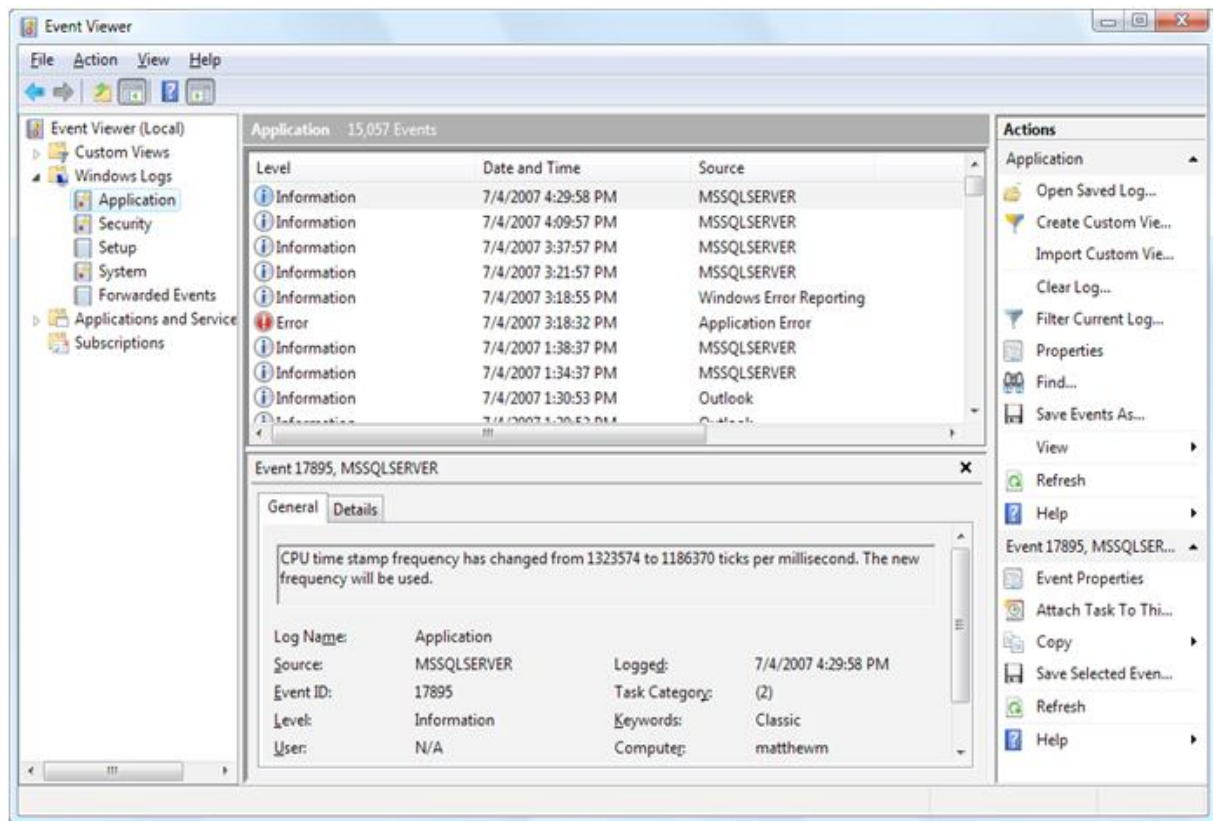


Figure 5. The Event Viewer

Writing to the Event Log

You can interact with event logs in an ASP.NET page by using the classes in the `System.Diagnostics` namespace. First, import the namespace at the beginning of your code-behind file:

using System.Diagnostics;

The following example rewrites the simple `ErrorTest` page to use event logging:

```
public partial class ErrorTestLog : Page
{
```

```

protected void cmdCompute_Click(Object sender, EventArgs e)

{

    try

    {

        decimal a, b, result;

        a = Decimal.Parse(txtA.Text);

        b = Decimal.Parse(txtB.Text);

        result = a / b;

        lblResult.Text = result.ToString();

        lblResult.ForeColor = System.Drawing.Color.Black;

    }

    catch (Exception err)

    {

        lblResult.Text = "<b>Message:</b> " + err.Message + "<br /><br />";

        lblResult.Text += "<b>Source:</b> " + err.Source + "<br /><br />";

        lblResult.Text += "<b>Stack Trace:</b> " + err.StackTrace;

        lblResult.ForeColor = System.Drawing.Color.Red;

        // Write the information to the event log.

        EventLog log = new EventLog();

        log.Source = "DivisionPage";

        log.WriteEntry(err.Message, EventLogEntryType.Error);

    }

```

}

}

The event log record will now appear in the Event Viewer utility, as shown in Figure 6. Note that logging is intended for the system administrator or developer. It doesn't replace the code you use to notify the user and explain that a problem has occurred.

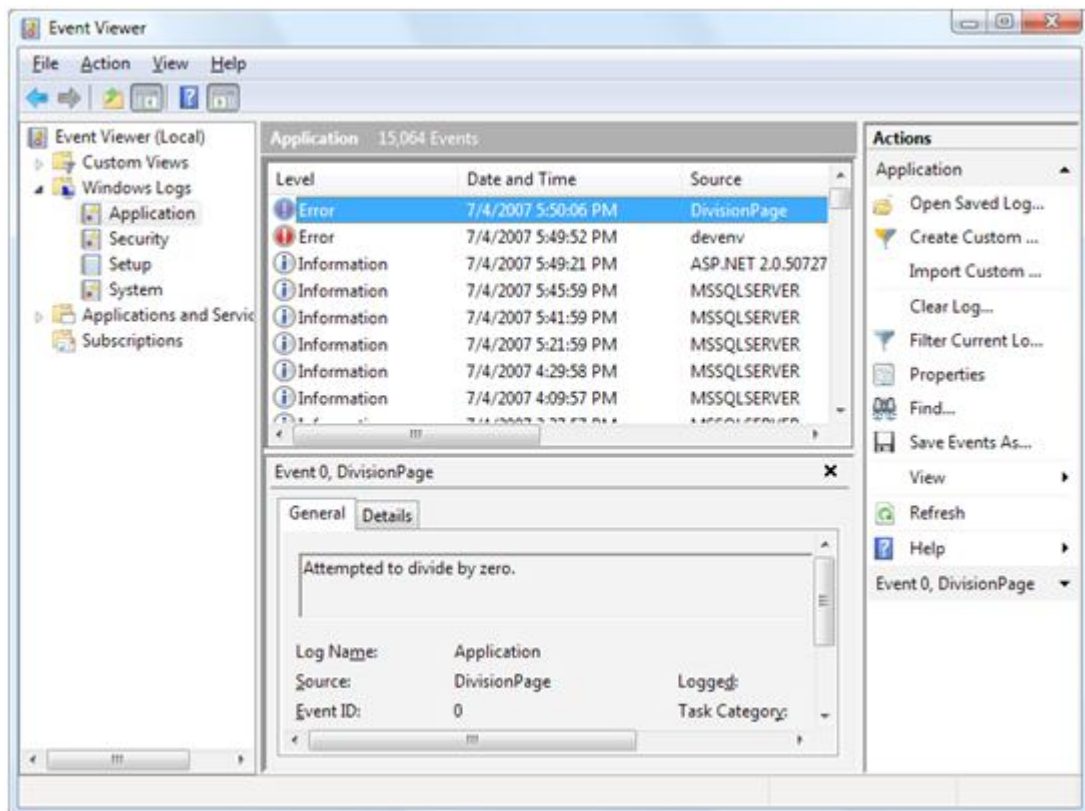


Figure 6. An event record

Custom Logs

You can also log errors to a custom log. For example, you could create a log with your company name and add records to it for all your ASP.NET applications. You might even want to create an individual log for a particularly large application and use the Source property of each entry to indicate the page (or webservice method) that caused the problem.

Accessing a custom log is easy—you just need to use a different constructor for the `EventLog` class to specify the custom log name. You also need to register an *event source* for the log.

This initial step needs to be performed only once—in fact, you’ll receive an error if you try to create the same event source. Typically, you’ll use the name of the application as the event source. Here’s an example that uses a custom log named ProseTech and registers the event source

DivideByZeroApp:

// Register the event source if needed.

if (!EventLog.SourceExists("ProseTech"))

{

// This registers the event source and creates the custom log,

// if needed.

EventLog.CreateEventSource("DivideByZeroApp", "ProseTech");

}

// Open the log. If the log doesn't exist,

// it will be created automatically.

EventLog log = new EventLog("ProseTech");

log.Source = "DivideByZeroApp";

log.WriteEntry(err.Message, EventLogEntryType.Error);

If you specify the name of a log that doesn’t exist when you use the `CreateEventSource()` method, the system will create a new, custom event log for you the first time you write an entry. To see a newly created event log in the Event Viewer tool, you’ll need to exit Event Viewer and restart it. Custom logs appear in a separate group named Applications and Services Logs, as shown in Figure 7.

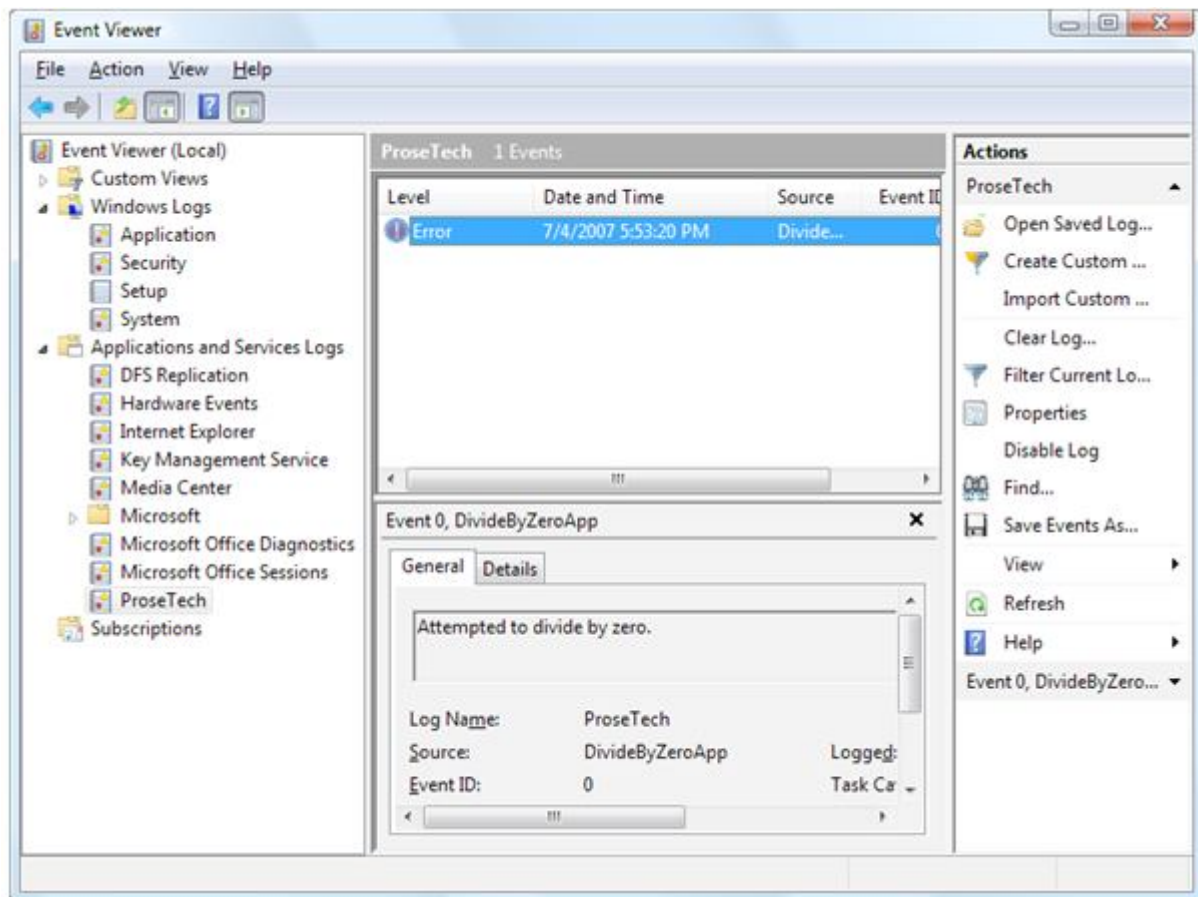


Figure 7. A custom log

You can use this sort of code anywhere in your application. Usually, you'll use logging code when responding to an exception that might be a symptom of a deeper problem.

5.7 A Custom Logging Class

Rather than adding too much logging code in the catch block, a better approach is to create a separate class that handles the event logging grunt work. You can then use that class from any web page, without duplicating any code. To use this approach, begin by creating a new code file in the App_Code subfolder of your website. You can do this in Visual Studio by choosing Website > Add New Item. In the Add New Item dialog box, choose Class, pick a suitable file name, and then click Add.

Here's an example of a class named MyLogger that handles the event logging details:

```

public class MyLogger

{

    public void LogError(string pageInError, Exception err)

    {

        RegisterLog();

        EventLog log = new EventLog("ProseTech");

        log.Source = pageInError;

        log.WriteEntry(err.Message, EventLogEntryType.Error);

    }

    private void RegisterLog()

    {

        // Register the event source if needed.

        if (!EventLog.SourceExists("ProseTech"))

        {

            EventLog.CreateEventSource("DivideByZeroApp", "ProseTech");

        }

    }

}

```

Once you have a class in the App_Code folder, it's easy to use it anywhere in your website. Here's

how you might use the MyLogger class in a web page to log an exception:

```

try

{

```



```

// Risky code goes here.

}

catch (Exception err)

{

// Log the error using the logging class.

MyLogger logger = new MyLogger();

logger.LogError(Request.Path, err);

// Now handle the error as appropriate for this page.

lblResult.Text = "Sorry. An error occurred.";

}

```

If you write log entries frequently, you may not want to check whether the log exists every time you want to write an entry. Instead, you could create the event source once—when the application first startups—using an application event handler in the `global.asax` file.

Retrieving Log Information

One of the disadvantages of the event logs is that they're tied to the web server. This can make it difficult to review log entries if you don't have a way to access the server. This problem has several possible solutions. One interesting technique involves using a special administration page. This ASP.NET page can use the `EventLog` class to retrieve and display all the information from the event log. Figure 8 shows in a simple web page all the entries that were left by the `ErrorTestCustomLog` page. The results are shown using a label in a scrollable panel (a `Panel` control with the `Scrollbars` property set to `Vertical`)

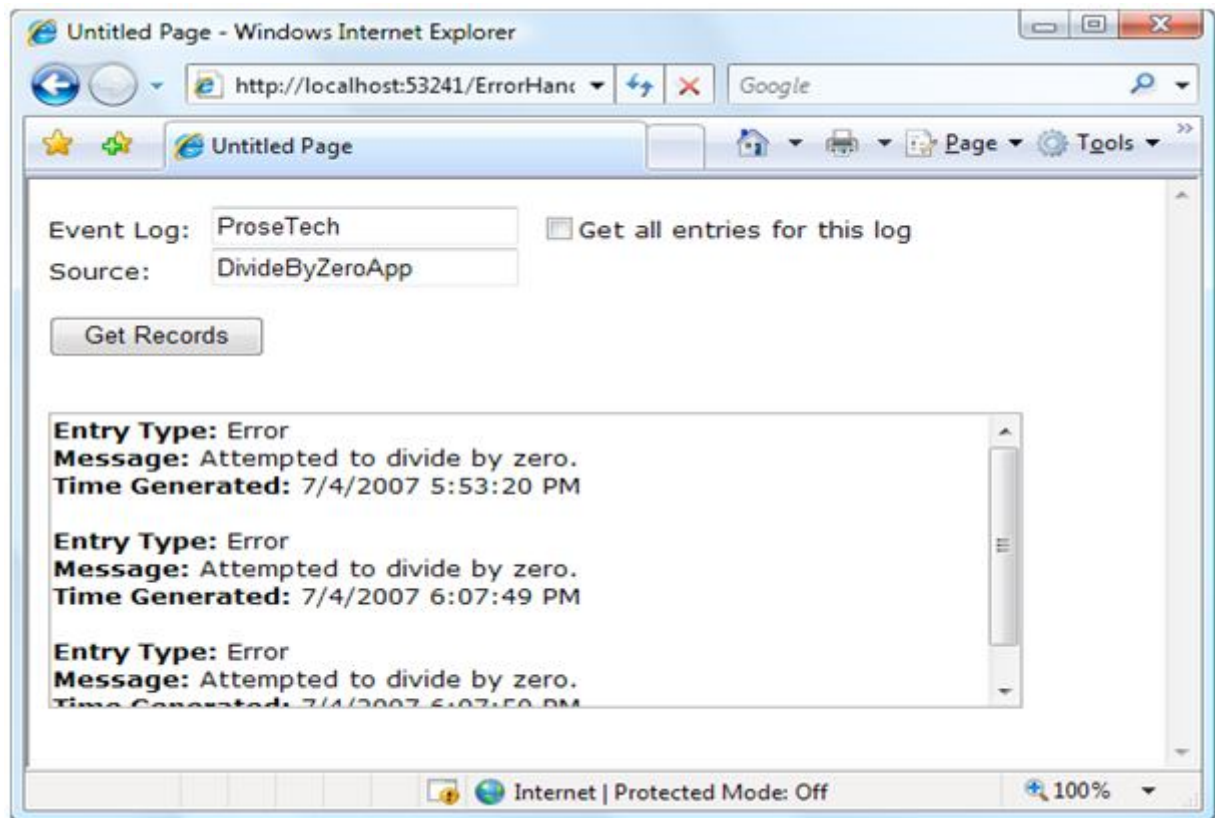


Figure 8. A log viewer page

Here's the web page code you'll need:

```
public partial class EventReviewPage : System.Web.UI.Page
{
    protected void cmdGet_Click(Object sender, EventArgs e)
    {
        lblResult.Text = "";
        // Check if the log exists.
        if (!EventLog.Exists(txtLog.Text))
        {
            lblResult.Text = "The event log " + txtLog.Text ;
            lblResult.Text += " doesn't exist.";
        }
    }
}
```

```

}

else

{

    EventLog log = new EventLog(txtLog.Text);

    foreach (EventLogEntry entry in log.Entries)

    {

        // Write the event entries to the page.

        if (chkAll.Checked ||

            entry.Source == txtSource.Text)

        {

            lblResult.Text += "<b>Entry Type:</b> ";

            lblResult.Text += entry.EntryType.ToString();

            lblResult.Text += "<br /><b>Message:</b> ";

            lblResult.Text += entry.Message;

            lblResult.Text += "<br /><b>Time Generated:</b> ";

            lblResult.Text += entry.TimeGenerated;

            lblResult.Text += "<br /><br />";

        }

    }

}

protected void chkAll_CheckedChanged(Object sender,

EventArgs e)

```

```

{
// The chkAll control has AutoPostBack = true.

if (chkAll.Checked)
{
txtSource.Text = "";
txtSource.Enabled = false;
}
else
{
txtSource.Enabled = true;
}
}
}
}

```

If you choose to display all the entries from the application log, the page will perform slowly. Two factors are at work here. First, it takes time to retrieve each event log entry; a typical application log can easily hold several thousand entries. Second, the code used to append text to the Label control is inefficient. Every time you add a new piece of information to the Label.Text property, .NET needs to generate a new String object. A better solution is to use the specialized System.Text.StringBuilder class, which is designed to handle intensive string processing with a lower overhead by managing an internal buffer or memory.

Here's the more efficient way you could write the string processing code:

```

// For maximum performance, join all the event
// information into one large string using the
// StringBuilder.

System.Text.StringBuilder sb = new System.Text.StringBuilder();

```

```

EventLog log = new EventLog(txtLog.Text);

foreach (EventLogEntry entry in log.Entries)
{
    // Write the event entries to the StringBuilder.

    if (chkAll.Checked ||

        entry.Source == txtSource.Text)
    {
        sb.Append("<b>Entry Type:</b> ");

        sb.Append(entry.EntryType.ToString());

        sb.Append("<br /><b>Message:</b> ");

        sb.Append(entry.Message);

        sb.Append("<br /><b>Time Generated:</b> ");

        sb.Append(entry.TimeGenerated);

        sb.Append("<br /><br />");
    }

    // Copy the complete text to the web page.

    lblResult.Text = sb.ToString();
}

```

5.8 Page Tracing

Visual Studio's debugging tools and ASP.NET's detailed error pages are extremely helpful when you're testing an application. However, sometimes you need a way to identify problems after you've deployed an application, when you don't have Visual Studio to rely on. ASP.NET provides a feature called *tracing* that gives you a far more convenient and flexible way to report diagnostic information.

Enabling Tracing

To use tracing, you need to explicitly enable it. There are several ways to switch on tracing. One of the easiest ways is by adding an attribute to the Page directive in the .aspx file:

```
<%@ Page Trace="true" ... %>
```

You can also enable tracing using the built-in Trace object (which is an instance of the `System.Web.TraceContext` class). Here's an example of how you might turn tracing on in the `Page_Load` event handler:

```
protected void Page_Load(Object sender, EventArgs e)  
  
{  
  
Trace.IsEnabled = true;  
  
}
```

This technique is useful because it allows you to enable or disable tracing for a page under specific circumstances that you test for in your code.

Tracing Information

ASP.NET tracing automatically provides a lengthy set of standards, formatted information. Figure 9 shows what this information looks like. To build this example, you can start with any basic ASP.NET page. Shown here is a rudimentary ASP.NET page with just a label and a button.

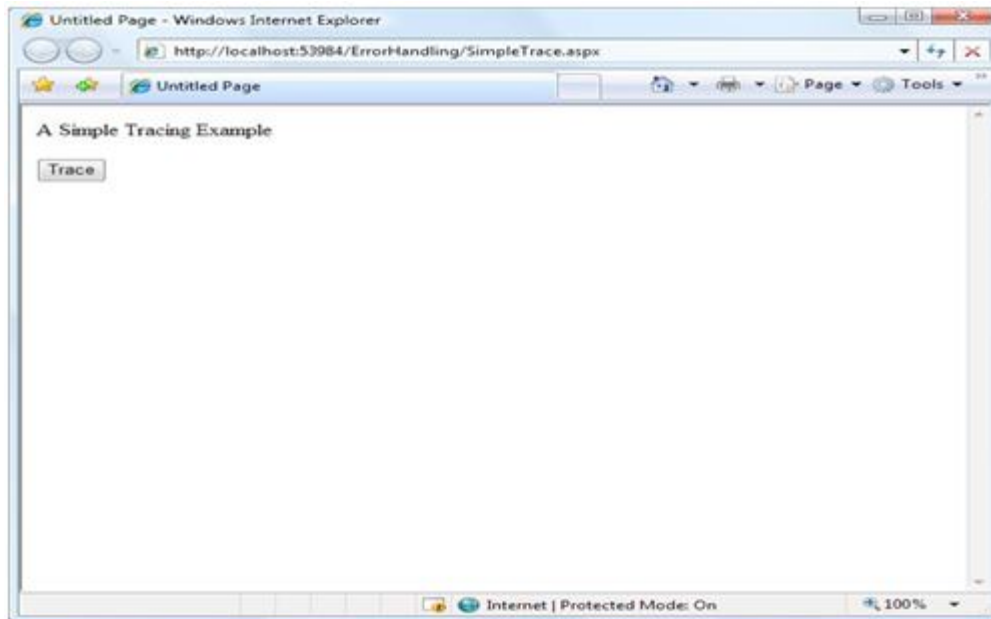


Figure 9. A simple ASP.NET page

On its own, this page does very little, displaying a single line of text. However, if you click the button, tracing is enabled by setting the `Trace.IsEnabled` property to true (as shown in the previous codesnippet). When the page is rendered, it will include a significant amount of diagnostic information, as shown in Figure 10.

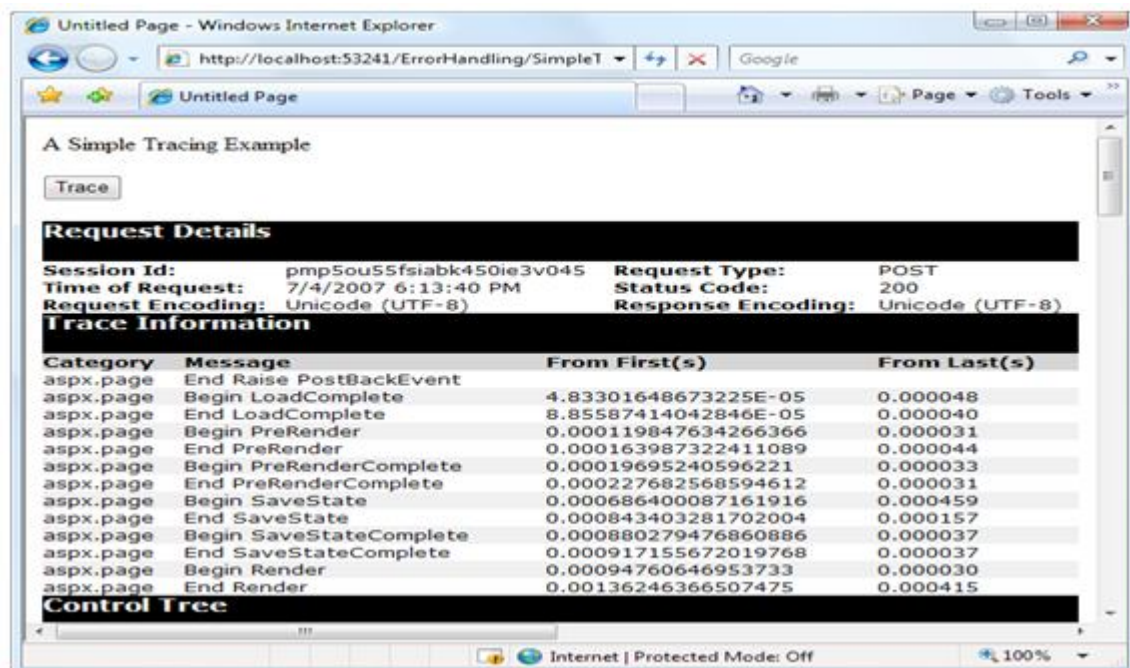


Figure 10. Tracing the simple ASP.NET page

Tracing information is provided in several different categories. Depending on your page, you may not see all the tracing information. For example, if the page request doesn't supply any query string parameters, you won't see the QueryString collection. Similarly, if there's no data being held in application or session state, you won't see those sections either.

Request Details

This section includes some basic information such as the current session ID, the time the web request was made, and the type of web request and encoding (see Figure 7-14). Most of these details are fairly uninteresting, and you won't spend much time looking at them. The exception is the session ID. By watching for when the session ID changes, you'll know when a new session is created. Sessions are used to store information for a specific user in between page requests.

Request Details			
Session Id:	pmp5ou55fsiabk450ie3v045	Request Type:	POST
Time of Request:	7/4/2007 6:13:40 PM	Status Code:	200
Request Encoding:	Unicode (UTF-8)	Response Encoding:	Unicode (UTF-8)

Figure 11. Request details

Trace Information

Trace information shows the different stages of processing that the page went through before being sent to the client. Each section has additional information about how long it took to complete, as a measure from the start of the first stage (From First) and as a measure from the start of the previous stage (From Last). If you add your own trace messages (a technique described shortly), they will also appear in this section.

Trace Information			
Category	Message	From First(s)	From Last(s)
aspx.page	End RaisePostBackEvent		
aspx.page	Begin LoadComplete	4.83301648673225E-05	0.000048
aspx.page	End LoadComplete	8.85587414042846E-05	0.000040
aspx.page	Begin PreRender	0.000119847634266366	0.000031
aspx.page	End PreRender	0.000163987322411089	0.000044
aspx.page	Begin PreRenderComplete	0.00019695240596221	0.000033
aspx.page	End PreRenderComplete	0.000227682568594612	0.000031
aspx.page	Begin SaveState	0.000686400087161916	0.000459
aspx.page	End SaveState	0.000843403281702004	0.000157
aspx.page	Begin SaveStateComplete	0.000880279476860886	0.000037
aspx.page	End SaveStateComplete	0.000917155672019768	0.000037
aspx.page	Begin Render	0.00094760646953733	0.000030
aspx.page	End Render	0.00136246366507475	0.000415

Figure 12. Trace information

5.9 Control Tree

The control tree shows you all the controls on the page, indented to show their hierarchy (which controls are contained inside other controls), as shown in Figure. In this simple page example, the control tree includes buttons named cmdWrite, cmdWrite_Category, cmdError, and cmdSession, all of which are explicitly defined in the web page markup. ASP.NET also adds literal controls automatically to represent spacing and any other static elements that aren't server controls (such as text or ordinary HTML tags). These controls appear in between the buttons in this example, and have automatically generated names like ctl00, ctl01, ctl02, and so on. One useful feature of this section is the Viewstate column, which tells you how many bytes of space are required to persist the current information in the control. This can help you gauge whether enabling control state is detracting from performance, particularly when working with data-bound controls such as the GridView.

Control Tree				
Control UniqueID	Type	Render Size Bytes (including children)	ViewState Size Bytes (excluding children)	ControlState Size Bytes (excluding children)
__Page	ASP.simpletrace_aspx	775	0	0
ctl02	System.Web.UI.LiteralControl	175	0	0
ctl00	System.Web.UI.HtmlControls.HtmlHead	46	0	0
ctl01	System.Web.UI.HtmlControls.HtmlTitle	33	0	0
ctl03	System.Web.UI.LiteralControl	14	0	0
form1	System.Web.UI.HtmlControls.HtmlForm	520	0	0
ctl04	System.Web.UI.LiteralControl	77	0	0
cmdTrace	System.Web.UI.WebControls.Button	67	0	0
ctl05	System.Web.UI.LiteralControl	12	0	0
ctl06	System.Web.UI.LiteralControl	20	0	0

Figure 13. Control tree

5.10 Session State and Application State

These sections display every item that is in the current session or application state. Each item in the appropriate state collection is listed with its name, type, and value. If you're storing simple pieces of string information, the value is straightforward—it's the actual text in the string. If you're storing an object, .NET calls the object's `ToString()` method to get an appropriate string representation. For complex objects that don't override `ToString()` to provide anything useful, the result may just be the classname. Figure shows the session state section after you've added two items to session state (an ordinary string and a `DataSet` object).

Session State		
Session Key	Type	Value
TestString	System.String	This is just a string.
MyDataSet	System.Data.DataSet	System.Data.DataSet

Figure 14. Session state

Request Cookies and Response Cookies

This section lists all the HTTP headers (see Figure 7-19). Technically, the headers are bits of information that are sent to the server as part of a request. They include information about the browser making the request, the types of content it supports, and the language it uses. In addition, the `Response.HeadersCollection` lists the headers that are sent to the client as part of a response (just before the actual HTML that's shown in the browser). The set of response headers is smaller, and it includes details such as the version of ASP.NET and the type of content that's being sent (text/html for web pages). Generally, you don't need to use the header information directly. Instead, ASP.NET takes this information into account automatically.

Headers Collection	
Name	Value
Cache-Control	no-cache
ConnectionKeep-Alive	
Content-Length	162
Content-Type	application/x-www-form-urlencoded
Accept	image/gif, image/x-bitmap, image/jpeg, image/pjpeg, application/x-ms-application, application/xpsdocument, application/xaml+xml, application/x-ms-xbap, application/vnd.ms-excel, application/powerpoint, application/msword, application/x-shockwave-flash, application/ag-plugin, */*
Accept-Encoding	gzip, deflate
Accept-Language	en-us
Host	localhost:53241
Referer	http://localhost:53241/ErrorHandler/SimpleTrace.aspx
User-Agent	Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.0; SLCC1; .NET CLR 2.0.50727; Media Center
UA-CPU	x86
Response Headers Collection	
Name	Value
X-AspNet-Version	2.0.50727
Cache-Control	private
Content-Type	text/html

Figure 15. Headers collection

Form Collection

This section lists the posted-back form information. The form information includes all the values that are submitted by web controls, like the text in a text box and the current selection in a list box. The ASP.NET web controls pull the information they need out of the form collection automatically, so you rarely need to worry about it. Figure shows the form values for the simple page. It includes the hidden view state field, another hidden field that's used for event validation (a low-level ASP.NET feature that helps prevent people from tampering with your web pages before posting them back), and a field for the cmdTrace button, which is the only web control on the page.

Form Collection	
Name	Value
__VIEWSTATE	/wEPDwUKMTQ2OTkzNDMyMWRk6PdTEsrgNkgFP9+LyoOOXsjohnM=
cmdTrace	Trace
__EVENTVALIDATION	/wEWAgl9lt3/BQLqhI2yDuBna1E/STtAj+7NCZA4/ezpy4LZ

Figure 16. Headers collection

5.11 Query String Collection

This section lists the variables and values submitted in the query string. You can see this information directly in the web page URL (in the address box in the browser). However, if the query string consists of several different values and contains a lot of information, it may be easier to review the individual items in the trace display. Figure shows the information for a page

that was requested with two query string values, one named search and the other named style. You can try this with the SimpleTrace.aspx page by typing in ?search=cat&style=full at the end of the URL in the address box of your web browser.

Querystring Collection	
Name	Value
search	cat
style	full

Figure 17. Query string collection

Server Variables

This section lists all the server variables and their contents. You don't generally need to examine this information. Note also that if you want to examine a server variable programmatically, you can do so by name with the built-in Request.ServerVariables collection or by using one of the more useful higher-level properties from the Request object.

Writing Trace Information

The default trace log provides a set of important information that can allow you to monitor some important aspects of your application, such as the current state contents and the time taken to execute portions of code. In addition, you'll often want to generate your own tracing messages. For example, you might want to output the value of a variable at various points in execution so you can compare it with an expected value. Similarly, you might want to output messages when the code reaches certain points in execution so you can verify that various procedures are being used (and are used in the order you expect). Once again, these are tasks you can also achieve using Visual Studio debugging, but tracing is an invaluable technique when you're working with a web application that's been deployed to a web server.

To write a custom trace message, you use the Write() method or the Warn() method of the built-in Trace object. These methods are equivalent. The only difference is that Warn() displays the message in red lettering, which makes it easier to distinguish from other messages in the list. Here's a code snippet that writes a trace message when the user clicks a button:

```
protected void cmdWrite_Click(Object sender, EventArgs e)  
  
{
```

```
Trace.Write("About to place an item in session state.");
```

```
Session["Test"] = "Contents";
```

```
Trace.Write("Placed item in session state.");
```

```
}
```

These messages appear in the trace information section of the page, along with the default messages that ASP.NET generates automatically.

Category	Message	From First(s)	From Last(s)
aspx.page	Begin PreInit		
aspx.page	End PreInit	5.7828578771883E-05	0.000058
aspx.page	Begin Init	0.000103365092490805	0.000046
aspx.page	End Init	0.000156444464310408	0.000053
aspx.page	Begin InitComplete	0.00019332065946929	0.000037
aspx.page	End InitComplete	0.000264558763753494	0.000071
aspx.page	Begin LoadState	0.000304507975175616	0.000040
aspx.page	End LoadState	0.000446984183744023	0.000142
aspx.page	Begin ProcessPostData	0.000496152443955966	0.000049
aspx.page	End ProcessPostData	0.000561523880828429	0.000065
aspx.page	Begin PreLoad	0.000600914362020871	0.000039
aspx.page	End PreLoad	0.000638628652524273	0.000038
aspx.page	Begin Load	0.000676622308142515	0.000038
aspx.page	End Load	0.000717688980023997	0.000041
aspx.page	Begin ProcessPostData Second Try	0.000754565175182879	0.000037
aspx.page	End ProcessPostData Second Try	0.000791720735456601	0.000037
aspx.page	Begin Raise ChangedEvents	0.000829155660845163	0.000037
aspx.page	End Raise ChangedEvents	0.000867428681578245	0.000038
aspx.page	Begin Raise PostBackEvent	0.000905701702311327	0.000038
	About to place an item in session state.	0.000993422348371092	0.000088
	Placed item in session state.	0.00105013346668361	0.000057
aspx.page	End Raise PostBackEvent	0.00109175886879478	0.000042
aspx.page	Begin LoadComplete	0.00112975252441302	0.000038
aspx.page	End LoadComplete	0.00116774618003126	0.000038
aspx.page	Begin PreRender	0.00121328269375018	0.000046

Figure 18. Custom trace messages

You can also use an overloaded method of Write() or Warn() that allows you to specify the category. A common use of this field is to indicate the current method, as shown in Figure.

Category	Message	From First(s)	From Last(s)
aspx.page	Begin PreInit		
aspx.page	End PreInit	5.7828578771883E-05	0.000058
aspx.page	Begin Init	0.000102806362261125	0.000045
aspx.page	End Init	0.000164825417755609	0.000062
aspx.page	Begin InitComplete	0.000201980978029331	0.000037
aspx.page	End InitComplete	0.000239974633647573	0.000038
aspx.page	Begin LoadState	0.000276292098576774	0.000036
aspx.page	End LoadState	0.000418209576915502	0.000142
aspx.page	Begin ProcessPostData	0.000462628630175064	0.000044
aspx.page	End ProcessPostData	0.000534984194918628	0.000072
aspx.page	Begin PreLoad	0.00057381594588139	0.000039
aspx.page	End PreLoad	0.000612088966614472	0.000038
aspx.page	Begin Load	0.000650361987347554	0.000038
aspx.page	End Load	0.000690869928999356	0.000041
aspx.page	Begin ProcessPostData Second Try	0.000728584219502758	0.000038
aspx.page	End ProcessPostData Second Try	0.00076629851000616	0.000038
aspx.page	Begin Raise ChangedEvents	0.000804571530739242	0.000038
aspx.page	End Raise ChangedEvents	0.000843682646816844	0.000039
aspx.page	Begin RaisePostBackEvent	0.000882793762894446	0.000039
aspx.page	About to place an item in session state.	0.00201310501753715	0.001130
aspx.page	Placed item in session state.	0.00501153079511502	0.002998
aspx.page	End RaisePostBackEvent	0.00508863556681087	0.000077
aspx.page	Begin LoadComplete	0.00512802604800331	0.000039
aspx.page	End LoadComplete	0.00516546097339187	0.000037
aspx.page	Begin PreRender	0.00520373399412495	0.000038
aspx.page	End PreRender	0.00525262288922195	0.000049
aspx.page	Begin PreRenderComplete	0.00529145464018472	0.000039
aspx.page	End PreRenderComplete	0.00532805147022876	0.000037

Figure 19. A categorized tracemessage

protected void cmdWriteCategory_Click(Object sender, EventArgs e)

```
{
    Trace.Write("cmdWriteCategory_Click",
        "About to place an item in session state.");
    Session["Test"] = "Contents";
    Trace.Write("cmdWriteCategory_Click",
        "Placed item in session state.");
}
```

Alternatively, you can supply category and message information with an exception object that will automatically be described in the trace log, as shown in Figure.

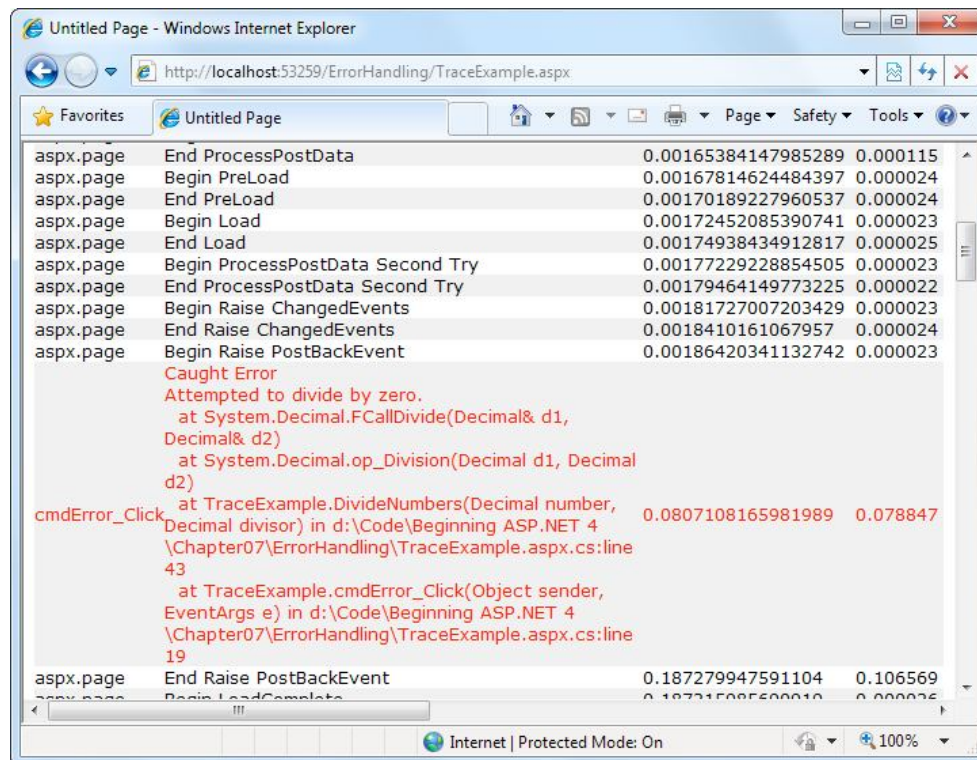


Figure 20. An exception trace message

protected void cmdError_Click(Object sender, EventArgs e)

```
{
    try
    {
        DivideNumbers(5, 0);
    }
    catch (Exception err)
    {
        Trace.Warn("cmdError_Click", "Caught Error", err);
    }
}
```

```

private decimal DivideNumbers(decimal number, decimal divisor)
{
    return number/divisor;
}

```

By default, trace messages are listed in the order they were written by your code. Alternatively, you can specify that messages should be sorted by category using the `TraceMode` attribute in the `Page` directive:

```
<%@ Page Trace="true" TraceMode="SortByCategory" %>
```

or the `TraceMode` property of the `Trace` object in your code:

```
Trace.TraceMode = TraceMode.SortByCategory;
```

Application-Level Tracing

Application-level tracing allows you to enable tracing for an entire application. However, the tracing information won't be displayed in the page. Instead, it will be collected and stored in memory for a short amount of time. You can review the recently traced information by requesting a special URL. Application-level tracing provides several advantages. The tracing information won't be mangled by the formatting and layout in your web page, you can compare trace information from different requests, and you can review the trace information that's recorded for someone else's request. To enable application-level tracing, you need to modify settings in the `web.config` file, as shown here:

```

<configuration>

<system.web>

<trace enabled="true" requestLimit="10" pageOutput="false" />

...

</system.web>

</configuration>

```

Table lists the tracing options.

Table Tracing Options

Attribute	Values	Description
Enabled	true, false	Turns application-level tracing on or off.
requestLimit	Any integer (for example, 10)	Stores tracing information for a maximum number of HTTP requests. Unlike page-level tracing, this allows you to collect a batch of information from multiple requests. When the maximum is reached, ASP.NET may discard the information from the oldest request (which is the default behavior) or the information from the new request, depending on the mostRecent setting.
pageOutput	true, false	Determines whether tracing information will be displayed on the page (as it is with page-level tracing). If you choose false, you'll still be able to view the collected information by requesting trace.axd from the virtual directory where your application is running.
traceMode	SortByTime, SortByCategory	Determines the sort order of trace messages. The default is SortByTime.
localOnly	true, false	Determines whether tracing information will be shown only to local clients (clients using the same computer) or can be shown to remote clients as well. By default,
mostRecent	true, false	Keeps only the most recent trace messages if true. When the requestLimit maximum is reached, the information for the oldest request is abandoned every time a new request is received. If false (the default), ASP.NET stops collecting new trace messages when the limit is reached.

To view tracing information, you request the trace.axd file in the web application's root directory. This file doesn't actually exist; instead, ASP.NET automatically intercepts the request and interprets it as a request for the tracing information. It will then list the most recent collected requests, provided you're making the request from the local machine or have enabled remote tracing.

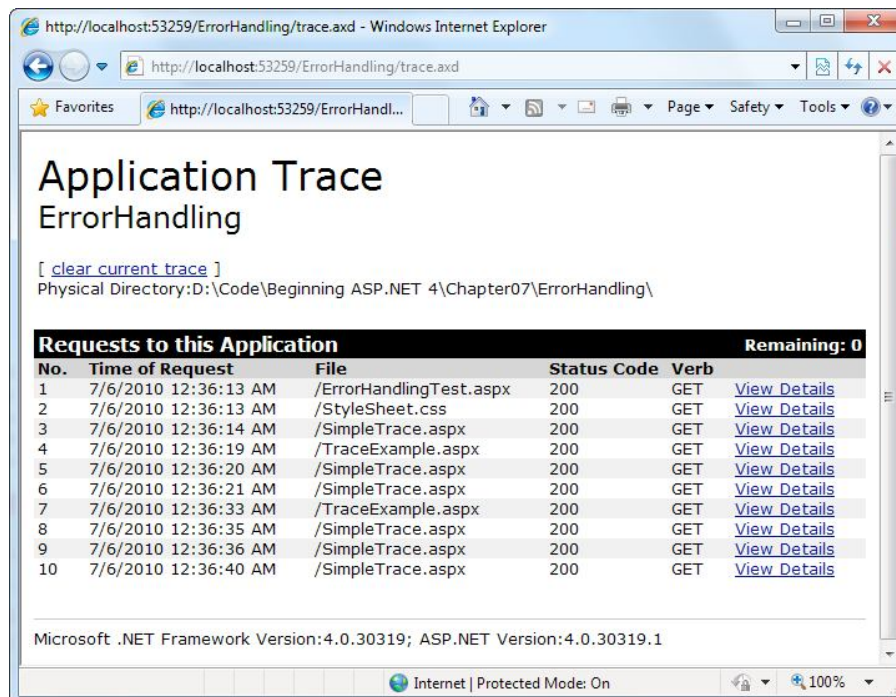


Figure 21. Traced application requests

You can see the detailed information for any request by clicking the View Details link. This provides a useful way to store tracing information for a short period of time and allows you to review it without needing to see the actual pages.

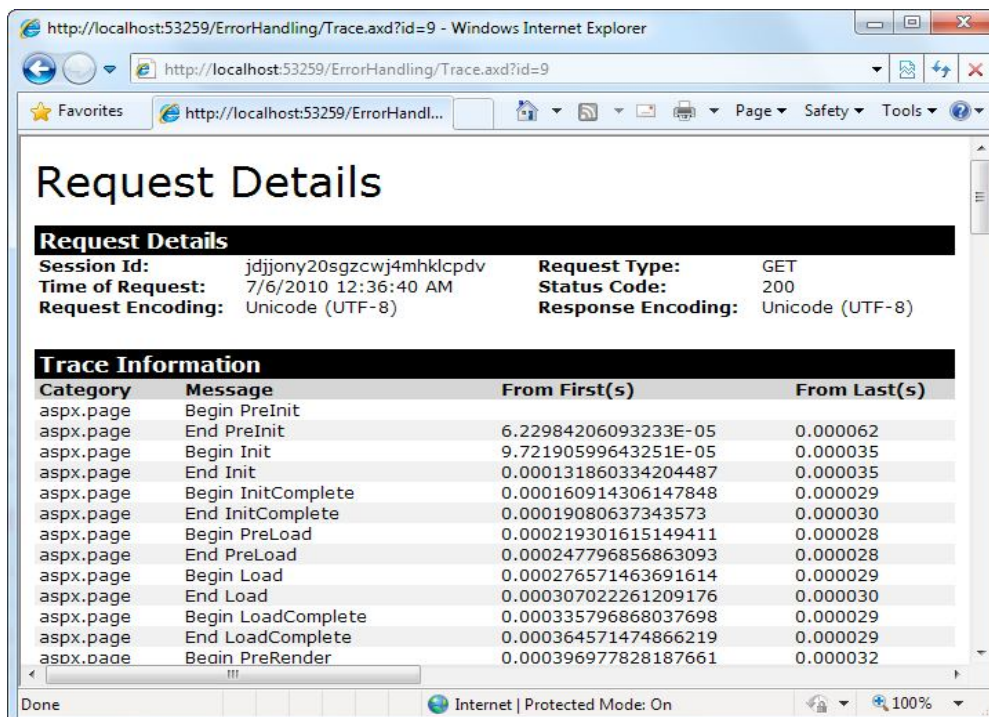


Figure 22. One of the traced application requests

5.12 Summary

One of the most significant differences between an ordinary website and a professional web application is often in how it deals with errors. In this chapter, you learned the different lines of defense you can use in .NET, including structured error handling, logging, and tracing.

Keywords

Page Class, Application Events, Exception Chain, Throwexception, EventLog, log class, Query String Collection, Trace

5.13 Check your progress

1. Which log used to track operating system events?
 - A. Application
 - B. Security
 - C. System**
 - D. Setup
2. Which among the following is considered as .NET Exception class?
 - A. Exception
 - B. StackUnderflow Exception**
 - C. File bound Exception
 - D. All of the mentioned
3. Which of these keywords is not a part of exception handling?
 - A. Try
 - B. Finally
 - C. Thrown**
 - D. Catch

4. Which of these keywords must be used to monitor exceptions?

A. try

B. finally

C. Throw

D. Catch

5.14 Model Questions:

1. Write short notes on exception handling.

2. What is page tracing?

3. How to throw your own exception?

4. How to write in log file?

5. How to catch exception?

LESSON - 6

STATE MANAGEMENT

Structure

- 6.1 Introduction
- 6.2 Objectives
- 6.3 The Problem of State
- 6.4 Making View State Secure
- 6.5 Getting More Information from the Source Page
- 6.6 Cookies
- 6.7 Session State
- 6.8 SQL Server
- 6.9 Validation
- 6.10 Summary
- 6.11 Check your progress
- 6.12 Model Questions

6.1 Introduction

In a web application, it's a different story. Thousands of users can simultaneously run the same application on the same computer (the web server), each one communicating over a stateless HTTP connection. There are different storage options, including view state, session state, and custom cookies.

The most significant difference between programming for the Web and programming for the desktop is *state management*—how you store information over the lifetime of your application. Understanding these state limitations is the key to creating efficient web applications. In this chapter, you'll see how you can use ASP.NET's state management features to store information

carefully and consistently. You'll explore different storage options, including view state, session state, and custom cookies. You'll also consider how to transfer information from page to page using cross-page posting and the query string.

6.2 Learning objectives

- ✓ Understand view state
- ✓ Transfer information between pages
- ✓ Understand cookie
- ✓ Understand session state
- ✓ Understand application state
- ✓ Understand validation control

6.3 The Problem of State

A professional ASP.NET site might look like a continuously running application, but that's really just a clever illusion. In a typical web request, the client connects to the web server and requests a page. When the page is delivered, the connection is severed, and the web server discards all the page objects from memory.

This stateless design has one significant advantage. Because clients need to be connected for only a few seconds at most, a web server can handle a huge number of nearly simultaneous requests without a performance hit. However, if you want to retain information for a longer period of time so it can be used over multiple post backs or on multiple pages, you need to take additional steps.

This stateless design has one significant advantage. Because clients need to be connected for only a few seconds at most, a web server can handle a huge number of nearly simultaneous requests without a performance hit. However, if you want to retain information for a longer period of time so it can be used over multiple post backs or on multiple pages, you need to take additional steps.

View State

One of the most common ways to store information is in view state. View state uses a hidden field that ASP.NET automatically inserts in the final, rendered HTML of a web page. It's a perfect place to store information that's used for multiple postbacks in a single web page. Web controls store most of their property values in view state, provided you haven't switched ViewState off.

View state isn't limited to web controls. Your web page code can add bits of information directly to the view state of the containing page and retrieve it later after the page is posted back. The type of information you can store includes simple data types and your own custom objects.

The View State Collection

The View State property of the page provides the current view state information. This property provides an instance of the State Bag collection class. The State Bag is a dictionary collection, which means every item is stored in a separate "slot" using a unique string name. For example, consider this code:

```
// This keyword refers to the current Page object. It's optional.
```

```
this.ViewState["Counter"] = 1;
```

If currently no item has the name Counter, a new item will be added automatically. If an item is already stored under the name Counter, it will be replaced. When retrieving a value, you use the key name. You also need to cast the retrieved value to the appropriate data type using the casting syntax. This extra step is required because the ViewState collection stores all items as basic objects, which allows it to handle many different data types. Here's the code that retrieves the counter from view state and converts it to an integer:

```
int counter;
```

```
counter = (int)this.ViewState["Counter"];
```

A View State Example

The following example is a simple counter program that records how many times a button is clicked.

```
public partial class SimpleCounter : System.Web.UI.Page

{

protected void cmdIncrement_Click(Object sender, EventArgs e)

{

int counter;

if (ViewState["Counter"] == null)

{

counter = 1;

}

else

{

counter = (int)ViewState["Counter"] + 1;

}

ViewState["Counter"] = counter;

lblCount.Text = "Counter: " + counter.ToString();

}

}
```

Figure 1 shows the output for this page

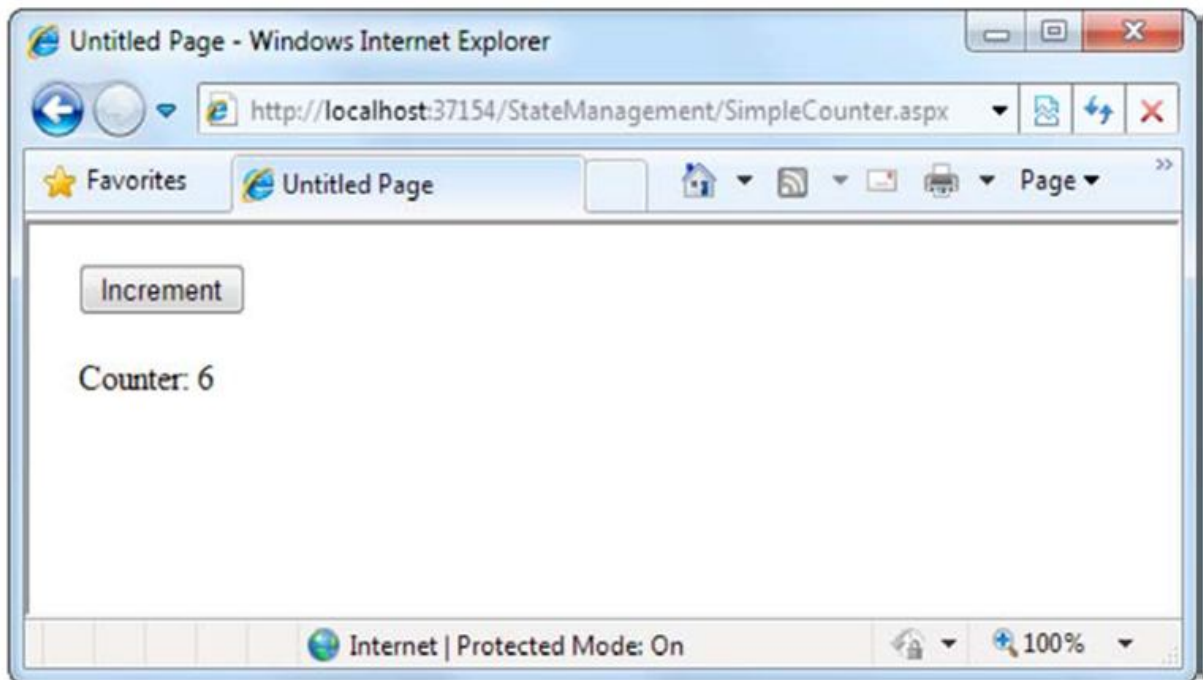


Figure 1. A simple view state counter

6.4 Making View State Secure

The view state information is stored in a single jumbled string that looks like this:

```
<input type="hidden" name="__VIEWSTATE" id="__VIEWSTATE"
      value="dDw3NDg2NTI5MDg7Oz4=" />
```

The view state information is simply patched together in memory and converted to a *Base64 string*. A clever hacker could reverse-engineer this string and examine your view state data in a matter of seconds.

Tamper-Proof View State

If you want to make view state more secure, you have two choices. First, you can make sure the viewstate information is tamperproof by instructing ASP.NET to use a *hash code*. A hash code is sometimes described as a cryptographically strong checksum. Hash codes are actually enabled by default, so if you want this functionality, you don't need to take any extra steps.

Private View State

Even when you use hash codes, the view state data will still be readable by the user. The view state tracks information that's often provided directly through other controls. However, if your view state contains some information you want to keep secret, you can enable view state encryption.

You can turn on encryption for an individual page using the `ViewStateEncryptionMode` property of the `Page` directive:

```
<%@Page ViewStateEncryptionMode="Always" %>
```

Or you can set the same attribute in a configuration file to configure view state encryption for all the

pages in your website:

```
<configuration>
```

```
<system.web>
```

```
<pages viewStateEncryptionMode="Always" />
```

```
...
```

```
</system.web>
```

```
</configuration>
```

Either way, this enforces encryption. You have three choices for your view state encryption setting—always encrypt (Always), never encrypt (Never), or encrypt only if a control specifically requests it (Auto).

Retaining Member Variables

Any information you set in a member variable for an ASP.NET page is automatically abandoned when the page processing is finished and the page is sent to the client. The basic principle is to save all member variables to view state when the `Page.PreRender` event occurs and retrieve them when the `Page.Load` event occurs.

1. Transferring Information Between Pages

One of the most significant limitations with view state is that it's tightly bound to a specific page. If the user navigates to another page, this information is lost. This problem has several solutions, and the best approach depends on your requirements.

Cross-Page Posting

A cross-page post back is a technique that extends the post back mechanism. The infrastructure that supports cross-page post backs is a property named `PostBackUrl`, which is defined by the `IButtonControl` interface and turns up in button controls such as `ImageButton`, `LinkButton`, and `Button`. To use cross-posting, you simply set `PostBackUrl` to the name of another webform.

The infrastructure that supports cross-page postbacks is a property named `PostBackUrl`, which is defined by the `IButtonControl` interface and turns up in button controls such as `ImageButton`, `LinkButton`, and `Button`. To use cross-posting, you simply set `PostBackUrl` to the name of another webform. When the user clicks the button, the page will be posted to that new URL with the values from all the input controls on the current page.

Here's an example—a page named `CrossPage1.aspx` that defines a form with two text boxes and a button. When the button is clicked, it posts to a page named `CrossPage2.aspx`.

```
<%@ Page Language="C#" AutoEventWireup="true" CodeFile="CrossPage1.aspx.cs"
```

```
Inherits="CrossPage1" %>
```

```
<html xmlns="http://www.w3.org/1999/xhtml">
```

```
<head runat="server">
```

```
<title>CrossPage1</title>
```

```
</head>
```

```
<body>
```

```
<form id="form1" runat="server" >
```

```
<div>
```

```
First Name:
```

```
<asp:TextBox ID="txtFirstName" runat="server"></asp:TextBox>
```

```
<br />
```

Last Name:

```
<asp:TextBox ID="txtLastName" runat="server"></asp:TextBox>
```

```
<br />
```

```
<br />
```

```
<asp:Button runat="server" ID="cmdPost"
```

```
PostBackUrl="CrossPage2.aspx" Text="Cross-Page Postback" /><br />
```

```
</div>
```

```
</form>
```

```
</body>
```

```
</html>
```

Figure 2 shows how it appears in the browser

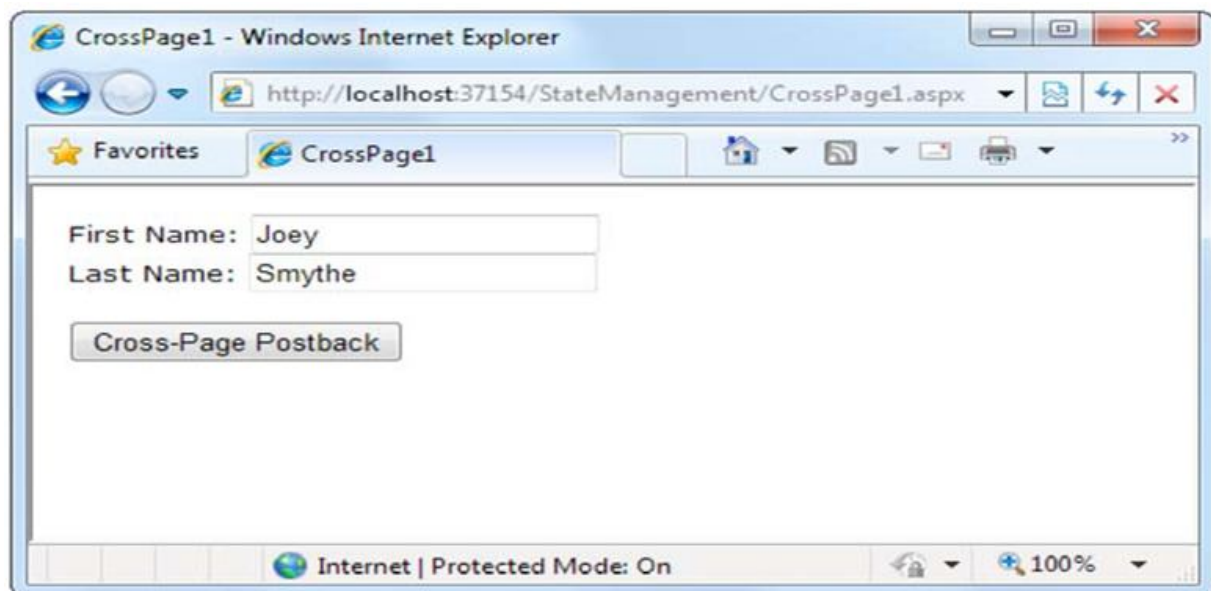


Figure 2. The source of a cross-page postback

Now if you load this page and click the button, the page will be posted back to CrossPage2.aspx. At this point, the CrossPage2.aspx page can interact with CrossPage1.aspx using the Page.PreviousPage property. Here's an event handler that grabs the title from the previous page and displays it:

```
public partial class CrossPage2 : System.Web.UI.Page
{
    protected void Page_Load(object sender, EventArgs e)
    {
        if (PreviousPage != null)
        {
            lblInfo.Text = "You came from a page titled " +
                PreviousPage.Title;
        }
    }
}
```

Figure 3 shows what you'll see when CrossPage1.aspx posts to CrossPage2.aspx.

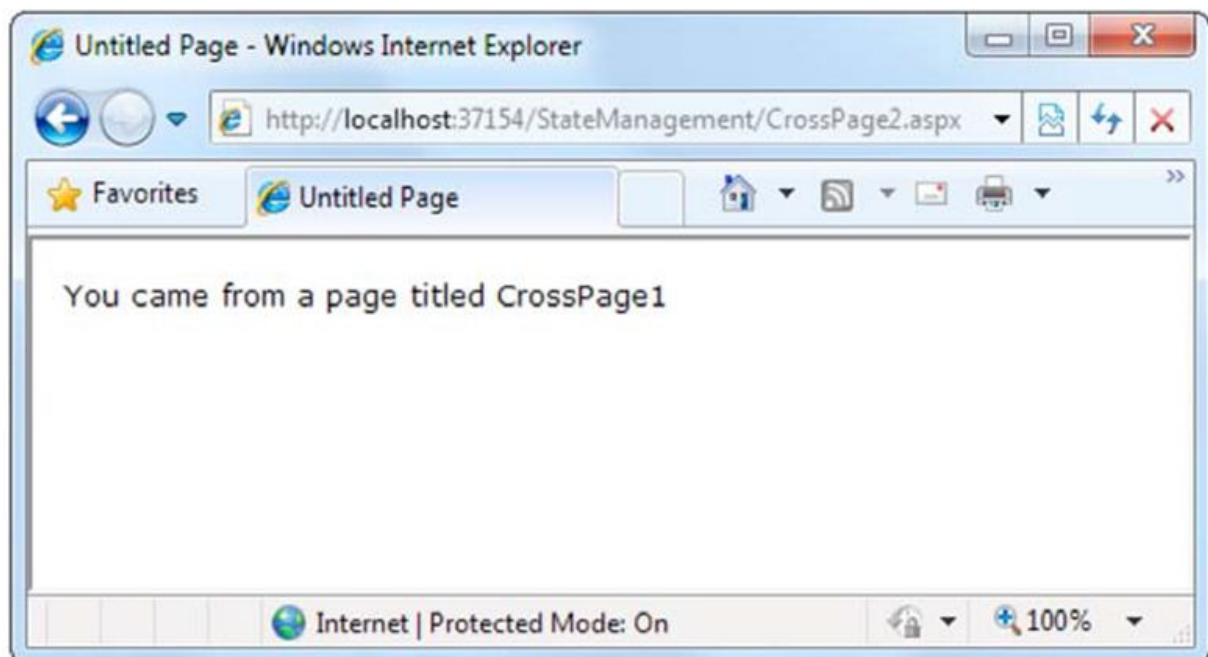


Figure 3. The target of a cross-page postback

6.5 Getting More Information from the Source Page

To get more specific details, such as control values, you need to cast the `PreviousPage` reference to the appropriate page class. Here's an example that handles this situation properly, by checking first whether the `PreviousPage` object is an instance of the expected class:

```
protected void Page_Load(object sender, EventArgs e)

{

    CrossPage1 prevPage = PreviousPage as CrossPage1;

    if (prevPage != null)

    {

        // (Read some information from the previous page.)

    }

}
```

The target page can access the `Title` property of the previous page without performing any casting. That's because the `Title` property is defined as part of the base `System.Web.UI.Page` class, and so every web page includes it. However, to get access to the more specialized `FullName` property, you need to cast the previous page to the right page class or use the `PreviousPageType` directive.

ASP.NET uses some interesting sleight of hand to make cross-page postbacks work. The first time the second page accesses `Page.PreviousPage`, ASP.NET needs to create the previous page object. To do this, it actually starts the page processing but interrupts it just before the `PreRender` stage, and it doesn't let the page render any HTML output.

However, this still has some interesting side effects. For example, all the page events of the previous page are fired, including `Page.Load` and `Page.Init`, and the `Button.Click` event also fires for the button that triggered the cross-page postback. ASP.NET fires these events because they might be needed to return the source page to the state it was last in, just before it triggered the cross-page postback.

The Query String

Another common approach is to pass information using a query string in the URL. This approach is commonly found in search engines. For example, if you perform a search on the Google website, you'll be redirected to a new URL that incorporates your search parameters. Here's an example:

<http://www.google.ca/search?q=organic+gardening>

The query string is the portion of the URL after the question mark. In this case, it defines a single variable named *q*, which contains the string *organic+gardening*. The advantage of the query string is that it's lightweight and doesn't exert any kind of burden on the server. However, it also has several limitations:

- Information is limited to simple strings, which must contain URL-legal characters.
- Information is clearly visible to the user and to anyone else who cares to eavesdrop on the Internet.
- The enterprising user might decide to modify the query string and supply new values, which your program won't expect and can't protect against.
- Many browsers impose a limit on the length of a URL (usually from 1KB to 2KB). For that reason, you can't place a large amount of information in the query string and still be assured of compatibility with most browsers.

To store information in the query string, you need to place it there yourself. Unfortunately, you have no collection-based way to do this. Instead, you'll need to insert it into the URL yourself. Here's an example that uses this approach with the `Response.Redirect()` method:

```
// Go to newpage.aspx. Submit a single query string argument
```

```
// named recordID, and set to 10.
```

```
Response.Redirect("newpage.aspx?recordID=10");
```

You can send multiple parameters as long as they're separated with an ampersand (&):

```
// Go to newpage.aspx. Submit two query string arguments:
```

```
// recordID (10) and mode (full).
```

```
Response.Redirect("newpage.aspx?recordID=10&mode=full");
```

The receiving page has an easier time working with the query string. It can receive the values from the QueryString dictionary collection exposed by the built-in Request object:

```
string ID = Request.QueryString["recordID"];
```

Note that information is always retrieved as a string, which can then be converted to another simple data type. Values in the QueryString collection are indexed by the variable name. If you attempt to retrieve a value that isn't present in the query string, you'll get a null reference.

A Query String Example

When the user chooses an item by clicking the appropriate item in the list, the user is forwarded to a new page. This page displays the received ID number. This provides a quick and simple query string test with two pages. In a sophisticated application, you would want to combine some of the data control features. The first page provides a list of items, a check box, and a submission button.

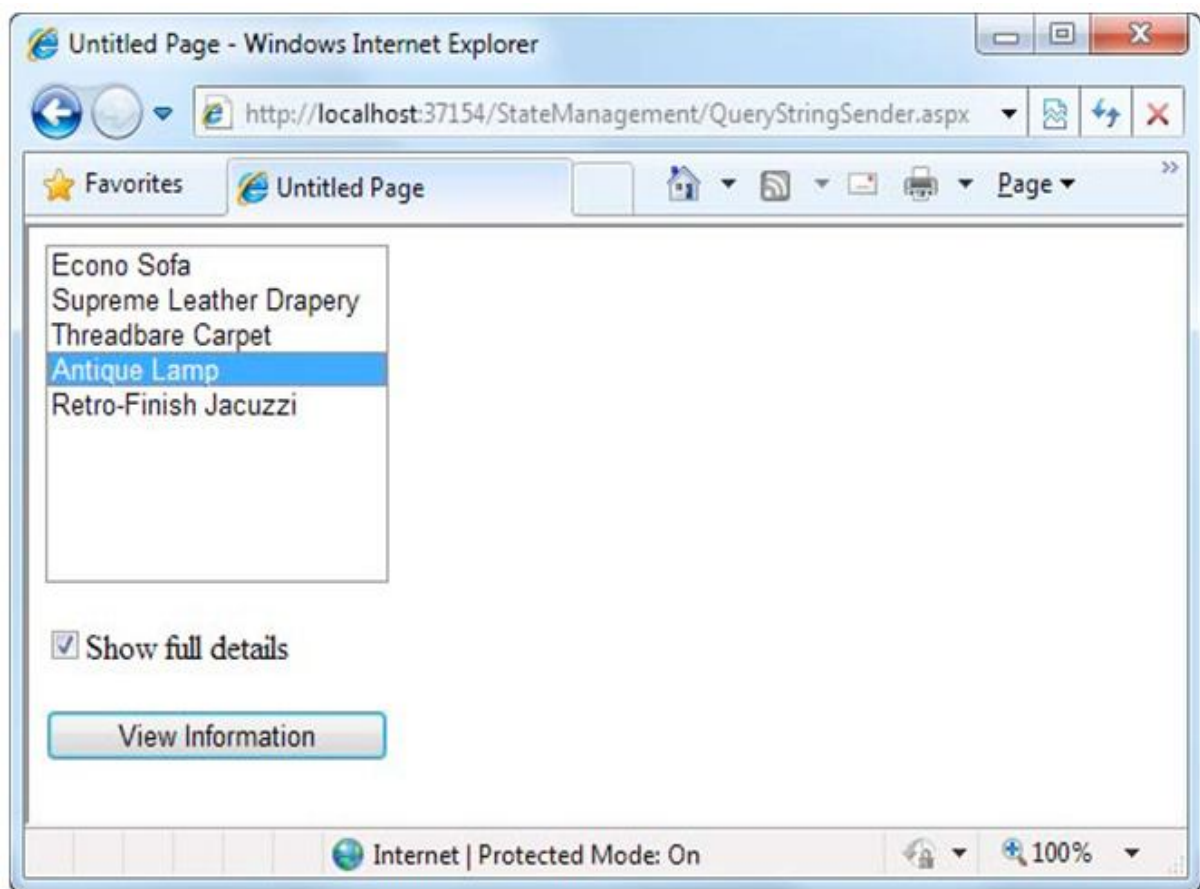


Figure 4. A query string sender

Here's the code for the first page:

```
public partial class QueryStringSender : System.Web.UI.Page
{
    protected void Page_Load(Object sender, EventArgs e)
    {
        if (!this.IsPostBack)
        {
            // Add sample values.

            lstItems.Items.Add("Econo Sofa");

            lstItems.Items.Add("Supreme Leather Drapery");

            lstItems.Items.Add("Threadbare Carpet");

            lstItems.Items.Add("Antique Lamp");

            lstItems.Items.Add("Retro-Finish Jacuzzi");
        }
    }

    protected void cmdGo_Click(Object sender, EventArgs e)
    {
        if (lstItems.SelectedIndex == -1)
        {
            lblError.Text = "You must select an item.";
        }
        Else{
```

// Forward the user to the information page,

// with the query string data.

string url = "QueryStringRecipient.aspx?";

url += "Item=" + lstItems.SelectedItem.Text + "&;

url += "Mode=" + chkDetails.Checked.ToString();

Response.Redirect(url);

}

}

}

Here's the code for the recipient page:

public partial class QueryStringRecipient : System.Web.UI.Page

{

protected void Page_Load(Object sender, EventArgs e)

{

lblInfo.Text = "Item: " + Request.QueryString["Item"];

***lblInfo.Text += "
Show Full Record: ";***

lblInfo.Text += Request.QueryString["Mode"];

}

}



Figure 5. A query string recipient

URL Encoding

One potential problem with the query string is that some characters aren't allowed in a URL. In fact, the list of characters that are allowed in a URL is much shorter than the list of allowed characters in an HTML document. All characters must be alphanumeric or one of a small set of special characters (including \$, -, ., +, !, *, '(),). Some characters have special meaning. For example, the ampersand (&) is used to separate multiple query string parameters, the plus sign (+) is an alternate way to represent a space, and the number sign (#) is used to point to a specific bookmark in a web page. If you try to send query string values that include any of these characters, you'll lose some of your data. You can test this with the previous example by adding items with special characters in the list box.

To avoid potential problems, it's a good idea to perform *URL encoding* on text values before you place them in the query string. With URL encoding, special characters are replaced by escaped character sequences starting with the percent sign (%), followed by a two-digit hexadecimal representation. For example, the & character becomes %26. The only exception is the space character, which can be represented as the character sequence %20 or the + sign. To perform URL encoding, you use the `UrlEncode()` and `UrlDecode()` methods of the `HttpServerUtility` class.

To avoid potential problems, it's a good idea to perform *URL encoding* on text values before you place them in the query string. With URL encoding, special characters are replaced by escaped character sequences starting with the percent sign (%), followed by a two-digit

hexadecimal representation. Foreexample, the & character becomes %26. The only exception is the space character, which can be represented as the character sequence %20 or the + sign. To perform URL encoding, you use the `UrlEncode()` and `UrlDecode()` methods of the `HttpServerUtility` class.

6.6 Cookies

Cookies provide another way that you can store information for later use. Cookies are small files that are created in the web browser's memory (if they're temporary) or on the client's hard drive (if they're permanent). One advantage of cookies is that they work transparently without the user being aware that information needs to be stored. They also can be easily used by any page in your application and even be retained between visits, which allows for truly long-term storage. They suffer from some of the same drawbacks that affect query strings—namely, they're limited to simple string information, and they're easily accessible and readable if the user finds and opens the corresponding file. These factors make them a poor choice for complex or private information or large amounts of data.

Before you can use cookies, you should import the `System.Net` namespace so you can easily work with the appropriate types:

```
using System.Net;
```

Cookies are fairly easy to use. Both the `Request` and `Response` objects (which are provided through `Page` properties) provide a `Cookies` collection. The important trick to remember is that you retrieve cookies from the `Request` object, and you set cookies using the `Response` object.

To set a cookie, just create a new `HttpCookie` object. You can then fill it with string information (using the familiar dictionary pattern) and attach it to the current web response:

```
// Create the cookie object.
```

```
HttpCookie cookie = new HttpCookie("Preferences");
```

```
// Set a value in it.
```

```
cookie["LanguagePref"] = "English";
```

```
// Add another value.
```

```
cookie["Country"] = "US";
```

// Add it to the current web response.

```
Response.Cookies.Add(cookie);
```

A cookie added in this way will persist until the user closes the browser and will be sent with every request. To create a longer-lived cookie, you can set an expiration date:

// This cookie lives for one year.

```
cookie.Expires = DateTime.Now.AddYears(1);
```

You retrieve cookies by cookie name using the Request.Cookies collection:

```
HttpCookie cookie = Request.Cookies["Preferences"];
```

// Check to see whether a cookie was found with this name.

// This is a good precaution to take,

// because the user could disable cookies,

// in which case the cookie will not exist.

```
string language;
```

```
if (cookie != null)
```

```
{
```

```
language = cookie["LanguagePref"];
```

```
}
```

The only way to remove a cookie is by replacing it with a cookie that has an expiration date that has already passed. This code demonstrates the technique:

```
HttpCookie cookie = new HttpCookie("Preferences");
```

```
cookie.Expires = DateTime.Now.AddDays(-1);
```

```
Response.Cookies.Add(cookie);
```

A Cookie Example

The next example shows a typical use of cookies to store a customer name (Figure 8-8). To try this example, begin by running the page, entering a name, and clicking the Create Cookie button. Then, close the browser, and request the page again. The second time, the page will find the cookie, read the name, and display a welcome message.

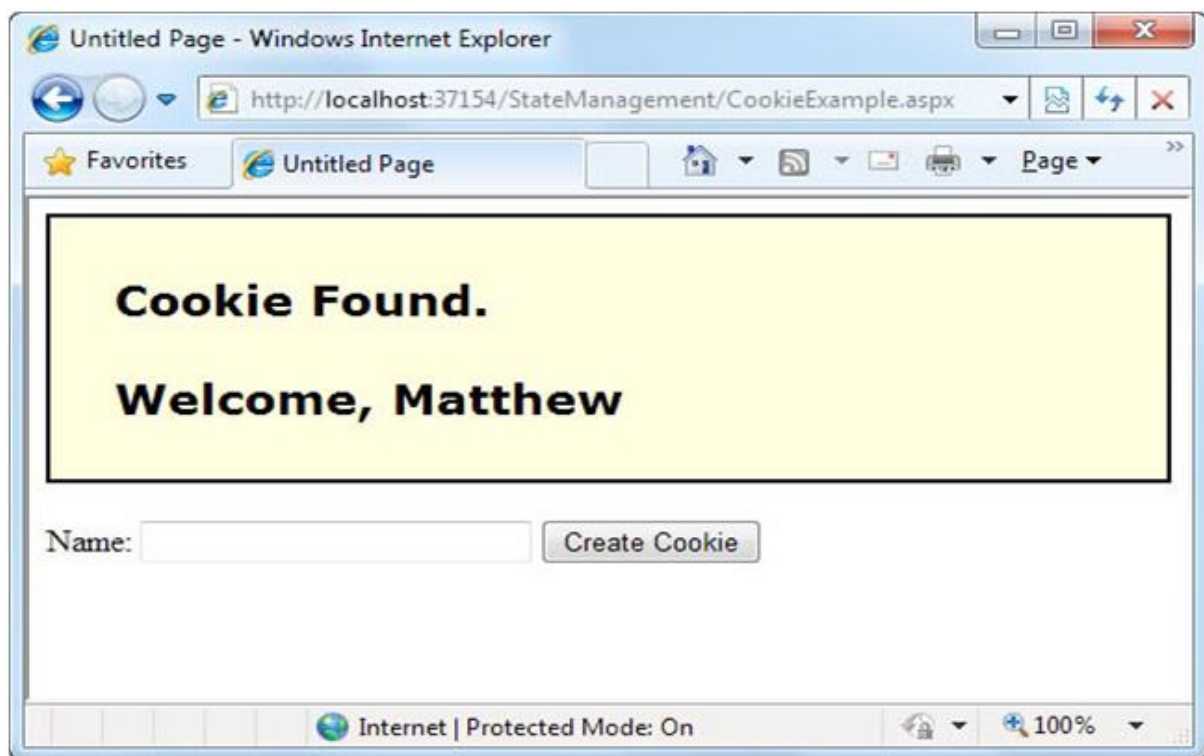


Figure 6. *Displaying information from a custom cookie*

Here's the code for this page:

```
public partial class CookieExample : System.Web.UI.Page
{
    protected void Page_Load(Object sender, EventArgs e)
    {
        HttpCookie cookie = Request.Cookies["Preferences"];
```

```

if (cookie == null)

{

    lblWelcome.Text = "<b>Unknown Customer</b>";

}

else

{

    lblWelcome.Text = "<b>Cookie Found.</b><br /><br />";

    lblWelcome.Text += "Welcome, " + cookie["Name"];

}

}

protected void cmdStore_Click(Object sender, EventArgs e)

{

    // Check for a cookie, and only create a new one if

    // one doesn't already exist.

    HttpCookie cookie = Request.Cookies["Preferences"];

    if (cookie == null)

    {

        cookie = new HttpCookie("Preferences");

    }

    cookie["Name"] = txtName.Text;

    cookie.Expires = DateTime.Now.AddYears(1);

```

```

Response.Cookies.Add(cookie);

lblWelcome.Text = "<b>Cookie Created.</b><br /><br />";

lblWelcome.Text += "New Customer: " + cookie["Name"];

}

}

```

6.7 Session State

An application might need to store and access complex information such as custom data objects, which can't be easily persisted to a cookie or sent through a query string. Or the application might have stringent security requirements that prevent it from storing information about a client in view state or in a custom cookie. In these situations, you can use ASP.NET's built-in session state facility.

Session state management is one of ASP.NET's premiere features. It allows you to store any type of data in memory on the server. The information is protected, because it is never transmitted to the client, and it's uniquely bound to a specific session. Every client that accesses the application has a different session and a distinct collection of information. Session state is ideal for storing information such as the items in the current user's shopping basket when the user browses from one page to another.

Session Tracking

ASP.NET tracks each session using a unique 120-bit identifier. ASP.NET uses a proprietary algorithm to generate this value, thereby guaranteeing (statistically speaking) that the number is unique and it's random enough that a malicious user can't reverse-engineer or "guess" what session ID a given client will be using. This ID is the only piece of session-related information that is transmitted between the web server and the client.

When the client presents the session ID, ASP.NET looks up the corresponding session, retrieves the objects you stored previously, and places them into a special collection so they can be accessed in your code. This process takes place automatically.

For this system to work, the client must present the appropriate session ID with each request. You can accomplish this in two ways:

Using cookies: In this case, the session ID is transmitted in a special cookie (named `ASP.NET_SessionId`), which ASP.NET creates automatically when the session collection is used. This is the default.

Using modified URLs: In this case, the session ID is transmitted in a specially modified (or *munged*) URL. This allows you to create applications that use session state with clients that don't support cookies.

Session state doesn't come for free. Though it solves many of the problems associated with other forms of state management, it forces the server to store additional information in memory. This extramemory requirement, even if it is small, can quickly grow to performance-destroying levels as hundreds or thousands of clients access the site. In other words, you must think through any use of session state. A careless use of session state is one of the most common reasons that a web application can't scale to serve a large number of clients.

Using Session State

You can interact with session state using the `System.Web.SessionState.HttpSessionState` class, which is provided in an ASP.NET web page as the built-in `Session` object. The syntax for adding items to the collection and retrieving them is basically the same as for adding items to a page's view state.

For example, you might store a `DataSet` in session memory like this:

```
Session["InfoDataSet"] = dsInfo;
```

You can then retrieve it with an appropriate conversion operation:

```
dsInfo = (DataSet)Session["InfoDataSet"];
```

Of course, before you attempt to use the `dsInfo` object, you'll need to check that it actually exists—in other words, that it isn't a null reference. If the `dsInfo` is null, it's up to you to regenerate it.

Session state is global to your entire application for the current user. However, session state can be lost in several ways:

- If the user closes and restarts the browser.

- If the user accesses the same page through a different browser window, although the session will still exist if a web page is accessed through the original browser window. Browsers differ on how they handle this situation.
- If the session times out due to inactivity. More information about session timeout can be found in the configuration section.
- If your web page code ends the session by calling the `Session.Abandon()` method.

The first two cases, the session actually remains in memory on the web server, because ASP.NET has no idea that the client has closed the browser or changed windows. The session will linger in memory, remaining inaccessible, until it eventually expires.

Table describes the key methods and properties of the `HttpSessionState` class.

Table. *HttpSessionState Members*

Member	Description
Count	Provides the number of items in the current session collection.
IsCookieless	Identifies whether the session is tracked with a cookie or modified URLs.
IsNewSession	Identifies whether the session was created only for the current request. If no information is in session state, ASP.NET won't bother to track the session or create a session cookie. Instead, the session will be re-created with every request
Keys	Gets a collection of all the session keys that are currently being used to store items in the session state collection
Mode	Provides an enumerated value that explains how ASP.NET stores session state information. This storage mode is determined based on the web.config settings discussed in the "Session State Configuration" section later in this chapter.
SessionID	Provides a string with the unique session identifier for the current client.
Timeout	Determines the number of minutes that will elapse before the current session is abandoned, provided that no more requests are received from

the client. This value can be changed programmatically, letting you make the session collection longer when needed.

Abandon()	Cancels the current session immediately and releases all the memory it occupied. This is a useful technique in a logoff page to ensure that server memory is reclaimed as quickly as possible.
Clear()	Removes all the session items but doesn't change the current session identifier.

A Session State Example

The next example uses session state to store several Furniture data objects. The data object combines a few related variables and uses a special constructor so it can be created and initialized in one easy line. Rather than use full property procedures, the class takes a shortcut and uses public member variables so that the code listing remains short and concise.

```
public class Furniture
{
public string Name;
public string Description;
public decimal Cost;
public Furniture(string name, string description,
decimal cost)
{
Name = name;
Description = description;
Cost = cost;
}
}
```

Three Furniture objects are created the first time the page is loaded, and they're stored in session state. The user can then choose from a list of furniture piece names. When a selection is made, the corresponding object will be retrieved, and its information will be displayed, as shown in Figure 7.

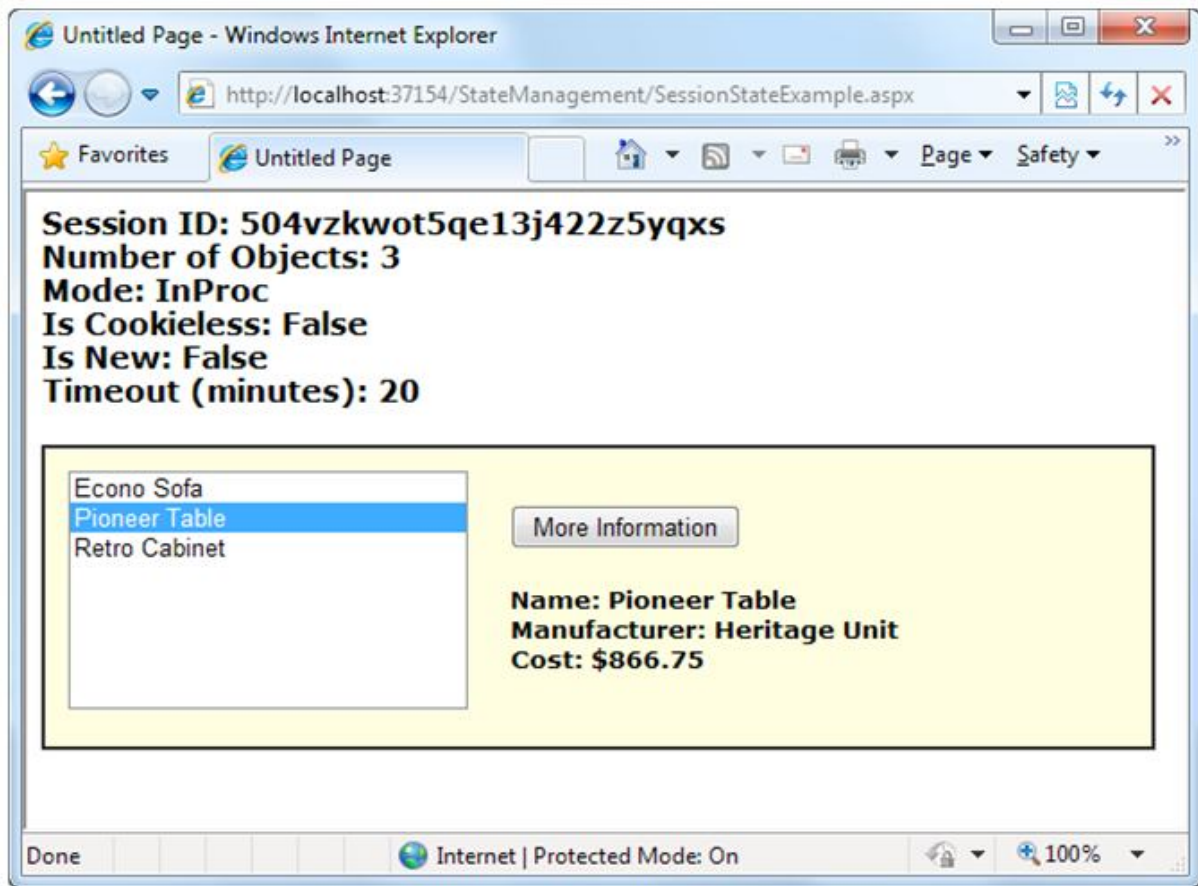


Figure 7. A session state example with data objects

```
public partial class SessionStateExample : System.Web.UI.Page
{
    protected void Page_Load(Object sender, EventArgs e)
    {
        if (!this.IsPostBack)
        {
            // Create Furniture objects.
```

```

Furniture piece1 = new Furniture("Econo Sofa",
    "Acme Inc.", 74.99M);

Furniture piece2 = new Furniture("Pioneer Table",
    "Heritage Unit", 866.75M);

Furniture piece3 = new Furniture("Retro Cabinet",
    "Sixties Ltd.", 300.11M);

// Add objects to session state.

Session["Furniture1"] = piece1;

Session["Furniture2"] = piece2;

Session["Furniture3"] = piece3;

// Add rows to list control.

lstItems.Items.Add(piece1.Name);

lstItems.Items.Add(piece2.Name);

lstItems.Items.Add(piece3.Name);

}

// Display some basic information about the session.

// This is useful for testing configuration settings.

lblSession.Text = "Session ID: " + Session.SessionID;

lblSession.Text += "<br />Number of Objects: ";

lblSession.Text += Session.Count.ToString();

lblSession.Text += "<br />Mode: " + Session.Mode.ToString();

```

252

```
lblSession.Text += "<br />Is Cookieless: ";

lblSession.Text += Session.IsCookieless.ToString();

lblSession.Text += "<br />Is New: ";

lblSession.Text += Session.IsNewSession.ToString();

lblSession.Text += "<br />Timeout (minutes): ";

lblSession.Text += Session.Timeout.ToString();

}

protected void cmdMoreInfo_Click(Object sender, EventArgs e)

{

    if (lstItems.SelectedIndex == -1)

    {

        lblRecord.Text = "No item selected.";

    }

    else

    {

        // Construct the right key name based on the index.

        string key = "Furniture" +

            (lstItems.SelectedIndex + 1).ToString();

        // Retrieve the Furniture object from session state.

        Furniture piece = (Furniture)Session[key];

        // Display the information for this object.

        lblRecord.Text = "Name: " + piece.Name;
```

```

lblRecord.Text += "<br />Manufacturer: ";

lblRecord.Text += piece.Description;

lblRecord.Text += "<br />Cost: " + piece.Cost.ToString("c");

}

}

}

```

It's also a good practice to add a few session-friendly features in your application. For example, you could add a logout button to the page that automatically cancels a session using the `Session.Abandon()` method. This way, the user will be encouraged to terminate the session rather than just close the browser window, and the server memory will be reclaimed faster.

Session State Configuration

You configure session state through the `web.config` file for your current application (which is found in the same virtual directory as the `.aspx` web page files). The configuration file allows you to set advanced options such as the timeout and the session state mode.

The following listing shows the most important options that you can set for the `<sessionState>` element. Keep in mind that you won't use all of these details at the same time. Some settings apply only to certain session state *modes*, as you'll see shortly.

```

<?xml version="1.0" ?>

<configuration>

  <system.web>

    <!-- Other settings omitted. -->

    <sessionState

      cookieless="UseCookies"

      cookieName="ASP.NET_SessionId"

      regenerateExpiredSessionId="false"

```

```

timeout="20"

mode="InProc"

stateConnectionString="tcpip=127.0.0.1:42424"

stateNetworkTimeout="10"

sqlConnectionString="data source=127.0.0.1;Integrated Security=SSPI"

sqlCommandTimeout="30"

allowCustomSqlDatabase="false"

customProvider=""

compressionEnabled="false"

/>

</system.web>

</configuration>

```

Cookieless

You can set the cookieless setting to one of the values defined by the `HttpCookieMode` enumeration, as described in Table.

Table. *HttpCookieMode Values*

Value	Description
UseCookies	Cookies are always used, even if the browser or device doesn't support cookies or they are disabled. This is the default. If the device does not support cookies, session information will be lost over subsequent requests, because each request will get a new ID.
UseUri	Cookies are never used, regardless of the capabilities of the browser or device. Instead, the session ID is stored in the URL.
UseDeviceProfile	ASP.NET chooses whether to use cookieless sessions by examining the <code>BrowserCapabilities</code> object. The drawback is that this object indicates

what the device should support—it doesn't take into account that the user may have disabled cookies in a browser that supports them.

AutoDetect

ASP.NET attempts to determine whether the browser supports cookies by attempting to set and retrieve a cookie (a technique commonly used on the Web). This technique can correctly determine whether a browser supports cookies but has them disabled, in which case cookieless mode is used instead.

Here's an example that forces cookieless mode (which is useful for testing):

```
<sessionState cookieless="UseUri" ... />
```

In cookieless mode, the session ID will automatically be inserted into the URL. When ASP.NET receives a request, it will remove the ID, retrieve the session collection, and forward the request to the appropriate directory. Figure 8 shows a munged URL.



Figure 8. A munged URL with the session ID

Because the session ID is inserted in the current URL, relative links also automatically gain the session ID. In other words, if the user is currently stationed on Page1.aspx and clicks a relative link to Page2.aspx, the relative link includes the current session ID as part of the URL. The same is true if you call `Response.Redirect()` with a relative URL, as shown here:

```
Response.Redirect("Page2.aspx");
```

Figure 9 shows a sample website (included with the online samples in the `CookielessSessions` directory) that tests cookieless sessions. It contains two pages and uses cookieless mode. The first page (`Cookieless1.aspx`) contains a `HyperLink` control and two buttons, all of which take you to a second page (`Cookieless2.aspx`). The trick is that these controls have

different ways of performing their navigation. Only two of them work with cookieless session—the third loses the current session.

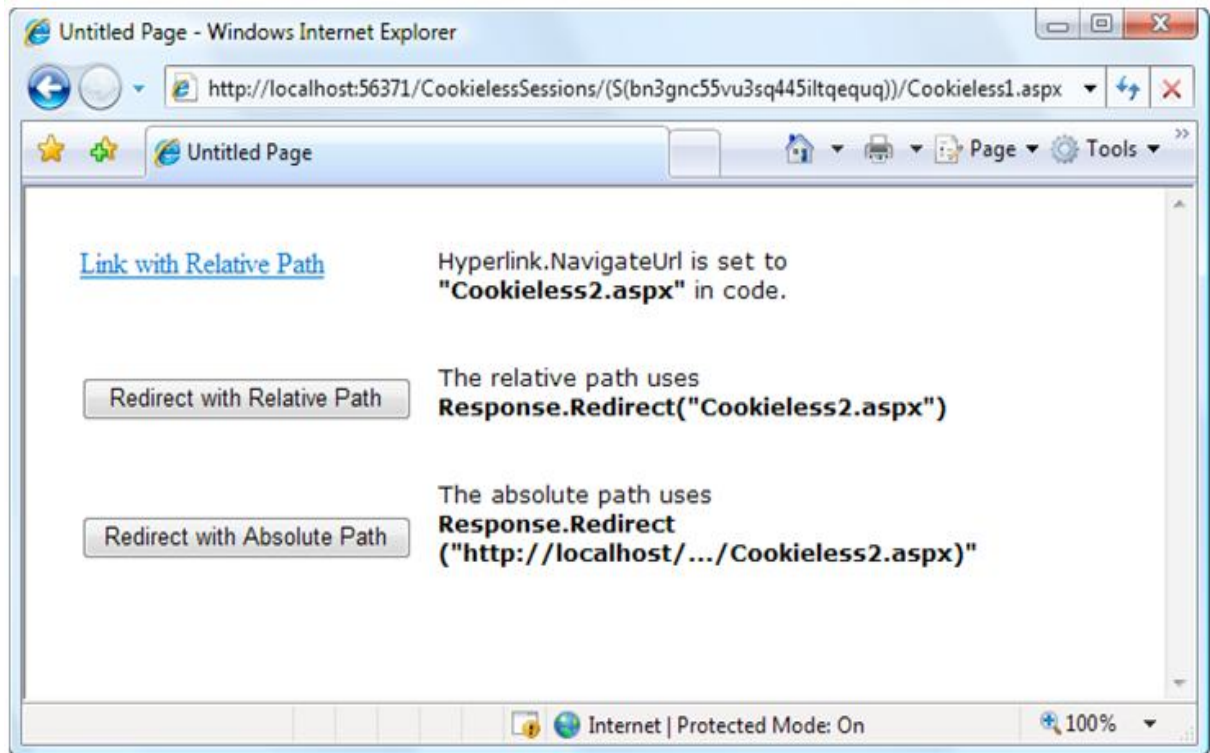


Figure 9. Three tests of cookieless sessions

The HyperLink control navigates to the page specified in its NavigateUrl property, which is set to the relative path `Cookieless2.aspx`. If you click this link, the session ID is retained in the URL, and the new page can retrieve the session information. This demonstrates that cookieless sessions work with relative links. The two buttons on this page use programmatic redirection by calling the `Response.Redirect()` method. The first button uses the relative path `Cookieless2.aspx`, much like the HyperLink control. This approach works with cookieless session state and preserves the munged URL with no extra steps required.

```
protected void cmdLink_Click(Object sender, EventArgs e)
{
    Response.Redirect("Cookieless2.aspx");
}
```

The only real limitation of cookieless state is that you cannot use absolute links (links that include the full URL, starting with `http://`). The second button uses an absolute link to demonstrate this problem. Because ASP.NET cannot insert the session ID into the URL, the session is lost.

```
protected void cmdLinkAbsolute_Click(Object sender, EventArgs e)
{
    Response.Redirect("http://localhost:56371/CookielessSessions/Cookieless2.aspx");
}
```

Now the target page checks for the session information but can't find it.

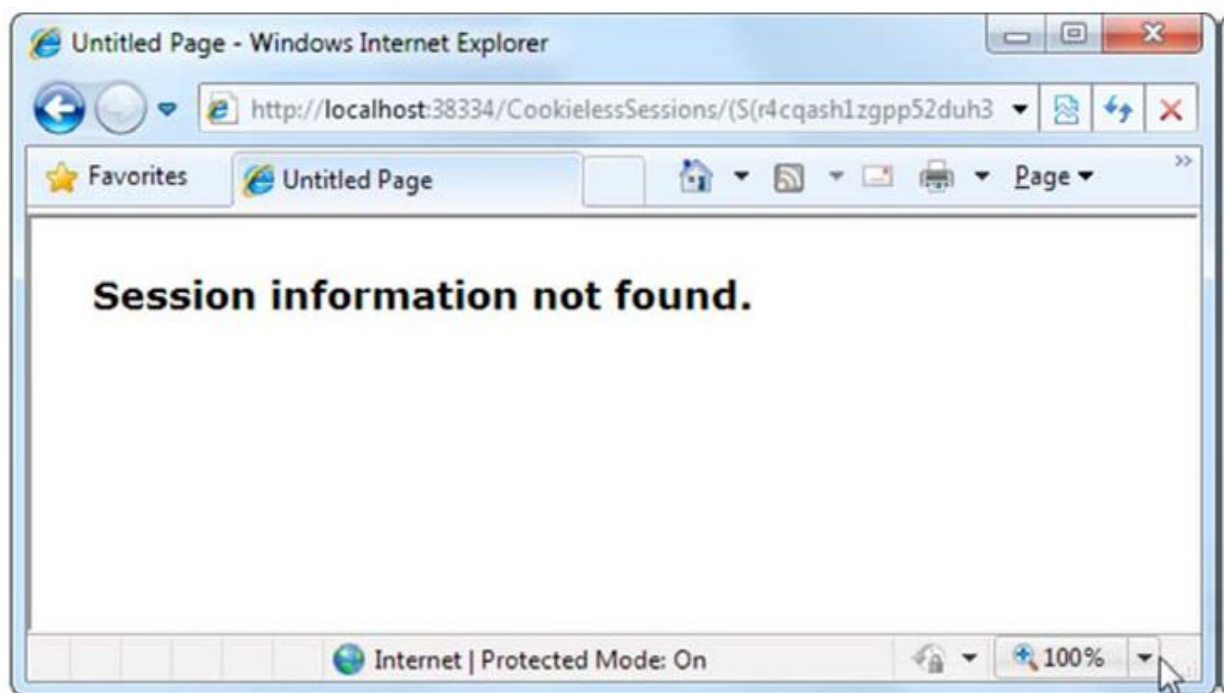


Figure 10. A lost session

Writing the code to demonstrate this problem in a test environment is a bit tricky. The problem is that Visual Studio's integrated web server chooses a different port for your website every time you start it. As a result, you'll need to edit the code every time you open Visual Studio so that your URL uses the right port number (such as 56371 in the previous example). There's another workaround. You can use some crafty code that gets the current URL from the page and just modifies the last part of it (changing the page name from `Cookieless1.aspx` to `Cookieless2.aspx`).

Here's how:

```
// Create a new URL based on the current URL (but ending with
// the page Cookieless2.aspx instead of Cookieless1.aspx.
string url = "http://" + Request.Url.Authority +
Request.Url.Segments[0] + Request.Url.Segments[1] +
"Cookieless2.aspx";
Response.Redirect(url);
```

Of course, if you deploy your website to a real virtual directory that's hosted by IIS, you won't use arbitrarily chosen port number anymore, and you won't experience this quirk.

Timeout

Another important session state setting in the web.config file is the timeout. This specifies the number of minutes that ASP.NET will wait, without receiving a request, before it abandons the session.

This setting represents one of the most important compromises of session state. A difference of minutes can have a dramatic effect on the load of your server and the performance of your application. Ideally, you will choose a timeframe that is short enough to allow the server to reclaim valuable memory after a client stops using the application but long enough to allow a client to pause and continue a session without losing it.

You can also programmatically change the session timeout in code. For example, if you know a session contains an unusually large amount of information, you may need to limit the amount of time the session can be stored. You would then warn the user and change the Timeout property. Here's a sample line of code that changes the timeout to 10 minutes:

```
Session.Timeout = 10;
```

Mode

The remaining session state settings allow you to configure ASP.NET to use different session state services, depending on the mode that you choose. The next few sections describe the modes you can choose from.

InProc

InProc is the default mode, and it makes the most sense for small websites. It instructs information to be stored in the same process as the ASP.NET worker threads, which provides the best performance but the least durability. If you restart your server, the state information will be lost. InProc mode won't work if you're using a *web farm*, which is a load-balancing arrangement that uses multiple web servers to run your website. In this situation, different web servers might handle consecutive requests from the same user. If the web servers use InProc mode, each one will have its own private collection of session data. The end result is that users will unexpectedly lose their sessions when they travel to a new page or post back the current one.

Off

This setting disables session state management for every page in the application. This can provide a slight performance improvement for websites that are not using session state.

StateServer

With this setting, ASP.NET will use a separate Windows service for state management. This service runs on the same web server, but it's outside the main ASP.NET process, which gives it a basic level of protection if the ASP.NET process needs to be restarted. The cost is the increased time delay imposed when state information is transferred between two processes. If you frequently access and change state information, this can make for a fairly unwelcome slowdown. When using the StateServer setting, you need to specify a value for the `stateConnectionString` setting. This string identifies the TCP/IP address of the computer that is running the StateServer service and its port number. This allows you to host the StateServer on another computer. If you don't change this setting, the local server will be used. Of course, before your application can use the service, you need to start it. The easiest way to do this is to use the Microsoft Management Console (MMC). Here's how:

1. *Select Start > Control Panel.*
2. *Open the Administrative Tools group, and then choose Computer Management.*
3. *In the Computer Management tool, go to the Services and Applications > Services node.*
4. *Find the service called ASP.NET State Service in the list, as shown in Figure 11*

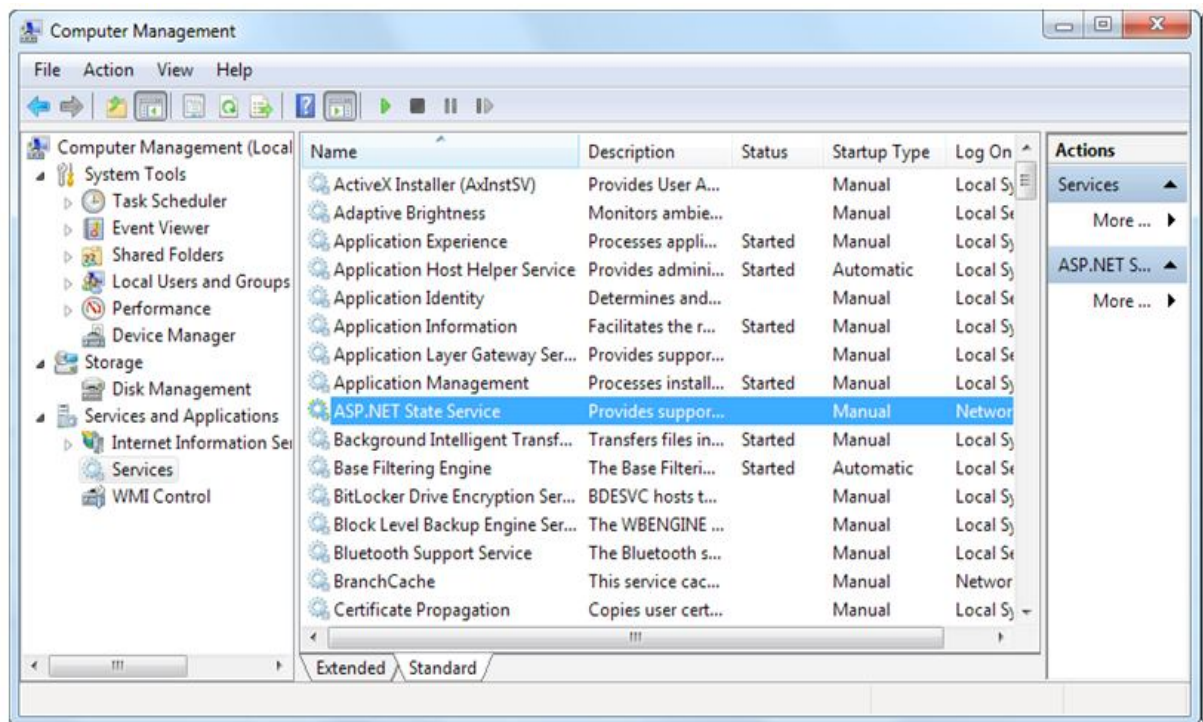


Figure 11. The ASP.NET state service

5. Once you find the service in the list, you can manually start and stop it by right-clicking it. Generally, you'll want to configure Windows to automatically start the service. Right-click it, select Properties, and modify the Startup Type, setting it to Automatic, as shown in Figure 12.

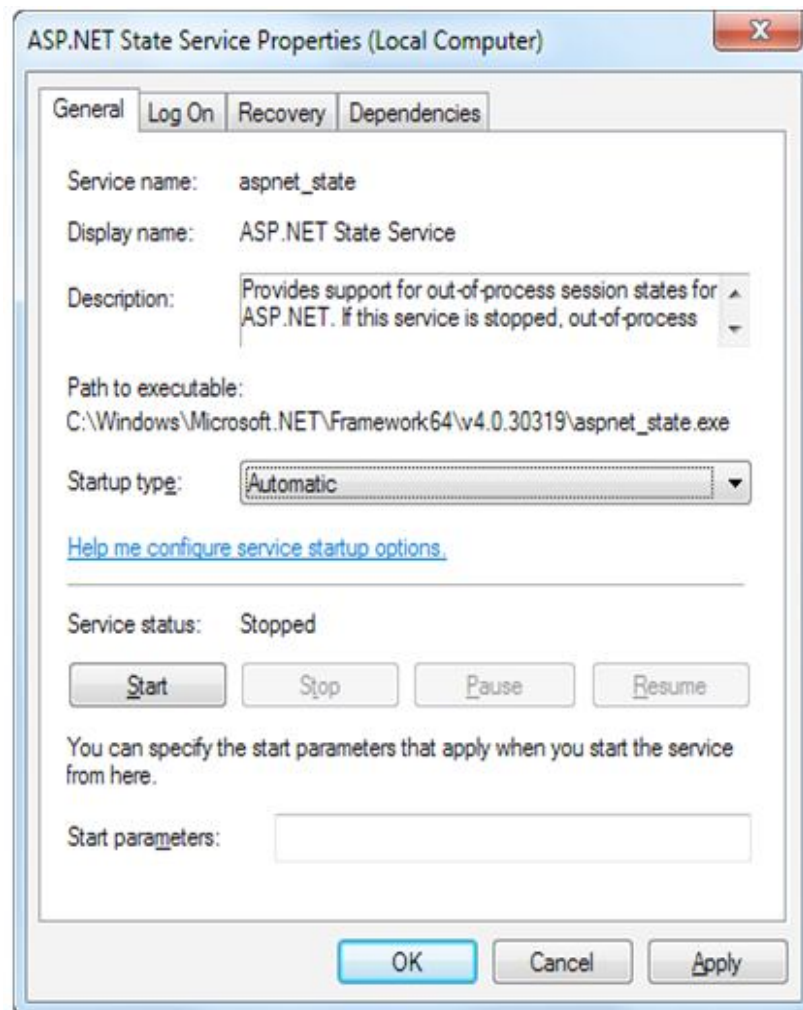


Figure 12. Changing the startup type

6.8 SQLServer

This setting instructs ASP.NET to use an SQL Server database to store session information, as identified by the `sqlConnectionString` attribute. This is the most resilient state store but also the slowest by far. To use this method of state management, you'll need to have a server with SQL Server installed.

When setting the `sqlConnectionString` attribute, you follow the same sort of pattern you use with ADO.NET data access. Generally, you'll need to specify a data source (the server address) and a user ID and password, unless you're using SQL integrated security.

In addition, you need to install the special stored procedures and temporary session databases. These stored procedures take care of storing and retrieving the session information.

ASP.NET includes a command-line tool that does the work for you automatically, called `aspnet_regsql.exe`. It's found in the `c:\Windows\Microsoft.NET\Framework\[Version]` directory (where [Version] is the current version of .NET, such as v4.0.30319). The easiest way to run `aspnet_regsql.exe` is to start by launching the Visual Studio command prompt (open the Start menu and choose Programs → Visual Studio 2010 → Visual

Studio Tools → Visual Studio Command Prompt). You can then type in an `aspnet_regsql.exe` command, no matter what directory you're in.

To use `aspnet_regsql.exe` to create a session storage database, you supply the `-ssadd` parameter. In addition, you use the `-S` parameter to indicate the database server name, and the `-E` parameter to log in to the database using the currently logged-in Windows user account.

Here's a command that creates the session storage database on the current computer, using the default database name `ASPState`:

```
aspnet_regsql.exe -S localhost -E -ssadd
```

This command uses the alias `localhost`, which tells `aspnet_regsql.exe` to connect to the database server on the current computer.

Once you've created your session state database, you need to tell ASP.NET to use it by modifying the `<sessionState>` section of the `web.config` file. If you're using a database named `ASPState` to store your session information (which is the default), you don't need to supply the database name. Instead, you simply have to indicate the location of the server and the type of authentication that ASP.NET should use to connect to it, as shown here:

```
<sessionState mode="SQLServer"
```

```
sqlConnectionString="data source=127.0.0.1;Integrated Security=SSPI"
```

```
... />
```

When using the `SQLServer` mode, you can also set an optional `sqlCommandTimeout` attribute that specifies the maximum number of seconds to wait for the database to respond before canceling the request. The default is 30 seconds.

Custom

When using custom mode, you need to indicate which session state store provider to use by supplying the `customProvider` attribute. The `customProvider` attribute indicates the name of the class. The class may be part of your web application (in which case the source code is placed in the `App_Code` subfolder), or it can be in an assembly that your web application is using (in which case the compiled assembly is placed in the `Bin` subfolder).

Creating a custom state provider is a low-level task that needs to be handled carefully to ensure security, stability, and scalability. Custom state providers are also beyond the scope of this book. However, other vendors may release custom state providers you want to use. For example, Oracle could provide a custom state provider that allows you to store state information in an Oracle database.

Compression

When you set `enableCompression` to true, session data is compressed before it's passed out of process. The `enableCompression` setting only has an effect when you're using out-of-process session state storage, because it's only in this situation that the data is serialized.

To compress and decompress session data, the web server needs to perform additional work. However, this isn't usually a problem, because compression is used in scenarios where web servers have plenty of CPU time to spare but are limited by other factors. There are two key scenarios where `sessionState.compression` makes sense:

When storing huge amounts of session state data in memory: Web server memory is a precious resource. Ideally, session state is used for relatively small chunks of information, while a database deals with the long-term storage of larger amounts of data. But if this isn't the case and if the out-of-process state server is hogging huge amounts of memory, compression is a potential solution.

When storing session state data on another computer: In some large-scale web applications, session state is stored out of process (usually in SQL Server) and on a separate computer. As a result, ASP.NET needs to pass the session information back and forth over a network connection. Clearly, this design reduces performance from the speeds you'll see when session state is stored on the webserver computer.

However, it's still the best compromise for some heavily trafficked webapplications with huge session state storage needs.

The actual amount of compression varies greatly depending on the type of data, but in testingMicrosoft saw clients achieve 30 percent to 60 percent size reductions, which is enough to improveperformance in these specialized scenarios.

Application State

Application state allows you to store global objects that can be accessed by any client. Application stateis based on the `System.Web.HttpApplicationState` class, which is provided in all web pages through thebuilt-in `Application` object.

Application state is similar to session state. It supports the same type of objects, retains informationon the server, and uses the same dictionary-based syntax. A common example with application state isa global counter that tracks how many times an operation has been performed by all the webapplication's clients.

For example, you could create a global.aspx event handler that tracks how many sessions have beencreated or how many requests have been received into the application. Or you can use similar logic inthe `Page.Load` event handler to track how many times a given page has been requested by variousclients. Here's an example of the latter:

```
protected void Page_Load(Object sender, EventArgs e)

{

// Retrieve the current counter value.

    int count = 0;

    if (Application["HitCounterForOrderPage"] != null)
    {

        count = (int)Application["HitCounterForOrderPage"];

    }

    // Increment the counter.
```

```

count++;

// Store the current counter value.

Application["HitCounterForOrderPage"] = count;

lblCounter.Text = count.ToString();

}

```

Once again, application state items are stored as objects, so you need to cast them when you retrieve them from the collection. Items in application state never time out. They last until the application or server is restarted or the application domain refreshes itself (because of automatic process recycling settings or an update to one of the pages or components in the application).

Application state isn't often used, because it's generally inefficient. In the previous example, the counter would probably not keep an accurate count, particularly in times of heavy traffic. For example, if two clients requested the page at the same time, you could have a sequence of events like this:

1. User A retrieves the current count (432).
2. User B retrieves the current count (432).
3. User A sets the current count to 433.
4. User B sets the current count to 433.

In other words, one request isn't counted because two clients access the counter at the same time. To prevent this problem, you need to use the `Lock()` and `Unlock()` methods, which explicitly allow only one client to access the Application state collection at a time.

```

protected void Page_Load(Object sender, EventArgs e)
{

// Acquire exclusive access.

Application.Lock();

```

```

int count = 0;

if (Application["HitCounterForOrderPage"] != null)
{
    count = (int)Application["HitCounterForOrderPage"];
}

count++;

Application["HitCounterForOrderPage"] = count;

// Release exclusive access.

Application.Unlock();

lblCounter.Text = count.ToString();
}

```

Unfortunately, all other clients requesting the page will be stalled until the Application collection is released. This can drastically reduce performance. Generally, frequently modified values are poor candidates for application state. In fact, application state is rarely used in the .NET world because its two most common uses have been replaced by easier, more efficient methods:

- In the past, application state was used to store application-wide constants, such as a database connection string. This type of constant can be stored in the web.config file, which is generally more flexible because you can change it easily without needing to hunt through web page code or recompile your application.
- Application state can also be used to store frequently used information that is time-consuming to create, such as a full product catalog that requires a database lookup. However, using application state to store this kind of information raises all sorts of problems about how to check whether the data is valid and how to replace it when needed. It can also hamper performance if the product catalog is too large.

6.9 Validation

The *validationcontrols*. These controls take a previously time-consuming and complicated task—verifying user input and reporting errors—and automate it with an elegant, easy-to-use collection of validators. Each validator has its own built-in logic. Some check for missing data, others verify that numbers fall in a predefined range, and so on. In many cases, the validation controls allow you to verify user input without writing a line of code.

Understanding Validation

As a seasoned developer, you probably realize users will make mistakes. What's particularly daunting is the range of possible mistakes that users can make. Here are some common examples: Users might ignore an important field and leave it blank.

- Users might try to type a short string of nonsense to circumvent a required field check, thereby creating endless headaches on your end. For example, you might get stuck with an invalid e-mail address that causes problems for your automatic e-mailing program.
- Users might make an honest mistake, such as entering a typing error, entering a nonnumeric character in a number field, or submitting the wrong type of information. They might even enter several pieces of information that are individually correct but when taken together are inconsistent (for example, entering a MasterCard number after choosing Visa as the payment type).
- Malicious users might try to exploit a weakness in your code by entering carefully structured wrong values. For example, they might attempt to cause a specific error that will reveal sensitive information. A more dramatic example of this technique is the *SQL injection attack*, where user-supplied values change the operation of a dynamically constructed database command.

A web application is particularly susceptible to these problems, because it relies on basic HTML input controls that don't have all the features of their Windows counterparts. For example, a common technique in a Windows application is to handle the `KeyPress` event of a text box, check to see whether the current character is valid, and prevent it from appearing if it isn't. This technique makes it easy to create a text box that accepts only numeric input.

This strategy isn't as easy in a server-side web page. To perform validation on the web server, you need to post back the page, and it just isn't practical to post the page back to the server every time the user types a letter. To avoid this sort of problem, you need to perform all your validation at once when a page (which may contain multiple input controls) is submitted. You

then need to create the appropriate user interface to report the mistakes. Some websites report only the first incorrect field, while others use a table, list, or window to describe them all. By the time you've perfected your validation strategy, you'll have spent a considerable amount of effort writing tedious code.

ASP.NET aims to save you this trouble and provide you with a reusable framework of validation controls that manages validation details by checking fields and reporting on errors automatically. These controls can even use client-side JavaScript to provide a more dynamic and responsive interface while still providing ordinary validation for older browsers (often referred to as *down-level* browsers).

6.10 The Validation Controls

ASP.NET provides five validator controls, which are described in Table. Four are targeted at specific types of validation, while the fifth allows you to apply custom validation routines. You'll also see a `ValidationSummary` control in the Toolbox, which gives you another option for showing a list of validation error messages in one place. You'll learn about the `ValidationSummary` later in this chapter (see the "Other Display Options" section).

Table. *Validator Controls*

Control Class	Description
<code>RequiredFieldValidator</code>	Validation succeeds as long as the input control doesn't contain an empty string.
<code>RangeValidator</code>	Validation succeeds if the input control contains a value within a specific numeric, alphabetic, or date range.
<code>CompareValidator</code>	Validation succeeds if the input control contains a value that matches the value in another input control, or a fixed value that you specify.
<code>RegularExpressionValidator</code>	Validation succeeds if the value in an input control matches a specified regular expression.
<code>CustomValidator</code>	Validation is performed by a user-defined function.

Each validation control can be bound to a single input control. In addition, you can apply more than one validation control to the same input control to provide multiple types of validation. You use the `RangeValidator`, `CompareValidator`, or `RegularExpressionValidator`, validation will automatically succeed if the input control is empty, because there is no value to validate. If this isn't the behavior you want, you should also add a `RequiredFieldValidator` and link it to the same input control.

This ensures that two types of validation will be performed, effectively restricting blank values.

Server-Side Validation

You can use the validator controls to verify a page automatically when the user submits it or manually in your code. The first approach is the most common. When using automatic validation, the user receives a normal page and begins to fill in the input controls. When finished, the user clicks a button to submit the page. Every button has a `CausesValidation` property, which can be set to true or false. What happens when the user clicks the button depends on the value of the `CausesValidation` property:

- If `CausesValidation` is false, ASP.NET will ignore the validation controls, the page will be posted back, and your event handling code will run normally.
- If `CausesValidation` is true (the default), ASP.NET will automatically validate the page when the user clicks the button. It does this by performing the validation for each control on the page. If any control fails to validate, ASP.NET will return the

page with some error information, depending on your settings. Your click event handling code may or may not be executed—meaning you'll have to specifically check in the event handler whether the page is valid.

Client-Side Validation

In modern browsers (including Internet Explorer, Firefox, Safari, and Google Chrome), ASP.NET automatically adds JavaScript code for client-side validation. In this case, when the user clicks a `CausesValidation` button, the same error messages will appear without the page needing to be submitted and returned from the server. This increases the responsiveness of your web page. However, even if the page validates successfully on the client side, ASP.NET still revalidates it when it's received at the server. This is because it's easy for an experienced user to circumvent client-side

validation. For example, a malicious user might delete the block of JavaScript validation code and continue working with the page. By performing the validation at both ends, ASP.NET makes sure your application can be as responsive as possible while also remaining secure.

The Validation Controls

The validation controls are found in the `System.Web.UI.WebControls` namespace and inherit from the `BaseValidator` class. This class defines the basic functionality for a validation control. Table describes its key properties.

Table. *Properties of the BaseValidator Class*

Property	Description
ControlToValidate	Identifies the control that this validator will check. Each validator can verify the value in one input control. However, it's perfectly reasonable to "stack" validators—in other words, attach several validators to one input control to perform more than one type of error checking.
ErrorMessage and ForeColor	If validation fails, the validator control can display a text message (set by the <code>ErrorMessage</code> property). By changing the <code>ForeColor</code> , you can make this message stand out in angry red lettering.
Display	Allows you to configure whether this error message will be inserted into the page dynamically when it's needed (Dynamic) or whether an appropriate space will be reserved for the message (Static). Dynamic is useful when you're replacing several validators next to each other. That way, the space will expand to fit the currently active error indicators, and you won't be left with any unseemly whitespace. Static is useful when the validator is in a table and you don't want the width of the cell to collapse when no message is displayed. Finally, you can also choose <code>None</code> to hide the error message altogether.
IsValid	After validation is performed, this returns true or false depending on whether it succeeded or failed. Generally, you'll check the state of the entire page by looking at its <code>IsValid</code> property instead to find out if all the validation controls succeeded.

Enabled	When set to false, automatic validation will not be performed for this control when the page is submitted.
EnableClientScript	If set to true, ASP.NET will add JavaScript and DHTML code to allow client-side validation on browsers that support it.

When using a validation control, the only properties you need to implement are `ControlToValidate` and `ErrorMessage`. In addition, you may need to implement the properties that are used for your specific validator. Table outlines these properties.

Table. *Validator-Specific Properties*

Validator Control	Added Members
RequiredFieldValidator	None required
RangeValidator	MaximumValue, MinimumValue, Type
CompareValidator	ControlToCompare, Operator, Type, ValueToCompare
RegularExpressionValidator	ValidationExpression
CustomValidator	ClientValidationFunction, ValidateEmptyText, ServerValidate event

6.10 Summary

State management is the art of retaining information between requests. Usually, this information is user-specific (such as a list of items in a shopping cart, a user name, or an access level), but sometimes it's global to the whole application (such as usage statistics that track site activity). Because ASP.NET uses a disconnected architecture, you need to explicitly store and retrieve state information with each request. The approach you choose to store this data can dramatically affect the performance, scalability, and security of your application.

Keywords

View state, Cookie, session state, Application state, validation control.

6.11 Check your understanding here

1. Which object in ASP.NET provides a global storage mechanism for state data that needs to be accessible to all pages in a given Web application?
 - A. Session
 - B. **Application**
 - C. ViewState
 - D. None of the above
2. What is/are the advantages of Session State
 - A. It helps to maintain user data to all over the application and can store any kind of object.
 - B. Stores every client data separately.
 - C. Session is secure and transparent from user.
 - D. **All of the above**
3. What are the client-side state management options that ASP.NET supports?
 - A. Application
 - B. Session
 - C. **QueryString**
 - D. Option a and b are correct
4. When a User's Session times out which event should you respond to?
 - A. Application_Start
 - B. **Session_End**
 - C. Session_Start
 - D. Application_End

6.12 Model Questions

1. What is Session Identifier?
2. How to transfer information between pages?
3. What is cookie?
4. Write notes on validation controls
5. What is view state? Explain

Reference Books

1. Mathew Mac Donald, "Beginning ASP.NET 4 in C# 2010" Apress 2010

LESSON - 7

RICH CONTROLS, USER CONTROL AND GRAPHICS

Structure

- 7.1 Introduction
- 7.2 Objectives
- 7.3 Rich Controls
- 7.4 User controls
- 7.5 Dynamic Graphics
- 7.6 Summary
- 7.7 Check your progress
- 7.8 Model Questions

7.1 Introduction

In this chapter, you'll start with the Calendar, which provides slick date-selection functionality. Next, you'll consider the AdRotator, which gives you an easy way to insert a randomly selected image into a web page. Then how to create sophisticated pages with multiple views using two advanced container controls: the MultiView and the Wizard. Finally, user control and graphics.

7.2 Learning Objectives



Work efficiently with calendar features.

AdRotator, gives you an easy way to insert a randomly selected image into a web page.

The user can create a multipreview using, the MultiView and the Wizard.

The user able to use common user interfaces across asp.net web application.

The user can draw by using graphics by using its properties.

7.3 Rich Controls

ASP.NET provides large set of controls. These controls are divided into different categories, depends upon their functionalities. The followings control comes under the rich control's category.

CALENDAR CONTROL

Calendar control provides you lots of property and events. By using these properties and events you can perform the following task with calendar control.

- Selecting the date
- Formatting the calendar
- Restricting the date

Selecting the date

You set the type of selection through the `Calendar.SelectionMode` property. You may also need to set the `Calendar.FirstDayOfWeek` property to configure how a week is selected. When you allow multiple date selection, you need to examine the `SelectedDates` property, which provides a collection of all the selected dates. You can loop through this collection using the `foreach` syntax. The following code demonstrates this technique:

```
lblDates.Text = "You selected these dates:";

foreach (DateTime dt in MyCalendar.SelectedDates)

{ lblDates.Text += dt.ToLongDateString() + " "; }
```

Formatting the Calendar

The Calendar control provides a whole host of formatting-related properties. Calendar control provides a whole host of formatting-related properties. The following table1 represent the *Properties for Calendar Styles*.

Table1 represent the *Properties for Calendar Styles*.

Member	Description
DayHeaderStyle	The style for the section of the Calendar that displays the days of the week.
DayStyle	The default style for the dates in the current month.
NextPrevStyle	The style for the navigation controls in the title section that move from month to month.
OtherMonthDayStyle	The style for the dates that aren't in the currently displayed month. These dates are used to "fill in" the calendar grid.
SelectedDayStyle	The style for the selected dates on the calendar.
SelectorStyle	<i>The</i> style for the week and month date selection controls.
TitleStyle	The style for the title section.
TodayDayStyle	The style for the date designated as today (represented by the TodayDate property of the Calendar control).
WeekendDayStyle	The style for dates that fall on the weekend.

7.3.1. Restricting the date

The DayRender event is extremely powerful. Besides allowing you to tailor what dates are selectable, it also allows you to configure the cell where the date is located through the e.Cell property.

For example, you could highlight an important date or even add information. Here's an example that highlights a single day—the fifth of May—by adding a new Label control in the table cell for that day:

```
protected void MyCalendar_DayRender(Object source, DayRenderEventArgs e)----
{
```

```
// Check for May 5 in any year, and format it.

if (e.Day.Date.Day == 5 && e.Day.Date.Month == 5)
{
    e.Cell.BackColor = System.Drawing.Color.Yellow;

    // Add some static text to the cell.

    Label lbl = new Label();

    lbl.Text = "<br />My Birthday!";

    e.Cell.Controls.Add(lbl);
}
}
```

Figure1 shows the resulting calendar display.



Figure1 .Highlighting a day

The Calendar control provides two other useful events: Selection Changed and Visible Month Changed. These occur immediately after the user selects a new day or browses to a new month (using the next month and previous month links). You can react to these events and update other portions of the web page to correspond to the current calendar month.

For example, you could design a page that lets you schedule a meeting in two steps. First, you choose the appropriate day. Then, you choose one of the available times on that day. The following code demonstrates this approach, using a different set of time values if a Monday is selected in the calendar than it does for other days:

```
protected void MyCalendar_SelectionChanged(Object source, EventArgs e)
{
    lstTimes.Items.Clear();

    switch (MyCalendar.SelectedDate.DayOfWeek)
    {
        case DayOfWeek.Monday:
            // Apply special Monday schedule.
            lstTimes.Items.Add("10:00");
            lstTimes.Items.Add("10:30");
            lstTimes.Items.Add("11:00");
            break;
        default:
            lstTimes.Items.Add("10:00");
            lstTimes.Items.Add("10:30");
            lstTimes.Items.Add("11:00");
            lstTimes.Items.Add("11:30");
            lstTimes.Items.Add("12:00");
    }
}
```



```

lstTimes.Items.Add("12:30");

break;

}

}

```

To try these features of the Calendar control, run the Appointment.aspx page from the onlinesamples. This page provides a formatted Calendar control that restricts some dates, formats others specially, and updates a corresponding list control when the selection changes.

Table gives you an at-a-glance look at almost all the members of the Calendar control class.

Table2. *Calendar Members*

Member	Description
Caption and CaptionAlign	Gives you an easy way to add a title to the calendar. By default, the caption appears at the top of the title area, just above the month heading. However, you can control this to some extent with the CaptionAlign property.
CellPadding	ASP.NET creates a date in a separate cell of an invisible table. Cell Padding is the space, in pixels, between the border of each cell and its contents.
CellSpacing	The space, in pixels, between cells in the same table.
DayNameFormat	Determines how days are displayed in the calendar header.
FirstDayOfWeek	Determines which day is displayed in the first column of the calendar. The values are any day name from the FirstDayOfWeek enumeration (such as Sunday).
NextMonthText andPrevMonthText	NextPrevFormat is set to CustomText. NextPrevFormat Sets the text that the user clicks to move to the next or previous month.

SelectedDate and SelectedDates	Sets or gets the currently selected date as a DateTime object. If you allow multiple date selection, the SelectedDates property will return a collection of DateTime objects, one for each selected date. You can use collection methods such as Add, Remove, and Clear to change the selection.
SelectionMode	Determines how many dates can be selected at once. The default is Day, which allows one date to be selected.
SelectMonthText andSelectWeekText	The text shown for the link that allows the user to select an entire month or week. These properties don't apply if the SelectionMode is Day.
ShowDayHeader, ShowGridLines, ShowNextPrevMonth, and ShowTitle	<i>These</i> Boolean properties allow you to configure whether various parts of the calendar are shown, including the day titles, gridlines between every day, the previous/next month navigation links, and the title section.
TitleFormat	Configures how the month is displayed in the title area.
TodaysDate	Sets which day should be recognized as the current date and formatted with the TodayDayStyle.
VisibleDate	Gets or sets the date that specifies what month will be displayed in the calendar.
DayRender Event	Occurs once for each day that is created and added to the currently visible month before the page is rendered.
Selection Changedevent	Occurs when the user selects a day, a week, or an entire month by clicking the date selector controls.
VisibleMonth Changedevent	Occurs when the user clicks the next or previous month navigation controls to move to another month.

DayHeaderStyle	The style for the section of the Calendar that displays the days of the week (as column headers).
DayStyle	The default style for the dates in the current month.
NextPrevStyle	The style for the navigation controls in the title section that move from month to month.
OtherMonthDayStyle	The style for the dates that aren't in the currently displayed month. These dates are used to "fill in" the calendar grid. For example, the first few cells in the topmost row may display the last few days from the previous month.
SelectedDayStyle	The style for the selected dates on the calendar.
SelectorStyle	<i>The</i> style for the week and month date selection controls.
TitleStyle	The style for the title section.
TodayDayStyle	The style for the date designated as today.
WeekendDayStyle	The style for dates that fall on the weekend.

THE ADROTATOR:

AdRotator control is used to display different advertisements randomly in a page. The list of advertisements is stored in either an XML file or in a database table. Lots of websites uses AdRotator control to display the advertisements on the web page.

To create an advertisement list, first add an XML file to your project.

```
<?xml version="1.0" encoding="utf-8" ?>
```

```
<Advertisements>
```

```
  <Ad>
```

```
    <ImageUrl><" /Images/logo1.png</ImageUrl>
```

```
    <NavigateUrl>http://www.xxxxx.com</NavigateUrl>
```

<AlternateText>Advertisement</AlternateText>

<Impressions>100</Impressions>

<Keyword>banner</Keyword>

</Ad>

<Ad>

<ImageUrl><“ /Images/logo2.png</ImageUrl>

<NavigateUrl>http://www.xxxxx.com</NavigateUrl>

<AlternateText>Advertisement</AlternateText>

<Impressions>100</Impressions>

<Keyword>banner</Keyword>

</Ad>

<Ad>

<ImageUrl><“ /Images/logo3.png</ImageUrl>

<NavigateUrl>http://www.xxxxx.com</NavigateUrl>

<AlternateText>Advertisement</AlternateText>

<Impressions>100</Impressions>

<Keyword>banner</Keyword>

</Ad>

<Ad>

<ImageUrl><“ /Images/logo4.png</ImageUrl>

<NavigateUrl>http://www.xxxxx.com</NavigateUrl>

<AlternateText>Advertisement</AlternateText>

<Impressions>50</Impressions>

```
<Keyword>banner</Keyword>
```

```
</Ad>
```

```
</Advertisements>
```

In the given XML file 'Images' is the name of the folder, where we stored all the images to display. Now set the AdRotator control's AdvertisementFile property. Set the path of the XML file that you created above to AdRotator control's AdvertisementFile property.

Table 3: AdvertisementFileelement

Element	Description
ImageUrl	The image that will be displayed. This can be a relative link (a file in the current directory) or a fully qualified Internet URL.
NavigateUrl	The link that will be followed if the user clicks the banner. This can be a relative or fully qualified URL.
AlternateText	The text that will be displayed instead of the picture if it cannot be displayed. This text will also be used as a tooltip in some newer browsers.
Impressions	A number that sets how often an advertisement will appear. This number is relative to the numbers specified for other ads.
Keyword	A keyword that identifies a group of advertisements.

7.3.2. The AdRotator Class

The actual AdRotator class provides a limited set of properties. The following table represent Special Frame Targets:

Table 4: special frame targets

Target	Description
_blank	The link opens a new unframed window.
_parent	The link opens in the parent of the current frame.

`_self` The link opens in the current frame.

`_top` The link opens in the topmost frame of the current window

PAGES WITH MULTIPLE VIEWS:

MultiView control can be used when you want to create a tabbed page. In many situations, a web form may be very long, and then you can divide a long form into multiple sub forms. MultiView control is made up of multiple view controls. You can put multiple ASP.NET controls inside view controls. One View control is displayed at a time and it is called as the active view. View control does not work separately. It is always used with a Multiview control.

If working with Visual Studio 2010 or later, you can drag and drop a MultiView control onto the form. You can drag and drop any number of View controls inside the MultiView control. The number of view controls is depending upon the need of your application.

7.3.3. The MultiView control:

The MultiView control supports the following important properties:

ActiveViewIndex:

It is used to determine which view will be active or visible.

7.3.4. Views

It provides the collection of View controls contained in the MultiView control. For understand the Multiview control, first we will create a user interface as given below. In the given example, in Multiview control, we have taken three separate View control.

1. In First step we will design to capture Product details
2. In Second step we will design to capture Order details
3. Next, we will show summary for confirmation.

Here we have not used any database programming to save the data into the database. We will show only the confirmation page, that data has been saved. Figure2: Multi Views

MultiView1	
Step 1 - Product Details Product ID Product Name Price/Unit Next >>	View1
Step 2 - Order Details Order ID Quantity Previous << Next >>	View2
Step 3 - Summary Product Details Product ID : [lblProductID] Product Name : [lblProductName] Price/Unit : [lblPrice] Order Details Order ID : [lblOrderID] Quantity : [lblQuantity] Previous << Submit >>	View3

MultiViewControlDemo.aspx file

```
<%@ Page Language="C#" AutoEventWireup="true" CodeFile="RichControl.aspx.cs"
Inherits="RichControl" %>
```

```
<! DOCTYPE html>
```

```
<html xmlns="http://www.w3.org/1999/xhtml">
```

```
<head runat="server">
```

```
<title></title>
```

```
</head>
```

```
<body>
```

```
<form id="form1" runat="server">
```

```
<div>
```

```
<asp:MultiView ID="MultiView1" runat="server">
```

```
<asp:View ID="View1" runat="server">
```

```

<table style="border:1px solid black">

<tr>

    <td colspan="2">

        <h2>Step 1 - Product Details</h2>

    </td> </tr> <tr>

    <td>Product ID</td>

    <td>

        <asp:TextBox ID="txtProductID" runat="server"></asp:TextBox>

    </td> </tr>

    <td>Product Name</td>

    <td>

        <asp:TextBox ID="txtProductName" runat="server"></asp:TextBox>

    </td> </tr>

    <td>Price/Unit</td>

    <td>

        <asp:TextBox ID="txtProductPrice" runat="server"></asp:TextBox>

    </td> </tr>

    <td colspan="2" style="text-align:right">

        <asp:Button ID="btnStep2" runat="server"

            Text="Next >>" onclick="btnStep2_Click" />

    </td>

```



```

    </tr>

</table>

</asp:View>

<asp:View ID="View2" runat="server">

    <table style="border:1px solid black">

<tr>

    <td colspan="2">

        <h2>Step 2 - Order Details</h2>

    </td> </tr>

    <tr>

        <td>Order ID</td>

        <td>

            <asp:TextBox ID="txtOrderID" runat="server"></asp:TextBox>

        </td> </tr>

        <tr>

            <td>Quantity</td>

            <td>

                <asp:TextBox ID="txtQuantity" runat="server"></asp:TextBox>

            </td> </tr>

            <tr>

                <td>

                    <asp:Button ID="btnBackToStep1" runat="server" Text=" << Previous"

                        onclick="btnBackToStep1_Click" />

                </td>

```

```

<td style="text-align:right">

    <asp:Button ID="btnStep3" runat="server" Text="Next >>"

        onclick="btnGoToStep3_Click" />

</td>        </tr>

</table>

</asp:View>

<asp:View ID="View3" runat="server">

    <table style="border: 1px solid black">

        <tr>

            <td colspan="2"><h2>Step 3 - Summary</h2></td>

        </tr>        <tr>

            <td colspan="2"><h3>Product Details</h3></td>

        </tr>        <tr>

            <td>Product ID</td>

            <td>

                <asp:Label ID="lblProductID" runat="server"></asp:Label>

            </td> </tr>        <tr>

            <td>Product Name</td>

            <td>

                <asp:Label ID="lblProductName" runat="server"></asp:Label>

            </td> </tr>        <tr>

            <td>Price/Unit</td>

            <td>


```

```

        <asp:Label ID="lblPrice" runat="server"></asp:Label>

    </td> </tr>    <tr>

    <td colspan="2"><h3>Order Details</h3></td>

</tr>    <tr>

    <td>Order ID</td>

    <td>

        <asp:Label ID="lblOrderID" runat="server"></asp:Label>

    </td> </tr>    <tr>

    <td>Quantity</td>

    <td>

        <asp:Label ID="lblQuantity" runat="server"></asp:Label>

    </td> </tr>    <tr>    <td>

        <asp:Button ID="btnBackToStep2" runat="server" OnClick="btnBackToStep2_Click"
style="height: 26px" Text="<<Previous" />

    </td>

    <td style="text-align:right">

        <asp:Button ID="btnSubmit" runat="server" Text="Submit >>"
OnClick="btnSubmit_Click"/>

    </td>    </tr>

</table>

</asp:View>    </asp:MultiView>

</div>    </form>

</body></html>

```

MultiViewControlDemo.aspx.cs file

```
using System;

using System.Text;

public partial class RichControl : System.Web.UI.Page
{
    protected void Page_Load(object sender, EventArgs e)
    {
        if (! IsPostBack)
        {
            MultiView1.ActiveViewIndex = 0;
        }
    }

    protected void btnStep2_Click(object sender, EventArgs e)
    {
        MultiView1.ActiveViewIndex = 1;
    }

    protected void btnBackToStep1_Click(object sender, EventArgs e)
    {
        MultiView1.ActiveViewIndex = 0;
    }

    protected void btnGoToStep3_Click(object sender, EventArgs e)
    {
        MultiView1.ActiveViewIndex = 2;
    }
}
```

```

lblProductID.Text = txtProductID.Text;

lblProductName.Text = txtProductName.Text;

lblPrice.Text = txtProductPrice.Text;

lblOrderID.Text = txtOrderID.Text;

lblQuantity.Text = txtQuantity.Text;
}

protected void btnSubmit_Click(object sender, EventArgs e)
{
    Response.Redirect("SaveData.aspx");
}

protected void btnBackToStep2_Click(object sender, EventArgs e)
{
    MultiView1.ActiveViewIndex = 1;
}}

```

ActiveViewIndex property of MultiView control is zero based.

Table 5: Recognized Command Names for the MultiView

Command Name	MultiView	Field Description
PrevView	PreviousViewCommandName	Moves to the previous view.
NextView	NextViewCommandName	Moves to the next view.
SwitchViewByID	SwitchViewByIDCommandName	Moves to the view with a specific ID (string name).
SwitchViewByIndex	SwitchViewByIndexCommandName	Moves to the view with a specific numeric index.

Wizard Control

This control is same as MultiView control but the main difference is that, it has inbuilt navigation buttons.

The wizard control enables you to design a long form in such a way that you can work in multiple sub form. You can perform the task in a step-by-step process. It reduces the work of developers to design multiple forms. It enables you to create multi step user interface. Wizard control provides with built-in previous/next functionality.

Wizard Steps:

The StepType associated with each WizardStep determines the type of navigation buttons that will be displayed for that step. The StepTypes are:

- Start:
- Step:
- Finish:
- Complete:
- Auto:

Drag the Wizard control on the web page from toolbox, you will get the following code.

```
<asp:Wizard ID="Wizard1" runat="server" Height="75px" Width="140px">
```

```
    <WizardSteps>
```

```
        <asp:WizardStep runat="server" title="Step 1">
```

```
        </asp:WizardStep>
```

```
        <asp:WizardStep runat="server" title="Step 2">
```

```
        </asp:WizardStep>
```

```

        </WizardSteps>

</asp:Wizard>

```

You can put WizardStep according to application need.

WizardStep Properties:

Table 6: wizard step properties.

Property	Description
Title	This name is used for the text of the links in the sidebar.
StepType	The type of step, as a value from the WizardStepType enumeration.
AllowReturn	Indicates whether the user can return to this step.

The first three steps allow the user to configure the greeting card, and the final step shows the generated card.

- 1) First, the controls aren't set to automatically post back.
- 2) Another change is that no navigation buttons exist. That's because the wizard adds these details automatically based on the step type.
- 3) The final step, which shows the complete card, doesn't provide any navigation links because the StepType is set to Complete.

consider a wizard that again uses the GreetingCardMaker example.

```

<asp:Wizard ID="Wizard1" runat="server" ActiveStepIndex="0"
BackColor="LemonChiffon" BorderStyle="Groove" BorderWidth="2px" CellPadding="10">
<WizardSteps>
<asp:WizardStep runat="server" Title="Step 1 - Colors">
    Choose a foreground (text) color:<br />
<asp:DropDownList ID="lstForeColor" runat="server" />
<br />

```

Choose a background color:

<asp:DropDownList ID="lstBackColor" runat="server" />

</asp:WizardStep>

<asp:WizardStep runat="server" Title="Step 2 - Background">

Choose a border style:

<asp:RadioButtonList ID="lstBorder" runat="server" RepeatColumns="2" />

<asp:CheckBox ID="chkPicture" runat="server"

Text="Add the Default Picture" />

</asp:WizardStep>

<asp:WizardStep runat="server" Title="Step 3 - Text">

Choose a font name:

<asp:DropDownList ID="lstFontName" runat="server" />

Specify a font size:

<asp:TextBox ID="txtFontSize" runat="server" />

Enter the greeting text below:

<asp:TextBox ID="txtGreeting" runat="server"


```

TextMode="MultiLine" />

</asp:WizardStep>

<asp:WizardStep runat="server" StepType="Complete" Title="Greeting Card">

<asp:Panel ID="pnlCard" runat="server" HorizontalAlign="Center">

<br />

<asp:Label ID="lblGreeting" runat="server" />

<asp:Image ID="imgDefault" runat="server" Visible="False" />

</asp:Panel>

</asp:WizardStep>

</WizardSteps>

</asp:Wizard>

```

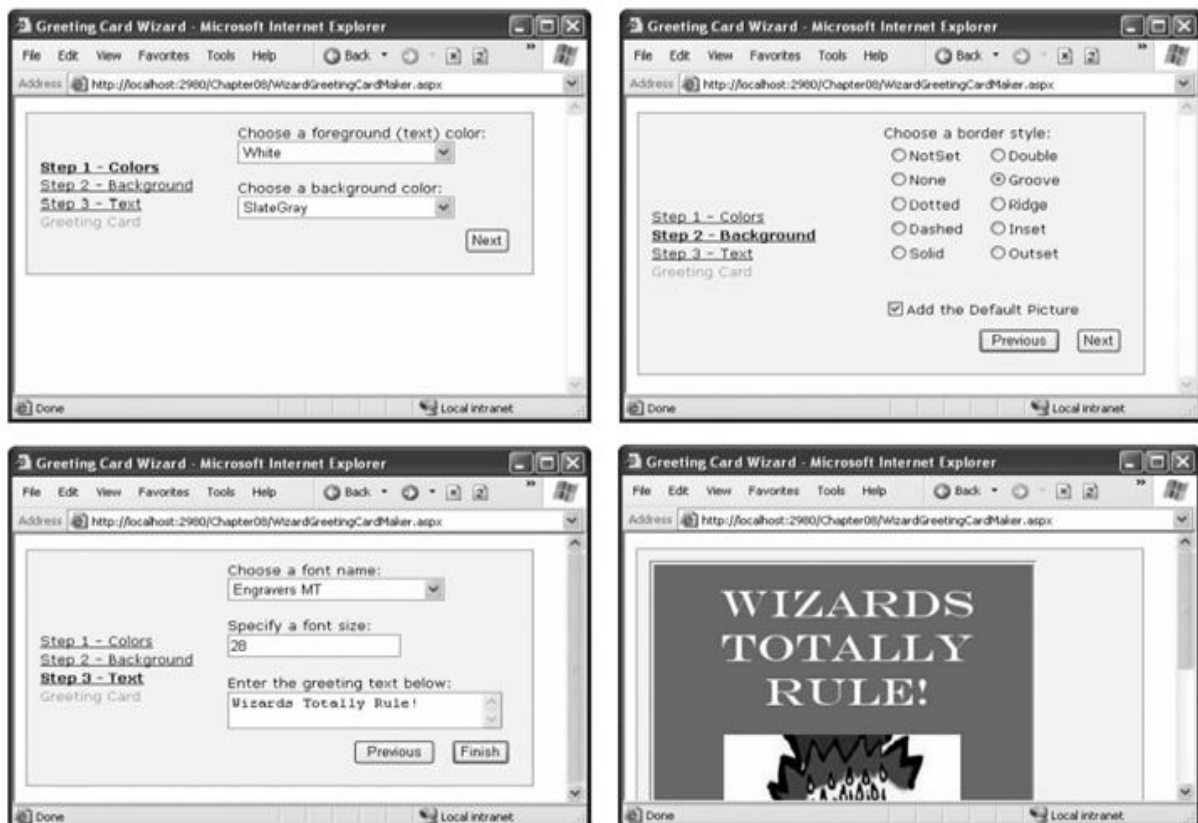


Figure 3 shows the wizard steps.

Wizard Events:

You can write the code that underpins your wizard by responding to several events (as listed in the below table:

Table7: wizard events

Event	Description
ActiveStepChanged	Occurs when the control switches to a new step
CancelButtonClick	Occurs when the Cancel button is clicked.
FinishButtonClick	Occurs when the Finish button is clicked.
NextButtonClick and PreviousButtonClick	Occurs when the Next or Previous button is clicked in any step.

On the whole, two wizard programming models exist:

Commit-as-you-go: This makes sense if each wizard step wraps an atomic operation that can't be reversed. For example, if you're processing an order that involves a credit card authorization followed by a final purchase, you can't allow the user to step back and edit the credit card number. To support this model, you set the `AllowReturn` property to false on some or all steps. You may also want to respond to the `ActiveStepChanged` event to commit changes for each step.

Commit-at-the-end: This makes sense if each wizard step is collecting information for an operation that's performed only at the end. For example, if you're collecting user information and plan to generate a new account once you have all the information, you'll probably allow a user to make changes midway through the process. You execute your code for generating the new account when the wizard ends by reacting to the `FinishButtonClick` event.

Formatting the Wizard:

You can use styles to apply formatting options to various portions of the Wizard control just as you can use styles to format parts of rich data controls such as the `GridView`. Table8 lists all the styles you can use

Table 8: Table lists all styles

Style	Description
ControlStyle	Applies to all sections of the Wizard control.
HeaderStyle	Applies to the header section of the wizard, which is visible only if you set some text in the HeaderText property.
BorderStyle	Applies to the border around the Wizard control.
SideBarStyle	Applies to the sidebar area of the wizard.
SideBarButtonStyle	Applies to just the buttons in the sidebar.
StepStyle	Applies to the section of the control where you define the step content.
NavigationStyle	Applies to the bottom area of the control where the navigation buttons are displayed.
NavigationButtonStyle	Applies to just the navigation buttons in the navigation area.
StartNextButtonStyle	StartNextButtonStyle Applies to the Next navigation button on the first step (when StepType is Start).
StepPreviousButtonStyle	Applies to the Previous navigation button on intermediate steps.
FinishPreviousButtonStyle	Applies to the Previous navigation button on the last step
FinishCompleteButtonStyle	Applies to the Complete navigation button on the last step
CancelButtonStyle	Applies to the Cancel button, if you have Wizard.DisplayCancelButton set to true.

Validation with the Wizard

The FinishButtonClick, NextButtonClick, PreviousButtonClick, and SideBarButtonClick events are cancellable. That means that you can use code like this to prevent the requested navigation action from taking place:

```
protected void Wizard1_NextButtonClick (object sender, WizardNavigationEventArgs e)
{
    // Perform some sort of check.

    if (e.NextStepIndex == 1 &&txtName.Text == "")
    {
        // Cancel navigation and display a message elsewhere on the page

        . e.Cancel = true;

        lblInfo.Text = You cannot move to the next step until you supply your name.";
    }
}
```

7.4 User Controls

User controls behaves like miniature ASP.NET pages or web forms, which could be used by many other pages. In fact, the UserControl class and the Page class both inherit from the same base classes, which is why they share so many of the same methods and events, as shown in the inheritance diagram in Figure 4. These are derived from the System.Web.UI.UserControl class. These controls have the following characteristics:

- They have an .ascx extension.
- They may not contain any <html>, <body>, or <form> tags.
- They have a Control directive instead of a Page directive.

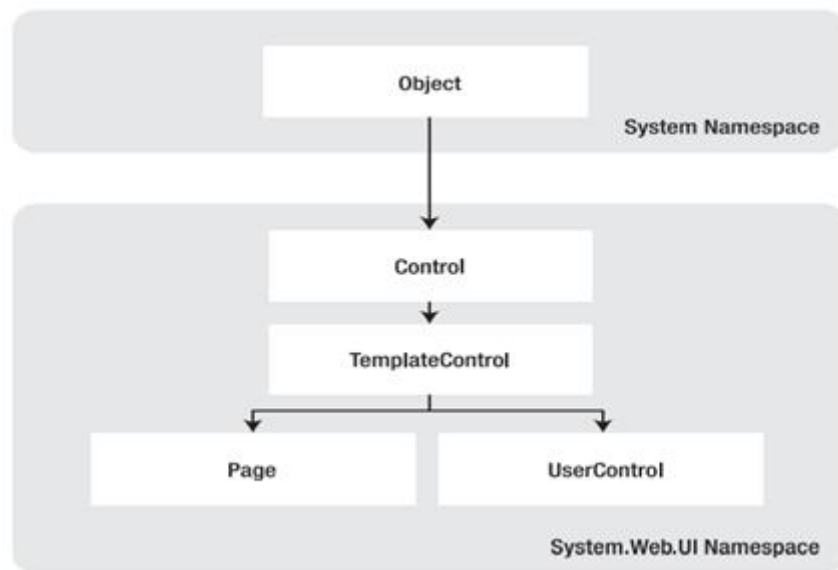


Figure 4: The Page and UserControl inheritance chain

Creating a Simple User Control:

The code-behind class for this sample user control is similarly straightforward. It uses the `UserControl.Load` event to add some text to the label: public partial class Footer:

```

System.Web.UI.UserControl

{

protected void Page_Load(Object sender, EventArgs e)

{

lblFooter.Text = "This page was served at ";

lblFooter.Text += DateTime.Now.ToString();

}

}
  
```

To test this user control, you need to insert it into a web page. This is a two-step process. First, you need to add a `Register` directive to the page that will contain the user control. Second, you can now add the user control whenever you want (and as many times as you want) in the page by inserting its control tag. Consider this page example:

```

<%@ Page Language="C#" AutoEventWireup="true"
CodeFile="FooterHost.aspx.cs" Inherits="FooterHost"%>

<%@ Register TagPrefix="apress" TagName="Footer" Src="Footer.ascx" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
"http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">

<head runat="server">

<title>Footer Host</title>

</head>

<body>

<form id="form1" runat="server">

<div>

<h1>A Page With a Footer</h1><hr />

Static Page Text<br /><br />

<apress:Footer id="Footer1" runat="server" />

</div>

</form>

</body>

</html>

```

This example demonstrates a simple way that you can create a header or footer and reuse it in all the pages in your website just by adding a user control. In the case of your simple footer, you won't save much code. However, this approach will become much more useful for a complex control with extensive formatting or several contained controls.

Types of user controls exist:

- 1) independent
- 2) integrated.

Independent User Control:

Independent user controls don't interact with the rest of the code on your form. The Footer user control is one such example. Another example might be a LinkMenu control that contains a list of buttons offering links to other pages. This LinkMenu user control can handle the events for all the buttons and then run the appropriate Response.

The following sample defines a simple control that presents an attractively formatted list of links:

```
<%@ Control Language="C#" AutoEventWireup="true"
```

```
CodeFile="LinkMenu.ascx.cs" Inherits="LinkMenu" %>
```

```
<div>
```

```
Products:<br />
```

```
<asp:HyperLink id="lnkBooks" runat="server"
```

```
NavigateUrl="MenuHost.aspx?product=Books">Books
```

```
</asp:HyperLink><br />
```

```
<asp:HyperLink id="lnkToys" runat="server"
```

```
NavigateUrl="MenuHost.aspx?product=Toys">Toys
```

```
</asp:HyperLink><br />
```

```
<asp:HyperLink id="lnkSports" runat="server"
```

```
NavigateUrl="MenuHost.aspx?product=Sports">Sports
```

```
</asp:HyperLink><br />
```

```

<asp:HyperLink id="lnkFurniture" runat="server"
NavigateUrl="MenuHost.aspx?product=Furniture">Furniture
</asp:HyperLink>
</div>

```

To test this menu, you can use the following MenuHost.aspx web page. It includes two controls: the Menu control and a Label control that displays the product query string parameter. Both are positioned using a table.

```

<%@ Page Language="C#" AutoEventWireup="true"
CodeFile="FooterHost.aspx.cs" Inherits="FooterHost"%>
<%@ Register TagPrefix="apress" TagName="Footer" Src="Footer.ascx" %>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
"http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
<title>Footer Host</title>
</head>
<body>
<form id="form1" runat="server">
<div>
<h1>A Page With a Footer</h1><hr />
Static Page Text<br /><br />
<apress:Footer id="Footer1" runat="server" />
</div>
</form>

```



```
</body>
```

```
</html>
```

Top of Form

Bottom of Form

When the MenuHost.aspx page loads, it adds the appropriate information to the lblSelection control:

```
protected void Page_Load(Object sender, EventArgs e)
```

```
{
```

```
    if (Request.Params["product"] != null)
```

```
{
```

```
    lblSelection.Text = "You chose: "; lblSelection.Text += Request.Params["product"];
```

```
}
```

```
}
```

Whenever you click a button, the page is posted back, and the text is updated.



Figure 5: Shows the end result

Integrated user controls:

Integrated user controls interact in one way or another with the web page that hosts them. The first step is to create an enumeration in the custom Footer class. Remember, an enumeration is simply a type of constant that is internally stored as an integer but is set in code by using one of the allowed names you specify. Variables that use the FooterFormat enumeration can take the value FooterFormat.LongDate or FooterFormat.ShortTime:

```
public enum FooterFormat  
  
{  
  
    LongDate, ShortTime  
  
}
```

The next step is to add a property to the Footer class that allows the web page to retrieve or set the current format applied to the footer. The actual format is stored in a private variable called `format`, which is set to the long date format by default when the control is first created.

```
private FooterFormat format = FooterFormat.LongDate;  
  
public FooterFormat Format  
  
{  
  
    get {  
  
        return format;  
  
    }  
  
    set {  
  
        format = value;  
  
    }  
  
}
```

Finally, the UserControl.Load event handler needs to take account of the current footer state and format the output accordingly. The following is the full Footer class code: public partial class Footer:

```
System.Web.UI.UserControl

{

public enum FooterFormat

{

    LongDate, ShortTime

}

private FooterFormat format = FooterFormat.LongDate;

public FooterFormat Format

{

    get {

        return format;

    }

    set {

        format = value;

    }

}

protected void Page_Load(Object sender, EventArgs e)

{

    lblFooter.Text = "This page was served at ";

    if (format == FooterFormat.LongDate)

    {
```

```
lblFooter.Text += DateTime.Now.ToLongDateString();  
  
}  
  
else if (format == FooterFormat.ShortTime)  
{  
    lblFooter.Text += DateTime.Now.ToShortTimeString();  
}  
  
}  
  
}
```

Figure6 shows an example page, which automatically sets the Format property for the user control to match a radio button selection whenever the page is posted back.

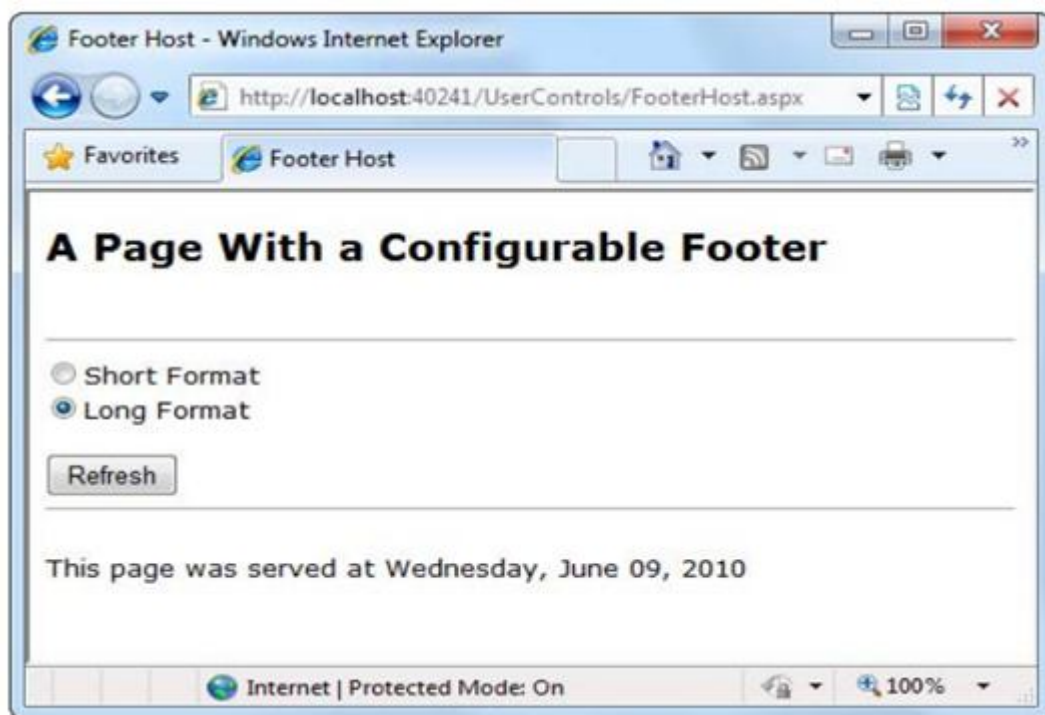


Figure 6: A modified footer

User Control Events:

The first step to create this control is to define the events. The .NET standard for events specifies that every event should use two parameters. The first one provides a reference to the control that sent the event, while the second incorporates any additional information. This additional information is wrapped into a custom EventArgs object, which inherits from the System.EventArgs class.

The LinkMenu2 control uses a single event, which indicates when a link is clicked:

```
public partial class LinkMenu2: System.Web.UI.UserControl

{

    public event EventHandlerLinkClicked; ...

}
```

This code defines an event named LinkClicked. The LinkClicked event has the signature specified by the System.EventHandler delegate, which includes two parameters—the event sender and an ordinary EventArgs object.

To fire the event, the LinkMenu2 control simply calls the event by name and passes in the two parameters, like this:

```
// Raise the LinkClicked event, passing a reference to

// the current object (the sender) and an empty EventArgs object.

LinkClicked(this, EventArgs.Empty

);
```

The following is the full user control code:

```
public partial class LinkMenu2: System.Web.UI.UserControl

{

    public event EventHandlerLinkClicked;
```

```
protected void Ink_Click(object sender, EventArgs e)

{

    // One of the LinkButton controls has been clicked.

    // Raise an event to the page.

    if (LinkClicked != null)

    {

        LinkClicked(this, EventArgs.Empty);

    }

}

}
```

Passing Information with Events:

The LinkClickedEventArgs class that follows allows the LinkMenu2 user control to pass the URL that the user selected through a Url property. It also provides a Cancel property. If set to true, the user control will stop its processing immediately. But if Cancel remains false (the default), the user control will send the user to the new page.

Here's the complete user control code. It implements one more feature. After the event has been raised and handled by the web page, the LinkMenu2 checks the Cancel property. If it's false, it goes ahead and performs the redirect using Reponse.Redirect().

```
public partial class LinkMenu2: System.Web.UI.UserControl

{

    public event LinkClickedEventHandler LinkClicked;

    protected void Ink_Command(object sender, CommandEventArgs e)

    {
```

```

// One of the LinkButton controls has been clicked.

// Raise an event to the page.

if (LinkClicked != null)

{ //

Pass along the link information.

LinkClickedEventArgs args = new LinkClickedEventArgs((string)e.CommandArgument);

LinkClicked(this, args);

// Perform the redirect.

if (!args.Cancel)

{

// Notice we use the Url from the LinkClickedEventArgs

// object, not the original link. That means the web page.

// can change the link if desired before the redirect.

Response.Redirect(args.Url);

}

}

}

}

```

Finally, you need to update the code in the web page (where the user control is placed) so that its event handler uses the new signature. In the following code, the LinkClicked event handler checks the URL and allows it in all cases except one:

```

protected void LinkClicked(object sender, LinkClickedEventArgs e)

{

```

```
if (e.Url == "Menu2Host.aspx?product=Furniture")  
{  
    lblClick.Text = "This link is not allowed."; e.Cancel = true;  
}  
  
else  
{  
    // Allow the redirect, and don't make any changes to the URL.  
}  
}
```

If you click the Furniture link, you'll see the message shown in Figure



Figure 7: Handling a user control event in the page

7.5 Dynamic Graphics

One of the features of the .NET Framework is GDI+, a set of classes designed for drawing images. You can use GDI+ in a Windows or an ASP.NET application to create dynamic graphics. The following steps are

Basic Drawing:

To create the bitmap, declare a new instance of the `System.Drawing.Bitmap` class. You must specify the height and width of the image in pixels. The next step is to create a GDI+ graphics context for the image, which is represented by a `System.Drawing.Graphics` object. This object provides the methods that allow you to render content to the in-memory bitmap.

To create a `Graphics` object from an existing `Bitmap` object, you just use the static `Graphics.FromImage()` method, as shown here:

```
Graphics g = Graphics.FromImage(image);
```

Table 9 lists some of the most fundamental methods of the `Graphics` class:

Method	Description
<code>DrawArc()</code>	Draws an arc representing a portion of an ellipse specified by a pair of coordinates, a width, and a height
<code>DrawBezier()</code> and <code>DrawBeziers()</code>	Draws the infamous and attractive Bezier curve, which is defined by four control points.
<code>DrawClosedCurve()</code>	Draws a curve and then closes it off by connecting the end points.
<code>DrawCurve()</code>	Draws a curve (technically, a cardinal spline).
<code>DrawEllipse()</code>	Draws an ellipse defined by a bounding rectangle specified by a pair of coordinates, a height, and a width.
<code>DrawIcon()</code> and <code>DrawIconUnstretched()</code>	Draws the icon represented by an <code>Icon</code> object and (optionally) stretches it to fit a given rectangle.
<code>DrawImage()</code> and <code>DrawImageUnscaled()</code>	Draws the image represented by an <code>Image</code> -derived object (such as a <code>Bitmap</code> object) and (optionally) stretches it to fit a given rectangle.
<code>DrawLine()</code> and <code>DrawLines()</code>	Draws one or more lines. Each line connects the two points specified by a coordinate pair.

<code>DrawPie()</code>	Draws a “piece of pie” shape defined by an ellipse specified by a coordinate pair, a width, a height, and two radial lines.
<code>DrawPolygon()</code>	Draws a multisided polygon defined by an array of points.
<code>DrawRectangle()</code> and <code>DrawRectangles()</code>	Draws one or more ordinary rectangles. Each rectangle is defined by a starting coordinate pair, a width, and a height.
<code>DrawString()</code>	Draws a string of text in a given font.
<code>DrawPath()</code>	Draws a more complex shape that’s defined by the Path object.
<code>FillClosedCurve()</code>	Draws a curve, closes it off by connecting the end points, and fills it.
<code>FillEllipse()</code>	Fills the interior of an ellipse.
<code>FillPie()</code>	Fills the interior of a “piece of pie” shape.
<code>FillPolygon()</code>	Fills the interior of a polygon.
<code>FillRectangle()</code> and <code>FillRectangles()</code>	Fills the interior of one or more rectangles.
<code>FillPath()</code>	Fills the interior of a complex shape that’s defined by the Path object.

Table 9 lists some fundamental methods of the Graphics class

Drawing a Custom Image:

It creates the in-memory Bitmap, gets the corresponding Graphics object, performs the painting, and then saves the image to the response stream.

```
protected void Page_Load(Object sender, EventArgs e)
{
    // Create an in-memory bitmap where you will draw the image.

    // The Bitmap is 300 pixels wide and 50 pixels high.

    Bitmap image = new Bitmap (300, 50);

    // Get the graphics context for the bitmap.
```

```
Graphics g = Graphics.FromImage(image);  
// Draw a solid yellow rectangle with a red border.  
g.FillRectangle(Brushes.LightYellow, 0, 0, 300, 50);  
g.DrawRectangle(Pens.Red, 0, 0, 299, 49);  
// Draw some text using a fancy font.  
Font font = new Font("Alba Super", 20, FontStyle.Regular);  
g.DrawString("This is a test.", font, Brushes.Blue, 10, 0);  
// Copy a smaller gif into the image from a file.  
System.Drawing.Image icon = Image.FromFile(Server.MapPath("smiley.gif"));  
g.DrawImageUnscaled(icon, 240, 0);  
// Render the entire bitmap to the HTML output stream.  
image.Save(Response.OutputStream, System.Drawing.Imaging.ImageFormat.Gif);  
// Clean up. g.Dispose(); image.Dispose();  
}
```

This example uses the `FillRectangle()`, `DrawRectangle()`, `DrawString()`, and `DrawImageUnscaled()` methods to create the complete drawing shown in Figure 8.



Figure 8: shows complete drawing.

Placing Custom Images Inside Web Pages:

Figure 9 shows a page that uses this dynamic graphic page, along with two Label controls. The page passes the query string argument Joe Brown to the page. The full Image.ImageUrl thus becomes GraphicalText.aspx?Name=Joe%20Brown, as shown here: Here is some content.



Figure 9: Dynamic graphic page

Image Format and Quality:

JPEG offers the best color support and graphics, although it uses compression that can lose detail and make text look fuzzy. In .NET, every GIF uses a fixed palette with 256 generic colors. In .NET, every GIF uses a fixed palette with 256 generic colors. the best format choice is PNG. PNG is an all-purpose format that always provides high quality by combining the lossless compression of GIFs with the rich color support of JPEGs.

The MemoryStream is always seekable, so you can save the image to this object. Once you've performed this step, you can easily copy the data from the MemoryStream to the Response.OutputStream. The only disadvantage is that this technique requires more memory because the complete rendered content of the graphic needs to be held in memory at once.

To implement this solution, start by importing the

```
System.IO namespace: using System.IO;
```

```
// Get the user name.

if (Request.QueryString["Name"] == null)

{

    // No name was supplied.

    // Don't display anything.

}

else

{

    string name = Request.QueryString["Name"];

    // Create an in-memory bitmap where you will draw the image.

    Bitmap image = new Bitmap(300, 50);

    // Get the graphics context for the bitmap.

    Graphics g = Graphics.FromImage(image);

    g.FillRectangle(Brushes.LightYellow, 0, 0, 300, 50);

    g.DrawRectangle(Pens.Red, 0, 0, 299, 49);

    // Draw some text based on the query string.

    Font font = new Font("Alba Super", 20, FontStyle.Regular);

    g.DrawString(name, font, Brushes.Blue, 10, 0);

    Response.ContentType = "image/png";

    // Create the PNG in memory.

    MemoryStream mem = new MemoryStream();

    image.Save(mem, System.Drawing.Imaging.ImageFormat.Png);

    // Write the MemoryStream data to the output stream.
```

```
mem.WriteTo(Response.OutputStream);  
g.Dispose();  
image.Dispose();  
}
```

As shown in the following figure Antialiasing smooths jagged edges in shapes and text. It works by adding shading at the border of an edge.

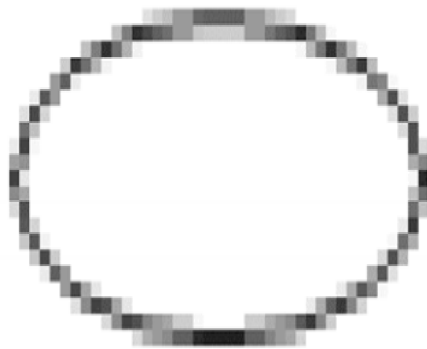


figure 10: Antialiasing with an ellipse

To use smoothing in your applications, you set the `SmoothingMode` property of the `Graphics` object. You can choose between `None`, `HighSpeed` (the default), `AntiAlias`, and `HighQuality` (which is similar to `AntiAlias` but uses other, slower optimizations that improve the display on LCD screens). The `Graphics.SmoothingMode` property is one of the few stateful `Graphics` class members.

7.6 Summary

This chapter showed you how the rich `Calendar`, `AdRotator`, `MultiView`, `Wizard` controls, user controls and graphics. can go far beyond the limitations of ordinary HTML elements. When you're working with these controls, you don't need to think about HTML at all. Instead, you can focus on the object model that's defined by the control.

Keywords

`Calendar`, `date`, `control`, `user`, `page`, `text`, `object`, `image`, `event`, `wizard`

7.7 Check your progress

1. Which web server control is used to display advertisements in ASP.NET a webpage?
 - i. Image
 - ii. Imagemap
 - iii. Panel
 - iv. **AdRotator**
2. Which of these is wizard step properties?
 - i. Title
 - ii. Stepwise
 - iii. Allow return
 - iv. **All of the above**
3. The wizard style Applies to all sections of the Wizard control is
 - i. **ControlStyle**
 - ii. HeaderStyle
 - iii. SideBarStyle
 - iv. BorderStyle
4. What class does the ASP.net web form class inherit from by default?
 - i. System.Web.UI.Page
 - ii. System.Web.UI.Form
 - iii. **System.Web.GUI.Page**
 - iv. System.Web.Form

5. Each line connects the two points specified by a coordinate pair _____

- i. DrawIcon() and DrawIconUnstretched()
- ii. DrawImage() and DrawImageUnscaled()
- iii. DrawLine() and DrawLines()
- iv. **DrawRectangle() and DrawRectangles()**

6. If we want to add graphics using asp.net which of the following web control will you use?

- i. Link
- ii. **AdRotator**
- iii. Grid
- iv. Layout

7.8 Model questions

- 1. What is meant by independent user controls?
- 2. List some of the drawing methods of the graphics class.
- 3. Explain wizard controls with its steps.
- 4. List some properties of calendar styles.
- 5. Define _blank, _parent, _self, _top.

Reference Book:

- 1. Mathew MacDonald, "Beginning ASP.NET 4 in C# 2010" Apress 2010.

LESSON - 8

SITEMAPS AND ADO.NET FUNDAMENTALS

Structure

- 8.1 Introduction
- 8.2 Objectives
- 8.3 Site Maps
- 8.4 URL Mapping and Routing
- 8.5 The Tree view control
- 8.6 The Menu control
- 8.7 ADO.NET Fundamentals
- 8.8 SQL Basics
- 8.9 Summary
- 8.10 Check your Progress
- 8.11 Model Questions

8.1 Introduction

In this chapter, you'll learn everything you need to know about the site map model and the navigation controls that work with it. you'll learn about ADO.NET and the family of objects that provides its functionality. ADO.NET is an object-oriented set of libraries that allows you to interact with data sources. Commonly, the data source is a database, but it could also be a text file, an Excel spreadsheet, or an XML file. For the purposes of this tutorial, we will look at ADO.NET as a way to interact with a database.

8.2 Learning objectives

- ✓ Know what *Sitemap sources* are, and how they compare to Web sources.
- ✓ Be able to create a Sitemap source.

- ✓ Understand how Sitemap sources gather metadata.
- ✓ Learned to explore the new navigation model
- ✓ Learned how to define site maps and bind the navigation data
- ✓ Learn what ADO.NET is.
- ✓ Understand what a data provider is.
- ✓ Understand what a connection object is.
- ✓ Understand what a command object is.
- ✓ Understand what a DataReader, DataSet and DataAdapter object is.

8.3 SITE MAPS

ASP.NET navigation is flexible, configurable, and pluggable.

A way to define the navigational structure of your website. This part is the XML site map, which is (by default) stored in a file.

- A convenient way to read the information in the site map file and convert it to an object model. The SiteMapDataSource control and the XmlSiteMapProvider perform this part.
- A way to use the site map information to display the user's current position and give the user the ability to easily move from one place to another. This part takes place through the navigation controls you bind to the SiteMapDataSource control, which can include breadcrumb links, lists, menus, and trees.

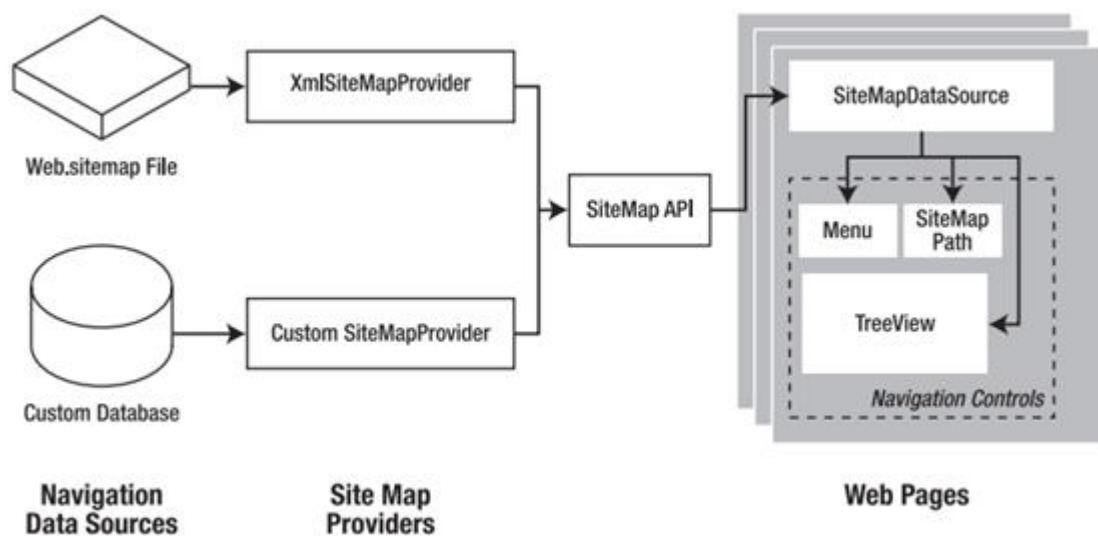


Figure 1: ASP.NET navigation with site maps

Defining a Site Map:

You can create a site map using a text editor such as Notepad, or you can create it in Visual Studio by selecting Website > Add New Item and then choosing the Site Map option. Before you can fill in the content in your site map file, you need to understand the rules that all ASP.NET site maps must follow. The following sections break these rules down piece by piece.

Rule 1: site maps begin with the element

Every Web.sitemap file begins by declaring the element and ends by closing that element. You place the actual site map information between the start and end tags:

```
<siteMapxmlns="http://schemas.microsoft.com/AspNet/SiteMap-File-1.0">
```

...

```
</siteMap>
```

Rule 2: each page is represented by a element

To insert a page into the site map, you add the element with some basic information. You add these three pieces of information using three attributes. The attributes are named title, description, and url, as shown here:

```
<siteMapNode title="Home" description="Home" url="~/default.aspx" />
```

this element ends with the characters `/>`. This indicates it's an empty element that represents a start tag and an end tag in one.

Rule 3: an element can contain other elements

To represent this in a site map file, you place one inside another. need to split your `<siteMapNode>` element into a start tag and an end tag:

```
<siteMapNode title="Home" description="Home" url="~/default.aspx">
```

```
<siteMapNode title="Products" description="Our products"
```

```
url="~/products.aspx" />
```

```
<siteMapNode title="Hardware" description="Hardware choices"
```

```
url="~/hardware.aspx" />
```

```
</siteMapNode>
```

Rule 4: every site map begins with a single

A site map must always have a single root node. All the other nodes must be contained inside this root-level node. The following site map is valid, because it has a single top-level node (Home), which contains two more nodes:

```
<siteMapxmlns="http://schemas.microsoft.com/AspNet/SiteMap-File-1.0">
```

```
<siteMapNode title="Products" de
```

```
scription="Our products" url="~/products.aspx" />
```

```
<siteMapNode title="Hardware" description="Hardware choices" url="~/hardware.aspx" />
```

```
</siteMap>
```

Rule 5: duplicate urls are not allowed

The default `SiteMapProvider` included with ASP.NET stores nodes in a collection, with each item indexed by its unique URL. For example, the following two nodes are acceptable, even

though they lead to the same page (products.aspx), because the two URLs have different query string arguments at the end:

```
<siteMapNode title="In Stock" description="Products that are available"
```

```
url="~/products.aspx?stock=1" />
```

```
<siteMapNode title="Not In Stock" description="Products that are on order"
```

```
url="~/products.aspx?stock=0" />
```

Seeing a simple site map in action:

The example defines a simple site map for a company named RevoTech.

```
<siteMapxmlns="http://schemas.microsoft.com/AspNet/SiteMap-File-1.0">
```

```
<siteMapNode title="Home" description="Home" url="~/default.aspx">
```

```
<siteMapNode title="Information" description="Learn about our company">
```

```
<siteMapNode title="About Us"
```

```
description="How RevoTech was founded"
```

```
url="~/aboutus.aspx" />
```

```
<siteMapNode title="Investing"
```

```
description="Financial reports and investor analysis"
```

```
url="~/financial.aspx" />
```

```
</siteMapNode>
```

```
<siteMapNode title="Products" description="Learn about our products">
```

```
<siteMapNode title="RevoStock"
```

```
description="Investment software for stock charting"
```

```
url="~/product1.aspx" />
```

```
<siteMapNode title="RevoAnalyze"
```

```
description="Investment software for yield analysis"
```

```
url="~/product2.aspx" />
```

```
</siteMapNode>
```

```
</siteMapNode>
```

```
</siteMap>
```

Binding an ordinary page to a site map:

The next step is to add the SiteMapDataSource control to your page. You can drag and drop it from the Data tab of the Toolbox. It creates a tag like this: The SiteMapDataSource control appears as a gray box on your page in Visual Studio, but it's invisible when you run the page. The last step is to add controls that are linked to the SiteMapDataSource.

The three navigation controls:

TreeView: The TreeView displays a “tree” of grouped links that shows your whole site map at a glance. **Menu:** The Menu displays a multilevel menu. By default, you'll see only the first level, but other levels pop up.

SiteMapPath: The SiteMapPath is the simplest navigation control—it displays the full path you need to take through the site map to get to the current page.

To connect a control to the SiteMapDataSource, you simply need to set its DataSourceID property to match the name of the SiteMapDataSource. For example, if you added a TreeView, you should tweak the tag so it looks like this: Figure 2 shows the result. When using the TreeView, the description information doesn't appear immediately. Instead, it's displayed as a tooltip when you hover over an item in the tree.

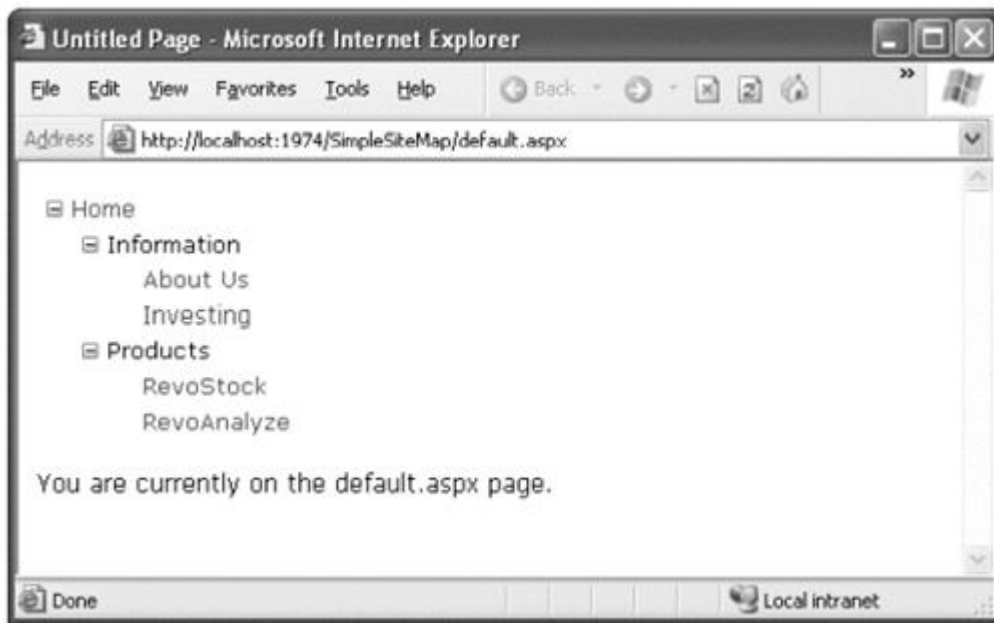


Figure 2: A site map in the TreeView

Binding a Master Page to a Site Map:

The easiest way to do this is to create a master page that includes the SiteMapDataSource and the navigation controls. define a basic structure in your master page that puts navigation controls on the left:

```
<%@ Master Language="C#" AutoEventWireup="true"
CodeFile="MasterPage.master.cs" Inherits="MasterPage" %>

<html>

<head runat="server">

<title>Navigation Test</title>

</head>

<body>

<form id="form1" runat="server">

<table>
```

326

```
<tr>

<td style="width: 226px;vertical-align: top;">

<asp:TreeView ID="TreeView1" runat="server"

DataSourceID="SiteMapDataSource1" />

</td>

<td style="vertical-align: top;">

<asp:ContentPlaceHolder id="ContentPlaceHolder1" runat="server" />

</td>

</tr>

</table>

<asp:SiteMapDataSource ID="SiteMapDataSource1" runat="server" />

</form>

</body>

</html>
```

Then, create a child with some simple static content:

```
<%@ Page Language="C#" MasterPageFile="~/MasterPage.master" AutoEventWireup="true"

CodeFile="default.aspx.cs" Inherits="_default" Title="Home Page" %>

<asp:Content ID="Content1" ContentPlaceHolderID="ContentPlaceHolder1"

runat="Server">

<br />

<br />

You are currently on the default.aspx page (Home).

</asp:Content>
```


In fact, while you're at it, why not create a second page so you can test the navigation between the two pages? You are currently on the product1.aspx page (RevoStock). Now you can jump from one page to another using the TreeView as in the Figure. The figure1 shows the home page as it initially appears, while the second shows the result of clicking the RevoStock link in the TreeView.

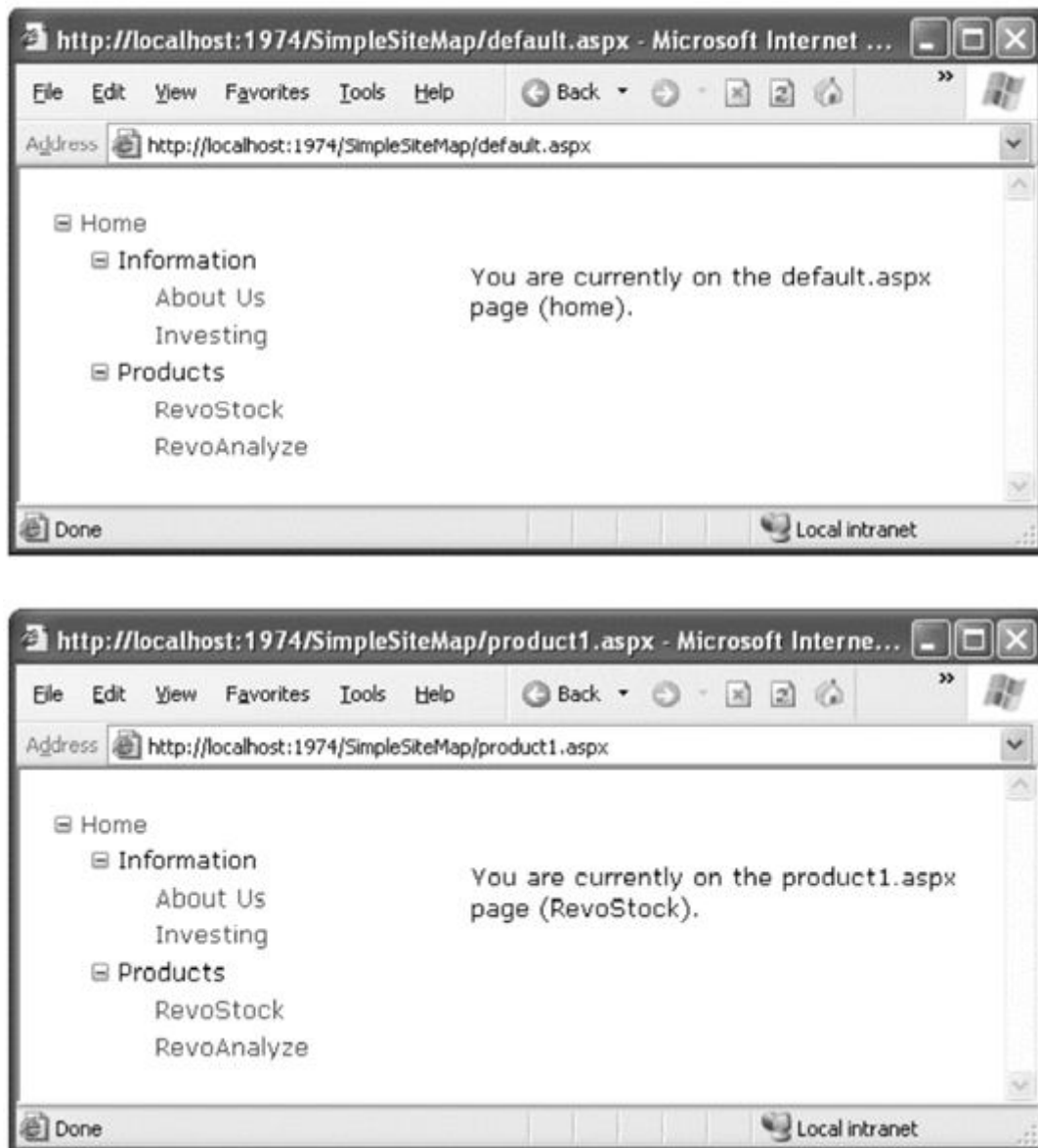


Figure 3: Navigating from page to page with the TreeView

Binding portions of a site map:

One way to clean this up is to configure the properties of the SiteMapDataSource. For example, you can set the ShowStartingNode property to false to hide the root node:

```
<asp:SiteMapDataSource ID="SiteMapDataSource1" runat="server"
```

```
ShowStartingNode="False" />
```



Figure 4: A site map without the root node

Showing subtrees:

The SiteMapDataSource has several properties that can help you configure the navigation tree to limit the display to just a specific branch. Typically, this is useful if you have a deeply nested tree. Table 1 describes the full set of properties.

Table 1. SiteMapDataSource Properties

Property	Description
ShowStartingNode	Set this property to false to hide the first (top-level) node that would otherwise appear in the navigation tree. The default is true.
StartingNodeUrl	Use this property to change the starting node. Set this value to the URL of the node that should be the first node in the navigation tree. This value must match the url attribute in the site map file exactly.

StartFromCurrentNode Set this property to true to set the current page as the starting node. The navigation tree will show only pages beneath the current page.

StartingNodeOffset Use this property to shift the starting node up or down the hierarchy. It takes an integer that instructs the SiteMapDataSource to move from the starting node down the tree (if the number is positive) or up the tree (if the number is negative).

The SiteMapDataSource properties work with multiple navigation levels, modify the Information nodes as shown here:

```
<siteMapNode title="Information" description="Learn about our company"
```

```
url="~/information.aspx">and change the Products node:
```

```
<siteMapNode title="Products" description="Learn about our products"
```

```
url="~/products.aspx">
```

Next, create the products.aspx and information.aspx pages.

Now, just bind two navigation controls. In this case, one TreeView is linked to each SiteMapDataSource in the markup for the master page:

Pages under the current page:

```
<asp:TreeView ID="TreeView1" runat="server"
```

```
DataSourceID="SiteMapDataSource1" />
```

```
<br />
```

The Information group of pages:


```
<asp:TreeView ID="TreeView2" runat="server"
```

```
DataSourceID="SiteMapDataSource2" />
```

Figure shows the result as you navigate from default.aspx down the tree to products1.aspx.

The first TreeView shows the portion of the tree under the current page, and the second TreeView is always fixed on the Information group.

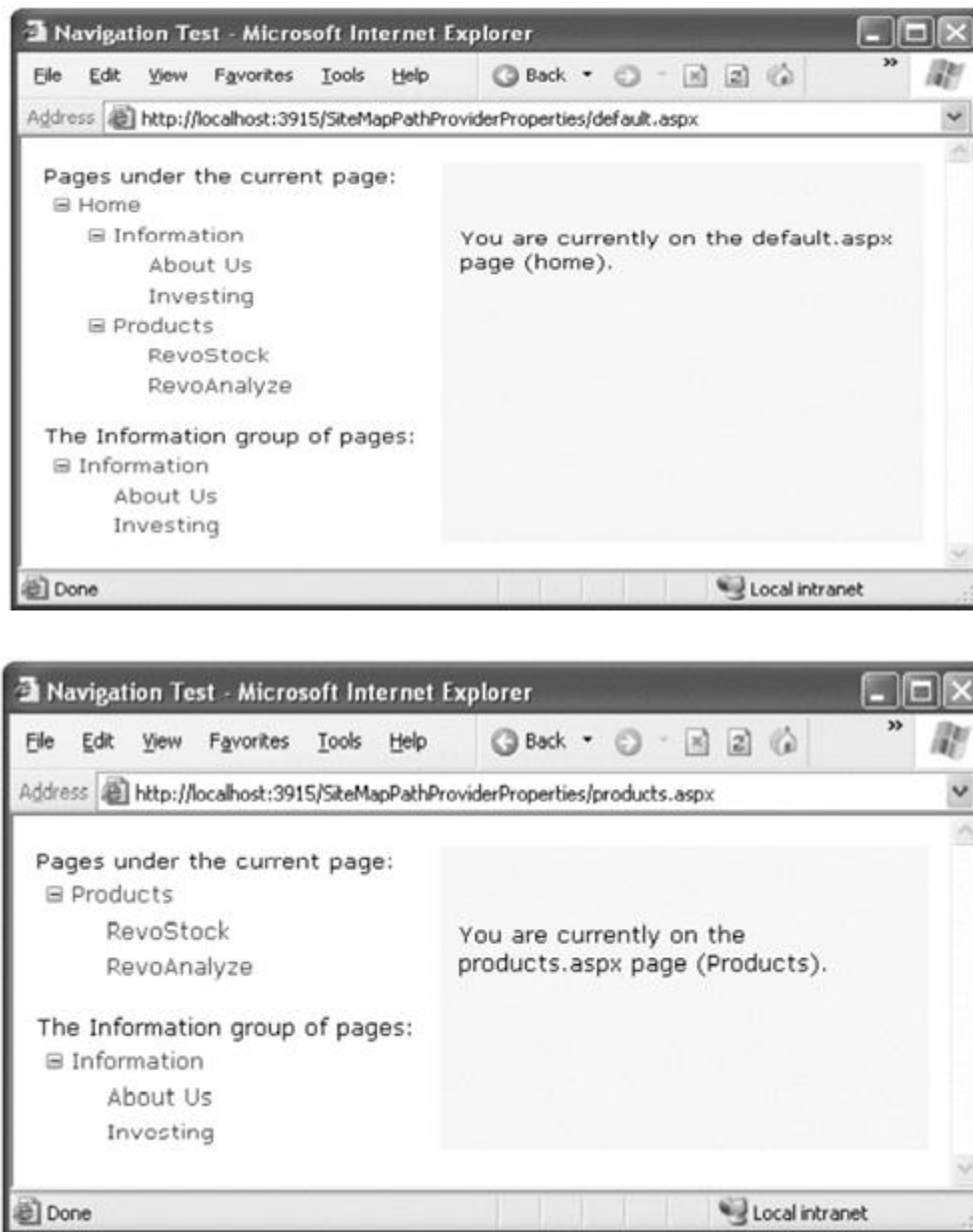


Figure 5: Showing portions of the site map

Using different site maps in the same file:

Imagine you want to have a dealer section and an employee section on your website. You might split this into two structures and define them both under different branches in the same file, like this:

```

<siteMapxmlns="http://schemas.microsoft.com/AspNet/SiteMap-File-1.0" >

<siteMapNode title="Root" description="Root" url="~/default.aspx">

<siteMapNode title="Dealer Home" description="Dealer Home"
url="~/default_dealer.aspx">

...

</siteMapNode>

<siteMapNode title="Employee Home" description="Employee Home"
url="~/default_employee.aspx">

...

</siteMapNode>

</siteMapNode>

</siteMap>

```

You can even make your life easier by breaking a single site map into separate files using the `siteMapFile` attribute, like this:

```

<siteMapxmlns="http://schemas.microsoft.com/AspNet/SiteMap-File-1.0" >

<siteMapNode title="Root" description="Root" url="~/default.aspx">

<siteMapNodesiteMapFile="Dealers.sitemap" />

<siteMapNodesiteMapFile="Employees.sitemap" />

</siteMapNode>

</siteMap>

```

The sitemap class:

To use it, you need to work with two classes from the System.Web namespace. The starting point is the SiteMap class, which provides the static properties CurrentNode (the site map node representing the current page) and RootNode (the root site map node). Both of these properties return a SiteMapNode object. Using the SiteMapNode object, you can retrieve information from the site map, including the title, description, and URL values. You can branch out to consider related nodes using the navigational properties in Table2:

Table2. SiteMapNode Navigational Properties

Property	Description
ParentNode	Returns the node one level up in the navigation hierarchy, which contains the current node. On the root node, this returns a null reference.
ChildNodes	Provides a collection of all the child nodes. You can check the HasChildNodes property to determine whether child nodes exist.
PreviousSibling	Returns the previous node that's at the same level
NextSibling	Returns the next node that's at the same level

consider the following code, which configures two labels on a page to show the heading and description information retrieved from the current node:

```
protected void Page_Load(object sender, EventArgs e)
{
    lblHead.Text = SiteMap.CurrentNode.Title;
    lblDescription.Text = SiteMap.CurrentNode.Description;
}
```

The code checks for the existence of sibling nodes, and if there aren't any in the required position, it simply hides the link:

```
protected void Page_Load(object sender, EventArgs e)
```

```
{  
  
    if (SiteMap.CurrentNode.NextSibling != null)  
  
    {  
  
        InkNext.NavigateUrl = SiteMap.CurrentNode.NextSibling.Url;  
  
        InkNext.Visible = true;  
  
    }  
  
    else  
  
    {  
  
        InkNext.Visible = false;  
  
    }  
  
}
```

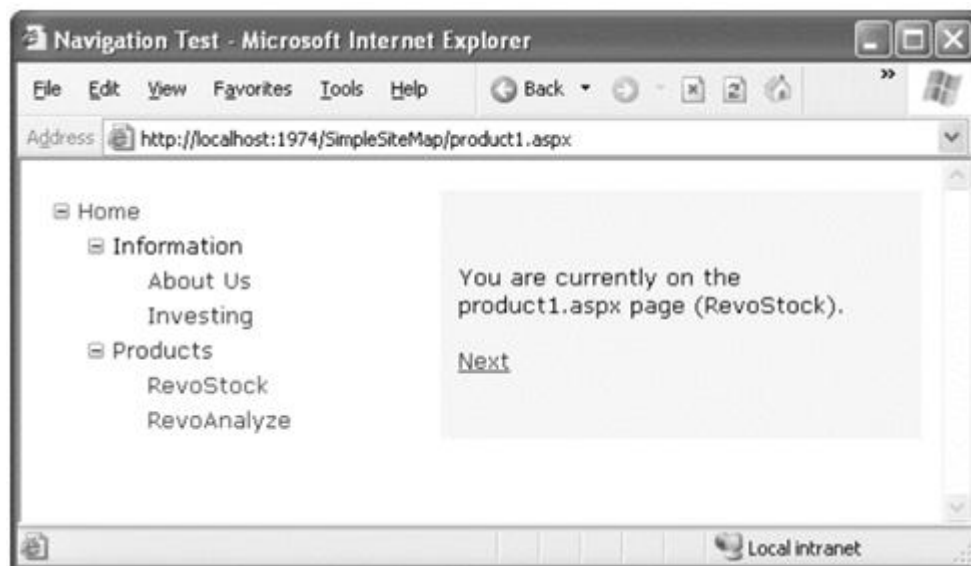


Figure 6: The Next link on the product1.aspx page

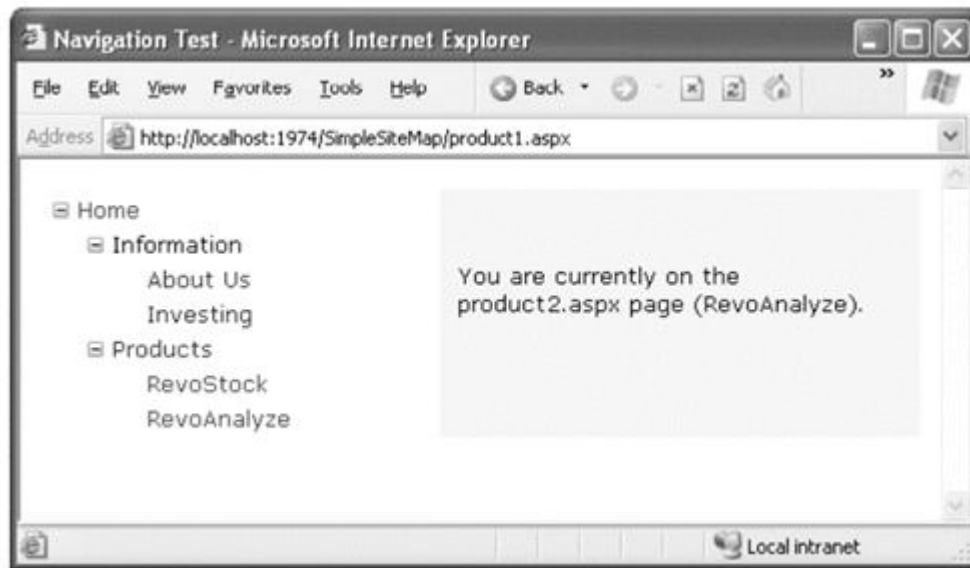


Figure 7: Link disappears when you navigate to product2.aspx

8.4 URL Mapping and Routing

Url mapping

The basic idea behind ASP.NET URL mapping is that you map a request URL to a different URL.

The mapping rules are stored in the web.config file, and they're applied before any other processing takes place. Of course, for ASP.NET to apply the remapping, it must be processing the request, which means the request URL must use a file type extension that's mapped to ASP.NET (such as .aspx).

You define URL mapping in the <urlMappings> section of the web.config file. You supply two pieces of information—the request URL (as the url attribute) and the new destination URL(mappedUrl). an example:

```
<configuration>

<system.web>

<urlMappings enabled="true">

<add url="~/category.aspx"
```



```

mappedUrl="~/default.aspx?category=default" />

<add url="~/software.aspx"

mappedUrl="~/default.aspx?category=software" />

</urlMappings>

...

</system.web>

</configuration>

```

First, the matching algorithm isn't case sensitive, so the capitalization of the request URL is always ignored. Second, any query string arguments in the URL are disregarded.

Url routing:

To register a route, you use the `RouteTable` class from the `System.Web.Routing` namespace. It provides a static property named `Routes`, which holds a collection of `Route` objects that are defined for your application. Initially, this collection is empty, but you can your custom routes by calling the `MapPageRoute()` method, which takes three arguments:

ROUTE NAME: This is a name that uniquely identifies the route. It can be whatever you want.

ROUTE URL: This specifies the URL format that browsers will use. Typically, a route URL consists of one or more pieces of variable information, separated by slashes, which are extracted and provided to your code.

PHYSICAL FILE: This is the target web form—the place where users will be redirected when they use the route. The information from the original `routeUrl` will be parsed and made available to this page as a collection through the `Page.RouteData` property. Here's an example that adds two routes to a web application when it first starts:

```

protected void Application_Start(object sender, EventArgs e)
{
    RouteTable.Routes.MapPageRoute("product-details", "product/{productID}", "~/productInfo.aspx");
    RouteTable.Routes.MapPageRoute("products-in-category", "products/category/{categoryID}", "~/products.aspx");
}

```

THE SITEMAPPATH CONTROL:

The TreeView shows the available pages, but it doesn't indicate where you're currently positioned. To solve this problem, it's common to use the TreeView in conjunction with the SiteMapPath control. Because the SiteMapPath is always used for displaying navigational information.

The SiteMapPath provides breadcrumb navigation, which means it shows the user's current location and allows the user to navigate up the hierarchy to a higher-level using links. Figure shows an example with a SiteMapPath control when the user is on the product1.aspx page. Using the SiteMapPath control, the user can return to the default.aspx page.



Figure 8. Breadcrumb navigation with SiteMapPath

Customizing the sitemappath:

The SiteMapPath has a subtle but important difference from other navigational controls such as the TreeView and Menu. Table lists some of its most commonly configured properties.

Table3. SiteMapPath Appearance-Related Properties.

Property	Description
ShowToolTips	Set this to false if you don't want the description text to appear when the user hovers over a part of the site map path.
ParentLevelsDisplayed	This sets the maximum number of levels above the current page that will be shown at once.

RenderCurrentNodeAsLink	If true, the portion of the page that indicates the current page is turned into a clickable link. By default, this is false because the user is already at the current page.
PathDirection	RootToCurrent (the default) and CurrentToRoot (which reverses the order of levels in the path).
PathSeparator	This indicates the characters that will be placed between each level in the path. The default is the greater-than symbol (>) and path separator is the colon (:).

Using sitemappath styles and templates:

A template is a bit of HTML (that you create) that will be shown for a specific part of the SiteMapPath control. For example, if you want to configure how the root node displays in a site map, you could create a SiteMapPath with as follows:

```
<asp:SiteMapPath ID="SiteMapPath1" runat="server">
<RootNodeTemplate>
<b>Root</b>
</RootNodeTemplate>
</asp:SiteMapPath>
```

This simple template does not use the title and URL information in the root node of the sitemap node. Instead, it simply displays the word Root in bold. Clicking the text has no effect.

To get the name of the current node, you need to write a data binding expression that retrieves the title. This data binding expression is bracketed between <%# and %> characters and uses a method named Eval() to retrieve information from a SiteMapNode object that represents a page. Here's what the template looks like:

```
<asp:SiteMapPath ID="SiteMapPath1" runat="server">
<CurrentNodeTemplate>
```

```
<i><%# Eval("Title") %></i>
```

```
</CurrentNodeTemplate>
```

```
</asp:SiteMapPath>
```

Data binding also gives you the ability to retrieve other information from the site map node, such as the description. Consider the following example:

```
<asp:SiteMapPath ID="SiteMapPath1" runat="server">
```

```
<PathSeparatorTemplate>
```

```
<asp:Image ID="Image1" ImageUrl="~/arrowright.gif"
```

```
runat="server" />
```

```
</PathSeparatorTemplate>
```

```
<RootNodeTemplate>
```

```
<b>Root</b>
```

```
</RootNodeTemplate>
```

```
<CurrentNodeTemplate>
```

```
<%# Eval("Title") %><br />
```

```
<small><i><%# Eval("Description") %></i></small>
```

```
</CurrentNodeTemplate>
```

```
</asp:SiteMapPath>
```

First, it uses the `PathSeparatorTemplate` to define a custom arrow image that's used between each part of the path. This template uses an `Image` control instead of an ordinary HTML `<img>` tag because only the `Image` understands the `~/` characters in the image URL, which represent the application's root folder.

Next, the SiteMapPath uses the RootNodeTemplate to supply a fixed string of bold text for the rootportion of the site map path. Finally, the CurrentNodeTemplate uses two data binding expressions to show two pieces of information—both the title of the node and its description. Figure9 shows the final result.

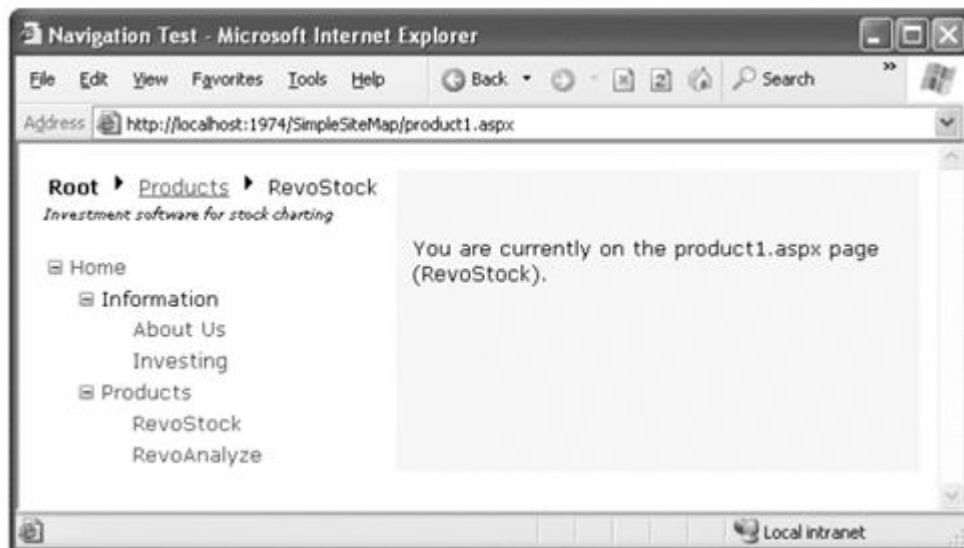


Figure9. A SiteMapPath with templates

Adding custom site map information:

You might want to insert additional node data for a number of reasons. This additional information might be descriptive information that you intend to display, or contextual information that describes how the link should work. For example, you could add an attribute specifying that the link should open in a new window.

For example, the following code shows a site map that uses a target attribute to indicate the frame where the link should open. In this example, one link is set with a target of `_blank` so it will open in a new browser window:

```
<siteMapNode title="RevoStock"
description="Investment software for stock charting"
url="~/product1.aspx" target="_blank" />
```

If you're using a template in your navigation control, you can bind directly to the new attribute. Here's an example with the SiteMapPath from the previous section:

```

<asp:SiteMapPath ID="SiteMapPath1" runat="server" Width="264px" Font-Size="10pt">

<NodeTemplate>

<a href='<%=# Eval("Url") %>' target='<%=# Eval("[target]") %>'>

<%=# Eval("Title") %>

</a>

</NodeTemplate>

</asp:SiteMapPath>

```

There's a slightly unusual detail in this example—the square brackets around the word [target]. You need to use this syntax to look up any custom attribute you add to the Web.sitemap file. If your navigation control doesn't support templates, you'll need to find another approach. For example, the TreeView doesn't support templates, but it fires a `TreeNodeDataBound` event each time an item is bound to the tree. You can react to this event to customize the current item. To apply the new target, use this code:

```

protected void TreeView1_TreeNodeDataBound(object sender, TreeNodeEventArgs e)
{
    SiteMapNode node = (SiteMapNode)e.Node.DataItem;

    e.Node.Target = node["target"];
}

```

As in the template, you can't retrieve the custom attribute from a strongly typed `SiteMapNode` property. Instead, you retrieve it by name using the `SiteMapNode` indexer.

8.5 The Treeview Control

TreeView Properties:

One of the most important properties is `ImageSet`, which lets you choose a predefined set of node icons. (Each set includes three icons: one for collapsed nodes, one for expanded nodes,

and one for nodes that have no children. Figure10 shows the same RevoStock navigation page you considered earlier, but this time with an ImageSet value of TreeViewImageSet.Faq.

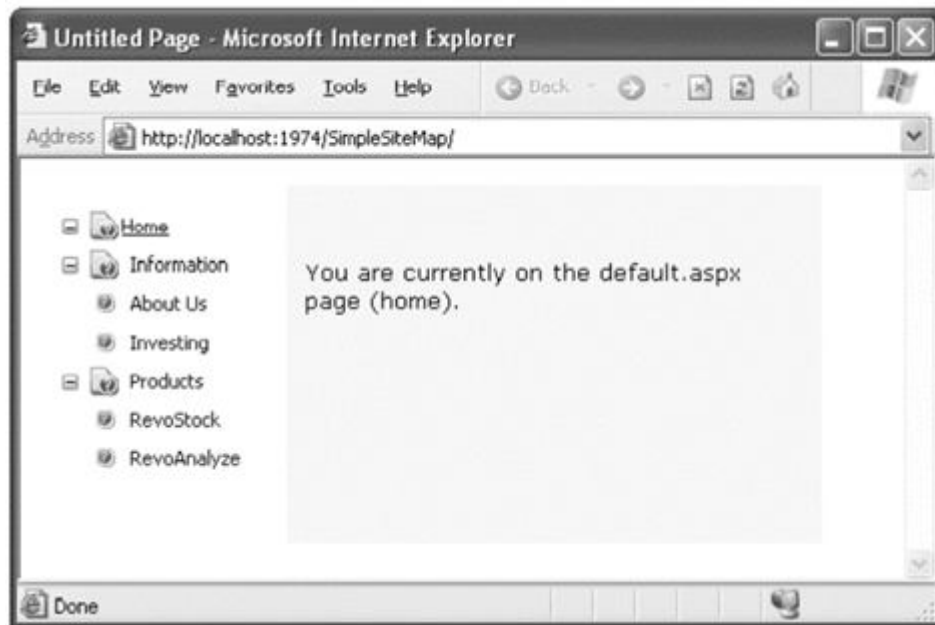


Figure10: shows a page with several TreeViews, each of which represents one of the options in the Auto Format window.



Figure 11. Different looks for a TreeView

Table4 lists some of the most useful properties of the TreeView.

Property	Description
MaxDataBindDepth	Determines how many levels the TreeView will show. By default, MaxDataBindDepth is -1.
ExpandDepth	specify how many levels of nodes will be visible at first.
NodeIndent	Sets the number of pixels between each level of nodes in the TreeView.
ImageSet	use a predefined collection of node images for collapsed, expanded, and nonexpandable nodes.
CollapseImageUrl, ExpandImageUrl, and NoExpandImageUrl	Sets the pictures that are shown next to nodes for collapsed nodes (CollapseImageUrl) and expanded nodes (ExpandImageUrl). The NoExpandImageUrl is used if the node doesn't have any children.
NodeWrap	Lets a node text wrap over more than one line when set to true.
ShowExpandCollapse	Hides the expand/collapse boxes when set to false.
ShowLines	ShowLines Adds lines that connect every node when set to true.
ShowCheckBoxes	Shows a check box next to every node when set to true.

Treeview styles:

The TreeNodeStyle class adds the node-specific style properties shown in Table. These properties deal with the node image and the spacing around a node.

Table5. TreeNodeStyle-Added Properties

Property	Description
ImageUrl	The URL for the image shown next to the node.
NodeSpacing.	The space (in pixels) between the current node and the node above and below.

VerticalPadding	The space (in pixels) between the top and bottom of the node text and border around the text.
HorizontalPadding	The space (in pixels) between the left and right of the node text and border around the text.
ChildNodesPadding	The space (in pixels) between the last child node of an expanded parent node and the following node.

One other property that comes into play is `TreeView.NodeIndent`, which sets the number of pixels of indentation in each subsequent level of the tree hierarchy. Figure 11 shows how these settings apply to a single node.

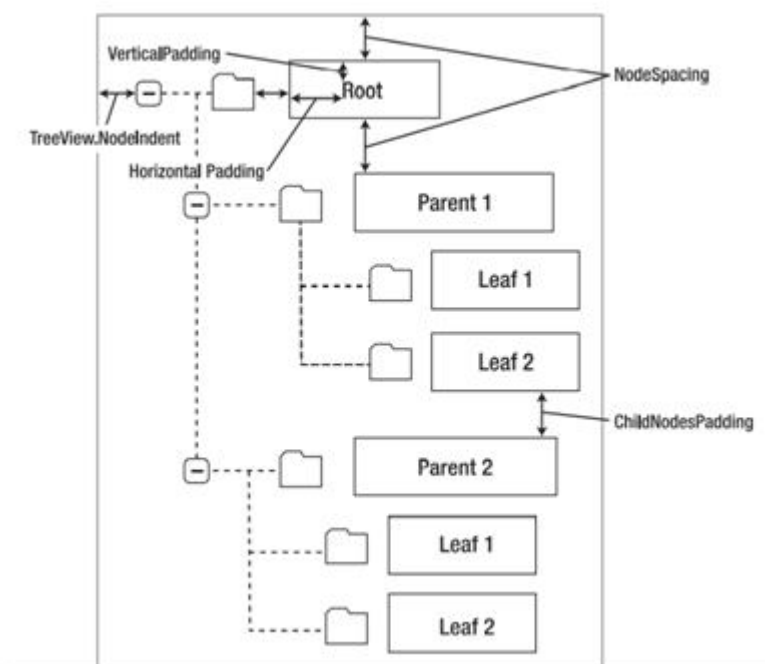


Figure 12. Node spacing

To apply a simple TreeView makeover, and to use the same style settings for each node in the TreeView, you apply style settings through the `TreeView.NodeStyle` property. For example, here's a TreeView that applies a custom font, font size, text color, padding, and spacing:

```
<asp:TreeView ID="TreeView1" runat="server" DataSourceID="SiteMapDataSource1">

<NodeStyle Font-Names="Tahoma" Font-Size="10pt" ForeColor="Blue"

HorizontalPadding="5px" NodeSpacing="0px" VerticalPadding="0px" />

</asp:TreeView>
```

Applying styles to node types

The TreeView allows you to individually control the styles for types of nodes. Table6 lists different TreeView styles and explains what nodes they affect.

Table6. TreeView Style Properties

Property	Description
NodeStyle	Applies to all nodes.
RootNodeStyle	Applies only to the first-level (root) node.
ParentNodeStyle	Applies to any node that contains other nodes, except root nodes.
LeafNodeStyle	Applies to any node that doesn't contain child nodes and isn't a root node.
SelectedNodeStyle	Applies to the currently selected node.
HoverNodeStyle	Applies to the node the user is hovering over with the mouse.

A sample TreeView that first defines a few standard style characteristics using the NodeStyle property.

```
<asp:TreeView ID="TreeView1" runat="server" DataSourceID="SiteMapDataSource1">

<NodeStyle Font-Names="Tahoma" Font-Size="10pt" ForeColor="Blue"

HorizontalPadding="5px" NodeSpacing="0px" VerticalPadding="0px" />

<ParentNodeStyle Font-Bold="False" />

<HoverNodeStyle Font-Underline="True" ForeColor="#5555DD" />
```

```
<SelectedNodeStyle Font-Underline="True" ForeColor="#5555DD" />
```

```
</asp:TreeView>
```

Applying styles to node levels

For example, here's a TreeView that differentiates levels by applying different amounts of spacing and different fonts:

```
<asp:TreeViewrunat="server" HoverNodeStyle-Font-Underline="True"
```

```
ShowExpandCollapse="False" NodeIndent="3" DataSourceID="SiteMapDataSource1">
```

```
<LevelStyles>
```

```
<asp:TreeNodeStyleChildNodesPadding="10" Font-Bold="True" Font-Size="12pt"
```

```
ForeColor="DarkGreen"/>
```

```
<asp:TreeNodeStyleChildNodesPadding="5" Font-Bold="True" Font-Size="10pt" />
```

```
<asp:TreeNodeStyleChildNodesPadding="5" Font-UnderLine="True"
```

```
Font-Size="10pt" />
```

```
</LevelStyles>
```

```
</asp:TreeView>
```

If you apply this to the category and product list shown in earlier examples, you'll see a page like the one shown in Figure 11.



Figure 13. A TreeView with styles

8.6 The Menu Control

The Menu control is used to create a menu of hierarchical data that can be used to navigate through the pages. The Menu control conceptually contains two types of items. First is StaticMenu that is always displayed on the page, SymanicMenu that appears when opens the parent item. Its properties like BackColor, ForeColor, BorderColor, BorderStyle, BorderWidth, Heitedough style propertes of <table, tr, td/> tag.

Table 7. Menu control properties

DataSourceID	Indicates the data source to be used (You can use .sitemap file as datasource).
Text	Indicates the text to display in the menu.
Tooltip	Indicates the tooltip of the menu item when you mouse over.
Value	Indicates the nondisplayed value (usually unique id to use in server-side events)
NavigateUrl	Indicates the target location to send the user when menu item is clicked. If not set you can handle MenuItemClick event to decide what to do.
Target	If NavigationUrl property is set, it indicates where to open the target location (in new window or same window).

Selectable	true/false. If false, this item can't be selected. Usually in case of this item has some child.
ImageUrl	Indicates the image that appears next to the menu item.
ImageToolTip	Indicates the tooltip text to display for image next to the item.
PopOutImageUrl	Indicates the image that is displayed right to the menu item when it has some subitems.
Target	If NavigationUrl property is set, it indicates where to open the target location (in new window or same window).

Menu styles

The key distinction that the Menu control encourages you to adopt is between static items (the root-level items that are displayed in the page when it's first generated) and dynamic items (the items in fly-out menus that are added when the user moves the mouse over a portion of the menu).

Table8. Menu Styles

Static Style	Dynamic Style	Description
StaticMenuStyle	DynamicMenuStyle.	Sets the appearance of the overall "box" in which all the menu items appear.
StaticMenuItemStyle	DynamicMenuItemStyle	Sets the appearance of individual menu items.
StaticSelectedStyle	DynamicSelectedStyle	Sets the appearance of the selected item.
StaticHoverStyle	DynamicHoverStyle	Sets the appearance of the item that the user is hovering over with the mouse.

Menu template

The full template that uses both types of data binding expressions to show the top-level of static menu items and the second level of pop-up (dynamic) items:

```

<asp:Menu ID="Menu1" runat="server" DataSourceID="SiteMapDataSource1">

<StaticItemTemplate>

<%# Eval("Text") %><br />

<small>

<%# GetDescriptionFromTitle(((MenuItem)Container.DataItem).Text) %>

</small>

</StaticItemTemplate>

<DynamicItemTemplate>

<%# Eval("Text") %><br />

<small>

<%# GetDescriptionFromTitle(((MenuItem)Container.DataItem).Text) %>

</small>

</DynamicItemTemplate>

</asp:Menu>

```

The next step is to create the `GetDescriptionFromTitle()` method in the code for your page class. This method belongs in the page that has the Menu control, which, in this example, is the master page. The `GetDescriptionFromTitle()` method must also have protected (or public) accessibility, so that ASP.NET can call it during the data binding process. The complete code that makes this example work:

```

protected string GetDescriptionFromTitle(string title)

{

    // This assumes there's only one node with this title.

    SiteMapNode startingNode = SiteMap.RootNode;

```

```
SiteMapNodematchNode = SearchNodes(startingNode, title);

if (matchNode == null)

{
    return null;
}

else

{
    return matchNode.Description;
}
}

private SiteMapNodeSearchNodes(SiteMapNode node, string title)
{
    if (node.Title == title)
    {
        return node;
    }

    else

    {
        // Perform recursive search.

        foreach (SiteMapNode child in node.ChildNodes)
        {
            SiteMapNodematchNode = SearchNodes(child, title);

            // Was a match found?
```

```

// If so, return it.

if (matchNode != null) return matchNode;

}

// All the nodes were examined, but no match was found.

return null;

}

}

```

8.7 ADO.NET Fundamentals

UNDERSTANDING DATABASES

In most ASP.NET applications, you'll need to use a database for some tasks. Here are some basic examples of data at work in a web application:

- E-commerce sites (like Amazon) use detailed databases to store product catalogs. They also track orders, customers, shipment records, and inventory information in a huge arrangement of related tables.
- Search engines (like Google) use databases to store indexes of page URLs, links and keywords.
- Knowledge bases (like Microsoft Support) use less structured databases that store vast quantities of information or links to various documents and resources.
- Media sites (like The New York Times) store their articles in databases.

CONFIGURING YOUR DATABASE

Browsing and modifying databases in visual studio

Using the Data Connections node in the Server Explorer, you can connect to existing databases or create new ones. Assuming you've installed the pubs database (see the readme.txt file for instructions), you can create a connection to it by following these steps:

1. Right-click the Data Connections node, and choose Add Connection. If the Choose Data

Source window appears, select Microsoft SQL Server and then click Continue.

2. If you're using SQL Server Express, you'll need to use the server name `localhost\SQLEXPRESS` instead, as shown in Figure. The `SQLEXPRESS` part indicates that you're connecting to a named instance of SQL Server. By default, this is the way that SQL Server Express configures itself when you first install it.

3. Click Test Connection to verify that this is the location of your database.

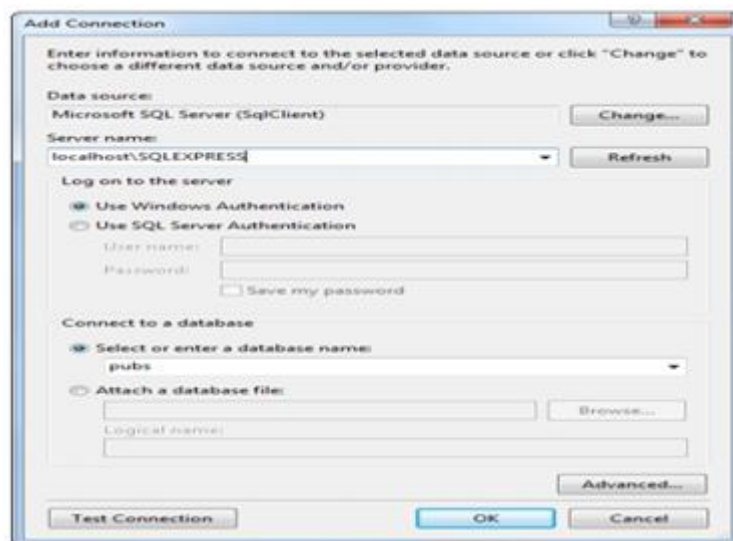


Figure14. Creating a connection in Visual Studio

4. In the Select or Enter a Database Name list, choose the pubs database. If you want to see more than one database in Visual Studio, you'll need to add more than one data connection.

5. Click OK. The database connection will appear in the Server Explorer window. For example, if you right-click a table and choose Show Table Data, you'll see a grid of records that you can browse and edit, as shown in Figure13.

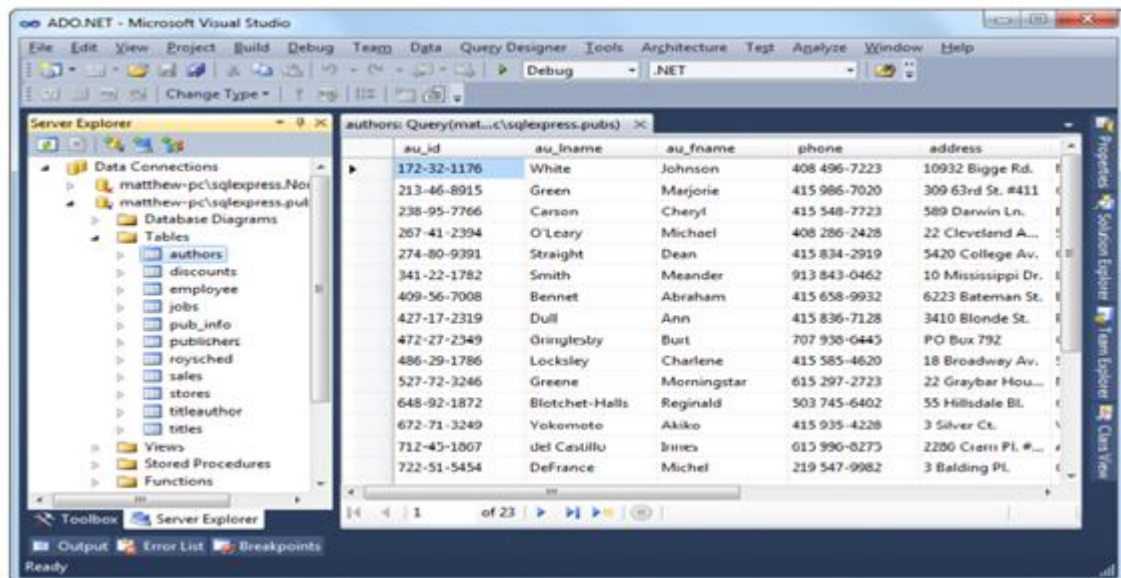


Figure15. Editing table data in Visual Studio

The sqlcmd command-line tool

Two commonly used sqlcmd parameters are `-S` (which specifies the location of your databaseserver) and `-i` (which supplies a script file with SQL commands that you want to run). For example, the downloadable code samples include a file named `InstPubs.sql` that contains the commands you need to create the pubs database and fill it with sample data. If you're using SQL Server Express, you can run the `InstPubs.sql` script using this command:

```
sqlcmd -S localhost\SQLEXPRESS -iInstPubs.sql
```

8.8 SQL BASICS

When working with ADO.NET, however, you'll probably use only the following standard types of SQL statements:

- A Select statement retrieves records.
- An Update statement modifies existing records.
- An Insert statement adds a new record.
- A Delete statement deletes existing records.

Running queries in visual studio

1. Right-click your connection, and choose New Query.
2. Choose the table (or tables) you want to use in your query from the Add Table dialog box (as shown in Figure14), click Add, and then click Close.

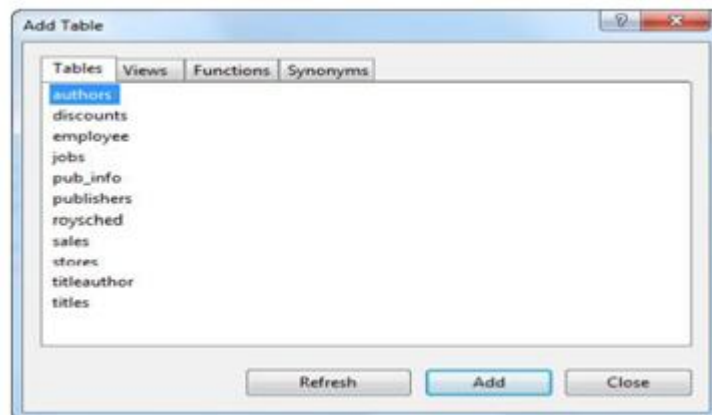


Figure16. Adding tables to a query

3. You can create your query by adding check marks next to the fields you want, or you can edit the SQL by hand in the lower portion of the window.
4. When you're ready to run the query, select Query Designer & Execute SQL from the menu. If you're selecting records, the results will appear at the bottom of the window. If you're deleting or updating records, a message box will appear informing you how many records were affected (see Figure15).

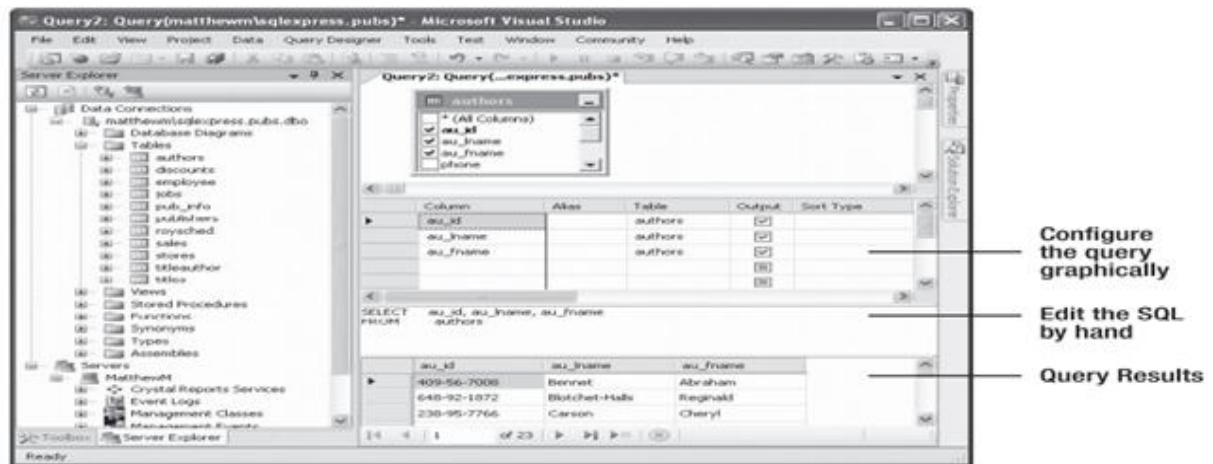


Figure17. Executing a query

The select statement

To retrieve one or more rows of data, you use a Select statement. A basic Select statement has the following structure:

SELECT [columns]

FROM [tables]

WHERE [search_condition]

ORDER BY [order_expression ASC | DESC]

2.3.3. A Sample Select Statement

SELECT * FROM Authors

- The asterisk (*) retrieves all the columns in the table.
- The From clause identifies that the Authors table is being used for this statement.

The Where Clause:

The following is an example with a different table and a more sophisticated Where statement:

SELECT * FROM Sales WHERE ord_date < '2000/01/01' AND ord_date > '1987/01/01'.

String Matching with the Like Operator

```
SELECT * FROM Stores WHERE stor_name LIKE 'B%'
```

Aggregate Queries

lists the most commonly used aggregate functions.

Table 9. SQL Aggregate Functions

Fuction	Description
Avg	Calculates the average of all values in a given numeric field
Sum	Calculates the sum of all values in a given numeric field
Max and min	Finds the minimum or maximum value in a number field
Count *	Returns the number of rows in the result set
Count(Distinct fieldname)	Returns the number of unique rows in the result set for the specified field

The SQL Update Statement

The SQL Update statement selects all the records that match a specified search expression and then modifies them all according to an update expression.

```
UPDATE [table] SET [update_expression] WHERE [search_condition]
```

The SQL Insert Statement

The SQL Insert statement adds a new record to a table with the information you specify. It takes the following form:

```
INSERT INTO [table] ([column_list]) VALUES ([value_list])
```

The SQL Delete Statement

The Delete statement is even easier to use. It specifies criteria for one or more rows that you want to remove.

```
DELETE FROM [table] WHERE [search_condition]
```

THE DATA PROVIDER MODEL

ADO.NET relies on the functionality in a small set of core classes. You can divide these classes into two groups: those that are used to contain and manage data (such as `DataSet`, `DataTable`, `DataRow`, and `DataRelation`) and those that are used to connect to a specific data source (such as `Connection`, `Command`, and `DataReader`). The second group of classes exists in several different flavors. Each set of data interaction classes is called an ADO.NET data provider.

Table 10. ADO.NET Namespaces for SQL Server Data Access

Namespace	Purpose
<code>System.Data.SqlClient</code>	Contains the classes you use to connect to a Microsoft SQL Server database and execute commands.
<code>System.Data.SqlTypes</code>	Contains structures for SQL Server–specific data types such as <code>SqlMoney</code> and <code>SqlDateTime</code> .
<code>System.Data</code>	This includes <code>DataSet</code> and <code>DataRelation</code> , which allow you to manipulate structured relational data.

DIRECT DATA ACCESS

To query information with simple data access, follow these steps:

1. Create `Connection`, `Command`, and `DataReader` objects.
2. Use the `DataReader` to retrieve information from the database, and display it in a control on a web form.
3. Close your connection.
4. Send the page to the user. At this point, the information your user sees and the information in the database no longer have any connection, and all the ADO.NET objects have been destroyed.

To add or update information, follow these steps:

1. Create new `Connection` and `Command` objects.
2. Execute the `Command` (with the appropriate SQL statement).

Figure17 shows a high-level look at how the ADO.NET objects interact to make direct data access work.²

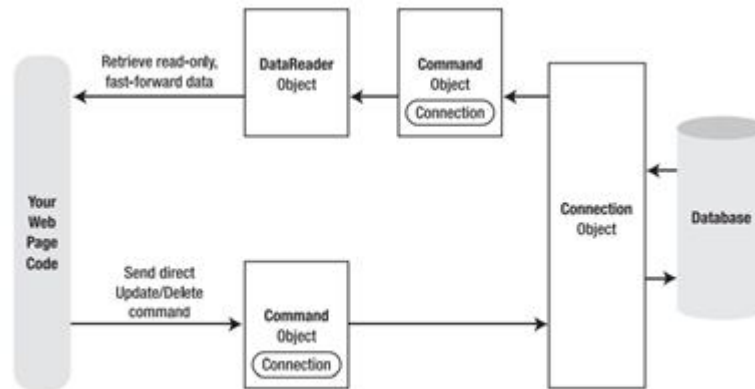


Figure18. Direct data access with ADO.NET

Creating a connection

When creating a Connection object, you need to specify a value for its `ConnectionString` property. This `ConnectionString` defines all the information the computer needs to find the data source, log in, and choose an initial database. s an example that uses a connection string to connect to SQL Server:

```
SqlConnection myConnection = new SqlConnection();

myConnection.ConnectionString = "Data Source=localhost;" +
    "Initial Catalog=Pubs;Integrated Security=SSPI";
```

If you're using SQL Server Express, your connection string will include an instance name, as shown here:

```
SqlConnection myConnection = new SqlConnection();

myConnection.ConnectionString = @"Data Source=localhost\SQLEXPRESS;" +
    "Initial Catalog=Pubs;Integrated Security=SSPI";
```


The connection string

The connection string is actually a series of distinct pieces of information separated by semicolons (;). Each separate piece of information is known as a connection string property. The following list describes some of the most commonly used connection string properties, including the three properties used in the preceding example:

- Initial catalog
- Integrated security
- ConnectionTimeout

windows authentication

With SQL Server authentication, SQL Server maintains its own user account information in the database. It uses this information to determine whether you are allowed to access specific parts of a database.

With integrated Windows authentication, SQL Server automatically uses the Windows account information for the currently logged-in process. In the database, it stores information about what database privileges each user should have.

user instance connections

Interestingly, SQL Server Express has a feature that lets you bypass the master list and connect directly to any database file, even if it's not in the master list of databases. This feature is called user instances. An example connection string that uses this approach:

```
myConnection.ConnectionString = @"Data Source=localhost\SQLEXPRESS;" +
    @"User Instance=True;AttachDBFilename=|DataDirectory|\Northwind.mdf;" +
    "Integrated Security=True";
```

storing the connection string

The <connectionStrings> section of the web.config file is a handy place to store your connection strings. Here's an example:

```
<configuration>
```

```
<connectionStrings>
```

```
<add name="Pubs" connectionString=
```

```
"Data Source=localhost;Initial Catalog=Pubs;Integrated Security=SSPI"/>
```

```
</connectionStrings>
```

```
...
```

```
</configuration>
```

making the connection

Before you can perform any database operations, you need to explicitly open your connection:

```
myConnection.Open();
```

Closing a connection is just as easy, as shown here:

```
myConnection.Close();
```

```
// Define the ADO.NET Connection object.
```

```
string connectionString =
```

```
    WebConfigurationManager.ConnectionStrings["Pubs"].ConnectionString;
```

```
SqlConnection myConnection = new SqlConnection(connectionString);
```

```
try
```

```
{
```

```
    using (myConnection)
```

```
    {
```

```
        // Try to open the connection.
```

```
        myConnection.Open();
```

```

lblInfo.Text = "<b>Server Version:</b> " + myConnection.ServerVersion;

lblInfo.Text += "<br /><b>Connection Is:</b> " +

myConnection.State.ToString();

}

}

catch (Exception err)

{

    // Handle an error by displaying the information.

    lblInfo.Text = "Error reading the database. ";

    lblInfo.Text += err.Message;

}

lblInfo.Text += "<br /><b>Now Connection Is:</b> ";

lblInfo.Text += myConnection.State.ToString();

```

The select command

The Connection object provides a few basic properties that supply information about the connection. To actually retrieve data, you need a few more ingredients:

- A SQL statement that selects the information you want
- A Command object that executes the SQL statement

The DataReader

- A DataReader or DataSet object to access the retrieved records

To create a DataReader, you use the ExecuteReader() method of the command object, as shown here:

// You don't need the new keyword, as the Command will create the DataReader.

```
SqlDataReadermyReader;
```

```
myReader = myCommand.ExecuteReader();
```

The next example demonstrates how you can use all the ADO.NET ingredients together to create a simple application that retrieves information from the Authors table. You can select an author record by last name using a drop-down list box, as shown in Figure 18.

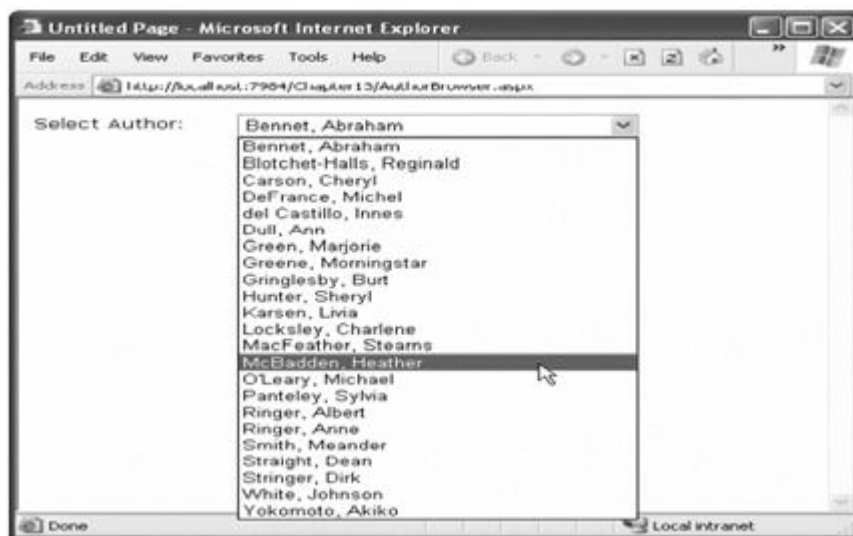


Figure 19. Selecting an author

Filling the List Box

The list box is filled when the Page.Load event occurs. Because the list box is set to persist its view state information, this information needs to be retrieved only once—the first time the page is displayed. It will be ignored on all postbacks. Here's the code that fills the list from the database:

```
protected void Page_Load(Object sender, EventArgs e)
{
    if (!this.IsPostBack)
    {
```

```
FillAuthorList();

}

}

private void FillAuthorList()

{

    lstAuthor.Items.Clear();

    // Define the Select statement.

    // Three pieces of information are needed: the unique id

    // and the first and last name.

    string selectSQL = "SELECT au_lname, au_fname, au_id FROM Authors";

    // Define the ADO.NET objects.

    SqlConnection con = new SqlConnection(connectionString);

    SqlCommand cmd = new SqlCommand(selectSQL, con);

    SqlDataReader reader;

    // Try to open database and read information.

    try

    {

        con.Open();

        reader = cmd.ExecuteReader();

        // For each item, add the author name to the displayed

        // list box text, and store the unique ID in the Value property.
```

364

```
while (reader.Read())

{

ListItem newItem = new ListItem();

newItem.Text = reader["au_lname"] + ", " + reader["au_fname"];

newItem.Value = reader["au_id"].ToString();

lstAuthor.Items.Add(newItem);

}

reader.Close();

}

catch (Exception err)

{

lblResults.Text = "Error reading list of names. ";

lblResults.Text += err.Message;

}

finally

{

con.Close();

}

}
```

Retrieving the Record

The record is retrieved as soon as the user changes the selection in the list box. To make this possible, the `AutoPostBack` property of the list box is set to true so that its change events are detected automatically.

```

protected void lstAuthor_SelectedIndexChanged(Object sender, EventArgs e)
{
    // Create a Select statement that searches for a record
    // matching the specific author ID from the Value property.
    string selectSQL;
    selectSQL = "SELECT * FROM Authors ";
    selectSQL += "WHERE au_id=" + lstAuthor.SelectedItem.Value + "";
    // Define the ADO.NET objects.
    SqlConnection con = new SqlConnection(connectionString);
    SqlCommand cmd = new SqlCommand(selectSQL, con);
    SqlDataReader reader;
    // Try to open database and read information.
    try
    {
        con.Open();
        reader = cmd.ExecuteReader();
        reader.Read();
        // Build a string with the record information,
        // and display that in a label.
        StringBuilder sb = new StringBuilder();
        sb.Append("<b>");
        sb.Append(reader["au_lname"]);
        sb.Append(", ");
        sb.Append(reader["au_fname"]);
    }
}

```

366

```
sb.Append("</b><br />");
sb.Append("Phone: ");
sb.Append(reader["phone"]);
sb.Append("<br />");
sb.Append("Address: ");
sb.Append(reader["address"]);
sb.Append("<br />");
sb.Append("City: ");
sb.Append(reader["city"]);
sb.Append("<br />");
sb.Append("State: ");
sb.Append(reader["state"]);
sb.Append("<br />");
lblResults.Text = sb.ToString();
reader.Close();
}
catch (Exception err)
{
    lblResults.Text = "Error getting author. ";
    lblResults.Text += err.Message;
}
finally
{
    con.Close();
```



```
}
}
```

DISPLAYING VALUES IN TEXT BOXES

Before you can update and insert records, you need to make a change to the previous example. Instead of displaying the field values in a single, fixed label, you need to show each detail in a separate text box. Figure19 shows the revamped page. It includes two new buttons that allow you to update the record (Update) or delete it (Delete), and two more that allow you to begin creating a new record (Create New) and then insert it (Insert New).

Figure20. A more advanced author manager

t boxes:

```
protected void lstAuthor_SelectedIndexChanged(Object sender, EventArgs e)
```

```
{
```

```
// Define ADO.NET objects.
```

```
string selectSQL;
```

```
selectSQL = "SELECT * FROM Authors ";
```

```
selectSQL += "WHERE au_id=" + lstAuthor.SelectedItem.Value + """;

SqlConnection con = new SqlConnection(connectionString);

SqlCommand cmd = new SqlCommand(selectSQL, con);

SqlDataReader reader;

// Try to open database and read information.

try
{
    con.Open();

    reader = cmd.ExecuteReader();

    reader.Read();

    // Fill the controls.

    txtID.Text = reader["au_id"].ToString();

    txtFirstName.Text = reader["au_fname"].ToString();

    txtLastName.Text = reader["au_lname"].ToString();

    txtPhone.Text = reader["phone"].ToString();

    txtAddress.Text = reader["address"].ToString();

    txtCity.Text = reader["city"].ToString();

    txtState.Text = reader["state"].ToString();

    txtZip.Text = reader["zip"].ToString();

    chkContract.Checked = (bool)reader["contract"];

    reader.Close();

    lblStatus.Text = "";
```

```

    }

    catch (Exception err)

    {

        lblStatus.Text = "Error getting author. ";

        lblStatus.Text += err.Message;

    }

    finally

    {

        con.Close();

    }

}

```

Adding a Record

To start adding a new record, click Create New to clear all the text boxes. Technically this step isn't required, but it simplifies the user's life:

```

protected void cmdNew_Click(Object sender, EventArgs e)

{

    txtID.Text = "";

    txtFirstName.Text = "";

    txtLastName.Text = "";

    txtPhone.Text = "";

    txtAddress.Text = "";

    txtCity.Text = "";

```

370

```
txtState.Text = "";

txtZip.Text = "";

chkContract.Checked = false;

lblStatus.Text = "Click Insert New to add the completed record.";

}
```

The Insert New button triggers the ADO.NET code that inserts the finished record using a dynamically generated Insert statement:

```
protected void cmdInsert_Click(Object sender, EventArgs e)

{

    // Perform user-defined checks.

    // Alternatively, you could use RequiredFieldValidator controls.

    if (txtID.Text == "" || txtFirstName.Text == "" || txtLastName.Text == "")

    {

        lblStatus.Text = "Records require an ID, first name, and last name.";

        return;

    }

    // Define ADO.NET objects.

    string insertSQL;

    insertSQL = "INSERT INTO Authors (";

    insertSQL += "au_id, au_fname, au_lname, ";

    insertSQL += "phone, address, city, state, zip, contract) ";

    insertSQL += "VALUES (";
```

```

insertSQL += txtID.Text + “, “;

insertSQL += txtFirstName.Text + “, “;

insertSQL += txtLastName.Text + “, “;

insertSQL += txtPhone.Text + “, “;

insertSQL += txtAddress.Text + “, “;

insertSQL += txtCity.Text + “, “;

insertSQL += txtState.Text + “, “;

insertSQL += txtZip.Text + “, “;

insertSQL += Convert.ToInt16(chkContract.Checked) + “)”;

SqlConnection con = new SqlConnection(connectionString);

SqlCommand cmd = new SqlCommand(insertSQL, con);

// Try to open the database and execute the update.

int added = 0;

try

{

con.Open();

added = cmd.ExecuteNonQuery();

lblStatus.Text = added.ToString() + “ records inserted.”;

}

catch (Exception err)

{

```

372

```
lblStatus.Text = "Error inserting record. ";  
  
lblStatus.Text += err.Message;  
  
}  
  
finally  
  
{  
  
con.Close();  
  
}  
  
// If the insert succeeded, refresh the author list.  
  
if (added > 0)  
  
{  
  
FillAuthorList();  
  
}  
  
}
```

Creating More Robust Commands

Two potentially serious drawbacks:

- Users may accidentally enter characters that will affect your SQL statement. Foreexample, if a value contains an apostrophe ('), the pasted-together SQL string will no longer be valid.
- Users might deliberately enter characters that will affect your SQL statement. Examples include using the single apostrophe to close a value prematurely and then following the value with additional SQL code.

Updating a Record

When the user clicks the Update button, the information in the text boxes is applied to the database as follows:

```

protected void cmdUpdate_Click(Object sender, EventArgs e)
{
    // Define ADO.NET objects.

    string updateSQL;

    updateSQL = "UPDATE Authors SET ";

    updateSQL += "au_fname=@au_fname, au_lname=@au_lname, ";

    updateSQL += "phone=@phone, address=@address, city=@city, state=@state, ";

    updateSQL += "zip=@zip, contract=@contract ";

    updateSQL += "WHERE au_id=@au_id_original";

    SqlConnection con = new SqlConnection(connectionString);

    SqlCommand cmd = new SqlCommand(updateSQL, con);

    // Add the parameters.

    cmd.Parameters.AddWithValue("@au_fname", txtFirstName.Text);

    cmd.Parameters.AddWithValue("@au_lname", txtLastName.Text);

    cmd.Parameters.AddWithValue("@phone", txtPhone.Text);

    cmd.Parameters.AddWithValue("@address", txtAddress.Text);

    cmd.Parameters.AddWithValue("@city", txtCity.Text);

    cmd.Parameters.AddWithValue("@state", txtState.Text);

    cmd.Parameters.AddWithValue("@zip", txtZip.Text);

    cmd.Parameters.AddWithValue("@contract", chkContract.Checked);

    cmd.Parameters.AddWithValue("@au_id_original",

lstAuthor.SelectedItem.Value);

```

374

```
// Try to open database and execute the update.

int updated = 0;

try
{
    con.Open();

    updated = cmd.ExecuteNonQuery();

    lblStatus.Text = updated.ToString() + " record updated.";
}

catch (Exception err)
{
    lblStatus.Text = "Error updating author. ";

    lblStatus.Text += err.Message;
}

finally
{
    con.Close();
}

// If the update succeeded, refresh the author list.

if (updated > 0)
{
    FillAuthorList();
}
}
```


Deleting a Record

When the user clicks the Delete button, the author information is removed from the database. The number of affected records is examined, and if the delete operation was successful, the FillAuthorList()

function is called to refresh the page.

```
protected void cmdDelete_Click(Object sender, EventArgs e)

{

    // Define ADO.NET objects.

    string deleteSQL;

    deleteSQL = "DELETE FROM Authors ";

    deleteSQL += "WHERE au_id=@au_id";

    SqlConnection con = new SqlConnection(connectionString);

    SqlCommand cmd = new SqlCommand(deleteSQL, con);

    cmd.Parameters.AddWithValue("@au_id", lstAuthor.SelectedItem.Value);

    // Try to open the database and delete the record.

    int deleted = 0;

    try

    {

        con.Open();

        deleted = cmd.ExecuteNonQuery();

    }

    catch (Exception err)
```

376

```
{  
  
lblStatus.Text = "Error deleting author. ";  
  
lblStatus.Text += err.Message;  
  
}  
  
finally  
  
{  
  
con.Close();  
  
}  
  
// If the delete succeeded, refresh the author list.  
  
if (deleted > 0)  
{  
  
FillAuthorList();  
  
}  
  
}
```

DISCONNECTED DATA ACCESS

When you use disconnected data access, you keep a copy of your data in memory using the `DataSet`. You connect to the database just long enough to fetch your data and dump it into the `DataSet`, and then you disconnect immediately. There are a variety of good reasons to use the `DataSet` to hold onto data in memory. Here are a few:

- You need to do something time-consuming with the data.
- You want to use ASP.NET data binding to fill a web control (like a `GridView`) with your data. Although you can use the `DataReader`, it won't work in all scenarios.
- You want to navigate backward and forward through your data while you're processing it. This

isn't possible with the `DataReader`, which goes in one direction only—forward.

- You want to navigate from one table to another. Using the `DataSet`, you can store several tables of information.
- You want to save the data to a file for later use. The `DataSet` includes two methods—`WriteXml()` and `ReadXml()`—that allow you to dump the content to a file and convert it back to a live database object later.

Selecting Disconnected Data

With disconnected data access, a copy of the data is retained in memory while your code is running. Figure shows a model of the `DataSet`. You fill the `DataSet` in much the same way that you connect a `DataReader`. However, although the `DataReader` holds a live connection, information in the `DataSet` is always disconnected. The following example shows how you could rewrite the `FillAuthorList()` method from the earlier example to use a `DataSet` instead of a `DataReader`. The changes are highlighted in bold.

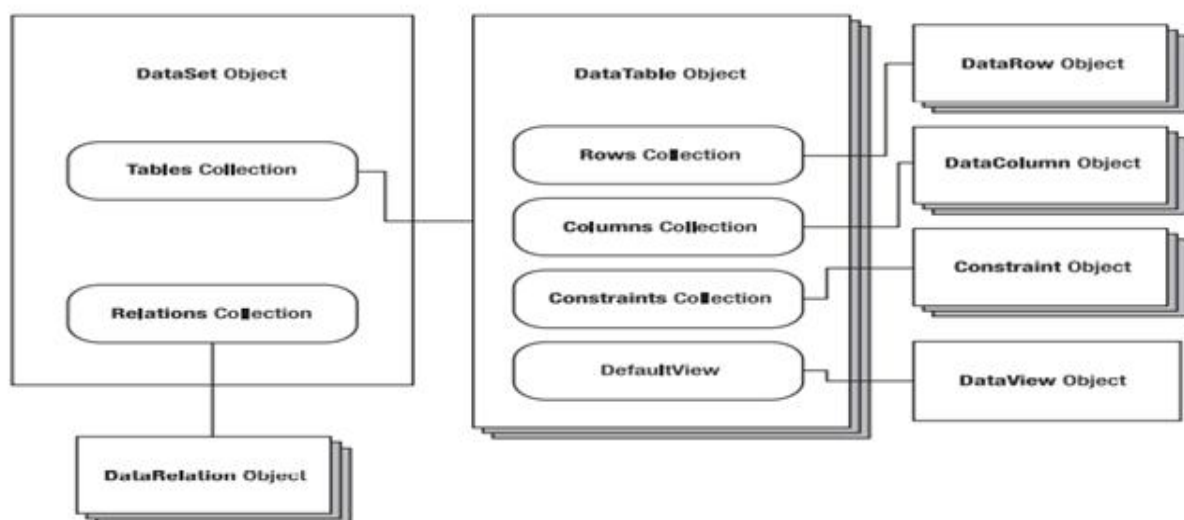


Figure 21. The `DataSet` family of objects

```

private void FillAuthorList()
{
    lstAuthor.Items.Clear();

```

378

```
// Define ADO.NET objects.

string selectSQL;

selectSQL = "SELECT au_lname, au_fname, au_id FROM Authors";

SqlConnection con = new SqlConnection(connectionString);

SqlCommand cmd = new SqlCommand(selectSQL, con);

SqlDataAdapter adapter = new SqlDataAdapter(cmd);

DataSet dsPubs = new DataSet();

// Try to open database and read information.

try

{

con.Open();

// All the information is transferred with one command.

// This command creates a new DataTable (named Authors)

// inside the DataSet.

adapter.Fill(dsPubs, "Authors");

}

catch (Exception err)

{

lblStatus.Text = "Error reading list of names. ";

lblStatus.Text += err.Message;

}
```

```

finally
{
con.Close();
}

foreach (DataRow row in dsPubs.Tables["Authors"].Rows)
{
ListItem newItem = new ListItem();

newItem.Text = row["au_lname"] + ", " +
row["au_fname"];

newItem.Value = row["au_id"].ToString();

lstAuthor.Items.Add(newItem);
}
}

```

The `DataAdapter.Fill()` method takes a `DataSet` and inserts one table of information. In this case, the table is named `Authors`, but any name could be used. That name is used later to access the appropriate table in the `DataSet`. To access the individual `DataRows`, you can loop through the `Rows` collection of the appropriate table. Each piece of information is accessed using the field name, as it was with the `DataReader`.

Selecting Multiple Tables

In the `pubs` database, authors are linked to titles using three tables. This arrangement (called a many-to-many relationship, shown in Figure 21)

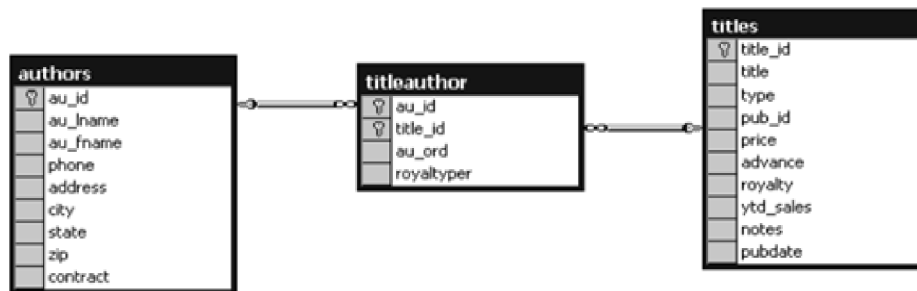


Figure 22. A many-to-many relationship

Defining Relationships

To create a `DataRelation`, you need to specify the linked fields from two different tables, and you need to give your `DataRelation` a unique name. The order of the linked fields is important. The first field is the parent, and the second field is the child (one-to-many relationship.) In this example, each book title can have more than one entry in the `TitleAuthor` table. Each author can also have more than one entry in the `TitleAuthor` table:

```
DataRelationTitles_TitleAuthor = new DataRelation("Titles_TitleAuthor",
dsPubs.Tables["Titles"].Columns["title_id"],
dsPubs.Tables["TitleAuthor"].Columns["title_id"]);

DataRelationAuthors_TitleAuthor = new DataRelation("Authors_TitleAuthor",
dsPubs.Tables["Authors"].Columns["au_id"],
dsPubs.Tables["TitleAuthor"].Columns["au_id"]);
```

Once you've create these `DataRelation` objects, you must add them to the `DataSet`:

```
dsPubs.Relations.Add(Titles_TitleAuthor);

dsPubs.Relations.Add(Authors_TitleAuthor);
```

The remaining code loops through the `DataSet`. However, unlike the previous example, which moved through one table, this example uses the `DataRelation` objects to branch to the other linked tables. It works like this:

1. Select the first record from the Author table.
2. Using the Authors_TitleAuthor relationship, find the child records that correspond to this author. To do so, you call the `DataRow.GetChildRows()` method, and pass in the `DataRelationship` object that models the relationship between the Authors and TitleAuthor table.
3. For each matching record in TitleAuthor, look up the corresponding Title record to get the full text title. To do so, you call the `DataRow.GetParentRows()` method and pass in the `DataRelationship` object that connects the TitleAuthor and Titles table.
4. Move to the next Author record, and repeat the process. The code is lean and economical:

```
foreach (DataRow rowAuthor in dsPubs.Tables["Authors"].Rows)
{
    lblList.Text += "<br /><b>" + rowAuthor["au_fname"];

    lblList.Text += " " + rowAuthor["au_lname"] + "</b><br />";

    foreach (DataRow rowTitleAuthor in
        rowAuthor.GetChildRows(Authors_TitleAuthor))
    {
        DataRow rowTitle =
            rowTitleAuthor.GetParentRows(Titles_TitleAuthor)[0];

        lblList.Text += "&nbsp;&nbsp; ";

        lblList.Text += rowTitle["title"] + "<br />";
    }
}
```

Figure 22 shows the final result.



Figure 23. Hierarchical information from two tables

8.9 Summary

Then considered three controls that are specifically designed for navigation data: the SiteMapPath, TreeView, and Menu. The SqlConnection object lets you manage a connection to a data source. SqlCommand objects allow you to talk to a data source and send commands to it. To have fast forward-only read access to data, use the SqlDataReader. If you want to work with disconnected data, use a DataSet and implement reading and writing to/from the data source with a SqlDataAdapter.

Keywords

Node, page, sitemap, treeview, url, nodes, menu, information, navigation, data adapter

8.10 Check your progress

1. Which Data Provider gives the maximum performance when connect to SQL Server?

a. The SqlClient data provider.

b. The OLE DB data provider.

c. The Oracle data provider

d. All of the above.

2. Which ADO.NET class provide Connected Environment?

a. DataReader

b. DataSet

c. Command

d. None of the above

3. How do you execute multiple SQL statements using a DataReader?

a. Call the ExecuteReader method of a single Command object twice.

b. Set the Command.CommandText property to multiple SQL statements delimited by a semicolon.

c. Set the Command.CommandType property to multiple result sets.

d. None of the above.

4. Which SqlCommand execution returns the value of the first column of the first row from a table?

a. ExecuteNonQuery

b. ExecuteReader

c. ExecuteXmlReader

d. ExecuteScalar

5. How do you determine the actual SQL data type of a SqlParameter (the type expected by the SQL Server)?

a. It is the .NET Framework data type in your application that the parameter represents.

b. It is the type of column or data in SQL Server that the command expects.

- c. It is the type of column in a DataTable that it represents.
- d. It is any type defined in the SqlDbType enumeration.

8.11 Model questions

1. How do we use stored procedure in ADO.NET?
2. Differences between “DataSet” and “DataReader”.
3. Explain the basic methods of DataAdapter.
4. What are the ways to create connection in ADO.NET?
5. Define route url and route name.
6. Explain breadcrumb navigation.

Reference Book:

1. Mathew MacDonald, “Beginning ASP.NET 4 in C# 2010” Apress 2010.

LESSON - 9

DATA BINDING

Structure

- 9.1 Introduction
- 9.2 Objectives
- 9.3 Fundamentals of Data Binding
- 9.4 Repeated value data binding
- 9.5 Data Binding with ADO.NET
- 9.6 Data Source Controls
- 9.7 Handling Errors
- 9.8 Summary
- 9.9 Check your Progress
- 9.10 Model Questions

9.1 Introduction

In this chapter, you'll learn how to use data binding to display data more efficiently. You'll also learn how you to retrieve your data from database without writing a line of ADO.NET code by using ASP.NET data source controls.

9.2 Learning Objectives

- ✓ Single value databinding allows you to add information anywhere on the page dynamically.
- ✓ Repeated value data binding allows the user to display the entire table by using a special control Data source property.
- ✓ The user can do multiple data binding using multiple data controls.

- ✓ SqlDataSource allows you to connect to any data source that has an ADO.NET data provider.
- ✓ The DataSet mode allows the user to perform advanced sorting, filtering, and caching settings that depend on the DataSet.

9.3 Fundamentals of DATA BINDING

Data binding is attaching data to the data controls from the database.

Types of ASP.NET Data Binding

Two types of ASP.NET data binding exist:

- Single-value binding.
- Repeated – value binding.

Single-Value, or “Simple,” Data Binding

- single-value data binding is used to add information anywhere on an ASP.NET page.
- It allows to undertake a variable, a property, or an expression and insert it dynamically into a page.
- It also allows user to place information into a control property or as plain text inside an HTML tag.

Repeated-Value, or “List,” Binding

- Repeated-value data binding allows user to display an entire table or just a single field from a table.
- It requires special controls which provide support i.e. a Data Source property.

How Data Binding Works

- Data binding works a little differently depending on using single-value or repeated-value binding.
- To use single-value binding, user must insert a data binding expression into the markup in the .aspx file.

- To use repeated-value binding, user must set one or more properties of a data control, need to initialize when the Page.Load event fires.
- Once specified data binding should be activated by calling the DataBind() method.
- The DataBind() method is a basic piece of functionality supplied in the Control class. It automatically binds a control and any child controls that it contains.
- With repeated-value binding, user can bind the whole page at once by calling the DataBind() method of the current Page object. Once this method called, all the data binding expressions in the page are evaluated and replaced with the specified value.
- The DataBind() method is called in the Page.Load event handler, if not called ASP.NET will ignore data binding expressions, and the client will receive a page that contains empty values.

SINGLE-VALUE DATA BINDING

Single-value data binding is really just a different approach to dynamic text. To use it, you add special data binding expressions into your .aspx files. These expressions have the following format:

```
<%#expression_goes_here%>
```

This isn't a script. If user try to write any code inside this tag, will receive an error. The only thing user can add is a valid data binding expression. For example, if a variable named country declared in public or protected in the page, then it can be written as

```
<%# Country %>
```

When user calls the DataBind() method for the page, this text will be replaced with the value for Country (for example, Spain). Similarly, user could use a property or a built-in ASP.NET object as follows:

```
<%# Request.Browser.Browser %>
```

This would substitute a string with the current browser name (for example, IE). Otherwise, call a function defined on user's page, or execute a simple expression, provided it returns a result that can be converted to text and displayed on the page. Thus, the following data binding expressions are all valid:

```
<%# GetUserName(ID) %>
```

```
<%# 1 + (2 * 20) %>
```

```
<%# "John " + "Smith" %>
```

All these data binding expressions should be placed in the markup portion of your .aspx file, not in the code-behind file.

A Simple Data Binding Example

First start with a variable defined in the Page class, which is called TransactionCount:

```
public partial class SimpleDataBinding : System.Web.UI.Page
{
    protected int TransactionCount;

    // (Additional code omitted.)
}
```

If the variable not declared in public or protected it will not be accessed and evaluated when binding expression is called. Now, assume that this value is set in the Page.Load event handler using some database lookup code. For testing purposes, the example skips this step and hard-codes a value:

```
protected void Page_Load(object sender, EventArgs e)
{
    // (You could use database code here
    // to look up a value for TransactionCount.)

    TransactionCount = 10;

    // Now convert all the data binding expressions on the page.

    this.DataBind();
}
```

DataBind() can be called without the this keyword, because the default object is the current Page object. However, using the this keyword makes it a bit clearer what object is being used. To make this data binding accomplish something, need to add a data binding expression. Usually, it's easiest to add this value directly to the markup in the .aspx file. To do so, click the Source button at the bottom of the web page designer window. Below figure 1 shows an example with a Label control.

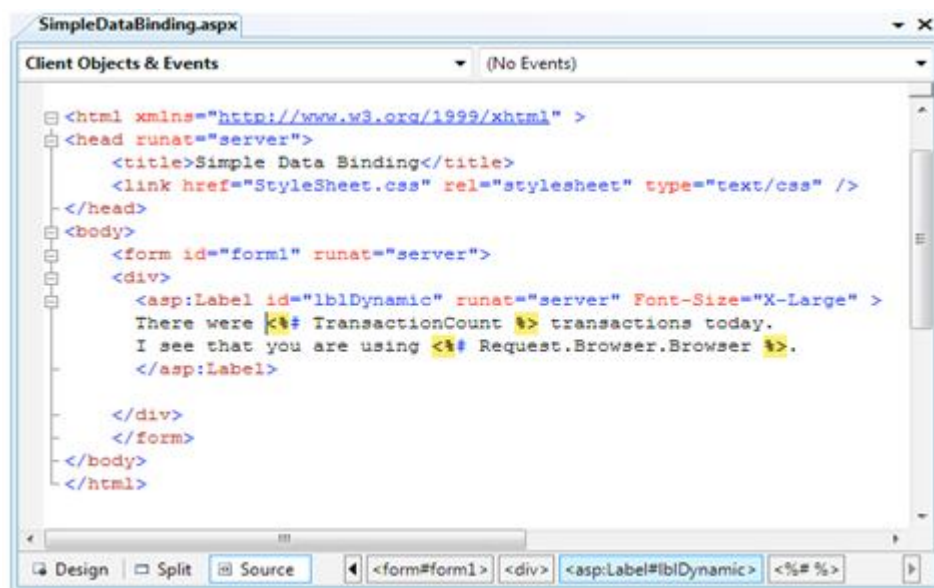


Figure 1. Source view in the web page designer

To add the expression, find the tag for the Label control. Modify the text inside the label as shown here:

```
<asp:Label id="lblDynamic" runat="server" Font-Size="X-Large">
```

```
There were <%# TransactionCount %> transactions today.
```

```
I see that you are using <%# Request.Browser.Browser %>.
```

```
</asp:Label>
```

This example uses two separate data binding expressions, which are inserted along with the normal static text. The first data binding expression references the TransactionCount variable, and the second uses the built-in Request object to determine some information about the user's browser. When user runs this page, the output looks like

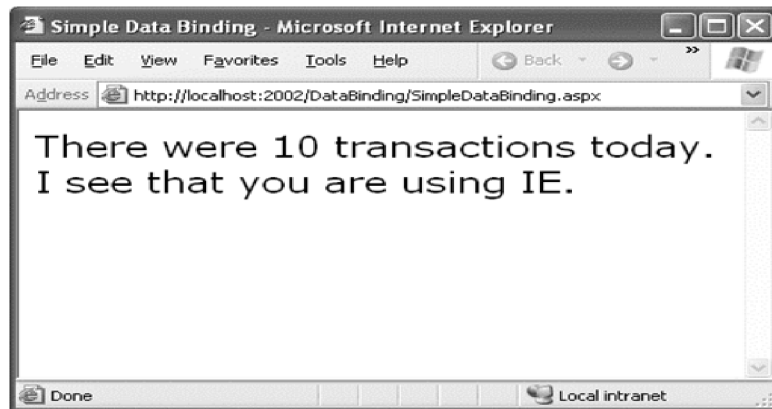


Figure 15-2. The result of data binding

The data binding expressions have been automatically replaced with the appropriate values. If the page is posted back, additional code can be used to modify Transaction Count, and as long as the `DataBind()` method is called, that information will be popped into the page in the data binding expression you've defined.

If, user forget to call the `DataBind()` method, the data binding expressions will be ignored. The Transaction Count value and browser name will not be displayed in the page, window that looks like the below figure3



Figure3. The non-data-bound page

When using single-value data binding, you need to consider when you should call the `DataBind()` method. For example, if `databind()` called before setting the `TransactionCount` variable, the corresponding expression would just be converted to 0. Remember, data binding is a one-way street. This means changing the `TransactionCount` variable after using the `DataBind()` method won't produce any visible effect. Unless user call the `DataBind()` method again, the displayed value won't be updated.

Simple Data Binding with Properties

However, you can also use single-value data binding can be used to set other types of information on the page, including control properties. To do this, you simply have to know where to put the data binding expression in the web page markup. For example, consider the following page, which defines a variable named URL and uses it to point to a picture in the application directory:

```
public partial class DataBindingUrl : System.Web.UI.Page
{
    protected string URL;

    protected void Page_Load(Object sender, EventArgs e)
    {
        URL = "Images/picture.jpg";

        this.DataBind();
    }
}
```

You can now use this URL to create a label, as shown here:

```
<asp:Label id="lblDynamic" runat="server"><%# URL %></asp:Label>
```

You can also use it for a check box caption:

```
<asp:CheckBox id="chkDynamic" Text="<%# URL %>" runat="server" />
```

or you can use it for a target for a hyperlink:

```
<asp:Hyperlink id="lnkDynamic" Text="Click here!" NavigateUrl="<%# URL %>"
runat="server" />
```

You can even use it for a picture:

```
<asp:Image id="imgDynamic" ImageUrl="<%# URL %>" runat="server" />
```

The only trick is that you need to edit these control tags manually. The page would look like as in the figure4.

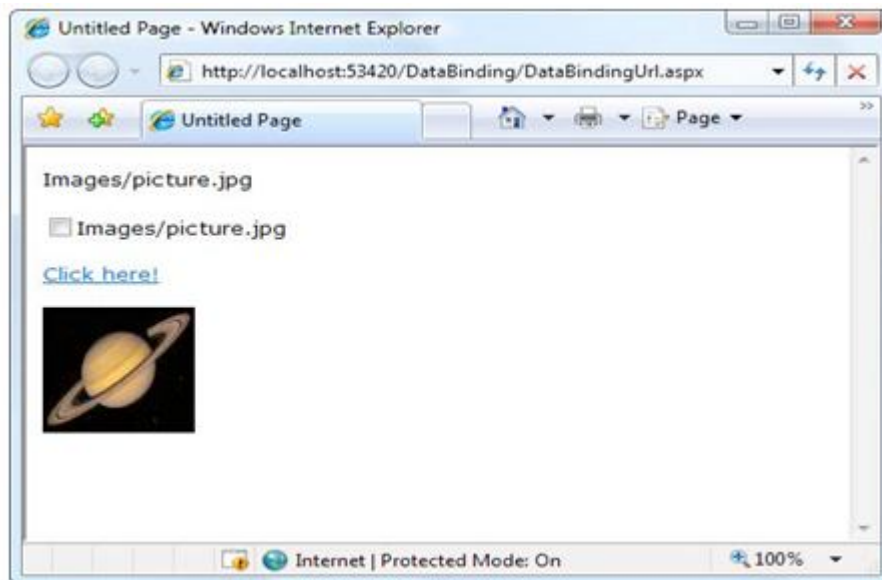


Figure4. Multiple ways to bind the same data

Problems with Single-Value Data Binding

Single-value data binding techniques has some drawbacks.

Putting code into a page's user interface: One of ASP.NET's great advantages is that it allows developers to separate the user interface code (the HTML and control tags in the .aspx file) from the actual code used for data access and all other tasks (in the code-behind file). Overenthusiastic use of single-value data binding can allow encourage you to start coding function calls and even operations into your page. If not carefully managed, this can lead to complete disorder.

Fragmenting code: When using data binding expressions, it may not be obvious where the functionality resides for different operations. This is particularly a problem if you blend both approaches. For example, if you use data binding to fill a control and also modify that control directly in code. If the page code changes, or a variable or function is removed or renamed, the corresponding data binding expression could stop providing valid information without any explanation or even an obvious error. All of these details make it more difficult to maintain your code, and make it more difficult for multiple developers to work together on the same project.

Using Code Instead of Simple Data Binding: Without using single-value data binding, you can accomplish the same thing using code. For example, you could use the following event handler to display the same output as the first label example:

```
protected void Page_Load(Object sender, EventArgs e)
{
    TransactionCount = 10;

    lblDynamic.Text = "There were " + TransactionCount.ToString();

    lblDynamic.Text += " transactions today. ";

    lblDynamic.Text += "I see that you are using " + Request.Browser.Browser;
}
```

This code dynamically fills in the label without using data binding. When you use data binding expressions, you end up complicating your markup with additional details about your code (such as the names of the variables in your code-behind class). When you use the code-only approach, you end up doing the reverse—complicating your code with additional details about the page markup (like the text you want to display).

REPEATED-VALUE DATA BINDING

Repeated-value data binding works with the ASP.NET list controls. To use repeated-value binding, you link one of these controls to a data source (such as a field in a data table). When you call `DataBind()`, the control automatically creates a full list using all the corresponding values. This saves from writing code that loops through the array or data table and manually adds elements to a control.

To create a data expression for list binding, you need to use a list control that explicitly supports data binding. Luckily, ASP.NET provides a number of list controls like `ListBox`, `DropDownList`, `CheckBoxList`, and `RadioButtonList`. These web controls provide a list for a single field of information.

HtmlSelect: This server-side HTML control represents the HTML `<select>` element and works essentially the same way as the `ListBox` web control. Generally used for backward compatibility.

GridView, DetailsView, FormView, and ListView: These rich web controls allow you to provide repeating lists or grids that can display more than one field of information at a time. For example, if you bind one of these controls to a full-fledged table in a DataSet, you can display the values from multiple fields. These controls offer the most powerful and flexible options for data binding.

- With repeated-value data binding, you can write a data binding expression in your .aspx file, or you can apply the data binding by setting control properties.
- In the case of the simpler list controls, you can set properties by using code in a code-behind file or by modifying the control tag in the .aspx file, possibly with the help of Visual Studio's Properties window.
- Simple list controls, you don't use any `<%# expression %>` data binding expressions.

Data Binding with Simple List Controls

In some ways, data binding to a list control is the simplest kind of data binding. You need to follow only three steps:

1. Create and fill some kind of data object. You have numerous options, including an array, the basic ArrayList and Hashtable collections, the strongly typed List and Dictionary collections, and the DataTable and DataSet objects. Essentially, you can use any type of collection that supports the IEnumerable interface, although you'll discover each class has specific advantages and disadvantages.

2. Link the object to the appropriate control. To do this, you need to set only a couple of properties, including DataSource. If you're binding to a full DataSet, you'll also need to set the DataMember property to identify the appropriate table you want to use.

3. Activate the binding. As with single-value binding, you activate data binding by using the DataBind() method, either for the specific control or for all contained controls at once by using the DataBind() method for the current page.

This process is the same whether you're using the ListBox, the DropDownList, the CheckBoxList, the RadioButtonList, or even the HtmlSelect control. All these controls provide the same properties and work the same way, the only difference is in the way they appear on the final web page.

Simple List Binding Example: To try this type of data binding, add a ListBox control to a new web page. Next, import the `System.Collections` namespace in your code. Finally, use the `Page.Load` event handler to create an `ArrayList` collection to use as a data source as follows:

```
ArrayList fruit = new ArrayList();

fruit.Add("Kiwi");

fruit.Add("Pear");

fruit.Add("Mango");

fruit.Add("Blueberry");

fruit.Add("Apricot");

fruit.Add("Banana");

fruit.Add("Peach");

fruit.Add("Plum");

lstItems.DataSource = fruit; //linking collection to ListBox control
```

Because an `ArrayList` is a straightforward, unstructured type of object, this is all the information you need to set. If `DataTable` or a `DataSet` used additional information need to specify. To activate the binding, use the `DataBind()` method: `this.DataBind();`

You could also use `lstItems.DataBind()` to bind just the ListBox control. Output will be like



Figure5. A data-bound list

This technique saves few lines of code. In a more realistic application, however, you might be using a function that returns a ready-made collection to you:

```
ArrayList fruit;
```

```
fruit = GetFruitsInSeason("Summer");
```

In this case, it is extremely simple to add the extra two lines needed to bind and display the collection in the window:

```
lstItems.DataSource = fruit;
```

```
this.DataBind();
```

or you could even change it to the following, even more compact, code:

```
lstItems.DataSource = GetFruitsInSeason("Summer");
```

```
this.DataBind();
```

Strongly Typed Collections

- You can use data binding with the Hashtable and ArrayList, two of the more useful collection classes in the System.Collections namespace

- Ideal in cases where you want your collection to hold just a single type of object (for example, just strings).
- When you use the generic collections, you choose the item type from your collection objects "locked in".
- if you try to add another type of object which doesn't belong to collection object will result in compile time error.
- When you pull an item no need to write casting code to convert the object type, the compiler already knows about the type of object.

To use a generic collection, you must import the right namespace:

using System.Collections.Generic

The generic version of the ArrayList class is named List. Here is how you create a List collection object that can only store strings:

```
List<string> fruit = new List<string>();
```

```
fruit.Add("Kiwi");
```

```
fruit.Add("Pear");
```

The only real difference is that you need to specify the type of data you want to use when you declare the List object.

Multiple Binding

You can bind the same data list object to multiple different controls. Consider the following example, which compares all the types of list controls at your disposal by loading them with the same information:

```
protected void Page_Load(Object sender, EventArgs e)
```

```
{
```

```
// Create and fill the collection.
```

398

```
List<string> fruit = new List<string>();

fruit.Add("Kiwi");

fruit.Add("Pear");

fruit.Add("Mango");

fruit.Add("Blueberry");

fruit.Add("Apricot");

fruit.Add("Banana");

fruit.Add("Peach");

fruit.Add("Plum");

// Define the binding for the list controls.

MyListBox.DataSource = fruit;

MyDropDownListBox.DataSource = fruit;

MyHtmlSelect.DataSource = fruit;

MyCheckBoxList.DataSource = fruit;

MyRadioButtonList.DataSource = fruit;

// Activate the binding.

this.DataBind();

}
```

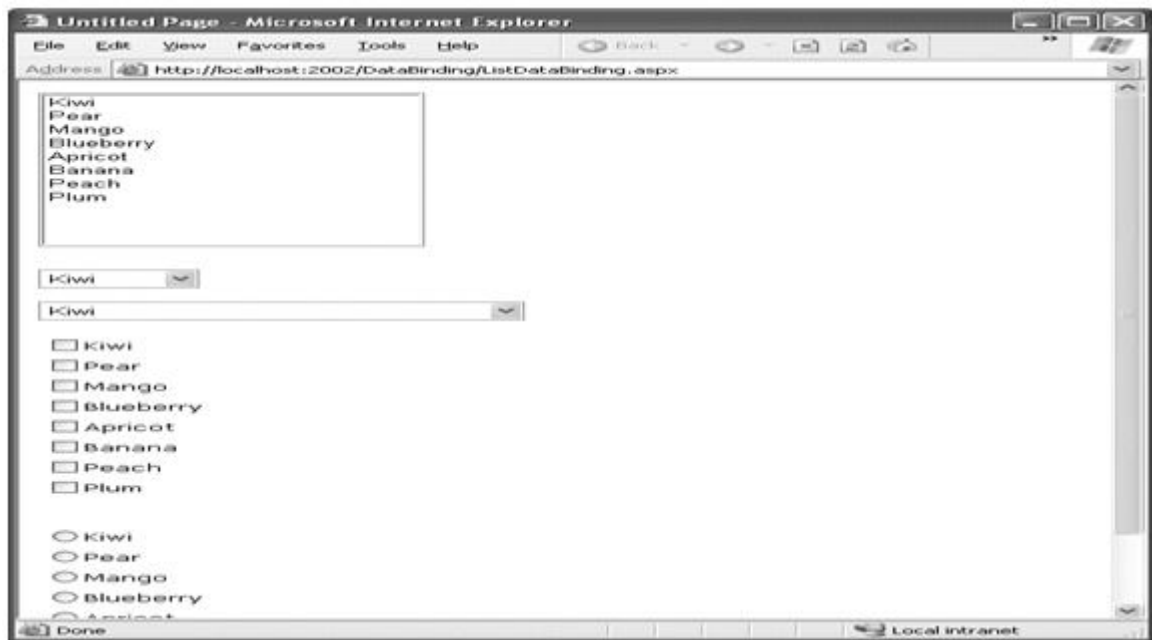



Figure6. Multiple bound lists

Data Binding with a Dictionary Collection

- A dictionary collection is a special kind of collection in which every item is indexed with a specific key.
- Dictionary collections always need keys.
- This makes it easier to retrieve the item you want by using its key.
- Dictionary collections allows you to frequently retrieve a single specific item.

You can use two basic dictionary-style collections in .NET. The Hashtable collection allows you to store any type of object and use any type of object for the key values. The Dictionary collection (in the System.Collections.Generic namespace) uses generics to provide the same locking in the behavior as the List collection. You choose the item type and the key type upfront to prevent errors and reduce the amount of casting code you need to write.

The following example uses the Dictionary collection class, which it creates once the first time the page is requested. You create a Dictionary object in much the same way you create an ArrayList or List collection. The only difference is that you need to supply a unique key for every item. This example uses the lazy practice of assigning a sequential number for each key:

400

```
protected void Page_Load(Object sender, EventArgs e)

{

    if (!this.IsPostBack)

    {

        // Use integers to index each item. Each item is a string.

        Dictionary<int, string> fruit = new Dictionary<int, string>();

        fruit.Add(1, "Kiwi");

        fruit.Add(2, "Pear");

        fruit.Add(3, "Mango");

        fruit.Add(4, "Blueberry");

        fruit.Add(5, "Apricot");

        fruit.Add(6, "Banana");

        fruit.Add(7, "Peach");

        fruit.Add(8, "Plum");

        // Define the binding for the list controls.

        MyListBox.DataSource = fruit;

        // Choose what you want to display in the list.

        MyListBox.DataTextField = "Value";

        // Activate the binding.

        this.DataBind();

    }

}
```

There's one new detail here. Its this line:

```
MyListBox.DataTextField = "Value";
```

Each item in a dictionary-style collection has both a key and a value associated with it. By default ToString() is called when property is not specified .it controls exactly what appears in list. The page will now appear as expected, with all thefruit names.

Using the DataValueField Property

Along with the DataTextField property, all list controls that support data binding also provide aDataValueField property, which adds the corresponding information to the value attribute in the controlelement. This allows you to store extra (undisplayed) information that you can access later. For example,you could use these two lines to define your data binding with the previous example:

```
MyListBox.DataTextField = "Value";
```

```
MyListBox.DataValueField = "Key";
```

The control will appear the same, with a list of all the fruit names in the collection. However, if youlook at the rendered HTML that's sent to the client browser, you will see that value attributes have been setwith the corresponding numeric key for each item:

```
<select name="MyListBox" id="MyListBox" >
```

```
<option value="1">Kiwi</option>
```

```
<option value="2">Pear</option>
```

```
<option value="3">Mango</option>
```

```
<option value="4">Blueberry</option>
```

```
<option value="5">Apricot</option>
```

```
<option value="6">Banana</option>
```

```
<option value="7">Peach</option>
```

```
<option value="8">Plum</option>
```

```
</select>
```

You can retrieve this value later using the `SelectedItem` property to get additional information. Foreexample, you could set the `AutoPostBack` property of the list control to true, and add the following code:

```
protected void MyListBox_SelectedIndexChanged(Object sender,
EventArgs e)
{
    lblMessage.Text = "You picked: " + MyListBox.SelectedItem.Text;
    lblMessage.Text += " which has the key: " + MyListBox.SelectedItem.Value;
}
```

The result is shown below

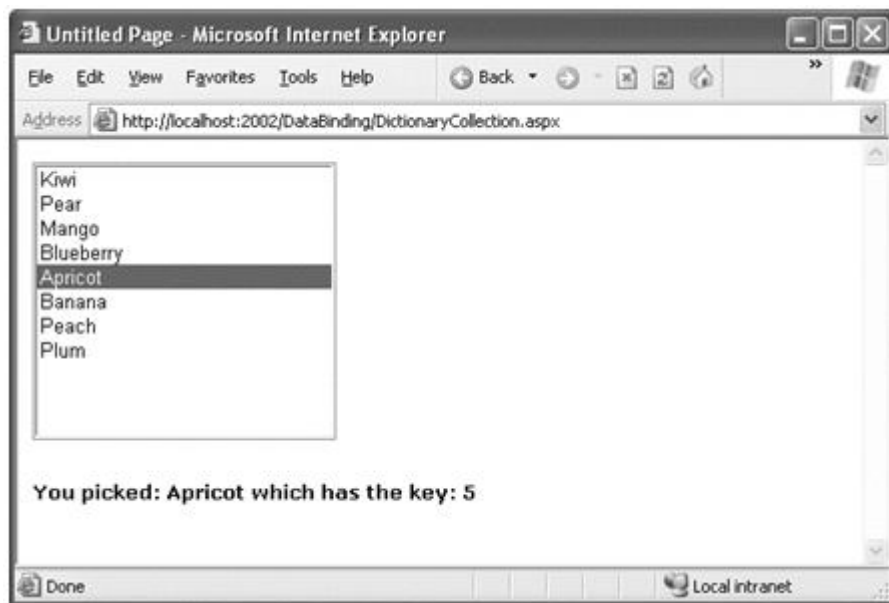


Figure7. Binding to the key and value properties

9.6 Data Binding with ADO.NET

- When you're using data binding with the information drawn from a database, the data binding process takes place in the same three steps.

- First you create your data source.
- Which will be a `DataReader` or `DataSet` object. A `DataReader` generally offers the best performance, but it limits your data binding to a single control. It traverses results from beginning to end only, can't go backwards. So it can't be used in another data binding operation.
- For this reason, a `DataSet` is a more common choice.

The next example creates a `DataSet` and binds it to a list. In this example, the `DataSet` is filled by hand. To fill a `DataSet` by hand, you need to follow several steps:

1. First, create the `DataSet`.
2. Next, create a new `DataTable`, and add it to the `DataSet.Tables` collection.
3. Next, define the structure of the table by adding `DataColumn` objects (one for each field) to the `DataTable.Columns` collection.
4. Finally, supply the data. You can get a new, blank row that has the same structure as your `DataTable` by calling the `DataTable.NewRow()` method. You must then set the data in all its fields, and add the `DataRow` to the `DataTable.Rows` collection.

Here is how the code unfolds:

```
// Define a DataSet with a single DataTable.
```

```
DataSet dsInternal = new DataSet();
```

```
dsInternal.Tables.Add("Users");
```

```
// Define two columns for this table.
```

```
dsInternal.Tables["Users"].Columns.Add("Name");
```

```
dsInternal.Tables["Users"].Columns.Add("Country");
```

```
// Add some actual information into the table.
```

```
DataRow rowNew = dsInternal.Tables["Users"].NewRow();
```

```
rowNew["Name"] = "John";  
  
rowNew["Country"] = "Uganda";  
  
dsInternal.Tables["Users"].Rows.Add(rowNew);  
  
rowNew = dsInternal.Tables["Users"].NewRow();  
  
rowNew["Name"] = "Samantha";  
  
rowNew["Country"] = "Belgium";  
  
dsInternal.Tables["Users"].Rows.Add(rowNew);  
  
rowNew = dsInternal.Tables["Users"].NewRow();  
  
rowNew["Name"] = "Rico";  
  
rowNew["Country"] = "Japan";  
  
dsInternal.Tables["Users"].Rows.Add(rowNew);
```

Next, you bind the DataTable from the DataSet to the appropriate control. Because list controls can only show a single column at a time, you also need to choose the field you want to display for each item by setting the DataTextField property:

```
// Define the binding.  
  
lstUser.DataSource = dsInternal.Tables["Users"];  
  
lstUser.DataTextField = "Name";
```

Alternatively, you could use the entire DataSet for the data source, instead of just the appropriate table. In that case, you would have to select a table by setting the controls DataMember property. This is an equivalent approach, but the code is slightly different:

```
// Define the binding.  
  
lstUser.DataSource = dsInternal;  
  
lstUser.DataMember = "Users";
```

```
lstUser.DataTextField = "Name";
```

```
this.DataBind(); // last step is to activate the binding:
```

The final result is a list with the information from the specified database field, as below

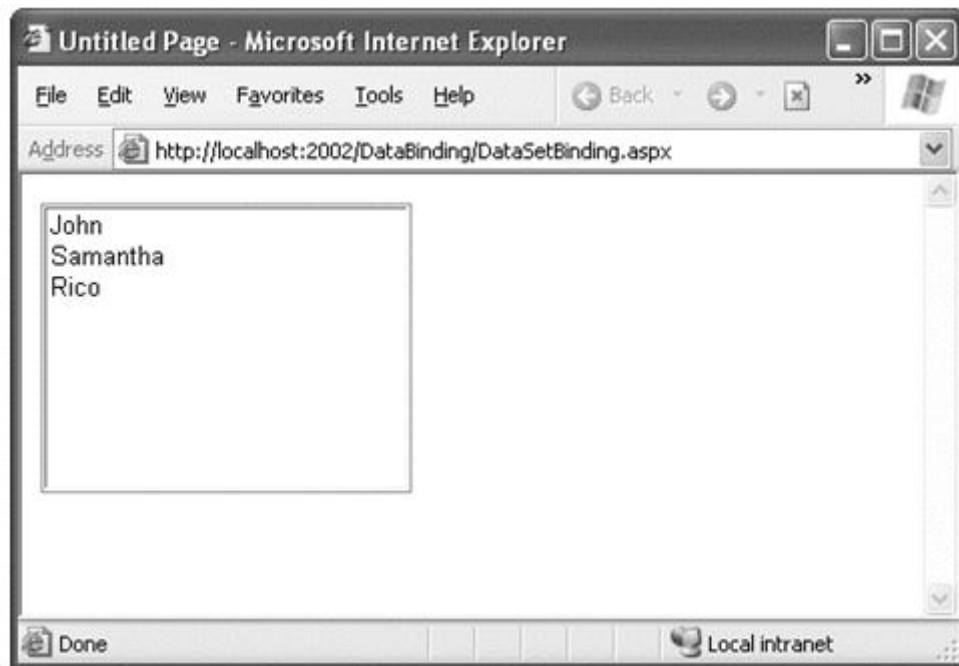


Figure8. DataSet binding

Creating a Record Editor

This example allows the user to select a record and update one piece of information by using data-bound list controls. The first step is to add the connection string to your web.config file. This example uses the Products table from the Northwind database included with many versions of SQL Server. Here is how you can define the connection string for SQL Server Express:

```
<configuration>
```

```
<connectionStrings>
```

```
<add name="Northwind" connectionString=
```

```
"Data Source=localhost\SQLEXPRESS;Initial Catalog=Northwind;Integrated Security=SSPI" /
```

```
>
```

```
</connectionStrings>
```

```
...
```

```
</configuration>
```

- To use the full version of SQL Server, remove the \SQLEXPRESS portion. To use a database server on

another computer, supply the computer name for the Data Source connection string property

- The next step is to retrieve the connection string and store it in a private variable in the Page class so that every part of your page code can access it easily. Once you have imported the `System.Web.Configuration` namespace, you can create a member variable in your code-behind class that is defined like this:

```
private string connectionString = WebConfigurationManager. ConnectionStrings
["Northwind"]. ConnectionString;
```

- The next step is to create a drop-down list that allows the user to choose a product for editing. The `Page.Load` event handler takes care of this task. Retrieving the data, binding it to the drop-down list control, and then activating the binding. Before you go any further, make sure you have imported the `System.Data.SqlClient` namespace, which allows you to use the SQL Server provider to retrieve data.

```
protected void Page_Load(Object sender, EventArgs e)
```

```
{
```

```
if (!this.IsPostBack)
```

```
{
```

```
// Define the ADO.NET objects for selecting products from the database.
```

```
string selectSQL = "SELECT ProductName, ProductID FROM Products";
```

```
SqlConnection con = new SqlConnection(connectionString);
```

```
SqlCommand cmd = new SqlCommand(selectSQL, con);
```



```
// Open the connection.

con.Open();

// Define the binding.

lstProduct.DataSource = cmd.ExecuteReader();

lstProduct.DataTextField = "ProductName";

lstProduct.DataValueField = "ProductID";

// Activate the binding.

this.DataBind();

con.Close();

// Make sure nothing is currently selected in the list box.

lstProduct.SelectedIndex = -1;

}

}
```

- Once again, the list is only filled the first time the page is requested.
- If the page is posted back, the list keeps its current entries. This reduces the amount of database work, and keeps the page working quickly and efficiently. The result will be shown as below,



Figure9. Product choices

The drop-down list enables `AutoPostBack`, so as soon as the user makes a selection, a `IstProduct.SelectedItemChanged` event fires. At this point, your code performs the following tasks:

It reads the corresponding record from the `Products` table and displays additional information about it in a label. In this case, a `Join` query links information from the `Products` and `Categories` tables. The code also determines what the category is for the current product. This is the piece of information it will allow the user to change.

It reads the full list of `CategoryNames` from the `Categories` table and binds this information to a different list control. Initially, this list is hidden in a panel with its `Visible` property set to `false`. The code reveals the content of this panel by setting `Visible` to `true`.

It highlights the row in the category list that corresponds to the current product. For example, if the current product is a `Seafood` category, the `Seafood` entry in the list box will be selected. The full listing is as follows:

```
protected void IstProduct_SelectedIndexChanged(object sender, EventArgs e)
```

```

{
// Create a command for selecting the matching product record.

string selectProduct = "SELECT ProductName, QuantityPerUnit, " +
"CategoryName FROM Products INNER JOIN Categories ON " +
"Categories.CategoryID=Products.CategoryID " +
"WHERE ProductID=@ProductID";

// Create the Connection and Command objects.

SqlConnection con = new SqlConnection(connectionString);

SqlCommand cmdProducts = new SqlCommand(selectProduct, con);

cmdProducts.Parameters.AddWithValue("@ProductID",
lstProduct.SelectedItem.Value);

// Retrieve the information for the selected product.

using (con)
{
con.Open();

SqlDataReader reader = cmdProducts.ExecuteReader();

reader.Read();

// Update the display.

lblRecordInfo.Text = "<b>Product:</b> " +
reader["ProductName"] + "<br />";

lblRecordInfo.Text += "<b>Quantity:</b> " +
reader["QuantityPerUnit"] + "<br />";

lblRecordInfo.Text += "<b>Category:</b> " + reader["CategoryName"];

// Store the corresponding CategoryName for future reference.

```

410

```
string matchCategory = reader["CategoryName"].ToString();

// Close the reader.

reader.Close();

// Create a new Command for selecting categories.

string selectCategory = "SELECT CategoryName, " +
"CategoryID FROM Categories";

SqlCommand cmdCategories = new SqlCommand(selectCategory, con);

// Retrieve the category information, and bind it.

lstCategory.DataSource = cmdCategories.ExecuteReader();

lstCategory.DataTextField = "CategoryName";

lstCategory.DataValueField = "CategoryID";

lstCategory.DataBind();

// Highlight the matching category in the list.

lstCategory.Items.FindByText(matchCategory).Selected = true;

}

pnlCategory.Visible = true;

}
```

The end result is a window that updates itself dynamically whenever a new product is selected, as shown



Figure10. Product information

This example still has one more trick in store. If the user selects a different category and clicks `Update`, the change is made in the database. Of course, this means creating new `Connection` and `Command` objects, as follows:

```
protected void cmdUpdate_Click(object sender, EventArgs e)
```

```
{
```

```
// Define the Command.
```

```
string updateCommand = "UPDATE Products " +
```

CHAPTER 15 | DATABINDING

521

```
"SET CategoryID=@CategoryID WHERE ProductID=@ProductID";
```

```
SqlConnection con = new SqlConnection(connectionString);
```

```
SqlCommand cmd = new SqlCommand(updateCommand, con);
```

```
cmd.Parameters.AddWithValue("@CategoryID", lstCategory.SelectedItem.Value);
```

```

cmd.Parameters.AddWithValue("@ProductID", lstProduct.SelectedItem.Value);

// Perform the update.

using (con)
{
    con.Open();

    cmd.ExecuteNonQuery();
}
}

```

Using these controls, you can create sophisticated lists and grids that provide automatic features for selecting, editing, and deleting records.

9.6 Data Source Controls

Data source controls allow you to create data-bound pages without writing any data access code at all. The data source controls include any control that implements the `IDataSource` interface. The .NET Framework includes the following data source controls:

- ***SqlDataSource***: This data source allows you to connect to any data source that has an ADO.NET data provider. This includes SQL Server, Oracle, and OLE DB or ODBC data sources. When using this data source, you don't need to write the data access code.
- ***AccessDataSource***: This data source allows you to read and write the data in an Access database file (.mdb). However, its use is discouraged, because Access doesn't scale well to large numbers of users (unlike SQL Server Express).
- ***ObjectDataSource***: This data source allows you to connect to a custom data access class. This is the preferred approach for large-scale professional web applications, but it forces you to write much more code.
- ***XmlDataSource***: This data source allows you to connect to an XML file.
- ***SiteMapDataSource***: This data source allows you to connect to a .sitemap file.

that describes the navigational structure of your website.

- **EntityDataSource:** This data source allows you to query a database using the LINQ to Entities feature.
- **LinqDataSource:** This data source allows you to query a database using the LINQ to SQL feature, which is a similar (but somewhat less powerful) predecessor to LINQ to Entities.

You can find all the data source controls in the Data tab of the Toolbox in Visual Studio, with the exception of the AccessDataSource. When you drop a data source control onto your web page, it shows up as a gray box in Visual Studio

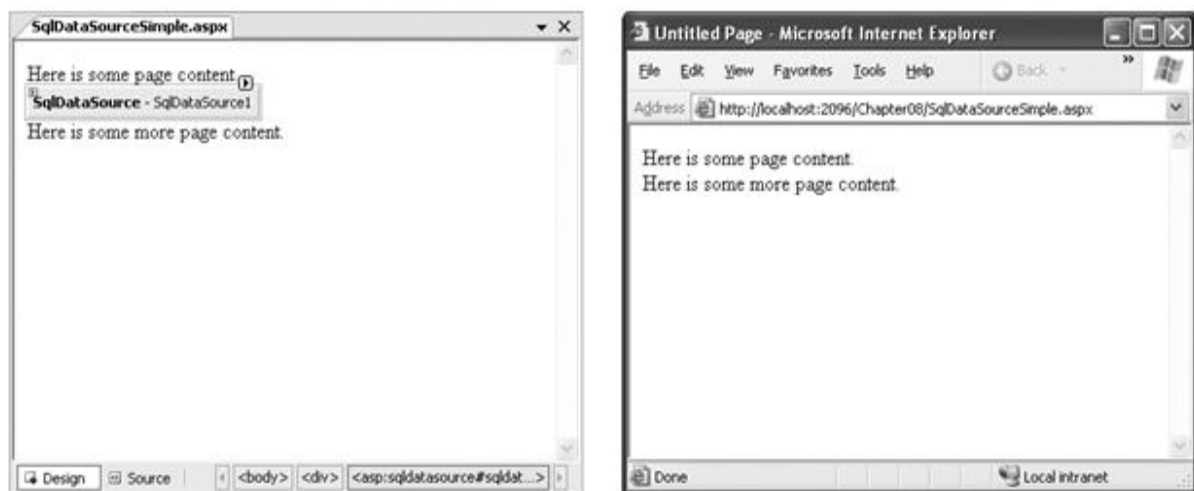


Figure 11. A data source control at design time and runtime

The Page Life Cycle with Data Binding

Data source controls can perform two key tasks:

- They can retrieve data from a data source and supply it to bound controls. When you use this feature, your bound controls are automatically filled with data. No need to call `DataBind()`.
- They can update the data source when edits take place. In order to use this feature, you must use one of ASP.NET's rich data controls, like the `GridView` or `DetailsView`.

The following steps explain the sequence of stages your page goes through in its lifetime.

1. The page object is created (based on the .aspx file).
2. The page life cycle begins, and the Page.Init and Page.Load events fire.
3. All other control events fire.
4. If the user is applying a change, the data source controls perform their update operations now. If a row is being updated, the Updating and Updated events fire. If a row is being inserted, the Inserting and Inserted events fire. If a row is being deleted, the Deleting and Deleted events fire.
5. The Page.PreRender event fires.
6. The data source controls perform their queries and insert the data they retrieve into the bound controls. This step happens the first time your page is requested and every time the page is posted back, ensuring you always have the most up-to-date data. The Selecting and Selected events fire at this point.
7. The page is rendered and disposed.

The **SqlDataSource**

Data source controls turn up in the .aspx markup portion of your web page like ordinary controls. Here is an example:

```
<asp:SqlDataSource ID="SqlDataSource1" runat="server" ... />
```

The **SqlDataSource** represents a database connection that uses an ADO.NET provider. However, this has a catch. The **SqlDataSource** needs a generic way to create the **Connection**, **Command**, and **DataReader** objects it requires. The only way this is possible is if your data provider includes something called a data provider factory. The factory has the responsibility of creating the provider-specific objects that the **SqlDataSource** needs to access the data source. Fortunately, .NET includes a data provider factory for each of its four data providers:

- `System.Data.SqlClient`
- `System.Data.OracleClient`

- `System.Data.OleDb`
- `System.Data.Odbc`

You can use all of these providers with the `SqlDataSource`. You choose your data source by setting the provider name. Here is a `SqlDataSource` that connects to a SQL Server database using the `SQLServer` provider:

```
<asp:SqlDataSourceProviderName="System.Data.SqlClient" ... />
```

The next step is to supply the required connection string. Without it, you cannot make any connections. Although you can hard-code the connection string directly in the `SqlDataSource` tag, it is always better to keep it in the `<connectionStrings>` section of the `web.config` file.

To refer to a connection string in your `.aspx` markup, you use a special syntax in this format:

```
<%%$ ConnectionStrings:[NameOfConnectionString] %>
```

This looks like a data binding expression, but it is slightly different. (For one thing, it begins with the character sequence `<%%$` instead of `<%#.`)

For example, if you have a connection string named `Northwind` in your `web.config` file that looks like this:

```
<configuration>
```

```
<connectionStrings>
```

```
<add name="Northwind" connectionString=
```

```
"Data Source=localhost\SQLEXPRESS;Initial Catalog=Northwind;Integrated Security=SSPI" /
> </connectionStrings>
```

```
...
```

```
</configuration>
```

you would specify it in the `SqlDataSource` using this syntax: `<asp:SqlDataSource ConnectionString="<%%$ ConnectionStrings:Northwind %>" ... />`

Once you have specified the provider name and connection string, the next step is to add the query logic that the `SqlDataSource` will use when it connects to the database.

Selecting Records

- You can use each `SqlDataSource` control you create to retrieve a single query.
- you can also add corresponding commands for deleting, inserting, and updating rows.
- The `SqlDataSource` command logic is supplied through four properties. `SelectCommand`, `InsertCommand`, `UpdateCommand`, and `DeleteCommand`. each of which takes a string.
- You need to define commands only for the types of actions you want to perform.

Here is a complete `SqlDataSource` that defines a `Select` command for retrieving product information from the `Products` table:

```
<asp:SqlDataSource ID="sourceProducts" runat="server"
ConnectionString="<%%$ ConnectionStrings:Northwind %>"
SelectCommand="SELECT ProductName, ProductID FROM Products"
/>
```

You can also set the `DataSourceID` property to point to the `SqlDataSource` using the `Properties` window, which provides a drop-down list of all the data sources on your current web page. At the same time, make sure you set the `DataTextField` and `DataValueField` properties. Once you make these changes, you will wind up with a control tag like this:

```
<asp:DropDownList ID="lstProduct" runat="server" AutoPostBack="True"
DataSourceID="sourceProducts" DataTextField="ProductName"
DataValueField="ProductID" />
```

When you run the page, the `DropDownList` control asks the `SqlDataSource` for the data it needs. At this point, the `SqlDataSource` executes the query you defined, fetches the information, and binds it to the `DropDownList`. The whole process unfolds automatically.

How the Data Source Controls Work

Data source controls can bind to a `DataReader` or a `DataSet`. The `DataSet` mode is almost always better, because it supports advanced sorting, filtering, and caching settings that depend on the `DataSet`. All these features are disabled in `DataReader` mode, you can use the `DataReader` mode with extremely large grids, because it is more memory-efficient. That's because the `DataReader` holds only one record in memory at a time, just long enough to copy the record's information to the linked control.

Another important fact to understand about the data source controls is that when you bind more than one control to the same data source, you cause the query to be executed multiple times. Second, you won't be binding more than one control to a data source. That's because the rich data controls, i.e. `GridView`, `DetailsView`, and `FormView`, have the ability to present multiple pieces of data in a flexible layout. If you use these controls, you will need to bind only one control, which allows you to steer clear of this limitation.

It is also important to remember that data binding is performed at the end of your web page processing, just before the page is rendered. This means the `Page.Load` event will fire, followed by any control events, followed by the `Page.PreRender` event. Only then will the data binding take place.

Parameterized Commands

Once you select a product, you want to execute another command to get the full details for that product. To make this work, you need two data sources. You should create the first `SqlDataSource`, which fetches limited information about every product then the second `SqlDataSource`, which gets more extensive information about a single product

```
<asp:SqlDataSource ID="sourceProductDetails" runat="server"
```

```
ProviderName="System.Data.SqlClient"
```

```
ConnectionString="<%%$ ConnectionStrings:Northwind %>"
```

```
SelectCommand="SELECT * FROM Products WHERE ProductID=@ProductID"
```

```
/>
```

But this example has a problem. It defines a parameter (@ProductID) that identifies the ID of the product you want to retrieve. It turns out you need to add a <SelectParameters> section to the SqlDataSource tag. Inside this section, you must define each parameter that is referenced by your SelectCommand and tell the SqlDataSource where to find the value it should use. You do that by mapping the parameter to a value in a control. Here is the corrected command:

```
<asp:SqlDataSource ID="sourceProductDetails" runat="server"
    ProviderName="System.Data.SqlClient"
    ConnectionString="<%= $ConnectionStrings:Northwind %>"
    SelectCommand="SELECT * FROM Products WHERE ProductID=@ProductID">
    <SelectParameters>
        <asp:ControlParameter ControlID="lstProduct" Name="ProductID"
            PropertyName="SelectedValue" />
    </SelectParameters>
</asp:SqlDataSource>
```

You always indicate parameters with an @ symbol, as in @City. You can define as many parameters as you want, but you must map each one to a value using a separate element in the SelectParameters collection. In this example, the value for the @ProductID parameter comes from the lstProduct.SelectedValue property. In other words, you are binding a value that's currently in a control to place it into a database command.

First, you can bind only a single data field to most simple controls such as lists. Second, each bound control makes a separate request to the SqlDataSource, triggering a separate database query. The solution is to use one of the rich data controls, such as the GridView, DetailsView, or FormView. These controls have the smarts to show multiple fields at once, in a highly flexible layout.

The DetailsView is a rich data control that's designed to show multiple fields in a data source. As long as its AutoGenerateRows is true (the default), it creates a separate row for each field, with the field caption and value. The result is shown below.

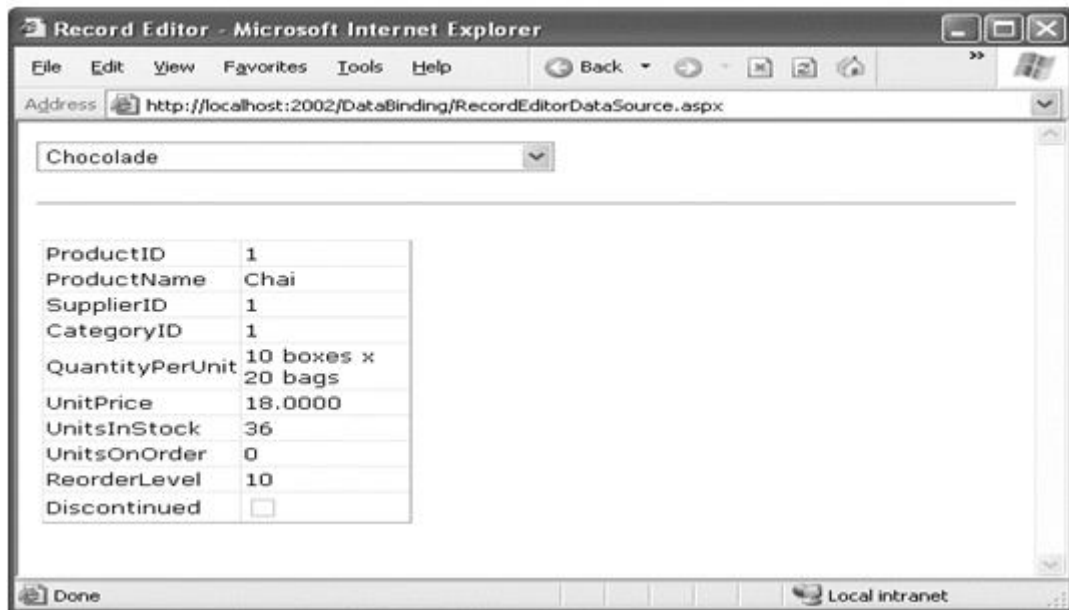


Figure12. Displaying full product information in a DetailsView

Here is the basic DetailsView tag that makes this possible:

```
<asp:DetailsView ID="detailsProduct" runat="server"
```

```
DataSourceID="sourceProductDetails" />
```

Other Types of Parameters

In the previous example, the @ProductID parameter in the second SqlDataSource is configured based on the selection in a drop-down list. This type of parameter, which links to a property in another control, is called a control parameter. But parameter values aren't necessarily drawn from other controls. You can map a parameter to any of the parameter types given below

Table1. Parameter Types

Source	Control tag	Description
Control Property	<asp:ControlParameter>	A property from another control on the page.
Query string value	<asp:QueryStringParameter>	A value from the current query string.

Session state value	<code><asp:SessionParameter></code>	A value stored in the current user's session.
Cookie value	<code><asp:CookieParameter></code>	A value from any cookie attached to the current request.
Profile value	<code><asp:ProfileParameter></code>	A value from the current user's profile
Routed URL value	<code><asp:RouteParameter></code>	A value from a routed URL. Routed URLs are an advanced technique that lets you map any URL to a different page
A form variable	<code><asp:FormParameter></code>	A value posted to the page from an input control.

For example, you could split the earlier example into two pages. In the first page, define a list control that shows all the available products:

```
<asp:SqlDataSource ID="sourceProducts" runat="server"
ProviderName="System.Data.SqlClient"
ConnectionString="<%%$ ConnectionStrings:Northwind %>"
SelectCommand="SELECT ProductName, ProductID FROM Products"
/>

<asp:DropDownList ID="lstProduct" runat="server" AutoPostBack="True"
DataSourceID="sourceProducts" DataTextField="ProductName"
DataValueField="ProductID" />
```

Now, you'll need a little extra code to copy the selected product to the query string and redirect the page. Here's a button that does just that:

```
protected void cmdGo_Click(object sender, EventArgs e)
{
```

```

if (lstProduct.SelectedIndex != -1)
{
    Response.Redirect(
        "QueryParameter2.aspx?prodID=" + lstProduct.SelectedValue);
}
}

```

Finally, the second page can bind the DetailsView according to the ProductID value that is supplied in the query string:

```

<asp:SqlDataSource ID="sourceProductDetails" runat="server"
    ProviderName="System.Data.SqlClient"
    ConnectionString="<%%$ ConnectionStrings:Northwind %>"
    SelectCommand="SELECT * FROM Products WHERE ProductID=@ProductID">
    <SelectParameters>
        <asp:QueryStringParameter Name="ProductID" QueryStringField="prodID" />
    </SelectParameters>
</asp:SqlDataSource>

<asp:DetailsView ID="detailsProduct" runat="server"
    DataSourceID="sourceProductDetails" />

```

Setting Parameter Values in Code

Sometimes you can set the parameter value using code just before the database operation takes place.

For example: Here is the markup that defines the list and its datasource:

```

<asp:SqlDataSource ID="sourceCustomers" runat="server"
ProviderName="System.Data.SqlClient"
ConnectionString="<%%$ ConnectionStrings:Northwind %>"
SelectCommand="SELECT CustomerID, ContactName FROM Customers"
/>

<asp:DropDownList ID="lstCustomers" runat="server"
DataSourceID="sourceCustomers" DataTextField="ContactName"
DataValueField="CustomerID" AutoPostBack="True">
</asp:DropDownList>

```

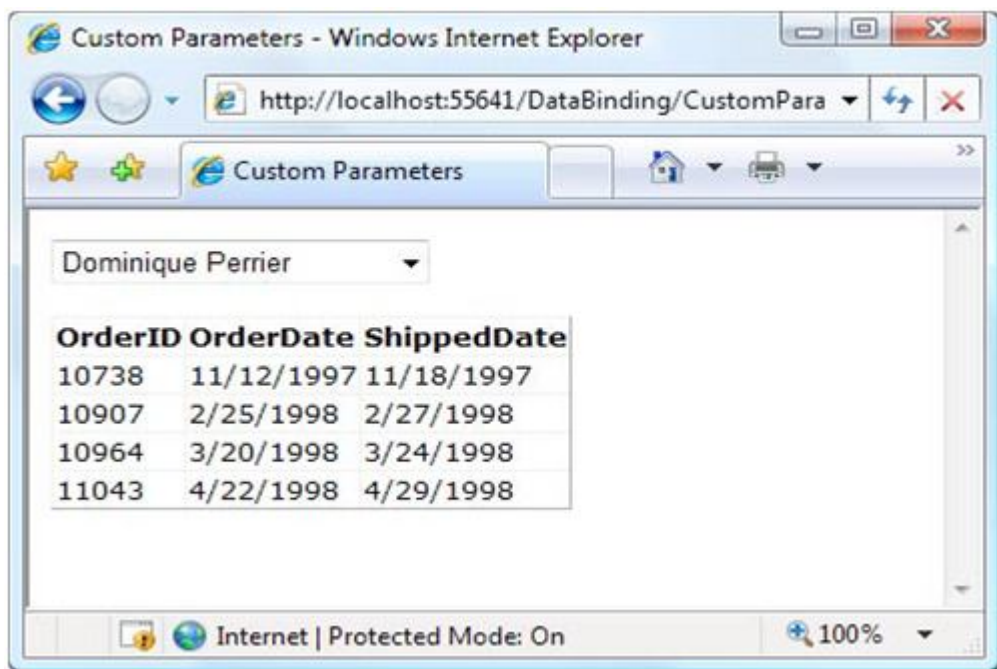


Figure13. Using parameters in a master-details page

When the user picks a customer from the list, the page is posted back (because AutoPostBack is set to true) and the matching orders are shown in a GridView underneath, using a second

data source. This data source pulls the CustomerID for the currently selected customer from the drop-down list using a ControlParameter:

```
<asp:SqlDataSource ID="sourceOrders" runat="server"
ProviderName="System.Data.SqlClient"
ConnectionString="<%%$ ConnectionStrings:Northwind %>"
SelectCommand="SELECT OrderID,OrderDate,ShippedDate FROM Orders WHERE
CustomerID=@CustomerID">
<SelectParameters>
<asp:ControlParameter Name="CustomerID"
ControlID="lstCustomers" PropertyName="SelectedValue" />
</SelectParameters>
</asp:SqlDataSource>
<asp:GridView ID="gridOrders" runat="server" DataSourceID="sourceOrders">
</asp:GridView>
```

The easiest way to solve this problem is to add a new parameter. one that you will be responsible for setting yourself:

```
<asp:SqlDataSource ID="sourceOrders" runat="server"
ProviderName="System.Data.SqlClient"
ConnectionString="<%%$ ConnectionStrings:Northwind %>"
SelectCommand="SELECT OrderID,OrderDate,ShippedDate FROM Orders WHERE
CustomerID=@CustomerID AND OrderDate>=@EarliestOrderDate"
OnSelecting="sourceOrders_Selecting">
```

```

<SelectParameters>

<asp:ControlParameter Name="CustomerID"

ControlID="lstCustomers" PropertyName="SelectedValue" />

<asp:Parameter Name="EarliestOrderDate" DefaultValue="1900/01/01" />

</SelectParameters>

</asp:SqlDataSource>

```

Now that you have created the parameter, you need to set its value before the command takes place. The `SqlDataSource` has a number of events that are perfect for setting parameter values. You can fill in parameters for a select operation by reacting to the `Selecting` event. Similarly, you can use the `Updating`, `Deleting`, and `Inserting` events when updating, deleting, or inserting a record. In these event handlers, you can access the command that is about to be executed, using the `Command` property of the `customEventArgs` object.

Here is the code that is needed to set the parameter value to a date that is seven days in the past, ensuring you see one week is worth of records:

```

protected void sourceOrders_Selecting(object sender,
SqlDataSourceSelectingEventArgs e)
{
e.Command.Parameters["@EarliestOrderDate"].Value =
DateTime.Today.AddDays(-7);
}

```

9.7 Handling Errors

- When you deal with coding or establishing connection to database or doing insert, update, delete operations unexpected error may occur.
- It is necessary to handle with errors in order to work efficiently.
- In the event handler, you can access the exception object through the

SqlDataSourceStatusEventArgs.Exception property. If you want to prevent the error from spreading any further, simply set the SqlDataSourceStatusEventArgs.ExceptionHandled property to true. Then, make sure, you show an appropriate error message on your web page to inform the user that the command was not completed. Here is an example:

```
protected void sourceProducts_Selected(object sender,
SqlDataSourceStatusEventArgs e)
{
    if (e.Exception != null)
    {
        lblError.Text = "An exception occurred performing the query.";
        // Consider the error handled.
        e.ExceptionHandled = true;
    }
}
```

Updating Records

You define the InsertCommand, DeleteCommand, and UpdateCommand in the same way you define the command for the SelectCommand property by using a parameterized query. For example, here is a revised version of the SqlDataSource for product information that defines a basic updatecommand to update every field:

```
<asp:SqlDataSource ID="sourceProductDetails" runat="server"
ProviderName="System.Data.SqlClient"
ConnectionString="<%%$ ConnectionStrings:Northwind %>"
SelectCommand="SELECT ProductID, ProductName, UnitPrice, UnitsInStock,
```

UnitsOnOrder, ReorderLevel, Discontinued FROM Products WHERE ProductID=@ProductID”

UpdateCommand=”UPDATE Products SET ProductName=@ProductName,
UnitPrice=@UnitPrice,

UnitsInStock=@UnitsInStock, UnitsOnOrder=@UnitsOnOrder, ReorderLevel=@ReorderLevel,
Discontinued=@Discontinued WHERE ProductID=@ProductID”>

<SelectParameters>

<asp:ControlParameterControlID=”lstProduct” Name=”ProductID”

PropertyName=”SelectedValue” />

</SelectParameters>

</asp:SqlDataSource>

No need to define parameter since @symbol is used as a preface ,it affects the same name as the field.It saves time. Most rich data controls make this fairlyeasy.with the DetailsView, it is simply a matter of setting the AutoGenerateEditButton property to true,as shown here:

<asp:DetailsView ID=”DetailsView1” runat=”server”

DataSourceID=”sourceProductDetails” AutoGenerateEditButton=”True” />

Now when you run the page, you will see an edit link. When clicked, this link switches the DetailsView into edit mode. All fields are changed to edit controls (typically text boxes), and the Edit link is replaced with an Update link and a Cancel link as shown below

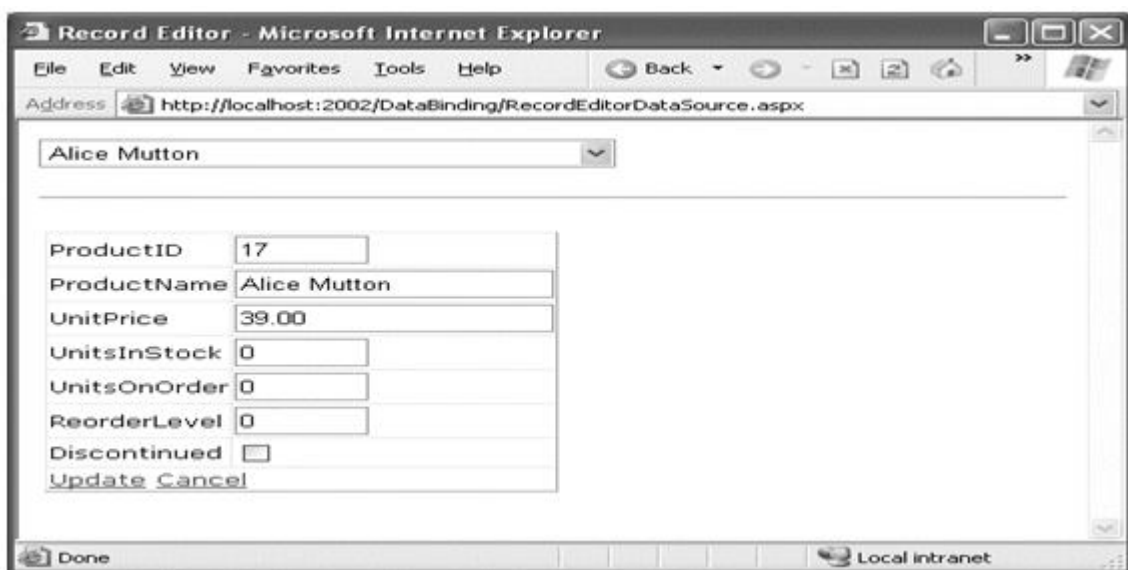


Figure14. Editing with the DetailsView

- Clicking the Cancel link returns the row to its initial state. Clicking the Update link triggers an update.
- The DetailsView extracts the field values, uses them to set the parameters in the SqlDataSource.UpdateParameters collection, and then triggers the SqlDataSource.UpdateCommand to apply the change to the database.
- You can create similar parameterized commands for the DeleteCommand and InsertCommand. To enable deleting and inserting, you need to set the Auto Generate Delete Button and Auto Generate Insert Button properties of the DetailsView to true.

Strict Concurrency Checking

The update command in the previous example matches the record based on its ID. You can tell this by examining the Where clause:

```
UpdateCommand="UPDATE Products SET ProductName=@ProductName,
UnitPrice=@UnitPrice,
UnitsInStock=@UnitsInStock, UnitsOnOrder=@UnitsOnOrder, ReorderLevel=@ReorderLevel,
Discontinued=@Discontinued WHERE ProductID=@ProductID"
```

The problem with this approach is that it opens the door to an update that overwrites the changes of another user, if these changes are made between the time your page is requested and the time your page commits its update.

One way to solve this problem is to use an approach called match-all-values concurrency. In this situation, your update command attempts to match every field. As a result, if the original record has changed, the update command won't find it and the update won't be performed at all. To use this approach, you need to add a Where clause that tries to match every field. Here is what the modified command would look like:

```
UpdateCommand="UPDATE Products SET ProductName=@ProductName,
UnitPrice=@UnitPrice,
UnitsInStock=@UnitsInStock, UnitsOnOrder=@UnitsOnOrder, ReorderLevel=@ReorderLevel,
Discontinued=@Discontinued WHERE ProductID=@ProductID AND
```

```

ProductName=@original_ProductName AND UnitPrice=@original_UnitPrice AND
UnitsInStock=@original_UnitsInStock AND UnitsOnOrder=@original_UnitsOnOrder AND
ReorderLevel=@original_ReorderLevel AND Discontinued=@original_Discontinued"

```

Before this command can work, you need to tell the `SqlDataSource` to maintain the old values from the data source and to give them parameter names that start with `original_`. You do this by setting two properties. First, set the `SqlDataSource.ConflictDetection` property to `ConflictOptions.CompareAllValues` instead of `ConflictOptions.OverwriteChanges` (the default). Next, set the `long-windedOldValuesParameterFormatString` property to the text `"original{0}"`. This tells the `SqlDataSource` to insert the text `original_` before the field name to create the parameter that stores the old value. Now your command will work as written.

The `SqlDataSource` doesn't raise an exception to notify you if no update is performed. So, if you use the command shown in this example, you need to handle the `SqlDataSource.Updated` event and check the `SqlDataSource.StatusEventArgs.AffectedRows` property. If it's 0, no records have been updated, and you should notify the user about the concurrency problem so the update can be attempted again, as shown here:

```

protected void sourceProductDetails_Updated(object sender,
SqlDataSourceStatusEventArgs e)
{
    if (e.AffectedRows == 0)
    {
        lblInfo.Text = "No update was performed. " +
        "A concurrency error is likely, or the command is incorrectly written.";
    }
    else
    {
        lblInfo.Text = "Record successfully updated.";
    }
}

```

```

}
}

```

if you run two copies of this page in two separate browser windows, begin editing in both of them, and then try to commit both updates.

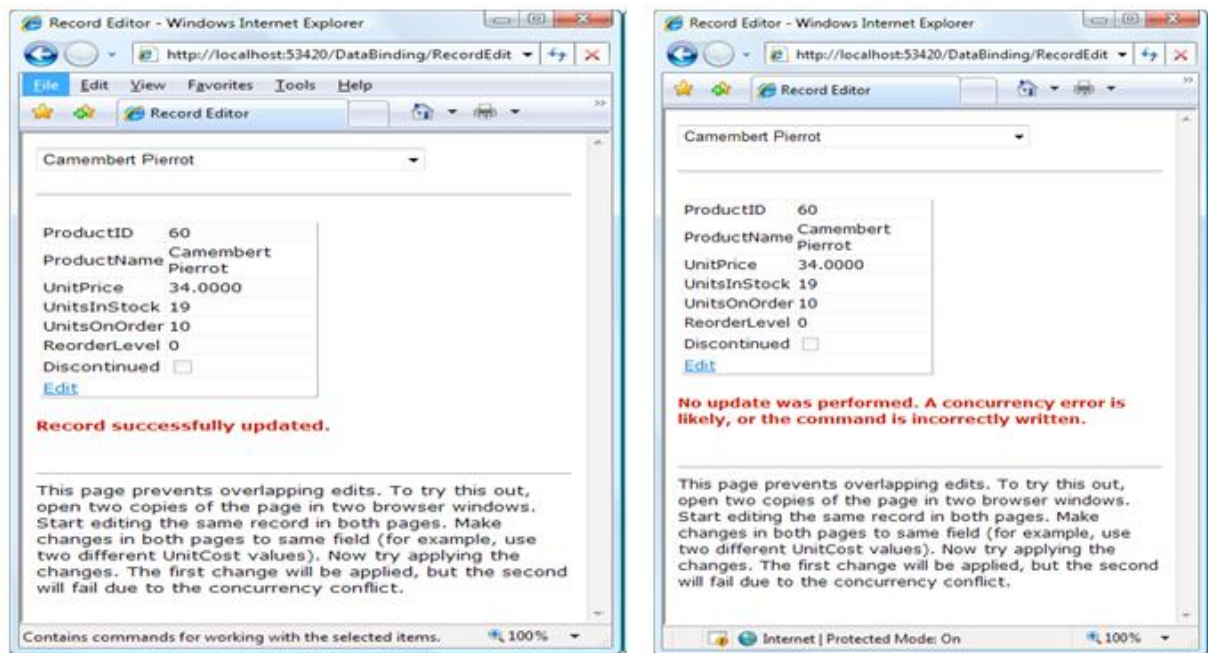


Figure15. A concurrency error in action

- Matching every field is an efficient approach for small records, not for the tables with huge amount of data
- you can match some of the fields (leaving out the ones with really big values) or you can add a timestamp field to your database table, and use that for concurrency checking.

Timestamps are special fields that the database uses to keep track of the state of a record. Whenever any change is made to a record, the database engine updates the timestamp field, giving it a new, automatically generated value. The purpose of a timestamp field is to make strict concurrency checking easier.

When you attempt to perform an update to a table that includes a timestamp field, you use a `Where` clause that matches the appropriate unique ID value (like `ProductID`) and the timestamp field:

```
UpdateCommand="UPDATE Products SET ProductName=@ProductName,  
UnitPrice=@UnitPrice,  
  
UnitsInStock=@UnitsInStock, UnitsOnOrder=@UnitsOnOrder,  
  
ReorderLevel=@ReorderLevel, Discontinued=@Discontinued  
  
WHERE ProductID=@ProductID AND RowTimestamp=@RowTimestamp"
```

The database engine uses the ProductID to look up the matching record. Then, it attempts to match the timestamp in order to update the record. If the timestamp matches, you know the record hasn't been changed. The actual value of the timestamp isn't important, because that's controlled by the database. You just need to know whether it's changed. Creating a timestamp is easy. In SQL Server, you create a timestamp field using the timestamp datatype. In other database products, timestamps are sometimes called row versions.

9.8 Summary

In this chapter, you learned a way to create dynamic text with simple data binding and also learned how ASP.NET builds on this infrastructure with much more useful features, including repeated-Value binding for quick-and-easy data display in a list control, and data source controls, which let you Create code-free bound pages. Using the techniques in this chapter, you can create a wide range of data-bound pages. You can create a page that incorporates record editing, sorting, and other more advanced tricks, the Data binding features you've learned in this chapter are just the first step.

Keywords: data binding, data source control, list, Grid View, controls

9.9 Check your progress here

1) If you are using the DataSet and you have to display the data in sorted order what will you do?

- a. Use Sort method of DataTable
- b. Use Sort method of DataSet
- c. Use DataView object with each sort
- d. Use datapaging and sort the data.

2) which of the following is true for ADO.NET DataSet?

- a. DataSet provides a disconnected view of a data source.
- b. Dataset enables to store data from multiple tables and multiple sources
- c. We can create relationship between the tables in a DataSet.
- d. All of above are true**

3) Which below is specified by the DataSource Property?

- a. Connection object
- b. DataAdapter object
- c. Database field**
- d. Dataset object

4) Valid Code for Creating a SqlConnection Object would be _____

a)SqlConnection conn =NEWSqlConnection("Data Source=(local);Initial Catalog=Northwind;Integrated Security=SSPI");

b)SqlConnect conn =NEWSqlConnection("Data Source=(local);Initial Catalog=Northwind;Integrated Security=SSPI");

c)SqlConnection conn =NEWSqlConnect("Data Source=(local);Initial Catalog=Northwind;Integrated Security=SSPI");

d)All of the mentioned

5) How many databinding types are there _____

- a) One
- b) Two**
- c)Three
- d)Four

9.10 Model Question

- 1) what is data binding and its types?
- 2) Describe single value and repeated value databinding?
- 3) Describe about .Net framework that includes data control?
- 4) write a code for insert, update and delete using databound list control.
- 5). Write few lines about sqldatasource with example.

Reference Book:

1. Mathew MacDonald, "Beginning ASP.NET 4 in C# 2010" Apress 2010.

LESSON - 10

THE DATA CONTROL

Structure

- 10.1 Introduction
- 10.2 Objectives
- 10.3 The Grid View
- 10.4 Formatting the Grid view
- 10.5 Sorting and Paging the grid view
- 10.6 Using grid view templates
- 10.7 Using Multiple templates
- 10.8 The detail view and Form view
- 10.9 Summary
- 10.10 Check your progress
- 10.11 Model Questions

10.1 Introduction

In this chapter, you'll concentrate on three more advanced controls—GridView, DetailsView, and FormView—that allow you to bind entire tables of data. The rich data controls are quite a bit different from the simple list controls—for one thing, they are designed exclusively for data binding. They also support higherlevel features such as selecting, editing, and sorting.

10.2 Learning Objectives

- ✓ Learn to configure columns and automatic generating columns.
- ✓ Formatting the gridview with styles and fields.
- ✓ Configure styles with visual studio.

- ✓ Explore Selecting and Editing with the Grid
- ✓ Learn to Sorting and paging the Grid
- ✓ Understand Grid view templates with handling and editing
- ✓ Understand The details view and form views

10.3 The Grid View

The GridView is an extremely flexible grid control that displays a multicolumn table. Each record in your data source becomes a separate row in the grid. Each field in the record becomes a separate column in the grid.

Automatically Generating Columns

Once you've added this GridView tag to your page, you can fill it with data. Here's an example that performs a query using the ADO.NET objects and binds the retrieved DataSet:

```
protected void Page_Load(object sender, EventArgs e)
{
    // Define the ADO.NET objects.

    string connectionString =
        WebConfigurationManager.ConnectionStrings["Northwind"].ConnectionString;

    string selectSQL = "SELECT ProductID, ProductName, UnitPrice FROM Products";

    SqlConnection con = new SqlConnection(connectionString);

    SqlCommand cmd = new SqlCommand(selectSQL, con);

    SqlDataAdapter adapter = new SqlDataAdapter(cmd);

    // Fill the DataSet.

    DataSet ds = new DataSet();

    adapter.Fill(ds, "Products");

    // Perform the binding.
```

```

GridView1.DataSource = ds;

GridView1.DataBind();

}

```

Figure1. shows the GridView this code creates.



ProductID	ProductName	UnitPrice
1	Chai	18.0000
2	Chang	19.0000
3	Aniseed Syrup	10.0000
4	Chef Anton's Cajun Seasoning	22.0000
5	Chef Anton's Gumbo Mix	21.3500
6	Grandma's Boysenberry Spread	25.0000
7	Uncle Bob's Organic Dried Pears	30.0000
8	Northwoods Cranberry Sauce	40.0000
9	Mishi Kobe Niku	97.0000
10	Ikura	31.0000
11	Queso Cabrales	21.0000
12	Queso Manchego La Pastora	38.0000
13	Konbu	6.0000
14	Tofu	23.2500
15	Genen Shouyu	15.5000

Figure 1. The bare-bones GridView

Defining Columns

By default, the `GridView.AutoGenerateColumns` property is true, and the GridView creates a column for each field in the bound `DataTable`. This automatic column generation is good for creating quick test pages, but it doesn't give you the flexibility you'll usually want. In all these cases, you need to set `AutoGenerateColumns` to false and define the columns in the section of the GridView control tag.

Each column can be any of several column types, as described in Table1. The order of your column tags determines the left-to-right order of columns in the GridView.

Table1. Column Types

Class	Description
BoundField	This column displays text from a field in the data source.

ButtonField	This column displays a button in this grid column.
CheckBoxField	This column displays a check box in this grid column. It's used automatically for true/false fields.
CommandField	This column provides selection or editing buttons.
HyperLinkField	This column displays its contents (a field from the data source or static text) as a hyperlink.
ImageField	This column displays image data from a binary field
TemplateField	This column allows you to specify multiple fields, custom controls, and arbitrary HTML using a custom template.

Here's a complete GridView declaration with explicit columns:

```
<asp:GridView ID="GridView1" runat="server" DataSourceID="sourceProducts"
AutoGenerateColumns="False">
<Columns>
<asp:BoundFieldDataField="ProductID" HeaderText="ID" />
<asp:BoundFieldDataField="ProductName" HeaderText="Product Name" />
<asp:BoundFieldDataField="UnitPrice" HeaderText="Price" />
</Columns>
</asp:GridView>
```

Explicitly defining columns has several advantages:

- You can easily fine-tune your column order, column headings, and other details by tweaking the properties of your column object.
- You can hide columns you don't want to show by removing the column tag.
- Without automatically generated columns, the GridView simply shows a few generic placeholder columns.
- You can add extra columns to the mix for selecting, editing, and more.

Configuring Columns

When you explicitly declare a bound field, you have the opportunity to set other properties. Table 2 lists these properties.

Table 2. BoundField Properties

Property	Description
DataField	Identifies the field (by name) that you want to display in this column.
DataFormatString	Formats the field. This is useful for getting the right representation of numbers and dates.
ApplyFormatInEditMode	If true, the DataFormat string is used to format the value even when the value appears in a text box in edit mode.
FooterText, HeaderText, and HeaderImageUrl	Sets the text in the header and footer region of the grid if this grid has a header (GridView.ShowHeader is true) and footer (GridView.ShowFooter is true). The header is most commonly used for a descriptive label such as the field name; the footer can contain a dynamically calculated value such as a summary.
ReadOnly	If true, it prevents the value for this column from being changed in edit mode. Primary key fields are often read-only.
InsertVisible	If true, it prevents the value for this column from being set in insert mode.
Visible	If false, the column won't be visible in the page.
SortExpression	Sorts your results based on one or more columns.
HtmlEncode	If true, all text will be HTML encoded to prevent special characters from mangling the page.
NullDisplayText	Displays the text that will be shown for a null value.
ConvertEmptyStringToNull	If true, converts all empty strings to null values.
ControlStyle, HeaderStyle, FooterStyle, and ItemStyle	Configures the appearance for just this column, overriding the styles for the row.

Generating Columns with Visual Studio

Once you've created your columns, you can also use some helpful design-time support to configure the properties of each column. To do this, select the GridView, and click the ellipsis next to the Columns property in the Properties window. You'll see a Fields dialog box that lets you add, remove, and refine your columns.

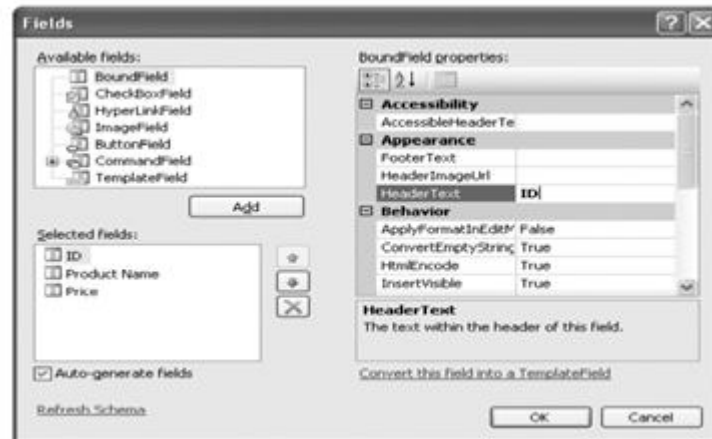


Figure2. Configuring columns in Visual Studio

understand the underpinnings of the GridView:

Formatting: How to format rows and data values

Selecting: How to let users select a row in the GridView and respond accordingly

Editing: How to let users commit record updates, inserts, and deletes

Sorting: How to dynamically reorder the GridView in response to clicks on a column header

Paging: How to divide a large result set into multiple pages of data

Templates: How to take complete control of designing, formatting, and editing by defining templates.

10.4 Formatting the Gridview

Formatting consists of several related tasks. First, you want to ensure that dates, currencies, and other number values are presented in the appropriate way. You handle this job with the DataFormatString property. Next, you'll want to apply the perfect mix of colors, fonts, borders, and alignment options to each aspect of the grid, from headers to data items. The GridView supports these features through styles.

Formatting Fields

Each `BoundField` column provides a `DataFormatString` property you can use to configure the appearance of numbers and dates using a format string. Format strings generally consist of a placeholder and a format indicator, which are wrapped inside curly brackets. A typical format string looks something like this:

```
{0:C}
```

0 represents the value that will be formatted, and the letter indicates a predetermined format style.

C means currency format, which formats a number as an amount of money.

Here's a column that uses this format string:

```
<asp:BoundField DataField="UnitPrice" HeaderText="Price"
DataFormatString="{0:C}" />
```

Table 3 shows some of the other formatting options for numeric values.

Table 3. Numeric Format Strings

Type	Format String	Example
Currency	{0:C}	\$1,234.50. Brackets indicate negative values: (\$1,234.50). The currency sign is locale-specific
Scientific (Exponential)	{0:E}	1.234500E+003
Percentage	{0:P}	45.6%
Fixed Decimal	{0:F?}	Depends on the number of decimal places you set. {0:F3} would be 123.400. {0:F0} would be 123.

You can find other examples in the MSDN Help. For date or time values, you'll find an extensive list. For example, if you want to write the `BirthDate` value in the format month/day/year (as in 12/30/08), you use the following column:

```
<asp:BoundFieldDataField="BirthDate" HeaderText="Birth Date"
```

```
DataFormatString="{0:MM/dd/yy}" />
```

Table4. Time and Date Format Strings

Type	Format String	Syntax	Example
Short Date	{0:d}	M/d/yyyy	10/30/2010
Long Date	{0:D}	dddd, MMMM dd, yyyy	Monday, January 30, 2010
Long Date and Short Time	{0:f} dddd	MMMM dd, yyyyHH:mm aa	Monday, January 30, 2010 10:00 AM
Long Date and Long Time	{0:F} dddd	MMMM dd, yyyy, HH:mm:ss aa	Monday, January 30 2010 10:00:23 AM
ISO Sortable Standard	{0:s}	yyyy-MM-ddTHH:mm:ss	2010-01-30T10:00:23
Month and Day	{0:M}	MMMM dd	January 30
General	{0:G}	M/d/yyyyHH:mm:ss aa	10/30/2010 10:00:23 AM

Using Styles

The GridView exposes a rich formatting model that's based on styles. Altogether, you can set eight GridView styles, as described in Table5.

Table5. GridView Styles

Style	Description
HeaderStyle	Configures the appearance of the header row that contains column titles, if you've chosen to show it.
RowStyle	Configures the appearance of every data row.
AlternatingRowStyle	If set, applies additional formatting to every other row.
SelectedRowStyle	Configures the appearance of the row that's currently selected.
EditRowStyle	Configures the appearance of the row that's in edit mode.
EmptyDataRowStyle	Configures the style that's used for the single empty row in the special case where the bound data object contains no rows.
FooterStyle	Configures the appearance of the footer row at the bottom of the GridView, if you've chosen to show it (if ShowFooter is true).

PagerStyle Configures the appearance of the row with the page links, if you've enabled paging (set `AllowPaging` to true).

Here's an example that changes the style of rows and headers in a GridView:

```
<asp:GridView ID="GridView1" runat="server" DataSourceID="sourceProducts"
AutoGenerateColumns="False">
<RowStyleBackColor="#E7E7FF" ForeColor="#4A3C8C" />
<HeaderStyleBackColor="#4A3C8C" Font-Bold="True" ForeColor="#F7F7F7" />
<Columns>
<asp:BoundFieldDataField="ProductID" HeaderText="ID" />
<asp:BoundFieldDataField="ProductName" HeaderText="Product Name" />
<asp:BoundFieldDataField="UnitPrice" HeaderText="Price" />
</Columns>
</asp:GridView>
```

In this example, every column is affected by the formatting changes. To create a column-specific style, you simply need to rearrange the control tags so that the formatting tag becomes a nested tag inside the appropriate column tag. Here's an example that formats just the ProductName column:

```
<asp:GridView ID="GridView2" runat="server" DataSourceID="sourceProducts"
AutoGenerateColumns="False" >
<Columns>
<asp:BoundFieldDataField="ProductID" HeaderText="ID" />
<asp:BoundFieldDataField="ProductName" HeaderText="Product Name">
<ItemStyleBackColor="#E7E7FF" ForeColor="#4A3C8C" />
<HeaderStyleBackColor="#4A3C8C" Font-Bold="True" ForeColor="#F7F7F7" />
</asp:BoundField>
```

```
<asp:BoundFieldDataField="UnitPrice" HeaderText="Price" />
```

```
</Columns>
```

```
</asp:GridView>
```

Figure3 compares these two examples. You can use a combination of ordinary style settings and column-specific style setting.

ID	Product Name	Price
1	Chai	18.0000
2	Chang	19.0000
3	Aniseed Syrup	10.0000
4	Chef Anton's Cajun Seasoning	22.0000
5	Chef Anton's Gumbo Mix	21.3500
6	Grandma's Boysenberry Spread	25.0000
7	Uncle Bob's Organic Dried Pears	30.0000
8	Northwoods Cranberry Sauce	40.0000
9	Mishi Kobe Niku	97.0000
10	Ikura	31.0000
11	Queso Cabrales	21.0000
12	Queso Manchego La Pastora	38.0000
13	Konbu	6.0000
14	Tofu	23.2500
15	Genen Shoyu	15.5000

ID	Product Name	Price
1	Chai	18.0000
2	Chang	19.0000
3	Aniseed Syrup	10.0000
4	Chef Anton's Cajun Seasoning	22.0000
5	Chef Anton's Gumbo Mix	21.3500
6	Grandma's Boysenberry Spread	25.0000
7	Uncle Bob's Organic Dried Pears	30.0000
8	Northwoods Cranberry Sauce	40.0000
9	Mishi Kobe Niku	97.0000
10	Ikura	31.0000
11	Queso Cabrales	21.0000
12	Queso Manchego La Pastora	38.0000
13	Konbu	6.0000
14	Tofu	23.2500
15	Genen Shoyu	15.5000

Figure3. Formatting the GridView

Configuring Styles with Visual Studio

To set style properties, you can use the Properties window to modify the style properties. For example, to configure the font of the header, expand the HeaderStyle property to show the nested Font property, and set that. The only limitation of this approach is that it doesn't allow you to set the style for individual columns—you must first call up the Fields dialog box shown in Figure by editing the Columns property. You can even set a combination of styles using a preset theme by clicking the Auto Format link in the GridView smart tag. Figure4 shows the Auto Format dialog box with some of the preset styles you can choose.

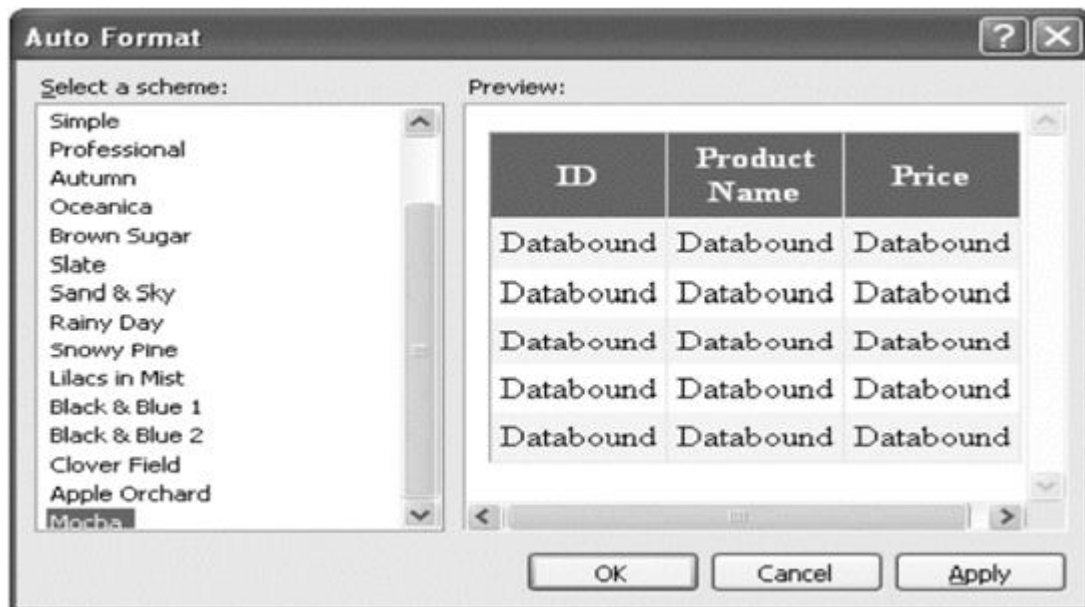


Figure4. Automatically formatting a GridView

Formatting-Specific Values

The `GridViewRow.DataItem` property provides the data object for the given row, and the `GridViewRow.Cells` collection allows you to retrieve the row content. You can use the `GridViewRow` to change colors and alignment, add or remove child controls, and so on. The following example handles the `RowDataBound` event and changes the background color to highlight high prices (those more expensive than \$50):

```
protected void GridView1_RowDataBound(object sender, GridViewRowEventArgs e)
{
    if (e.Row.RowType == DataControlRowType.DataRow)
    {
        // Get the price for this row.
        decimal price = (decimal)DataBinder.Eval(e.Row.DataItem, "UnitPrice");
        if (price > 50)
```

```

{
e.Row.BackColor = System.Drawing.Color.Maroon;

e.Row.ForeColor = System.Drawing.Color.White;

e.Row.Font.Bold = true;

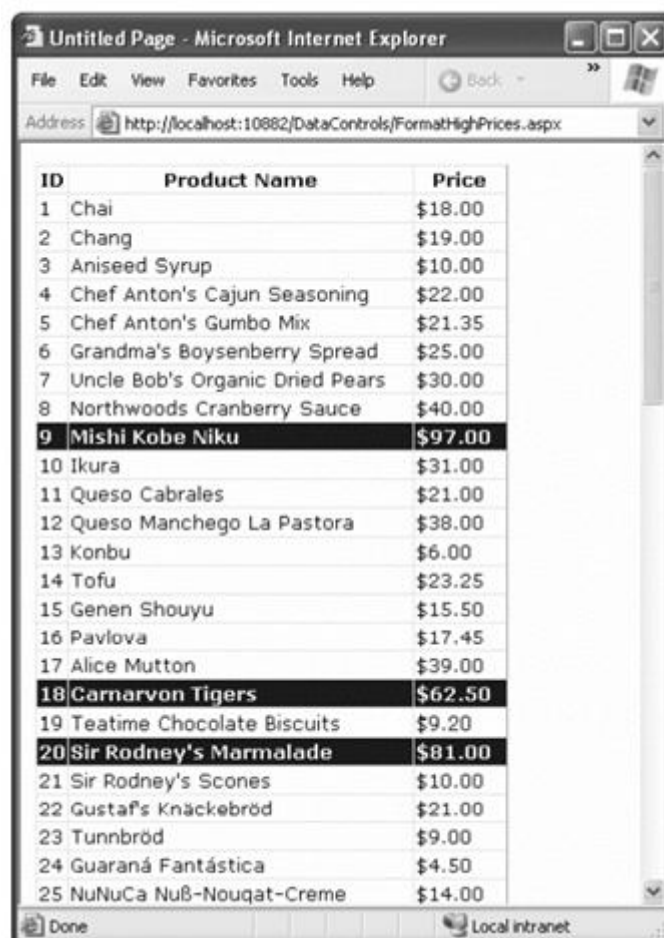
}

}

}

```

In this example, you're binding to the `SqlDataSource` in `DataSet` mode, which means each data item will be a `DataRowView` object. To avoid coding these details, which can make it more difficult to change your data access code, you can rely on the `DataBinder.Eval()` helper method, which understands all these types of data objects. Figure5 shows the resulting page.



ID	Product Name	Price
1	Chai	\$18.00
2	Chang	\$19.00
3	Aniseed Syrup	\$10.00
4	Chef Anton's Cajun Seasoning	\$22.00
5	Chef Anton's Gumbo Mix	\$21.35
6	Grandma's Boysenberry Spread	\$25.00
7	Uncle Bob's Organic Dried Pears	\$30.00
8	Northwoods Cranberry Sauce	\$40.00
9	Mishi Kobe Niku	\$97.00
10	Ikura	\$31.00
11	Queso Cabrales	\$21.00
12	Queso Manchego La Pastora	\$38.00
13	Konbu	\$6.00
14	Tofu	\$23.25
15	Genen Shouyu	\$15.50
16	Pavlova	\$17.45
17	Alice Mutton	\$39.00
18	Carnarvon Tigers	\$62.50
19	Teatime Chocolate Biscuits	\$9.20
20	Sir Rodney's Marmalade	\$81.00
21	Sir Rodney's Scones	\$10.00
22	Gustaf's Knäckebröd	\$21.00
23	Tunnbröd	\$9.00
24	Guaraná Fantástica	\$4.50
25	NuNuCa Nuß-Nougat-Creme	\$14.00

Figure5. Formatting individual rows based on values

SELECTING A GRIDVIEW ROW

With the GridView, selection happens almost automatically once you set up a few basics. To find out what item is currently selected (or to change the selection), you can use the `GridView.SelectedIndex` property.

Adding a Select Button

The GridView provides built-in support for selection. You simply need to add a `CommandField` column with the `ShowSelectButton` property set to true. You can then specify the text through the `SelectText` property or specify the link to the image through the `SelectImageUrl` property.

Here's an example that displays a select button:

```
<asp:CommandFieldShowSelectButton="True" ButtonType="Button"
SelectText="Select" />
```

And here's an example that shows a small clickable icon:

```
<asp:CommandFieldShowSelectButton="True" ButtonType="Image"
SelectImageUrl="select.gif" />
```

Figure 6 shows a page with a text select button (and product 14 selected).

	ID	Product Name	Price
Select	1	Chai	18.0000
Select	2	Chang	19.0000
Select	3	Aniseed Syrup	10.0000
Select	4	Chef Anton's Cajun Seasoning	22.0000
Select	5	Chef Anton's Gumbo Mix	21.3500
Select	6	Grandma's Boysenberry Spread	25.0000
Select	7	Uncle Bob's Organic Dried Pears	30.0000
Select	8	Northwoods Cranberry Sauce	40.0000
Select	9	Mishi Kobe Niku	97.0000
Select	10	Ikura	31.0000
Select	11	Queso Cabrales	21.0000
Select	12	Queso Manchego La Pastora	38.0000
Select	13	Konbu	6.0000
Select	14	Tofu	23.2500
Select	15	Genen Shouyu	15.5000
Select	16	Pavlova	17.4500

Figure6. GridView selection

First, the `GridView.SelectedIndexChanging` event fires, which you can intercept to cancel the operation. Next, the `GridView.SelectedIndex` property is adjusted to point to the selected row. Finally, the `GridView.SelectedIndexChanged` event fires, which you can handle if you want to manually update other controls to reflect the new selection.

Using a Data Field As a Select Button

To use this technique, remove the `CommandField` column, and add a `ButtonField` column instead. Then, set the `DataTextField` to the name of the field you want to use.

```
<asp:ButtonFieldButtonType="Button" DataTextField="ProductID" />
```

just configure the link to raise the `SelectedIndexChanged` event by

specifying a `CommandName` with the text `Select`, as shown here:

```
<asp:ButtonFieldCommandName="Select" ButtonType="Button"
```



```
DataTextField="ProductID" />
```

Now clicking the data field automatically selects the record.

Using Selection to Create Master-Details Pages

A typical master-details page has two GridView controls. The first GridView shows the master (or parent) table. When a user selects an item in the first GridView, the second GridView is filled with related records from the details (or parent) table. For example, a typical implementation of this technique might have a customers table in the first GridView. Select a customer, and the second GridView is filled with the list of orders made by that customer. The following example puts it all together. It defines two GridView controls. The first shows a list of categories. The second shows the products that fall into the currently selected category. Here's the page markup for this example:

```
Categories:<br />
```

```
<asp:GridView ID="gridCategories" runat="server" DataSourceID="sourceCategories"
```

```
DataKeyNames="CategoryID">
```

```
<Columns>
```

```
<asp:CommandField ShowSelectButton="True" />
```

```
</Columns>
```

```
<SelectedRowStyle BackColor="#FFCC66" Font-Bold="True"
```

```
ForeColor="#663399" />
```

```
</asp:GridView>
```

```
<asp:SqlDataSource ID="sourceCategories" runat="server"
```

```
ConnectionString="<%$ ConnectionStrings:Northwind %>"
```

```
SelectCommand="SELECT * FROM Categories"></asp:SqlDataSource>
```

```
<br />
```

Products in this category:


```
<asp:GridView ID="gridProducts" runat="server" DataSourceID="sourceProducts">
<SelectedRowStyleBackColor="#FFCC66" Font-Bold="True" ForeColor="#663399" />
</asp:GridView>

<asp:SqlDataSource ID="sourceProducts" runat="server"
ConnectionString="<%= $ConnectionStrings:Northwind %>"
SelectCommand="SELECT ProductID, ProductName, UnitPrice FROM Products WHERE
CategoryID=@CategoryID">
<SelectParameters>
<asp:ControlParameter Name="CategoryID" ControlID="gridCategories"
PropertyName="SelectedDataKey.Value" />
</SelectParameters>
</asp:SqlDataSource>
```

As you can see, you need two data sources, one for each GridView. The second data source uses aControlParameter that links it to the SelectedDataKey property of the first GridView. Best of all, you still don't need to write any code or handle the SelectedIndexChanged event on your own.

Figure 7 shows this example in action.

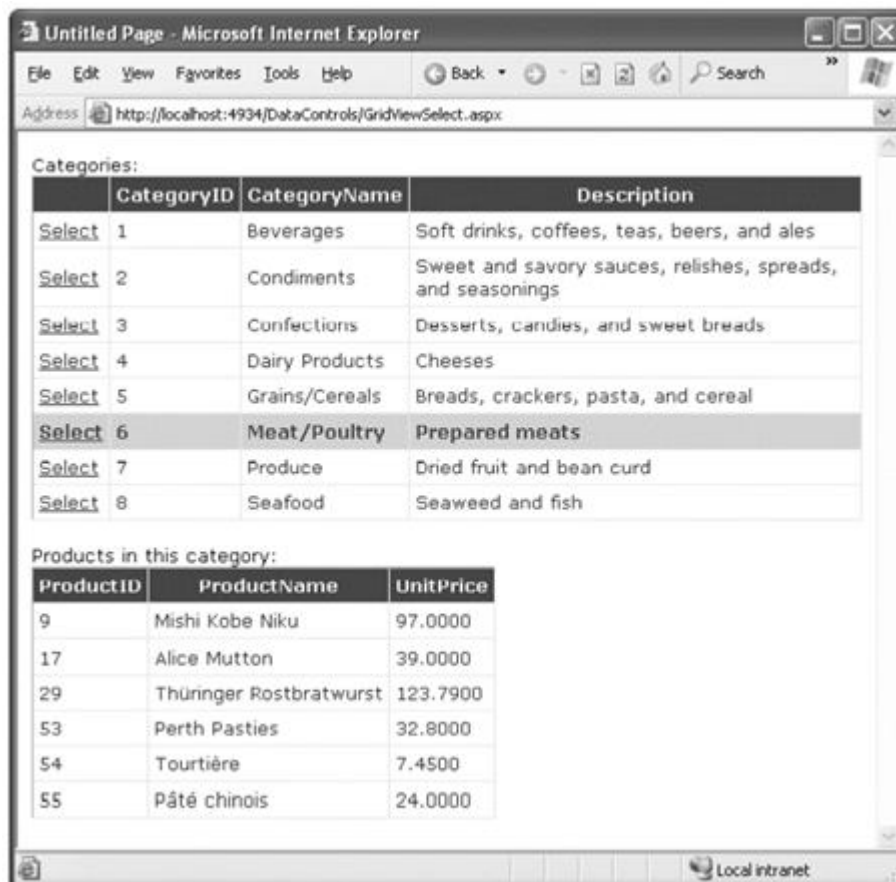


Figure7. A master-details page

Editing with the gridview

For selection, you use a CommandField column with the ShowSelectButton property set to true. To add edit controls, you follow almost the same step—once again, you use the CommandField column, but now you set ShowEditButton to true.

Here's an example of a GridView that supports editing:

```
<asp:GridView ID="gridProducts" runat="server" DataSourceID="sourceProducts"
AutoGenerateColumns="False" DataKeyNames="ProductID">
<Columns>
<asp:BoundField DataField="ProductID" HeaderText="ID" ReadOnly="True" />
```

```
<asp:BoundFieldDataField="ProductName" HeaderText="Product Name"/>
```

```
<asp:BoundFieldDataField="UnitPrice" HeaderText="Price" />
```

```
<asp:CommandFieldShowEditButton="True" />
```

```
</Columns>
```

```
</asp:GridView>
```

And here's a revised data source control that can commit your changes:

```
<asp:SqlDataSource id="sourceProducts" runat="server"
```

```
ConnectionString="<%$ ConnectionStrings:Northwind %>"
```

```
SelectCommand="SELECT ProductID, ProductName, UnitPrice FROM Products"
```

```
UpdateCommand="UPDATE Products SET ProductName=@ProductName,
```

```
UnitPrice=@UnitPrice WHERE ProductID=@ProductID" />
```

When you add a CommandField with the ShowEditButton property set to true, the GridView editing controls appear in an additional column. When you run the page and the GridView is bound and displayed, the edit column shows an Edit link next to every record (see Figure8).

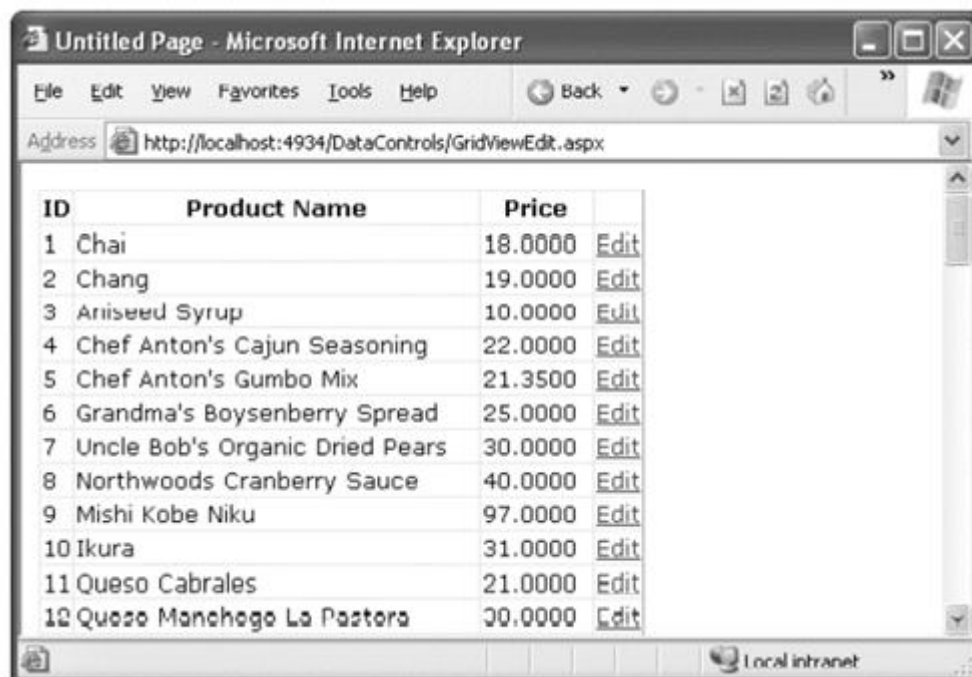


Figure8. The editing controls

When clicked, this link switches the corresponding row into edit mode. The Edit link is replaced with an Update link and a Cancel link (see Figure9).

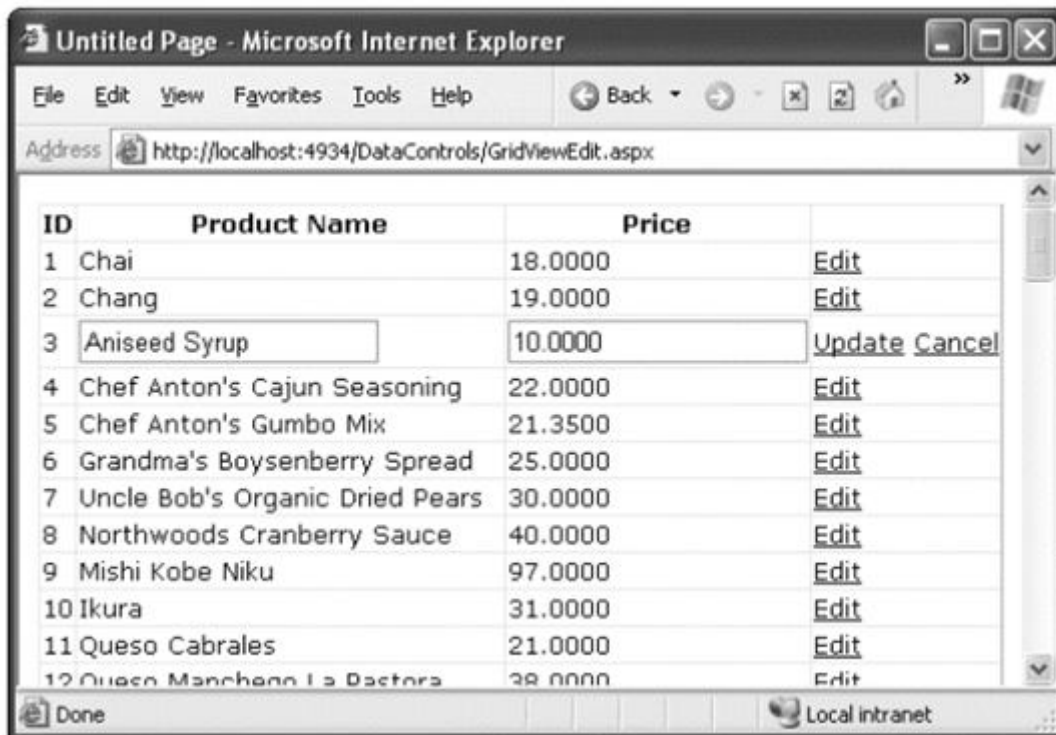


Figure9. Editing a record

The Cancel link returns the row to its initial state. The Update link passes the values to the `SqlDataSource.UpdateParameters` collection (using the field names) and then triggers the `SqlDataSource.Update()` method to apply the change to the database. To enable deleting, you need to add a column to the GridView that has the `ShowDeleteButton` property set to true. As long as your linked `SqlDataSource` has the `DeleteCommand` property filled in, these operations will work automatically.

10.5 Sorting and Paging the Gridview

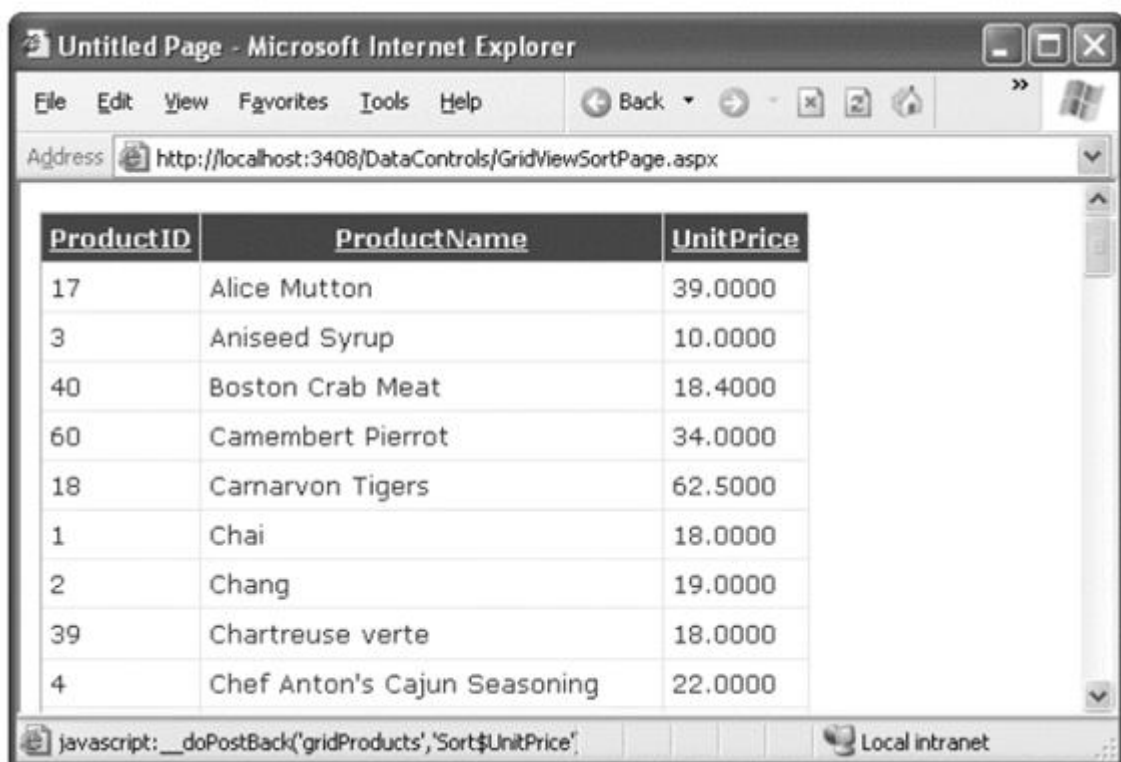
Sorting

To enable sorting, you must set the `GridView.AllowSorting` property to true. Next, you need to define a `SortExpression` for each column that can be sorted. In theory, a sort expression takes the form used in the `ORDER BY` clause of a SQL query and can use any syntax that's understood

by the data source control. For example, here's how you could define the ProductName column so it sorts by alphabetical ordering rows:

```
<asp:BoundField DataField="ProductName" HeaderText="Product Name"
SortExpression="ProductName" />
```

Figure10 shows an example with a grid that has sort expressions for all three columns, and is currently sorted by product name.



ProductID	ProductName	UnitPrice
17	Alice Mutton	39.0000
3	Aniseed Syrup	10.0000
40	Boston Crab Meat	18.4000
60	Camembert Pierrot	34.0000
18	Carnarvon Tigers	62.5000
1	Chai	18.0000
2	Chang	19.0000
39	Chartreuse verte	18.0000
4	Chef Anton's Cajun Seasoning	22.0000

Figure10. Sorting the GridView

In DataSet mode, the entire results are placed in a DataSet and then the records are copied from the DataSet into the bound control. If the data needs to be sorted, the sorting happens between these two steps—after the records are retrieved but before they're bound in the web page.

Sorting and Selecting

You must simply set the `GridView.EnablePersistedSelection` property true. Now, ASP.NET will ensure that the selected item is identified by its data key. As a result, the selected item will remain selected, even if it moves to a new position in the `GridView` after a sort.

Paging

When you use automatic paging, the full results are retrieved from the data source and placed into a `DataSet`. Once the `DataSet` is bound to the `GridView`, however, the data is subdivided into smaller groupings (for example, with 20 rows each), and only a single batch is sent to the user. The other groups are abandoned when the page finishes processing. When the user moves to the next page, the same process is repeated—in other words, the full query is performed once again.

The `GridView` extracts just one group of rows, and the page is rendered. To allow the user to skip from one page to another, the `GridView` displays a group of pager controls at the bottom of the grid. These pager controls could be previous/next links (often displayed as < and >) or number links (1, 2, 3, 4, 5, . . .) that lead to specific pages.

Table 6. Paging Members of the `GridView`

Property	Description
<code>AllowPaging</code>	Enables or disables the paging of the bound records. It is false by default.
<code>PageSize</code>	Gets or sets the number of items to display on a single page of the grid. The default value is 10.
<code>PageIndex</code>	Gets or sets the zero-based index of the currently displayed page, if paging is enabled.
<code>PagerSettings</code>	Provides a <code>PagerSettings</code> object that wraps a variety of formatting options for the pager controls.
<code>PagerStyle</code>	Provides a style object you can use to configure fonts, colors, and text alignment for the paging controls.

PageIndexChanging and
PageIndexChanged events

Occur when one of the page selection elements is clicked, just before the PageIndex is changed (PageIndexChanging) and just after (PageIndexChanged).

To use automatic paging, you need to set AllowPaging to true (which shows the page controls), and you need to set PageSize to determine how many rows are allowed on each page. Here's an example of a GridView control declaration that sets these properties:

```
<asp:GridView ID="GridView1" runat="server" DataSourceID="sourceProducts"
```

```
    PageSize="10" AllowPaging="True"
```

...

```
</asp:GridView>
```

Figure 11 shows an example with ten records per page (for a total of eight pages).

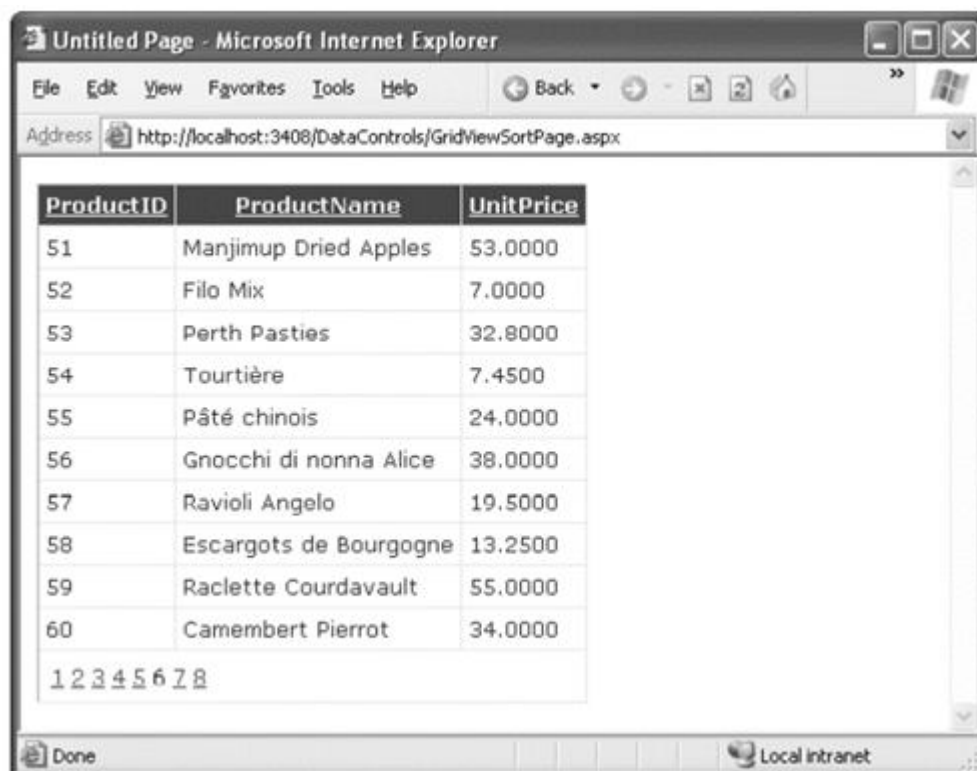


Figure 11. Paging the GridView

Paging and Selection

Set the `GridView.EnablePersistedSelection` property to `true`. Now, as you move from one page to another, the selection will automatically be removed from the `GridView` (and the `SelectedIndex` property will be set to `-1`). But if you move back to the page that held the originally selected row, that row will be re-selected.

10.6 Using Gridview Templates

The `TemplateField` allows you to define a completely customized template for a column. Inside the template you can add control tags, arbitrary HTML elements, and data binding expressions. For example, imagine you want to create a column that combines the in stock, on order, and reorder level information for a product. To accomplish this trick, you can construct an `ItemTemplate` like this:

```
<asp:TemplateFieldHeaderText="Status">

<ItemTemplate>

<b>In Stock:</b>

<%# Eval("UnitsInStock") %><br />

<b>On Order:</b>

<%# Eval("UnitsOnOrder") %><br />

<b>Reorder:</b>

<%# Eval("ReorderLevel") %>

</ItemTemplate>

</asp:TemplateField>
```

To create the data binding expressions, the template uses the `Eval()` method, which is a static method of the `System.Web.UI.DataBinder` class. `Eval()` is an indispensable convenience—it automatically retrieves the data item that's bound to the current row, uses reflection to find the matching field, and retrieves the value.

If you retrieve additional fields that are never bound to any template, no problem will occur. Here's the revised data source with these fields:

```

<asp:SqlDataSource ID="sourceProducts" runat="server"
ConnectionString="<%%$ ConnectionStrings:Northwind %>"
SelectCommand="SELECT ProductID, ProductName, UnitPrice, UnitsInStock,
UnitsOnOrder,ReorderLevel FROM Products"
UpdateCommand="UPDATE Products SET ProductName=@ProductName,
UnitPrice=@UnitPrice WHERE ProductID=@ProductID">
</asp:SqlDataSource>

```

Figure12 shows an example with several normal columns and the template column at the end.



The screenshot shows a web browser window titled "Untitled Page - Microsoft Internet Explorer". The address bar displays "http://localhost:3408/DataControls/GridViewTemplates.aspx". The main content area displays a GridView with four columns: "ID", "Product Name", "Price", and "Status". The "Status" column is a template column containing stock, order, and reorder information for each product.

ID	Product Name	Price	Status
1	Chai	18.0000	In Stock: 36 On Order: 0 Reorder: 10
2	Chang	19.0000	In Stock: 17 On Order: 40 Reorder: 25
3	Aniseed Syrup	10.0000	In Stock: 13 On Order: 70 Reorder: 25
4	Chef Anton's Cajun Seasoning	22.0000	In Stock: 53 On Order: 0 Reorder: 0
5	Chef Anton's Gumbo Mix	21.3500	In Stock: 0 On Order: 0 Reorder: 0
6	Grandma's Boysenberry Spread	25.0000	In Stock: 120 On Order: 0 Reorder: 25
7	Uncle Bob's Organic Dried Pears	30.0000	In Stock: 15 On Order: 0 Reorder: 10
8	Northwoods Cranberry Sauce	40.0000	In Stock: 6 On Order: 0 Reorder: 0

Figure12. A GridView with a template column

10.7 Using Multiple Templates

The TemplateField allows you to configure various aspects of its appearance with a number of templates. Inside every template column, you can use the templates listed in Table7.

Table 7. TemplateField Templates

Mode	Description
HeaderTemplate	Determines the appearance and content of the header cell.
FooterTemplate	Determines the appearance and content of the footer cell (if you set ShowFooter to true).
ItemTemplate	Determines the appearance and content of each data cell
AlternatingItemTemplate	Determines the appearance and content of even-numbered rows.
EditItemTemplate	Determines the appearance and controls used in edit mode.
InsertItemTemplate	Determines the appearance and controls used in edit mode

Editing Templates in Visual Studio

Visual Studio includes solid support for editing templates in the web page designer.

1. Create a GridView with at least one template column.
2. Select the GridView, and click Edit Templates in the smart tag. This switches the GridView into template edit mode.
3. In the smart tag, use the Display drop-down list to choose the template you want to edit (see Figure). You can choose either of the two templates that apply to the whole GridView (EmptyDataTemplate or PagerTemplate), or you can choose a specific template for one of the template columns.
4. Enter your content in the control. You can enter static content, drag and drop controls, and so on.
5. When you're finished, choose End Template Editing from the smart tag.

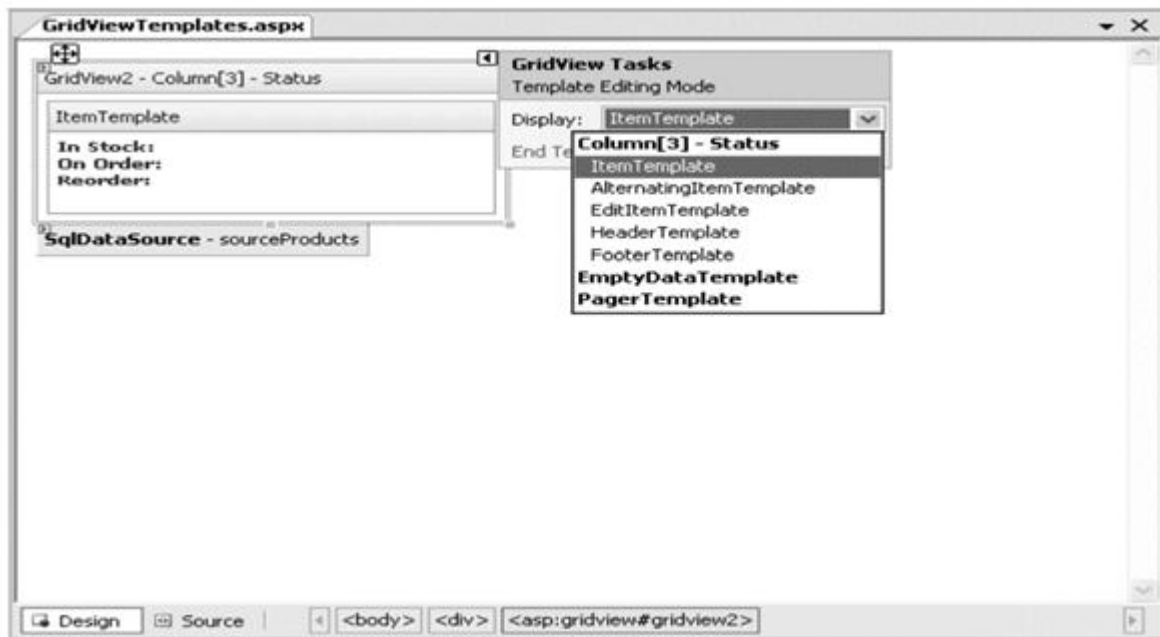


Figure13. Editing a template in Visual Studio

Handling Events in a Template

you want to add a clickable image link by adding an ImageButtoncontrol.

```
<asp:TemplateFieldHeaderText="Status">
```

```
<ItemTemplate>
```

```
<asp:ImageButton ID="ImageButton1" runat="server"
```

```
ImageUrl="statuspic.gif" />
```

```
</ItemTemplate>
```

```
</asp:TemplateField>
```

The GridView.RowCommand event serves this purpose, because it fires whenever any button is clicked in any template. This process, where a control event in a template is turned into an event in the containing control, is called *event bubbling*. you still need a way to pass information to the RowCommand event to identify the row where the action took place. The secret lies in two string properties that all button controls provide: CommandName and CommandArgument.

CommandName sets a descriptive name you can use to distinguish clicks on your ImageButton from clicks on other button controls in the GridView. Here's a template field that contains the revised ImageButton tag:

```
<asp:TemplateFieldHeaderText="Status">

<ItemTemplate>

<asp:ImageButton ID="ImageButton1" runat="server"

ImageUrl="statuspic.gif"

CommandName="StatusClick" CommandArgument='<%# Eval("ProductID") %>' />

</ItemTemplate>

</asp:TemplateField>
```

And here's the code you need in order to respond when an ImageButton is clicked:

```
protected void GridView1_RowCommand(object sender, GridViewCommandEventArgs e)

{

    if (e.CommandName == "StatusClick")

        lblInfo.Text = "You clicked product #" + e.CommandArgument.ToString();

}
```

Editing with a Template

The standard editing support has several limitations:

It's not always appropriate to edit values using a text box: Certain types of data are best handled with other controls (such as drop-down lists). Large fields need multiline text boxes, and so on.

You get no validation: It would be nice to restrict the editing possibilities so that currency figures can't be entered as negative numbers, for example. You can do that by adding validator controls to an EditItemTemplate.

The visual appearance is often ugly: A row of text boxes across a grid takes up too much space and rarely seems professional.

In a template column, you explicitly define the edit controls and their layout using the `EditItemTemplate`. This can be a somewhat laborious process. Here's the template column used earlier for stock information with an editing template:

```
<asp:TemplateFieldHeaderText="Status">

<ItemStyle Width="100px" />

<ItemTemplate>

<b>In Stock:</b>< %# Eval("UnitsInStock") %><br />

<b>On Order:</b>< %# Eval("UnitsOnOrder") %><br />

<b>Reorder:</b>< %# Eval("ReorderLevel") %>

</ItemTemplate>

<EditItemTemplate>

<b>In Stock:</b>< %# Eval("UnitsInStock") %><br />

<b>On Order:</b>< %# Eval("UnitsOnOrder") %><br /><br />

<b>Reorder:</b>

<asp:TextBox Text='< %# Bind("ReorderLevel") %>' Width="25px"

runat="server" id="txtReorder" />

</EditItemTemplate>

</asp:TemplateField>
```

Figure 14 shows the row in edit mode.

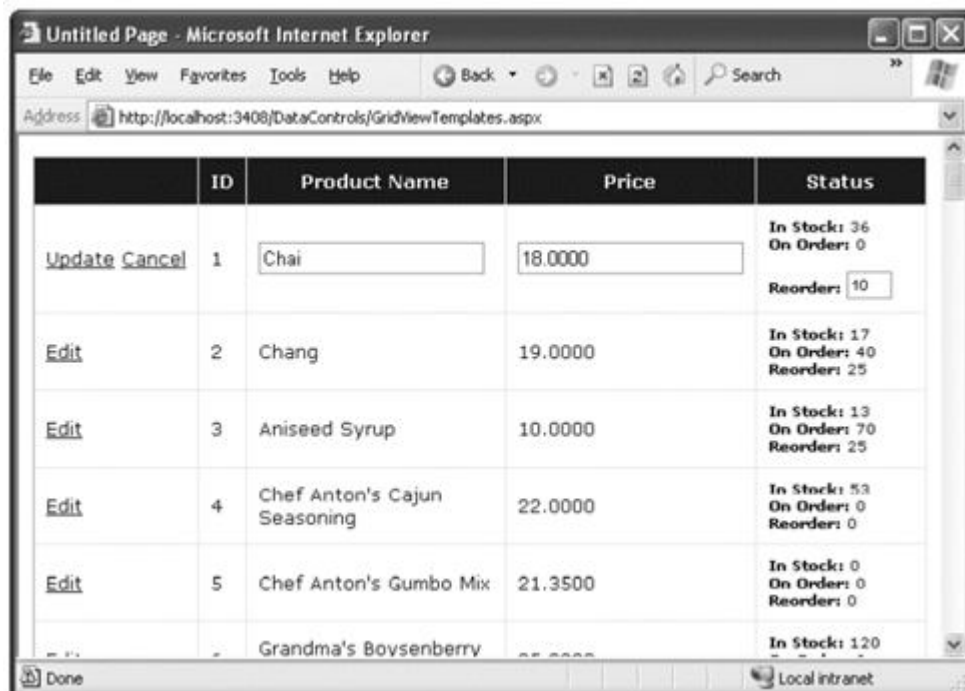


Figure14. Using an edit template

When binding an editable value to a control, you must use the Bind() method in your data binding expression instead of the ordinary Eval() method. Unlike the Eval() method, which can be placed anywhere in a page, the Bind() method must be used to set a control property. Only the Bind() method creates the two-way link, ensuring that updated values will be returned to the server. The modified SqlDataSource control with the correct command:

```
<asp:SqlDataSource ID="sourceProducts" runat="server"
```

```
ConnectionString="<%%$ ConnectionStrings:Northwind %>"
```

```
SelectCommand="SELECT ProductID, ProductName, UnitPrice, UnitsInStock,
```

```
UnitsOnOrder,ReorderLevel FROM Products"
```

```
UpdateCommand="UPDATE Products SET ProductName=@ProductName,  
UnitPrice=@UnitPrice,
```

```
ReorderLevel=@ReorderLevel WHERE ProductID=@ProductID">
```

```
</asp:SqlDataSource>
```

Editing with Validation

In the following example, a RangeValidator prevents changes that put the ReorderLevel at less than 0 or more than 100:

```
<asp:TemplateFieldHeaderText="Status">

<ItemStyle Width="100px" />

<ItemTemplate>

<b>In Stock:</b><%# Eval("UnitsInStock") %><br />

<b>On Order:</b><%# Eval("UnitsOnOrder") %><br />

<b>Reorder:</b><%# Eval("ReorderLevel") %>

</ItemTemplate>

<EditItemTemplate>

<b>In Stock:</b><%# Eval("UnitsInStock") %><br />

<b>On Order:</b><%# Eval("UnitsOnOrder") %><br /><br />

<b>Reorder:</b>

<asp:TextBox Text='<%# Bind("ReorderLevel") %>' Width="25px"

runat="server" id="txtReorder" />

<asp:RangeValidator id="rngValidator" MinimumValue="0" MaximumValue="100"

ControlToValidate="txtReorder" runat="server"

ErrorMessage="Value out of range." Type="Integer"/>

</EditItemTemplate>

</asp:TemplateField>
```

Figure 15. shows the validation at work.

	ID	Product Name	Price	Status
Update Cancel	1	Chai	18.0000	In Stock: 36 On Order: 0 Reorder: -5 Value out of range.
Edit	2	Chang	19.0000	In Stock: 17 On Order: 40 Reorder: 25
Edit	3	Aniseed Syrup	10.0000	In Stock: 13 On Order: 70 Reorder: 25
Edit	4	Chef Anton's Cajun Seasoning	22.0000	In Stock: 53 On Order: 0 Reorder: 0
Edit	5	Chef Anton's Gumbo Mix	21.3500	In Stock: 0 On Order: 0 Reorder: 0

Figure15.A GridView with a template column

Editing Without a Command Column

All you need to do is add a button control to the item template and set the `CommandName` to `Edit`. This automatically triggers the editing process, which fires the appropriate events and switches the row into edit mode.

```
<ItemTemplate>
```

```
<b>In Stock:</b><%# Eval("UnitsInStock") %><br />
```

```
<b>On Order:</b><%# Eval("UnitsOnOrder") %><br />
```

```
<b>Reorder:</b><%# Eval("ReorderLevel") %>
```

```
<br /><br />
```

```
<asp:LinkButton runat="server" Text="Edit"
```

```
CommandName="Edit" ID="LinkButton1" />
```

```
</ItemTemplate>
```

In the edit item template, you need two more buttons with `CommandName` values of `Update` and `Cancel`:

```

<EditItemTemplate>
<b>In Stock:</b><%# Eval("UnitsInStock") %><br />
<b>On Order:</b><%# Eval("UnitsOnOrder") %><br /><br />
<b>Reorder:</b>
<asp:TextBox Text='<%# Bind("ReorderLevel") %>' Width="25px"
runat="server" id="txtReorder" />
<br /><br />
<asp:LinkButton runat="server" Text="Update"
CommandName="Update" ID="LinkButton1" />
<asp:LinkButton runat="server" Text="Cancel"
CommandName="Cancel" ID="LinkButton2" CausesValidation="False" />
</EditItemTemplate>

```

The GridView editing events will fire and the data source controls will react in the same way as if you were using the automatically generated editing controls. Figure 16 shows the custom edit buttons.

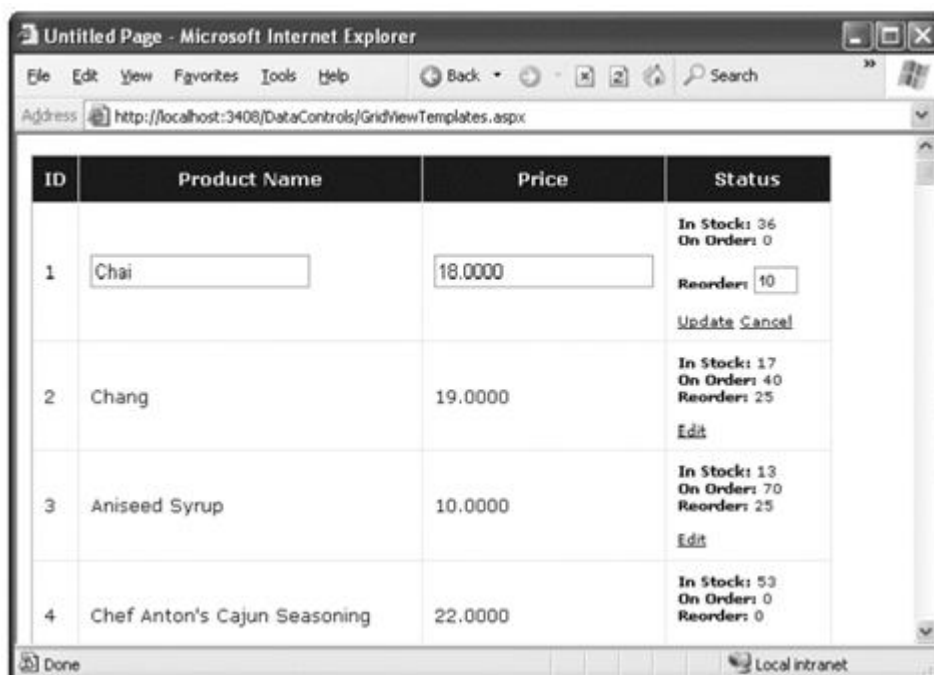


Figure 16. Custom edit controls

10.8 The Detailsview and Formview

The DetailsView renders its content inside a table, while the FormView gives you the flexibility to display your content without a table. But if you want to avoid the complexity of templates, the DetailsView gives you a simpler model that lets you build a multirow data display out of field objects, in much the same way that the GridView is built out of column objects. you can get up to speed with the DetailsView and the FormView quite quickly.

The DetailsView

The DetailsView also allows you to move from one record to the next using paging controls, if you've set the AllowPaging property to true. You can configure the paging controls using the PagerStyle and PagerSettings properties. Figure17 shows the DetailsView when it's bound to a set of product records, with full product information.

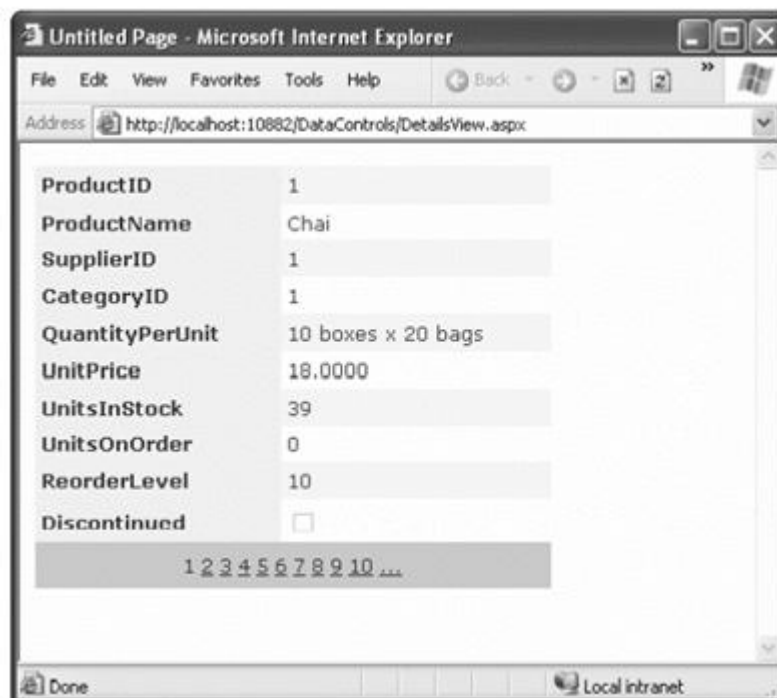


Figure17. The DetailsView with paging

Defining Fields

The DetailsView uses reflection to generate the fields it shows. The following code defines a DetailsView that shows product information. This tag creates the same grid of information shown in Figure, when AutoGenerateRows was set to true.

```
<asp:DetailsView ID="DetailsView1" runat="server" AutoGenerateRows="False"
DataSourceID="sourceProducts">

<Fields>

<asp:BoundFieldDataField="ProductID" HeaderText="ProductID"
ReadOnly="True" />

<asp:BoundFieldDataField="ProductName" HeaderText="ProductName" />
<asp:BoundFieldDataField="SupplierID" HeaderText="SupplierID" />
<asp:BoundFieldDataField="CategoryID" HeaderText="CategoryID" />
<asp:BoundFieldDataField="QuantityPerUnit" HeaderText="QuantityPerUnit" />
<asp:BoundFieldDataField="UnitPrice" HeaderText="UnitPrice" />
<asp:BoundFieldDataField="UnitsInStock" HeaderText="UnitsInStock" />
<asp:BoundFieldDataField="UnitsOnOrder" HeaderText="UnitsOnOrder" />
<asp:BoundFieldDataField="ReorderLevel" HeaderText="ReorderLevel" />
<asp:CheckBoxFieldDataField="Discontinued" HeaderText="Discontinued" />

</Fields>

...

</asp:DetailsView>
```

You can use the BoundField tag to set properties such as header text, formatting string,

editing behavior, and so on (refer to Table 16-2). In addition, you can use the ShowHeader property. When it's false, the header text is left out of the row, and the field data takes up both cells. The only difference is that instead of creating a dedicated column for editing controls, you simply set one of the Boolean properties of the DetailsView, such as AutoGenerateDeleteButton, AutoGenerateEditButton, and AutoGenerateInsertButton. The links for these tasks are added to the bottom of the DetailsView. When you add or edit a record, the DetailsView uses standard text box controls (see Figure 18). For more editing flexibility, you'll want to use templates with the DetailsView (by adding a TemplateField instead of a BoundField) or the FormView control (as described next).

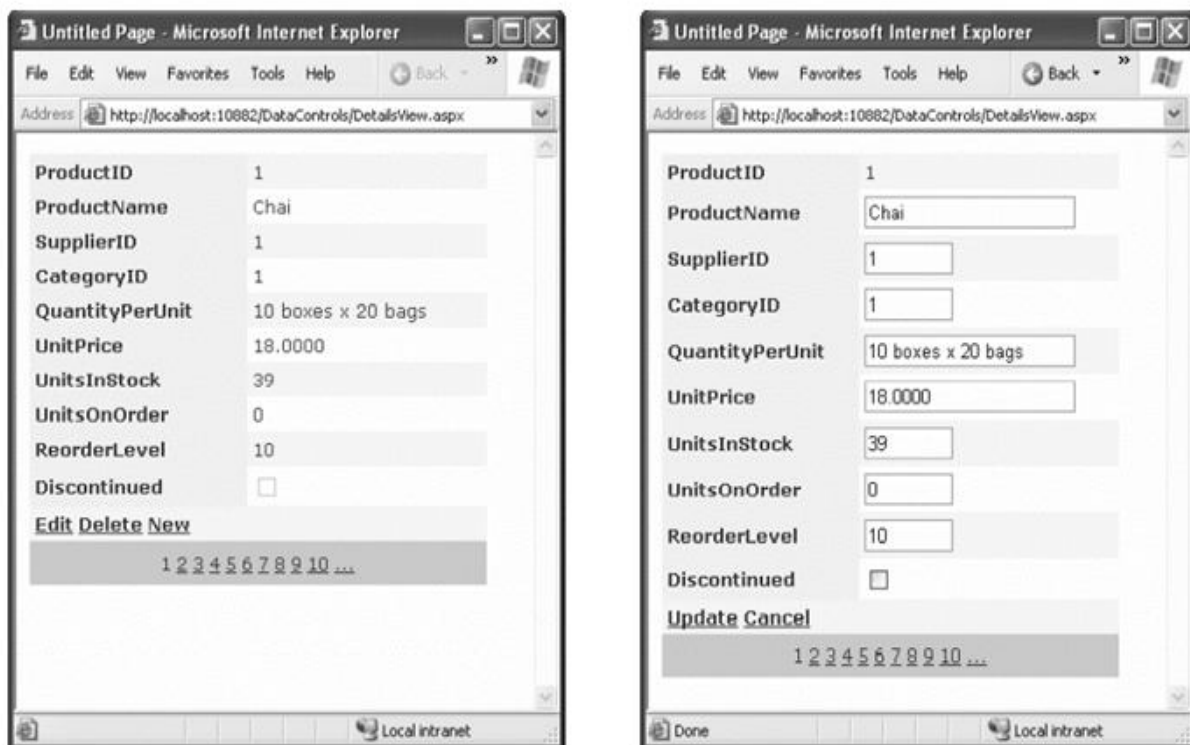


Figure 18. Editing in the DetailsView

The FormView

If you need the ultimate flexibility of templates, the FormView provides a template-only control for displaying and editing a single record. The beauty of the FormView template model is that it matches the model of the TemplateField in the GridView quite closely. This means you can work with the following templates:

- ItemTemplate
- EditItemTemplate
- InsertItemTemplate
- FooterTemplate
- HeaderTemplate
- EmptyDataTemplate
- PagerTemplate

Another option is to bind to a data source that returns just one record. Figure shows an example where a drop-down list control lets you choose a product, and a second data source shows the matching record in the FormView control.

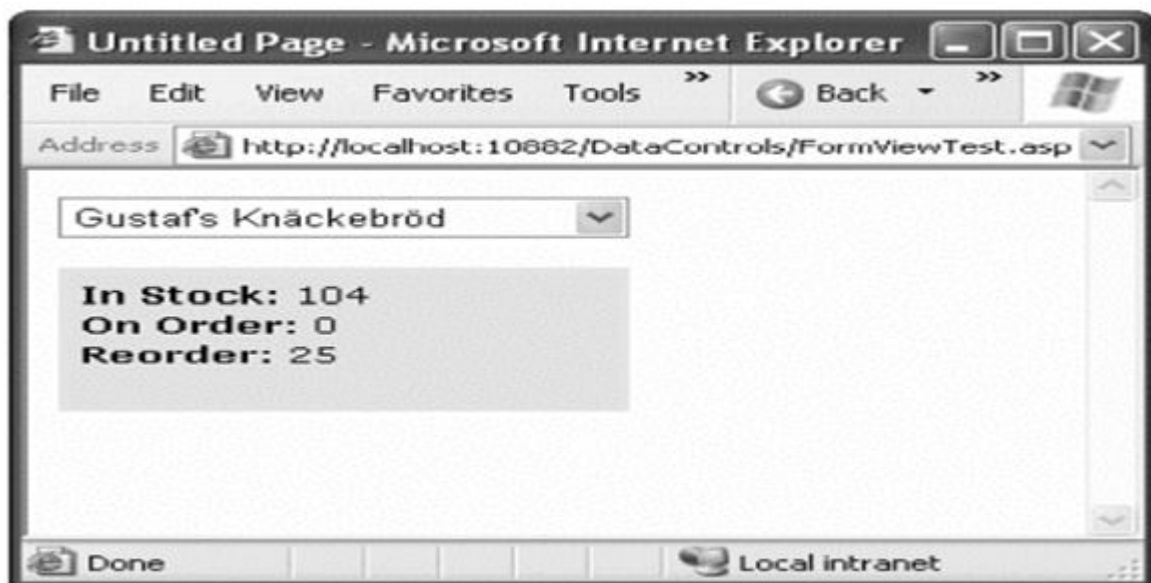


Figure 19. A FormView that shows a single record

Here's the markup you need to define the drop-down list and its data source:

```
<asp:SqlDataSource ID="sourceProducts" runat="server"
```

```
ConnectionString="<%%$ ConnectionStrings:Northwind %>"
```

```
SelectCommand="SELECT ProductID, ProductName FROM Products">
```

```
</asp:SqlDataSource>
```

```
<asp:DropDownList ID="lstProducts" runat="server"
```

```
AutoPostBack="True" DataSourceID="sourceProducts"
```

```
DataTextField="ProductName" DataValueField="ProductID" Width="184px">
```

```
</asp:DropDownList>
```

The FormView uses the template from the previous example. Here's the markup for the FormView (not including the template) and the data source that gets the full details for the selected product.

```
<asp:SqlDataSource ID="sourceProductFull" runat="server"
```

```
ConnectionString="<%%$ ConnectionStrings:Northwind %>"
```

```
SelectCommand="SELECT * FROM Products WHERE ProductID=@ProductID">
```

```
<SelectParameters>
```

```
<asp:ControlParameter Name="ProductID"
```

```
ControlID="lstProducts" PropertyName="SelectedValue" />
```

```
</SelectParameters>
```

```
</asp:SqlDataSource>
```

```
<asp:FormView ID="formProductDetails" runat="server"
```

```
DataSourceID="sourceProductFull"
```

```
BackColor="#FFE0C0" CellPadding="5">
```

```
<ItemTemplate>
```

```
...
```

```
</ItemTemplate>
```

```
</asp:FormView>
```

10.9 Summary

In this chapter, you considered everything you need to build rich data-bound pages. You took a detailed tour of the GridView and considered its support for formatting, selecting, sorting, paging, using templates, and editing. You also considered the DetailsView and the FormView, which allow you to display and edit individual records. Using these three controls, you can build all-in-one pages that display and edit data, without needing to write pages of ADO.NET code.

Keywords:

GridView, column, data, set, row, controls, figure, properties, editing, page

10. Check your understanding here

1. Which of the following denote New Data-bound Controls used with ASP.NET?

- a) GridView
- b) FormView
- c) SqlDataSource
- e) All the Above**

2. _____an item refers to the ability to click a row and have it change color (or become highlighted) to indicate that the user is currently working with this record.

- a. Selecting**
- b. Formatting
- c. Indexing
- d. None of the above

3. The GridView exposes a rich formatting model that's based on _____

- a. styles.**
- b. Forms

- c. Figure
- d. None of the above

4. Which control supports paging?

- a) Repeater
- b) Datagrid _**
- c) Both
- d) None

5. What tags one need to add within the asp:datagrid tags to bind columns manually?

- a) Set AutoGenerateColumns Property to false on the datagrid tag**
- b) Set AutoGenerateColumns Property to true on the datagrid tag
- c) It is not possible to do the operation
- d) Set AutomauanalColumns Property to true on the datagrid tag

10.11 Model questions

1. What are the advantages of explicitly defining columns?
2. Explain form view template model.
3. Write the steps for editing templates in visual studio.
4. Define event bubbling.
5. Explain paging.

Reference Book:

1. Mathew MacDonald, "Beginning ASP.NET 4 in C# 2010" Apress 2010.

LESSON - 11

XML

Structure

- 11.1 Introduction
- 11.2 Objectives
- 11.3 A Sample Code
- 11.4 XML Basics
- 11.5 XML Classes
- 11.6 XML Validation
- 11.7 XML Website Security
- 11.8 Summary
- 11.9 Check your Progress
- 11.10 Model Questions

11.1 Introduction

XML is designed as an all-purpose format for organizing data. In many cases, when you decide to use XML, you're deciding to store data in a standardized way, rather than creating your own new format conventions. The best way to understand the role XML plays is to consider the evolution of a simple file format without XML. XML is an all-purpose way to identify any type of data using elements. These elements use the same sort of format found in an HTML file, but while HTML elements indicate formatting, XML elements indicate content. (Because an XML file is just about data, there is no standardized way to display it in a browser, although Internet Explorer shows a collapsible view that lets you show and hide different portions of the document.)

11.2 Learning objectives

- ✓ Learn the XML classes, validation, display & transform
- ✓ Understand the website security

- ✓ Learn the security fundamentals
- ✓ Understanding the security, authentication & authorization
- ✓ Learn the forms authentication & windows authentication

11.3 A Sample code

The Super Pro Product List could use the following, clearer XML syntax:

```
<?xml version="1.0"?>

<SuperProProductList>

  <Product>

    <ID>1</ID>

    <Name>Chair</Name>

    <Price>49.33</Price>

    <Available>True</Available>

    <Status>3</Status>

  </Product>

  <Product>

    <ID>2</ID>

    <Name>Car</Name>

    <Price>43399.55</Price>

    <Available>True</Available>

    <Status>3</Status>

  </Product>

  <Product>
```

```

<ID>3</ID>

<Name>Fresh Fruit Basket</Name>

<Price>49.99</Price>

<Available>False</Available>

<Status>4</Status>

</Product>

</SuperProProductList>

```

This format is clearly understandable. Every product item is enclosed in a element, and every piece of information has its own element with an appropriate name. Elements are nested several layers deep to show relationships. Essentially, XML provides the basic element syntax, and you (the programmer) define the elements you want to use. That's why XML is often described as a meta language —it's a language you use to create your own language. In the SuperPro Product List example, this custom XML language defines elements such as,, and soon.

11.4 XML Basics

XML elements are composed of a start tag (like) and an end tag (like). Content is placed between the start and end tags. If you include a start tag, you must also include a corresponding end tag. The only other option is to combine the two by creating an empty element, which includes a forward slash at the end and has no content (like). This is similar to the syntax for ASP.NET controls.

- Whitespace between elements is ignored. That means you can freely use tabs and hard returns to properly align your information.
- You can use only valid characters in the content for an element. You can't enter special characters, such as the angle brackets (<>) and the ampersand (&), as content. Instead, you'll have to use the entity equivalents (such as < and > for angle brackets, and & for the ampersand). These equivalents will be automatically converted to the original characters when you read them into your program with the appropriate .NETclasses.
- XML elements are case sensitive, so and are completely different elements.

- All elements must be nested in a root element. In the SuperProProductList example, the root element is <SuperProProductList>. As soon as the root element is closed, the document is finished, and you cannot add anything else after it. In other words, if you omit the <SuperProProductList> element and start with a <Product> element, you'll be able to enter information for only one product; this is because as soon as you add the closing </Product>, the document is complete.
- Every element must be fully enclosed. In other words, when you open a subelement, you need to close it before you can close the parent. <Product><ID></ID></Product> is valid, but <Product><ID></Product></ID> isn't.
- XML documents must start with an XML declaration like. This signals that the document contains XML and indicates any special text encoding. However, many XML parsers work fine even if this detail is omitted.

Attributes

Attributes add extra information to an element. Instead of putting information into a subelement, you can use an attribute. In the XML community, deciding whether to use subelements or attributes—and what information should go into an attribute—is a matter of great debate, with no clear consensus. Here's the SuperProProductList example with ID and Name attributes instead of ID and Name subelements:

```
<?xml version="1.0"?>

<SuperProProductList>

  <Product ID="1" Name="Chair">

    <Price>49.33</Price>

    <Available>True</Available>

    <Status>3</Status>

  </Product>

  <Product ID="2" Name="Car">

    <Price>43399.55</Price>

    <Available>True</Available>
```

```

<Status>3</Status>

</Product>

<Product ID="3" Name="Fresh Fruit Basket">

<Price>49.99</Price>

<Available>False</Available>

<Status>4</Status>

</Product>

</SuperProProductList>

```

Comments

You can also add comments to an XML document. Comments go just about anywhere and are ignored for data processing purposes. Comments are bracketed by the character sequences. The following listing includes three valid comments:

```

<?xmlversion="1.0"?>

<SuperProProductList>

<!-- This is a test file. -->

<Product ID="1" Name="Chair">

<Price>49.33<!-- Why so expensive? --></Price>

<Available>True</Available>

<Status>3</Status>

</Product>

<!-- Other products omitted for clarity. -->

</SuperProProductList>

```

The only place you can't put a comment is embedded within a start or end tag (as in < my Data<!--A comment should not go here '!'</my Data>)

11.5 XMLCLASSES

.NET provides a rich set of classes for XML manipulation in several namespaces that start with System.Xml. One of the most confusing aspects of using XML with .NET is deciding which combination of classes you should use. Many of them provide similar functionality in a slightly different way, optimized for specific scenarios or for compatibility with specific standards.

The majority of the examples you'll explore use the types in the core System.Xml namespace. The classes here allow you to read and write XML files, manipulate XML data in memory, and even validate XML documents.

- Reading and writing XML directly, just like you read and write text files using XmlTextWriter and XmlTextReader. For sheer speed and efficiency, this is the best approach.
- Dealing with XML as a collection of in-memory objects using the XmlDocument class. If you need more flexibility than the XmlTextWriter and XmlTextReader provide or you just want a simpler, more straightforward model (and you don't need to squeeze out every last drop of performance), this is a good choice.
- Using the Xml control to transform XML content to displayable HTML. In the right situation—when all you want to do is display XML content using a prebuilt XSLT style sheet—this approach offers a useful shortcut.

XML Text Writer

One of the simplest ways to create or read any XML document is to use the basic XmlTextWriter and XmlTextReader classes. These classes work like their StreamWriter and StreamReader relatives, except that they write and read XML documents instead of ordinary text files. Then, you write to it or read from it, moving from top to bottom. Finally, you close it and get to work using the retrieved data in whatever way you'd like.

Before beginning this example, you'll need to import the namespaces for file handling and XML processing: using System.IO; using System.Xml;

Here's an example that creates a simple version of the SuperProProductList document:

```
// Place the file in the App_Data subfolder of the current website.
```

```
//The System.IO.Path class makes it easy to build the full filename. string file =
```

```

Path.Combine(Request.PhysicalApplicationPath, @"App_Data\SuperProProductList.xml");

FileStream fs = new FileStream(file, FileMode.Create); XmlTextWriter w = new XmlTextWriter(fs,
null); w.WriteStartDocument(); w.WriteStartElement("SuperProProductList");

w.WriteComment("This file generated by the XmlTextWriter class.");

//          Write the first product. w.WriteStartElement("Product");
w.WriteAttributeString("ID", "1"); w.WriteAttributeString("Name", "Chair");

w.WriteStartElement("Price"); w.WriteString("49.33"); w.WriteEndElement();
w.WriteEndElement();

//          Write the second product. w.WriteStartElement("Product");
w.WriteAttributeString("ID", "2"); w.WriteAttributeString("Name", "Car");

w.WriteStartElement("Price"); w.WriteString("43399.55"); w.WriteEndElement();
w.WriteEndElement();

//          Write the third product. w.WriteStartElement("Product");
w.WriteAttributeString("ID", "3"); w.WriteAttributeString("Name", "Fresh FruitBasket");

w.WriteStartElement("Price"); w.WriteString("49.99"); w.WriteEndElement();
w.WriteEndElement();

// Close the root element. w.WriteEndElement(); w.WriteEndDocument(); w.Close();

```

Dissecting the Code . . .

- You create the entire XML document by calling the methods of the `XmlTextWriter`, in the right order. To start a document, you always begin by calling `WriteStartDocument()`. To end it, you call `WriteEndDocument()`.
- The next step is writing the elements you need. You write elements in three steps. First, you write the start tag (like `<Product>`) by calling `WriteStartElement()`. Then you write attributes, elements, and text content inside. Finally, you write the end tag (like `</Product>`) by calling `WriteEndElement()`.
- The methods you use always work with the current element. So if you call `WriteStartElement()` and follow it up with a call to `WriteAttributeString()`, you are adding

an attribute to that element. Similarly, if you use `WriteString()`, you insert text content inside the current element, and if you use `WriteStartElement()` again, you write another element, nested inside the current element.

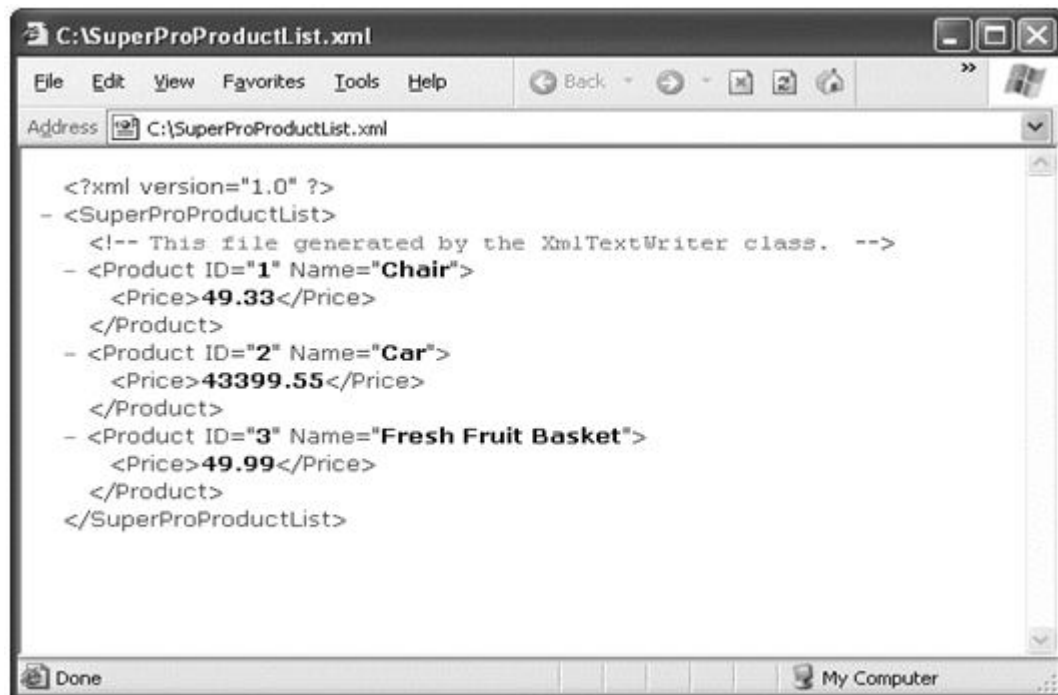


Figure1: SuperProProductList.xml

a. XML TextReader

Reading the XML document in your code is just as easy with the corresponding `XmlTextReader` class. The `XmlTextReader` moves through your document from top to bottom, one node at a time. You call the `Read ()` method to move to the next node. This method returns true if there are more nodes to read or false once it has read the final node. The current node is provided through the properties of the `XmlTextReader` class, such as `NodeType` and `Name`.

A node is a designation that includes comments, whitespace, opening tags, closing tags, content, and even the XML declaration at the top of your file. To get a quick understanding of nodes, you can use the `XmlTextReader` to run through your entire document from start to finish and display every node it encounters. The code for this task is as follows:

```

string file = Path.Combine(Request.PhysicalApplicationPath, @"Data\SuperProProductList.xml");
FileStream fs = new FileStream(file, FileMode.Open); xmlTextReader r = new XmlTextReader(fs);

```

480

```
{  
  
writer.Write("<b>Type:</b> ");  
  
writer.Write(r.NodeType.ToString());  
  
writer.Write("<br>");  
  
// The name is available when reading the opening and closing tags  
  
// for an element. It's not available when reading the inner content. if (r.Name != "")  
  
{  
  
writer.Write("<b>Name:</b> "); writer.Write(r.Name); writer.Write("<br>");  
  
}  
  
// The value is when reading the inner content. if (r.Value != "")  
  
{  
  
writer.Write("<b>Value:</b> "); writer.Write(r.Value); writer.Write("<br>");  
  
}  
  
if (r.AttributeCount > 0)  
  
{  
  
writer.Write("<b>Attributes:</b> "); for (int i = 0; i < r.AttributeCount; i++)  
  
{  
  
writer.Write(" "); writer.Write(r.GetAttribute(i)); writer.Write(" ");  
  
}  
  
writer.Write("<br>");  
  
}  
  
writer.Write("<br>");  
  
}
```

```
fs.Close();
```

```
// Copy the string content into a label to display it. lblXml.Text = writer.ToString();
```

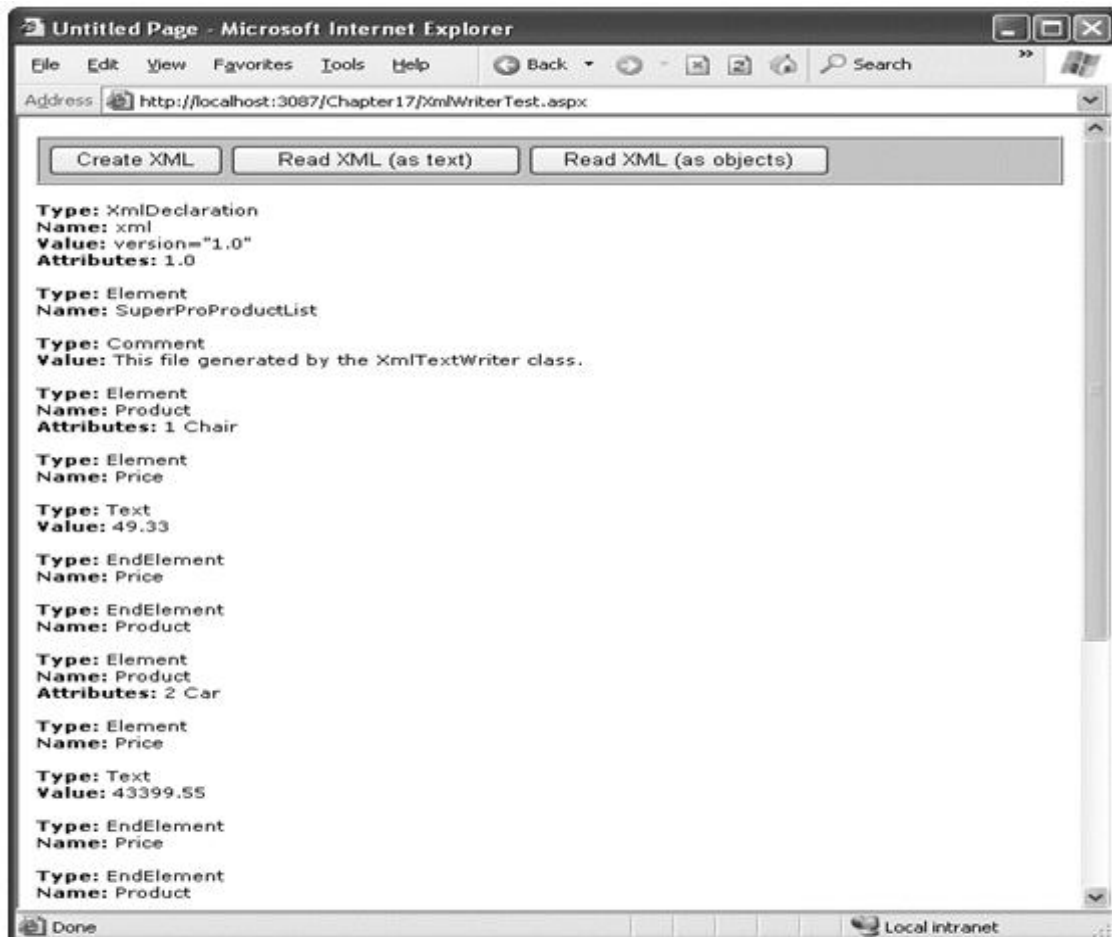


Figure 2 - ReadingXML structure

A typical application might read data from an XML file and place it directly into the corresponding objects. The next example (also a part of the XmlWriterTest.aspx page) shows how you can easily create a group of Product objects based on the SuperProProductList.xml file. This example uses the generic List collection, so you'll need to import the System.Collections.Generic namespace.

```
// Open a stream to the file.
```

```
String file = Path.Combine(Request.PhysicalApplicationPath,
@"App_Data\SuperProProductList.xml");
FileStream fs = new FileStream(file, FileMode.Open);
```

```

XmlTextReader r = new XmlTextReader(fs);

// Create a generic collection of products. List<Product> products = new List<Product>();

// Loop through the products. while (r.Read())
{
    if (r.NodeType == XmlNodeType.Element && r.Name == "Product")
    {
        Product newProduct = new Product ();

        newProduct.ID = Int32.Parse(r.GetAttribute("ID")); newProduct.Name = r.GetAttribute("Name");

        // Get the rest of the subtags for this product. while (r.NodeType != XmlNodeType.EndElement)
        {
            r.Read();

            // Look for Price subtags. if (r.Name == "Price")
            {
                while (r.NodeType != XmlNodeType.EndElement)
                {
                    r.Read();

                    if (r.NodeType == XmlNodeType.Text)
                    {
                        newProduct.Price = Decimal.Parse(r.Value);
                    }
                }
            }
        }

        // You could check for other Product nodes

        // (such as Available, Status, etc.) here.
    }
}

```

```
}  
  
// Add the product to the list. products.Add(newProduct);  
  
}  
  
}  
  
fs.Close();  
  
// Display the retrieved document. gridResults.DataSource=products; gridResults.DataBind();
```

Dissecting the Code . . .

- This code uses a nested looping structure. The outside loop iterates over all the products, and the inner loop searches through all the child elements of <Product>. (In this example, the code processes the <Price> element, and ignores everything else.) The looping structure keeps the code well organized.
- The EndElement node alerts you when a node is complete and the loop can end. Once all the information is read for a product, the corresponding object is added into the collection.
- All the information is retrieved from the XML file as a string. Thus, you need to use methods like Int32.Parse() to convert it to the right data type.
- Data binding is used to display the contents of the collection. A GridView set to generate columns automatically creates the table shown in Figure 3

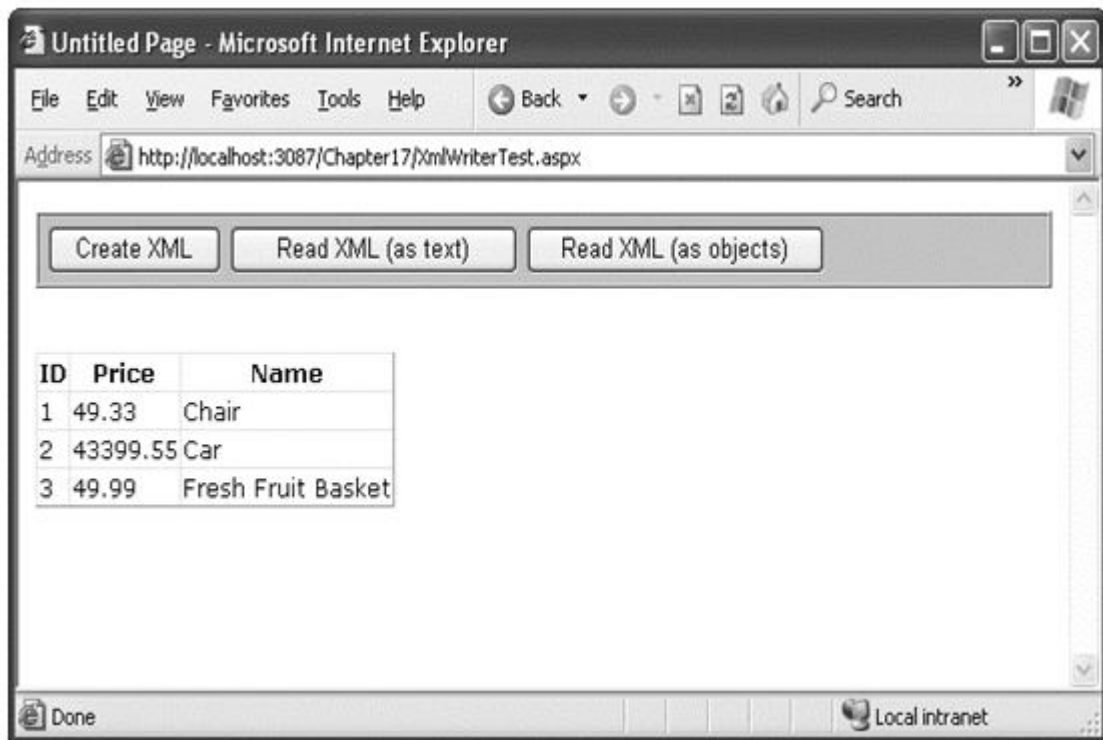


Figure3: Reading XML content

Reading an XML Document

The XDocument makes it easy to read and navigate XML content. You can use the static XDocument.Load() method to read XML documents from a file, URI, or stream, and you can use the static XDocument.Parse() method to load XML content from a string.

Once you have the XDocument with your content in memory, you can dig into the tree of nodes using a few key properties and methods of the XElement and XDocument class. Table 1 lists the most useful methods.

Table 1 - Useful Methods for XElement and XDocument

Method	Description
Attributes ()	Gets the collection of XAttribute objects for this element.
Attribute ()	Gets the XAttribute with the specific name.

Elements ()	Gets the collection of XElement objects that are contained by this element. (This is the top level only—these elements may in turn contain more elements.) Optionally, you can specify an element name, and only those elements will be retrieved.
Element ()	Gets the single XElement contained by this element that has a specific name (or null if there's no match). If there is more than one matching element, this method gets just the first one.
Descendants ()	Gets the collection of XElement objects that are contained by this element and (optionally) have the name you specify. Unlike the Elements () method, this method goes through all the layers of the document and finds elements at any level of the hierarchy.
Nodes ()	Gets all the XmlNode objects contained by this element. This includes elements and other content, such as comments. However, unlike the XmlTextReader class, the XmlDocument does not consider attributes to be nodes.
Descendant Nodes ()	Gets all the XmlNode object contained by this element. This method is like Descendants () in that it drills down through all the layers of nested elements.

11.6 XML Validation

XML has a rich set of supporting standards, many of which are far beyond the scope of this book. One of the most useful in this family of standards is XML Schema. XML Schema defines the rules to which a specific XML document should conform, such as the allowable elements and attributes, the order of elements, and the data type of each element. You define these requirements in an XML Schema document (XSD).

When you're creating an XML file on your own, you don't need to create a corresponding XSD file—instead, you might just rely on the ability of your code to behave properly. Although this is sufficient for tightly controlled environments, if you want to open your application to other programmers or allow it to interoperate with other applications, you should create an XSD.

Think of it this way: XML allows you to create a custom language for storing data, and XSD allows you to define the syntax of the language you create.

a) XMLNamespaces

Before you can create an XSD, you'll need to understand one other XML standard, called XML namespaces.

The core idea behind XML namespaces is that every XML markup language has its own namespace, which is used to uniquely identify all related elements. Technically, namespaces disambiguate elements by making it clear what markup language they belong to. For example, you could tell the difference between your SuperProProductList standard and another organization's product catalog because the two XML languages would use different namespaces.

Namespaces are particularly useful in compound documents, which contain separate sections, each with a different type of XML. In this scenario, namespaces ensure that an element in one namespace can't be confused with an element in another namespace, even if it has the same element name. Namespaces are also useful for applications that support different types of XML documents. By examining the namespace, your code can determine what type of XML document it's working with and can then process it accordingly.

Before you can place your XML elements in a namespace, you need to choose an identifying name for that namespace. Most XML namespaces use Universal Resource Identifiers (URIs). Typically, these URIs look like a web page URL. For example, `http://www.mycompany.com/mystandard` is a typical name for a namespace. Though the namespace looks like it points to a valid location on the Web, this isn't required (and shouldn't be assumed).

The reason that URIs are used for XML namespaces is because they are more likely to be unique. Typically, if you create a new XML markup, you'll use a URI that points to a domain or website you control. That way, you can be sure that no one else is likely to use that URI. For example,

To specify that an element belongs to a specific namespace, you simply need to add the `xmlns` attribute to the start tag and indicate the namespace. For example, the `<Price>` element shown here is part of the `http://www.SuperProProducts.com/SuperProProductList` namespace:

```
<Price xmlns="http://www.SuperProProducts.com/SuperProProductList"> 49.33
```



```
</Price>
```

If you don't take this step, the element will not be part of any namespace.

It would be cumbersome if you needed to type the full namespace URI every time you wrote an element in an XML document. Fortunately, when you assign a namespace in this fashion, it becomes the default namespace for all child elements. For example

```
<?xml version="1.0"?>
```

```
<SuperProProductList xmlns="http://www.SuperProProducts.com/SuperProProductList">
```

```
<Product>
```

```
<ID>1</ID>
```

```
<Name>Chair</Name>
```

```
<Price>49.33</Price>
```

```
<Available>True</Available>
```

```
<Status>3</Status>
```

```
</Product>
```

```
<!-- Other products omitted. -->
```

```
</SuperProProductList>
```

b) XML Schema Definition

An XSD, or schema, defines what elements and attributes a document should contain and the way these nodes are organized (the structure). It can also identify the appropriate data types for all the content.

Schema documents are written using an XML syntax with specific element names. All the XSD elements are placed in the namespace. Often, this namespace uses the prefix `xsd:` or `xs:`, as in the following example.

The full XSD specification is out of the scope of this chapter, but you can learn a lot from a simple example. The following is a slightly abbreviated SuperProProductList.xsd file that defines the rules for SuperProProductList documents:

```
<?xml version="1.0"?>

<xs:schema

targetNamespace="http://www.SuperProProducts.com/SuperProProductList" xmlns:xs="http://
/www.w3.org/2001/XMLSchema" elementFormDefault="qualified" >

<xs:element name="SuperProProductList">

<xs:complexType>

<xs:sequence maxOccurs="unbounded">

<xs:element name="Product">

<xs:complexType>

<xs:sequence>

<xs:element name="Price" type="xs:double" />

</xs:sequence>

<xs:attribute name="ID" use="required" type="xs:int" />

<xs:attribute name="Name" use="required" type="xs:string" />

</xs:complexType>

</xs:element>

</xs:sequence>

</xs:complexType>

</xs:element>

</xs:schema>
```

At first glance, this markup looks a bit intimidating. However, it's actually not as complicated as it looks. Essentially, this schema indicates that a SuperProProductList document consists of a list of

<Product> elements. Each <Product> element is a complex type made up of a string (Name), a decimal value (Price), and an integer (ID). This example uses the second version of the SuperProProductList document to demonstrate how to use attributes in a schema file.

Dissecting the Code

By examining the SuperProProductList.xsd schema, you can learn a few important points:

- Schema documents use their own form of XML markup. In the previous example, you'll quickly see that all the elements are placed in the `http://www.w3.org/2001/XMLSchemaNamespace` using the `xs` namespace prefix.
- Every schema document starts with a root <schema> element.
- The schema document must specify the namespace of the documents it can validate. It specifies this detail with the target Namespace attribute on the root <schema> element.
- The elements inside the <schema> element describe the structure of the target document. The <element> element represents an element, while the <attribute> element represents an attribute. To find out what the name of an element or attribute is, look at the name attribute. For example, you can tell quite easily that the first <element> has the name SuperProProductList. This indicates that the first element in the validated document must be <SuperProProductList>.
- If an element can contain other elements or has attributes, it's considered a complex type. Complex types are represented in a schema by the <complexType> element. The simplest complex type is a sequence, which is represented in a schema by the <sequence> element. It requires that elements are always in the same order—the order that's set in the schema document.
- When defining elements, you can define the maximum number of times an element can appear (using the maxOccurs attribute) and the minimum number of times it must occur (using the minOccurs attribute). If you leave out these details, the default value of

both is 1, which means that every element must appear exactly once in the target document. Use a maxOccurs value of unbounded if you want to allow an unlimited list. For example, this allows there to be an unlimited number of <Product> elements in the SuperProProductList catalog. However, the

- <Price> element must occur exactly once in each <Product>.
- When defining an attribute, you can use the use attribute with a value of required to make that attribute mandatory.
- When defining elements and attributes, you can specify the data type using the type attribute. The XSD standard defines 44 data types that map closely to the basic data types in .NET, including the double, int, and string data types used in this example.

Validating an XML Document

The following example shows you how to validate an XML document against a schema, using an XmlReader that has validation features built in.

The first step when performing validation is to import the System.Xml.Schema namespace, which contains types such as XmlSchema and XmlSchemaCollection:

```
using System.Xml.Schema;
```

You must perform two steps to create the validating reader. First, you create an XmlReaderSettings object that specifically indicates you want to perform validation. You do this by setting the ValidationType property and loading your XSD schema file into the Schemas collection, as shown here:

```
// Configure the reader to use validation. XmlReaderSettings settings = new
XmlReaderSettings(); settings.ValidationType = ValidationType.Schema;
```

```
// Create the path for the schema file.
```

```
string    schemaFile    =    Path.Combine(Request.PhysicalApplicationPath,
@"App_Data\SuperProProductList.xsd");
```

```
// Indicate that elements in the namespace
```

```
// http://www.SuperProProducts.com/SuperProProductList should be
```

```
// validated using the schema file. settings.Schemas.Add("http://www.SuperProProducts.com/
SuperProProductList",schemaFile);
```

Second, you need to create the validating reader using the static `XmlReader.Create()` method. This method has several overloads, but the version used here requires a `FileStream` (with the XML document) and the `XmlReaderSettings` object that has your validation settings:

```
// Open the XML file.
```

```
FileStream fs = new FileStream(file, FileMode.Open);
```

```
// Create the validating reader.
```

```
XmlReader r = XmlReader.Create(fs, settings);
```

The `XmlReader` in this example works in the same way as the `XmlTextReader` you've been using up until now, but it adds the ability to verify that the XML document follows the schema rules. This reader throws an exception (or raises an event) to indicate errors as you move through the XML file.

The following example shows how you can create a validating reader that uses the `SuperProProductList.xsd` file to verify that the XML in `SuperProProductList.xml` is valid:

```
// Set the validation settings.
```

```
XmlReaderSettings settings = new XmlReaderSettings(); settings.Schemas.Add("http://
www.SuperProProducts.com/SuperProProductList", schemaFile);
```

```
settings.ValidationType = ValidationType.Schema;
```

```
// Open the XML file.
```

```
FileStream fs = new FileStream(file, FileMode.Open);
```

```
// Create the validating reader.
```

```
XmlReader r = XmlReader.Create(fs, settings);
```

```
// Read through the document. while (r.Read())
```

```
{
```

```
// Process document here.

// If an error is found, an exception will be thrown.

}

fs.Close();
```

Using the current file, this code will succeed, and you'll be able to access each node in the document. However, consider what happens if you make the minor modification shown here:

```
<Product ID="A" Name="Chair">
```

Now when you try to validate the document, an `XmlSchemaException` (from the `System.Xml.Schema` namespace) will be thrown, alerting you to the invalid data type, as shown in Figure 4 .

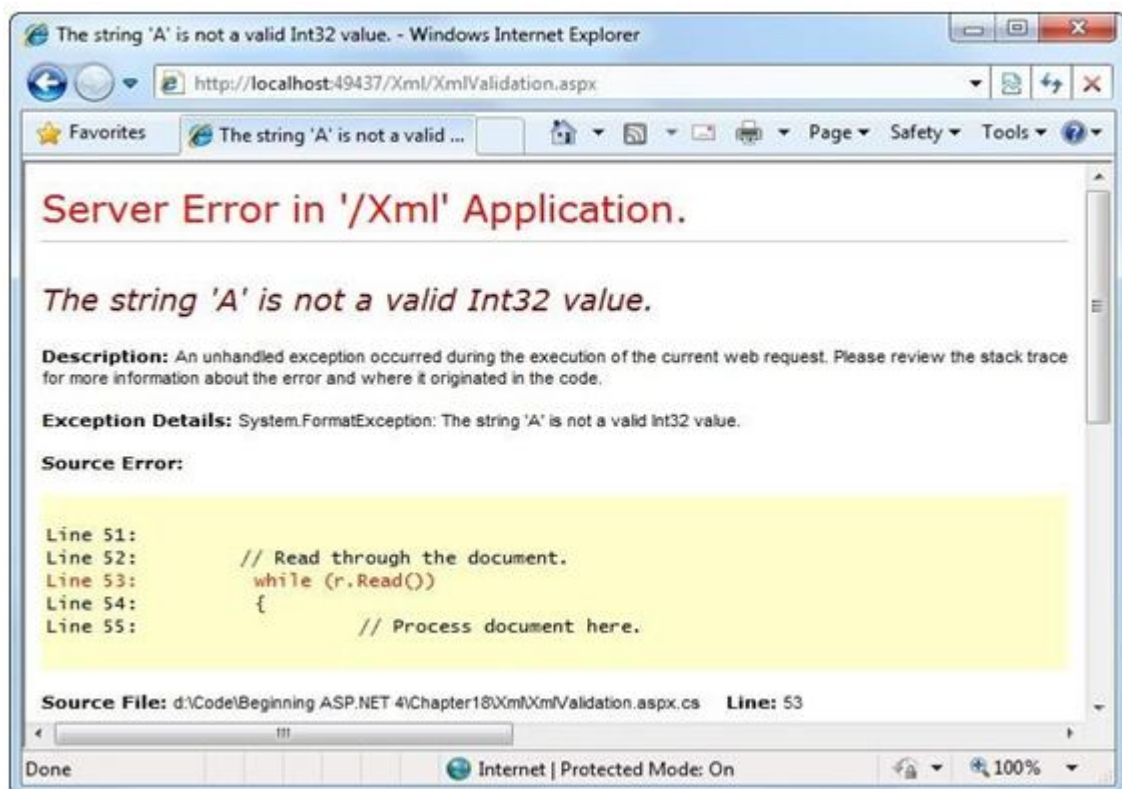


Figure 4: Xml Schema Exception

c) XML Display and Transforms

Another standard associated with XML is XSL Transformations (XSLT). XSLT allows you to create style sheets that can extract a portion of a large XML document or transform an XML document into another type of XML document. An even more popular use of XSLT is to convert an XML document into an HTML document that can be displayed in a browser.

XSLT is easy to use from the point of view of the .NET class library. All you need to understand is how to create an `XslCompiledTransform` object (found in the `System.Xml.Xsl` namespace). You use its `Load()` method to specify a style sheet and its `Transform()` method to output the result to a file or stream:

```
// Define the file paths this code uses. The XSLT file and the
// XML source file already exist, but the XML result file
// will be created by this code.

string    xsltFile      =    Path.Combine(Request.PhysicalApplicationPath,
@"App_Data\SuperProProductList.xsl");

string    xmlSourceFile =    Path.Combine(Request.PhysicalApplicationPath,
@"App_Data\SuperProProductList.xml");    string    xmlResultFile    =
Path.Combine(Request.PhysicalApplicationPath, @"App_Data\TransformedFile.xml");

// Load the XSLT style sheet.

XslCompiledTransform transformer = new XslCompiledTransform(); transformer.Load(xsltFile);

// Create a transformed XML file.

// SuperProProductList.xml is the starting point. transformer.Transform(xmlSourceFile,
xmlResultFile);
```

However, this doesn't spare you from needing to learn the XSLT syntax. Once again, the

intricacies of XSLT aren't directly related to core ASP.NET programming, so they're outside the scope of this book. To get started with XSLT, however, it helps to review a simple style sheet example. The following example shows an XSLT style sheet that transforms the no-namespace version of the `SuperProProductList` document into a formatted HTML table:

```

<?xml version="1.0" encoding="UTF-8" ?>

<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0" >

<xsl:template match="SuperProProductList">

<html>

<body>

<table border="1">

<xsl:apply-templates select="Product"/>

</table>

</body>

</html>

</xsl:template>

<xsl:template match="Product">

<tr>

<td><xsl:value-of select="@ID"/></td>

<td><xsl:value-of select="@Name"/></td>

<td><xsl:value-of select="Price"/></td>

</tr>

</xsl:template>

</xsl:stylesheet>

```

Every XSLT document has a root `xsl:stylesheet` element. The style sheet can contain one or more templates (the sample file `SuperProProductList.xslt` has two). In this example, the first template searches for the root `SuperProProductList` element. When it finds it, it outputs the

tags necessary to start an HTML table and then uses the `xsl:apply-templates` command to branch off and perform processing for any contained Product elements.

```
<xsl:template match="SuperProProductList">
```

```
<html>
```

```
<body>
```

```
<table border="1">
```

```
<xsl:apply-templates select="Product"/>
```

When that process is complete, the HTML tags for the end of the table will be written:

```
</table>
```

```
</body>
```

```
</html>
```

```
</xsl:template>
```

When processing each `<Product>` element, the value from the nested ID attribute, Name attribute, and `<Price>` element is extracted and written to the output using the `xsl:value-of` command. The at sign (@) indicates that the value is being extracted from an attribute, not a sub element. Every piece of information is written inside a table row.

```
<xsl:template match="Product">
```

```
<tr>
```

```
<td><xsl:value-of select="@ID"/></td>
```

```
<td><xsl:value-of select="@Name"/></td>
```

```
<td><xsl:value-of select="Price"/></td>
```

```
</tr>
```

```
</xsl:template>
```

For more advanced formatting, you could use additional HTML elements to format some text in bold or italics.

The final result of this process is the HTML file shown here:

```
<html>

<body>

<table border="1">

<tr>

<td>1</td>

<td>Chair</td>

<td>49.33</td>

</tr>

<tr>

<td>2</td>

<td>Car</td>

<td>43398.55</td>

</tr>

<tr>

<td>3</td>

<td>Fresh Fruit Basket</td>

<td>49.99</td>

</tr>

</table>

</body>

</html>
```

11.7 XML Website Security

ASP.NET provides an extensive security model that makes it easy to protect your web applications. Although this security model is powerful and profoundly flexible, it can appear confusing because of the many different layers that it includes. Much of the work in securing your application isn't writing code, but determining the appropriate places to implement your security strategy.

Understanding Security

The first step in securing your applications is deciding where you need security and what it needs to protect. For example, you may need to block access in order to protect private information. Or, maybe you just need to enforce a pay-for-content system. Perhaps you don't need any sort of security at all, but you want an optional login feature to provide personalization for frequent visitors. These requirements will determine the approach you use.

Security doesn't need to be complex, but it does need to be wide-ranging and multilayered. For example, consider an e-commerce website that allows users to view reports of their recently placed orders. You probably already know the first line of defense that this website should use—a login page that forces users to identify themselves before they can see any personal information. In this chapter, you'll learn how to use this sort of authentication system. However, it's important to realize that, on its own, this layer of protection is not enough to truly secure your system. You also need to protect the back-end database with a strong password, and you might even choose to encrypt sensitive information before you store it (scrambling so that it's unreadable without the right key to decrypt it). Steps like these protect your website from other attacks that get beyond your authentication system. For example, they can deter a disgruntled employee with an account on the local network, a hacker who has gained access to your network through the company firewall, or a careless technician who discards a hard drive used for data storage without erasing it first.

Authentication and Authorization

Two concepts form the basis of any discussion about security:

Authentication: This is the process of determining a user's identity and forcing users to prove they are who they claim to be. Usually, this involves entering credentials (typically a user name and password) into some sort of login page or window. These credentials are then authenticated against the Windows user accounts on a computer, a list of users in a file, or a back-end database.

Authorization: Once a user is authenticated, authorization is the process of determining whether that user has sufficient permissions to perform a given action (such as viewing a page or retrieving information from a database). Windows imposes some authorization checks (for example, when you open a file), but your code will probably want to impose its own checks (for example, when a user performs a task in your web application such as submitting an order, assigning a project, or giving a promotion).

Authentication and authorization are the two cornerstones of creating a secure user-based site. The Windows operating system provides a good analogy. When you first boot up your computer, you supply a user ID and password, thereby authenticating yourself to the system. After that point, every time you interact with a restricted resource (such as a file, database, registry key, and so on), Windows quietly performs authorization checks to ensure your user account has the necessary rights. You can use two types of authentication to secure an ASP.NET website:

Forms authentication: With forms authentication, ASP.NET is in charge of authenticating users, tracking them, and authorizing every request. Usually, forms authentication works in conjunction with a database where you store user information (such as user names and passwords), but you have complete flexibility. You could even store user information in a plain text file or write customized login code that calls a remote service. Forms authentication is the best and most flexible way to run a subscription site or e-commerce store.

Windows authentication: With Windows authentication, the web server forces every user to log in as a Windows user. (Depending on the specific configuration you use, this login process may take place automatically, as it does in the Visual Studio test web server, or it may require that the user type a name and password into a Login dialog box.) This system requires that all users have Windows user accounts on the server (although users could share accounts). This scenario is poorly suited for a public web application but is often ideal with an intranet or company-specific site designed to provide resources for a limited set of users.

Forms Authentication

ASP.NET uses the same approach in its forms authentication model. You are still responsible for creating the login page (although you can use a set of specially designed controls).

However, you don't need to create the security cookie manually, or check for it in secure pages, because ASP.NET handles these tasks automatically. You also benefit from ASP.NET's support

for sophisticated validation algorithms, which make it all but impossible for users to spoof their own cookies or try other hacking tricks to fool your application into giving them access. Figure 5 shows a simplified security diagram of the forms authentication model in ASP.NET.

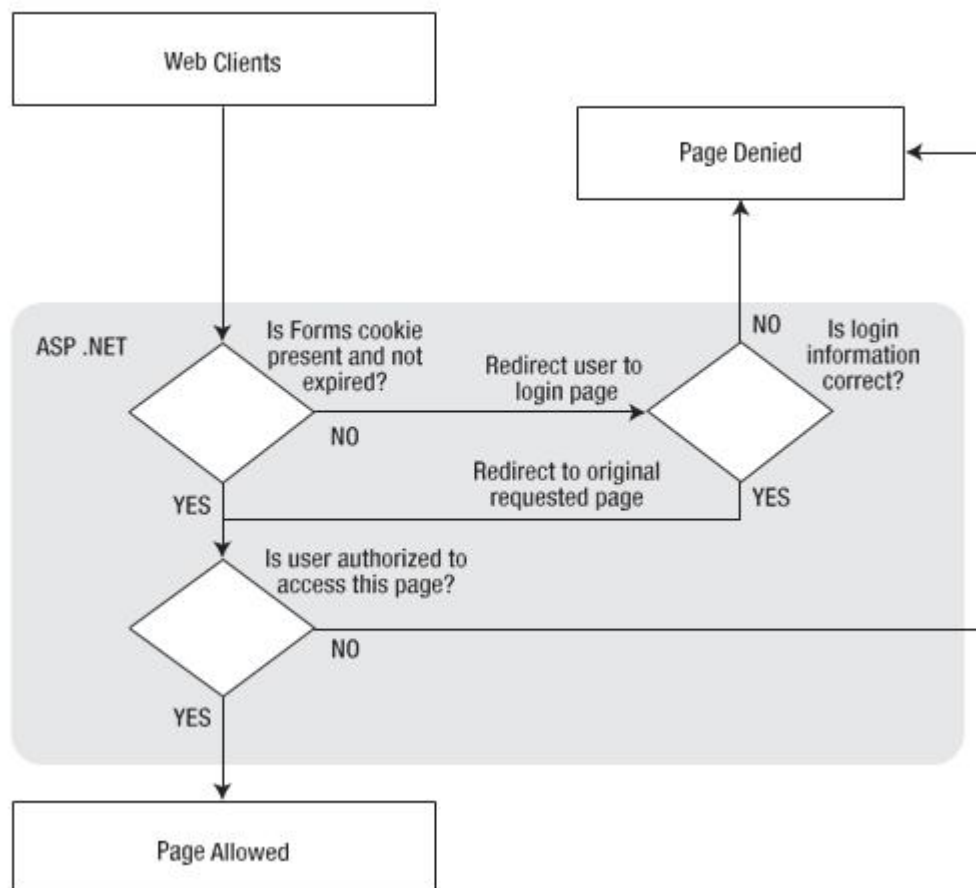


Figure 5 ASP.NET forms authentication

To implement forms-based security, you need to follow three steps:

1. Set the authentication mode to forms authentication in the web. Config file. (If you prefer a graphical tool, you can use the WAT during development or IIS Manager after deployment.)
2. Restrict anonymous users from a specific page or directory in your application.
3. Create the login page.

Table 1: Forms Authentication Settings

Attribute	Description
name	The name of the HTTP cookie to use for authentication (defaults to .ASPXAUTH). If multiple applications are running on the same web server, you should give each application's security cookie a unique name.
loginUrl	Your custom login page, where the user is redirected if no valid authentication cookie is found. The default value is Login.aspx.
protection	The type of encryption and validation used for the security cookie (can be All, None, Encryption, or Validation). Validation ensures the cookie isn't changed during transit, and encryption (typically Triple-DES) is used to encode its contents. The default value is All.
timeout	The number of idle minutes before the cookie expires. ASP.NET will refresh the cookie every time it receives a request. The default value is 30.
path	The path for cookies issued by the application. The default value (/) is recommended, because case mismatches can prevent the cookie from being sent with a request.

If you make these changes to an application's web.config file and request a page, you'll notice that nothing unusual happens, and the web page is served in the normal way. This is because even though you have enabled forms authentication for your application, you have not restricted anonymous users. In other words, you've chosen the system you want to use for authentication, but at the moment none of your pages needs authentication.

To control who can and can't access your website, you need to add access control rules to the <authorization> section of your web.config file. Here's an example that duplicates the default behavior:

```
<configuration>
```

```
<system.web>
```

```

<authentication mode="Forms">

<forms loginUrl="~/Login.aspx" />

</authentication>

>

<authorization>

<allow users="*"

/>

</authorization>

...

</system.web>

</configuration>

```

The asterisk (*) is a wildcard character that explicitly permits all users to use the application, even those who haven't been authenticated. But even if you don't include this line in your application's web.config file, this is still the behavior you'll see, because ASP.NET's default settings allow all users. (Technically, this behavior happens because there's an <allow users="*"> rule in the root web.config file. If you're curious, you can find this file a directory like c:\Windows\Microsoft.NET\Framework\ [Version]\Config, where [Version] is the version of ASP.NET that's installed, like v4.0.30319.)

To change this behavior, you need to explicitly add a more restrictive rule, as shown here:

```

<authorization>

<deny users="?" />

</authorization>

```

The question mark (?) is a wildcard character that matches all anonymous users. By including this rule in your web.config file, you specify that anonymous users are not allowed. Every user must be authenticated, and every user request will require the security cookie. If you request a

page in the application directory now, ASP.NET will detect that the request isn't authenticated. It will then redirect the request to the login page that's specified by the `loginUrl` attribute in the `web.config` file. (If you try this step right now, the redirection process will cause an error, unless you've already created the login page.)

Now consider what happens if you add more than one rule to the authorization section:

```
<authorization>
<allow users="*" />
<deny users="?" />
</authorization>
```

When evaluating rules, ASP.NET scans through the list from top to bottom and then continues with the settings in any `.config` file inherited from a parent directory, ending with the settings in the base `machine.config` file. As soon as it finds an applicable rule, it stops its search. Thus, in the previous case, it will determine that the rule `<allow users="*" />` applies to the current request and will not evaluate the second line. This means these rules will allow all users, including anonymous users. But consider what happens if these two lines are reversed:

```
<authorization>
<deny users="?" />
<allow users="*" />
</authorization>
```

The WAT

To use the WAT for this type of configuration, select **Website & ASP.NET Configuration** from the menu. Next, click the **Security** tab. You'll see the window shown in Figure which gives you links to set the authentication type, define authorization rules (using the **Access Rules** section), and enable role-based security.

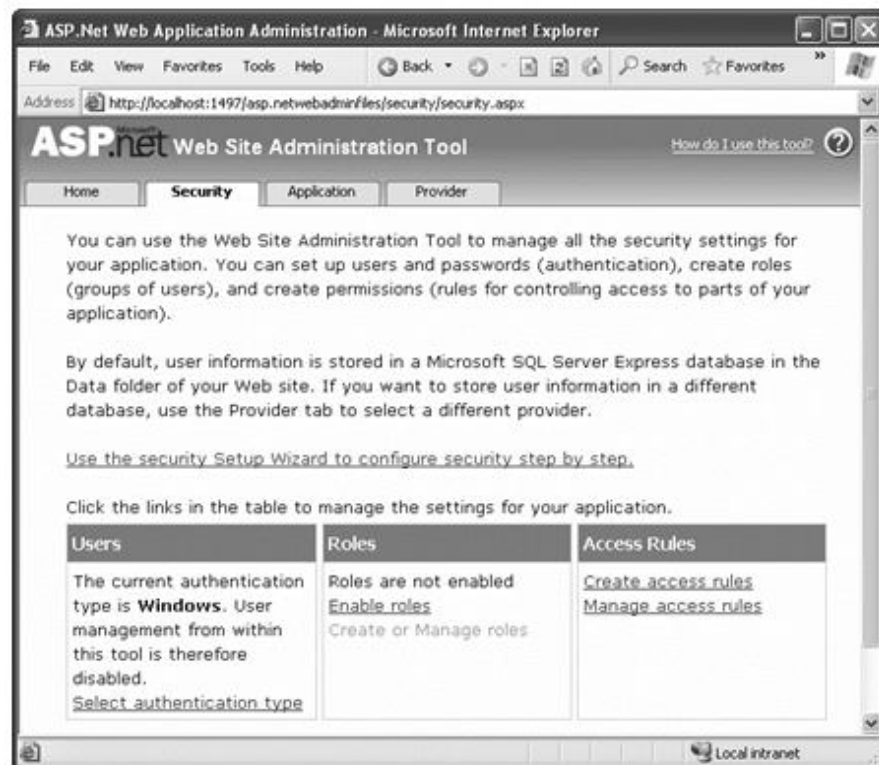


Figure 6: The Security tab in the WAT

To set an application to use forms authentication, follow these steps:

1. Click Select Authentication Type.
2. Choose the From the Internet option. (If you choose From a Local Network instead, you'd wind up using the built-in Windows authentication approach described later in the "Windows Authentication" section.)
3. Click Done. The appropriate <authorization> tag will be created in the web.config file.

The Login Page

Once you've specified the authentication mode and the authorization rules, you need to build the actual login page, which is an ordinary .aspx page that requests information from the user and decides whether the user should be authenticated.

ASP.NET provides a special FormsAuthentication class in the System.Web.Security namespace, which provides static methods that help manage the process. Table describes the most important methods of this class.

Table 2: Members of the Forms Authentication Class

<i>Member</i>	<i>Description</i>
FormsCookieName	Read-only property that provides the name of the forms authentication cookie.
FormsCookiePath	Read-only property that provides the path of the forms authentication cookie.
Authenticate()	Checks a user name and password against a list of accounts that can be entered in the web.config file.
RedirectFromLogin	
Page()	Logs the user into an ASP.NET application by creating the cookie, attaching it to the current response, and redirecting the user to the page originally requested.
	SignOut() Logs the user out of the ASP.NET application by removing the current encrypted cookie.
SetAuthCookie()	Logs the user into an ASP.NET application by creating and attaching the forms authentication cookie. Unlike the RedirectFromLoginPage() method, it doesn't forward the user back to the initially requested page.
GetRedirectUrl()	Provides the URL of the originally requested page. You could use this with SetAuthCookie() to log a user into an application and make a decision in your code whether to redirect to the requested page or use a more suitable default page.
GetAuthCookie()	Creates the authentication cookie but doesn't attach it to the current response. You can perform additional cookie customization and then add it manually to the response.

Windows Authentication

With Windows authentication, the web server takes care of the authentication process. ASP.NET simply makes this identity available to your code for your security checks.

When you use Windows authentication, you force users to log into IIS before they're allowed to access secure content in your website. The user login information can be transmitted in several ways (depending on the network environment, the requesting browser, and the way IIS is configured), but the end result is that the user is authenticated using a local Windows account. Typically, this makes Windows authentication best suited to intranet scenarios, in which a limited set of known users is already registered on a network server.

To implement Windows-based security with known users, you need to follow three steps:

1. Set the authentication mode to Windows authentication in the web.config file. (If you prefer a graphical tool, you can use the WAT during development or IIS Manager after deployment.)
2. Disable anonymous access for a directory by using an authorization rule.
3. Configure the Windows user accounts on your web server (if they aren't already present).

Web.config Settings

To use Windows authentication, you need to make sure the <authentication> element is set accordingly in your web.config file.

```
<configuration>
```

```
<system.web>
```

```
<authentication mode="Windows" />
```

```
<authorization>
```

```
<deny users="?" />
```

```
</authorization>
```

```
...
```

```
</system.web>
```

```
</configuration>
```

```
<allow users="WebServer\matthew" />
```

At the moment, there's only one authorization rule, which uses the question mark to refuse all anonymous users. This step is critical for Windows authentication (as it is for forms authentication). Without this step, the user will never be forced to log in.

Ideally, you won't even see the login process take place. Instead, Internet Explorer will pass along the credentials of the current Windows user, which the web server uses automatically. The VisualStudio integrated web server works this way. IIS also works this way, provided you've set up integrated Windows authentication and the browser supports it.

You can also add `<allow>` and `<deny>` elements to specifically allow or restrict users from specific files or directories. Unlike with forms authentication, you need to specify the name of the server or domain where the account exists. For example, this rule allows the user account `matthew`, which is defined on the computer named `WebServer`:

For a short cut, you can use `local host` (or just a period) to refer to an account on the current computer, as shown here: `<allow users=".\\matthew"/>`

You can also restrict certain types of users, provided their accounts are members of the same Windows group, by using the `roles` attribute: `<authorization>`

```
<deny users="?" />
```

```
<allow roles=".\\SalesAdministrator,\\SalesStaff" />
```

```
<deny users=".\\matthew" />
```

```
</authorization>
```

Table 3 Default Windows Roles

Role	Description
AccountOperator	Users with the special responsibility of managing the user accounts on a computer or domain.
Administrator	Users with complete and unrestricted access to the computer or domain. (If the web server computer uses user account control [UAC], Windows will hold back administrator privileges from administrator accounts, to reduce the risk of viruses and other malicious code.)

BackupOperator	Users who can override certain security restrictions only as part of backing up or restore operations.
Guest	Like the User role but even more restrictive. PowerUser Similar to Administrator but with some restrictions.
PrintOperator	Like User but with additional privileges for taking control of a printer.
Replicator	Like User but with additional privileges to support file replication in a domain.
SystemOperator	Similar to Administrator with some restrictions. Generally, system operators manage a computer.

A Windows Authentication Test

One of the nice features of Windows authentication is that no login page is required. Depending on the authentication protocol you're using (see Chapter 26 for the complete details), the login process may take place automatically or the browser may show a login dialog box. Either way, you don't need to perform any additional work. You can retrieve information about the currently logged-on user from the `Userobject`.

As you learned earlier, the `User` object provides identity information through the `User.Identity` property. Depending on the type of authentication, a different identity object is used, and each identity object can provide customized information. To get some additional information about the identity of the user who has logged in with Windows authentication, you can convert the generic `Identity` object to a `Windows Identity` object (which is defined in the `System.Security.Principal` namespace).

The following is a sample test page that uses Windows authentication. To use this code as written, you need to import the `System.Security.Principal` namespace (where the `Windows Identity` class is defined).

```
public partial class SecuredPage : System.Web.UI.Page
{
    protected void Page_Load(Object sender, EventArgs e)
    {
```

```
StringBuilder displayText = new System.Text.StringBuilder(); displayText.Append("You have reached the secured page, "); displayText.Append(User.Identity.Name);
```

```
WindowsIdentity winIdentity = (WindowsIdentity)User.Identity; displayText.Append("<br /><br />Authentication Type: "); displayText.Append(winIdentity.AuthenticationType); displayText.Append("<br />Anonymous: "); displayText.Append(winIdentity.IsAnonymous); displayText.Append("<br />Authenticated: "); displayText.Append(winIdentity.IsAuthenticated); displayText.Append("<br />Guest: "); displayText.Append(winIdentity.IsGuest); displayText.Append("<br />System: "); displayText.Append(winIdentity.IsSystem); displayText.Append("<br />Administrator: "); displayText.Append(User.IsInRole(@"BUILTIN\Administrators")
```

```
);
```

```
lblMessage.Text = displayText.ToString();
```

```
}
```

```
}
```

11.8 Summary

XML is designed as an all-purpose format for organizing data. XML elements are case sensitive, so and are completely different elements. The classes here allow you to read and write XML files, manipulate XML data in memory, and even validate XML documents. When you're creating an XML file on your own, you don't need to create a corresponding XSD file. XSLT allows you to create style sheets that can extract a portion of a large XML document or transform an XML document into another type. Although this security model is powerful and profoundly flexible, it can appear confusing because of the many different layers that it includes.

Keywords

Classes, Validation, Web security

11.9 Check your progress:

1.To start a document which calling statement is used in xmltextwriter?

- A. StartDocument()
- B. WriteStartDocument()**
- C. DocumentStart()
- D. WriteDocument()

2.Which statement is used to add another element in the nested current element?

- A. WrtiteString()
- B. WriteNestedElement()
- C. WriteStartElement()**
- D. WrtieStartNested()

3.When information are retrieved as string which method is used to convert to right data type?

- A. Int8.Parse()
- B. Int32.Parse()**
- C. Int16.Parse()
- D. Int32.Parse()

11.10 Model Questions

1. Write short notes on XML Classes?
2. Write a short note on XML Validation?
3. Write a short note on XML Display and Transform?
4. Explain the xml website security?
5. Explain in detail about authentication and authorization?

MODEL QUESTION PAPER
MASTER COMPUTER APPLICATIONS
FIRST YEAR - SECOND SEMESTER
CORE PAPER - X
INTER DISCIPLINARY - II
WEB BASED APPLICATION DEVELOPMENT

Time: 3 Hours

Maximum: 80 marks

Section A (10X2 = 20 marks)

Answer any 10 questions

1. Write any four features of .net framework 4.0
2. What is a web service? Give example
3. Define namespace. Write syntax for its creation
4. Name some XML classes
5. Give the steps for creating user controls
6. What is dynamic graphics?
7. Illustrate grid views
8. How data binding is done?
9. List the advantages of debugging.
10. Differentiate HTML and XML
11. Define cookie. Give example
12. What is an application state?

Section B (5X6= 30 marks)**Answer any five questions**

13. Describe ASP.net life cycle
14. Explain object based manipulation in C#
15. Explain repeated value data binding
16. Elaborate on URL mapping
17. Discuss on HTML control classes
18. How transferring of information between pages is done? Give example
19. Elucidate web control events

Section C (#X10 = 30 marks)**Answer any three questions**

20. Write a C# code to illustrate delegates and events
21. How do forms authentication done using XML
22. Illustrate calendar and ad rotator rich control usage
23. Give an account on exception handling
24. Create a web form using HTML CSS, XML code for student information registration